

大规模并行处理器编程

实践导向（第 5 版）

Wen-mei W. Hwu、David B. Kirk、Izzat El Hajj 著

中文学习译稿

非官方个人学习版本

原书版权归 Elsevier Inc. 及相关权利人所有

2026 年 8 月 25 日

译稿说明

本译稿以原书第五版为底本，目标是为中文读者提供一份便于阅读、编译和逐章校对的学习材料。它不是出版社授权的中文译本，也不替代原书。

本版本首先验证排版、术语和章节组织方式；正文翻译将按章节逐步完成。原书中的代码标识符、API 名称、公式和参考文献尽量保持原貌，必要时添加译者注。图表在没有适合的可再分发表文件时先以中文概念示意重绘。

目录

译稿说明	ii
第一章 引言	1
1.1 异构并行计算	1
1.1.1 多核与众线程路线	2
1.2 为什么需要更高速度或更多并行性?	6
1.3 加速真实应用	7
1.4 并行编程中的挑战	8
1.5 相关并行编程接口	10
1.6 总体目标	11
1.7 本书的组织结构	12
参考文献	15
 第一部分 基础概念	 16
第二章 异构数据并行计算	17
2.1 数据并行性	17
2.2 CUDA C++ 程序结构	19
2.3 向量加法示例	20
2.4 设备全局内存与数据传送	23
2.5 内核函数与线程	27
2.6 调用内核函数	31
2.7 编译	33
2.8 小结	33

2.9 练习	34
参考文献	37
第三章 多维网格与数据	38
3.1 多维网格组织	38
3.2 将线程映射到多维数据	41
3.3 图像模糊——一个更复杂的内核	48
3.4 矩阵乘法	51
3.5 小结	55
练习	55
第四章 计算架构与调度	57
4.1 现代 GPU 的架构	57
4.2 线程块调度	58
4.3 同步与透明可扩展性	59
4.4 Warp 与 SIMD 硬件	62
4.5 控制流分歧	65
4.6 Warp 调度与延迟容忍	67
4.7 资源划分与占用率	69
4.8 查询设备属性	70
4.9 小结	72
练习	73
参考文献	75
第五章 内存架构与数据局部性	76
5.1 内存带宽作为性能限制因素	76
5.2 CUDA 内存类型	80
5.3 通过分块减少内存流量	83
5.4 分块矩阵乘法内核	85
5.5 边界检查	89
5.6 内存使用对占用率的影响	92
5.7 小结	94

练习	95
参考文献	97
第六章 性能考量	98
6.1 全局内存访问合并	98
6.2 隐藏内存延迟	104
6.3 向量加载与存储	108
6.4 共享内存 bank 冲突	109
6.5 线程粗化	111
6.6 循环展开	113
6.7 双缓冲	116
6.8 优化检查清单	117
6.9 优化策略	121
6.10 小结	122
练习	122
参考文献	124
第二部分 并行模式	125
第七章 卷积	126
7.1 背景	126
7.2 并行卷积——一个基本内核	130
7.3 内存带宽考量	132
7.4 常量内存与缓存	133
7.5 带 halo 单元的分块卷积	137
7.6 使用缓存处理 halo 单元的分块卷积	141
7.7 小结	143
练习	144
第八章 模板计算与线程粗化	146
8.1 背景	146
8.2 并行模板：基本内核	149

8.3	内存带宽考量	150
8.4	模板扫描的共享内存分块	151
8.5	线程粗化	153
8.6	寄存器分块	156
8.7	小结	158
8.8	练习	158
第九章	直方图	160
9.1	背景	160
9.2	原子操作与基本直方图内核	162
9.3	原子操作的延迟与吞吐量	166
9.4	私有化	167
9.5	线程粗化	170
9.6	线程级私有化	173
9.7	小结	176
	练习	176
	参考文献	177
第十章	归约	178
10.1	背景	178
10.2	归约树	179
10.3	一个简单的归约内核	183
10.4	减少控制流分歧	185
10.5	减少内存访问分歧	188
10.6	减少全局内存访问	189
10.7	用 warp 级原语减少同步开销	190
10.8	用两阶段 warp 内归约进一步减少同步开销	194
10.9	任意长度输入的归约	196
10.10	用线程粗化降低开销	199
10.11	小结	201
	练习	201
	参考文献	202

第三部分 高级模式与应用	203
第十七章 稀疏矩阵计算	204
17.1 背景	204
17.2 COO 格式与最简单的 SpMV 内核	206
17.3 用 CSR 格式按行分组非零元素	209
17.4 用 ELL 格式改善内存访问合并	212
17.5 用混合 ELL-COO 格式调节填充量	215
17.6 用 JDS 格式降低控制分歧	217
17.7 用 CSC 格式支持按列访问	220
17.8 小结	222
练习	223
参考文献	224
术语表 (待扩充)	225

第一章 引言

1.1 异构并行计算

从计算诞生之初起，许多高价值应用就要求比计算设备所能提供的更多执行速度和资源。早期应用依靠处理器速度、内存速度和内存容量的提升，来增强应用层面的能力，例如提高天气预报的及时性、工程结构分析的准确性、计算机生成图形的逼真度，以及每秒处理的航班预订数或资金转账数。近年来，深度学习等新应用对执行速度和资源的需求甚至超过了最先进计算设备的供给。这些应用需求在过去五十年中推动了计算设备能力的快速发展，并且在可预见的未来仍将继续推动这种发展。

以单个中央处理器（central processing unit, CPU）为基础、看起来按照顺序执行指令的微处理器，例如 Intel 和 AMD 的 x86 处理器，在 20 世纪 80 年代和 90 年代依靠不断提高的时钟频率和硬件资源，推动了计算应用性能的快速提升和成本的持续下降。在这二十年的增长期内，单 CPU 微处理器把 GFLOPS（giga，即 10^9 次浮点运算每秒）带到了桌面计算机，把 TFLOPS（tera，即 10^{12} 次浮点运算每秒）带到了数据中心。这种对性能改进的不懈追求，使应用软件能够提供更多功能、更好的用户界面以及更有用的结果。用户一旦习惯了这些改进，就会进一步要求更多改进，从而形成计算机产业的正向（良性）循环。

然而，由于能耗和散热问题，这种发展势头在 2003 年之后放缓了。这些问题限制了时钟频率的提升，也限制了单个 CPU 在保持“按顺序执行指令”这一表象的同时、能够在每个时钟周期内完成的有效工作量。此后，几乎所有微处理器厂商都转向在每个芯片中使用多个物理 CPU（称为处理器核心）来提高处理能力。在这种模型下，传统 CPU 可以看作单核 CPU。要从多个处理器核心中获益，用户必须拥有多个指令序列——它们可以来自同一个应用，也可以来自不同应用——以便在这些处理器核心上同时执行。对于某个应用而言，要想利用多个处理器核心，就必须把其工作划分为多个能够同时在这些核心上执行的指令序列。这种从单 CPU 按顺序执行指令转向多核心并行执行多个指令序列的变化，对软件开发群体产生了深远影响。

传统上，绝大多数软件应用都是顺序程序；处理器的设计思想源自冯·诺伊曼在 1945 年发表的奠基性报告 [1]。人们可以借助程序计数器（program counter，在相关文献中也称 instruction pointer）这一概念，把这些程序的执行理解为沿着程序代码逐

条向前推进。程序计数器保存处理器将要执行的下一条指令的内存地址。应用以这种逐步顺序方式执行指令所形成的指令执行序列，在相关文献中称为执行线程（thread of execution），简称线程。本书后文会更正式地定义线程，并广泛使用这一概念；它的重要性由此可见。

历史上，大多数软件开发者依靠硬件进步——例如更高的时钟频率，以及硬件在幕后同时执行多条指令——来提高顺序应用的速度。随着新一代处理器问世，同一份软件通常会自动运行得更快；计算机用户也习惯了每一代微处理器都能让程序运行得更快。然而，这种预期已经十多年没有成为现实。顺序程序只能运行在某个处理器核心上，而单个核心在不同代际之间已经不再显著变快。随着性能改进停滞，顺序应用开发者无法再仅靠新微处理器的推出，为软件加入新的特性和能力，这也削弱了整个计算机产业的增长机会。

相反，在每一代微处理器上仍能持续获得显著性能提升的，是并行软件：多个执行线程协作完成工作，从而更快地完成任务。并行程序相对于顺序程序的这种优势急剧扩大，被称为“并发革命”[2]。并行编程并非新实践。高性能计算领域已经开发并行程序数十年；这些程序通常运行在规模庞大且价格昂贵的计算机上。只有少数精英应用能够证明使用这类昂贵计算机是合理的，因此并行编程一度只属于少数应用开发者。然而，当所有微处理器都变成并行计算机之后，需要以并行程序形式开发的应用数量急剧增加。软件开发者学习并行编程的需求也随之大幅增长，而这正是本书的重点。

1.1.1 多核与众线程路线

自 2003 年以来，半导体产业主要沿着两条微处理器设计路线发展 [3]：多核（multi-core）和众线程（many-thread）。多核路线试图在转向多个核心的同时，保持顺序程序的执行速度。多核处理器最初是双核处理器，之后核心数量随着每一代半导体工艺不断增加。例如，Intel 和 AMD 的多核服务器微处理器可以拥有超过 100 个处理器核心；每个核心都是支持乱序执行和多指令发射、实现完整 x86 指令集的处理器，并支持两个硬件线程的超线程技术，目标是最大化顺序程序的执行速度。另一个例子是 ARM Ampere 多核服务器处理器，最多可拥有 128 个处理器核心。

相比之下，众线程路线更加关注并行应用的执行吞吐量。众线程路线从大量线程起步，之后线程数量也随着每一代产品不断增加。近期的代表是 NVIDIA Hopper H100 图形处理器（graphics processing unit, GPU）：它拥有数十万个线程，这些线程在大量简单的顺序发射流水线上执行。自 2003 年以来，众线程处理器，尤其是 GPU，一直在浮点性能竞赛中领先。截至 2022 年，H100 GPU 的峰值浮点吞吐量为：64 位双精度 34 TFLOPS、32 位单精度 67 TFLOPS，以及 16 位半精度 1979 TFLOPS。相比之下，同年典型服务器级 CPU 的峰值浮点吞吐量只有几 TFLOPS。众线程 GPU 与多核 CPU 之间的峰值浮点计算吞吐量差距在过去几年持续扩大。需要注意的是，

这些数字不一定代表应用速度，而只是芯片中的执行资源在理论上能够支持的原始速度。

多核处理器和众线程处理器之间如此巨大的峰值性能差距，仿佛积累起了很大的“电势”；最终总会有某种变化发生。我们已经到达了这一刻。迄今为止，这种巨大的峰值性能差距已经促使许多应用开发者把软件中计算密集的部分迁移到 GPU 上执行。也许更重要的是，并行执行能力的大幅提升使深度学习等革命性新应用成为可能；这类应用天然由计算密集的部分组成。不出意外，这些计算密集部分也正是并行编程的首要目标：工作越多，就越有机会把工作分配给协作的并行工作者，也就是线程。

为什么众线程 GPU 与多核 CPU 之间会存在如此巨大的峰值性能差距？答案在于这两类处理器截然不同的基本设计理念，如图 1.1 所示。CPU 的设计（图 1.1 中的左侧部分）针对顺序代码性能进行了优化。算术单元和操作数数据传送逻辑经过设计，以尽量降低算术运算的有效延迟，但代价是每个单元占用更多芯片面积和功耗。较大的片上末级缓存用于捕获频繁访问的数据，把一部分长延迟内存访问转化为短延迟缓存访问。复杂的分支预测逻辑和执行控制逻辑则用来缓解条件分支指令带来的延迟。通过降低操作延迟，CPU 硬件降低了每个独立线程的执行延迟。然而，低延迟算术单元、复杂的操作数传送逻辑、大容量缓存和复杂控制逻辑都会消耗芯片面积与功耗；这些资源本可以用来提供更多算术执行单元和内存访问通道。这种设计方法通常称为面向延迟的设计（latency-oriented design）。

另一方面，GPU 的设计理念受到快速增长的电子游戏产业塑造。该产业带来了巨大的经济压力，要求在高级游戏的每一帧画面中完成海量浮点计算和内存访问。这样的需求促使 GPU 厂商设法把尽可能多的芯片面积和功耗预算用于浮点计算与内存访问吞吐量。

图形应用每秒需要完成海量浮点计算，例如视点变换和物体渲染，这一点很直观。此外，每秒进行海量内存访问同样重要，甚至可能更加重要。许多图形应用的速度受限于数据从内存系统送入处理器、以及从处理器送回内存系统的速率。GPU 必须能够在其 DRAM（动态随机存取存储器）与图形帧缓冲区之间搬运极大量数据，因为正是这种数据移动让视频显示对玩家而言丰富而有吸引力。游戏应用通常接受一种较为宽松的内存模型（即各种系统软件、应用和 I/O 设备对于内存访问应如何工作的共同预期），这也让 GPU 更容易支持对内存进行大规模并行访问。

相比之下，通用处理器必须满足传统操作系统、应用和 I/O 设备提出的要求。这些要求使并行内存访问更加困难，也使提高内存访问吞吐量——通常称为内存带宽（memory bandwidth）——更加困难。因此，图形芯片的内存带宽一直约为同时代 CPU 芯片的 10 倍，我们预计在一段时间内 GPU 仍将在内存带宽方面占据优势。

一个重要观察是：从功耗和芯片面积的角度看，降低延迟比提高吞吐量昂贵得多。例如，把算术单元数量加倍，可以把算术吞吐量加倍，代价大致是芯片面积和功耗加倍；但要把算术延迟减半，可能需要把电流加倍，芯片面积增加到两倍以上，功耗增

加到四倍。因此，GPU 的主流方案是优化大量线程的执行吞吐量，而不是降低单个线程的延迟。这种设计方法允许流水化的内存通道和算术操作承受较长延迟，从而节省芯片面积和功耗。内存访问硬件与算术单元所节省下来的面积和功耗，可以让 GPU 设计者在同一芯片上放置更多这样的单元，进而提高总体执行吞吐量。图 1.1 通过视觉方式展示了两设计方法的差异：CPU 设计包含较少但更大的算术单元和较少的内存通道，而 GPU 设计包含更多但更小的算术单元以及更多内存通道。

	CPU	GPU
设计取向	面向延迟：尽量降低单个线程的操作延迟	面向吞吐量：最大化大量线程的总体执行吞吐量
控制逻辑	复杂的控制逻辑和分支预测，减少控制与分支延迟	控制逻辑相对简化，通过大量可运行线程隐藏延迟
算术单元	较少但较大的算术单元	更多但较小的算术单元
缓存层次	靠近计算单元的小缓存，以及用于捕获频繁访问数据的较大末级缓存	较小的缓存，用于控制带宽需求并减少对 DRAM 的重复访问
片外内存	较少的内存通道连接 DRAM	较多的内存通道连接高带宽 DRAM
适合的工作	一个或少量线程的顺序工作	大量线程可协作的数据并行工作

图 1.1: CPU 与 GPU 的基本设计理念（依据原书图 1.1 重绘的概念示意）

GPU 应用软件通常需要使用大量并行线程编写。硬件可以利用大量线程，在其中一些线程等待长延迟内存访问或算术操作时，寻找其他可执行的工作。图 1.1 右侧所示的小型缓存用于控制这些应用的带宽需求，使访问相同内存数据的多个线程不必全部访问 DRAM。这种设计风格通常称为面向吞吐量的设计（throughput-oriented design）：它允许单个线程可能需要很长时间执行，同时努力最大化大量线程的总体执行吞吐量。

应当明确，GPU 是为并行、面向吞吐量的计算引擎而设计的；对于 CPU 擅长的某些任务，GPU 并不会表现良好。对于只有一个或很少线程的程序，具有更低操作延迟的 CPU 可以获得远高于 GPU 的性能。当程序拥有大量线程时，具有更高执行吞吐量的 GPU 则可以获得远高于 CPU 的性能。因此，应当预期许多应用会同时使用 CPU 和 GPU：在 CPU 上执行应用的顺序部分，在 GPU 上执行数值密集部分。这正是 NVIDIA 于 2007 年引入的 CUDA 编程模型被设计用来支持 CPU-GPU 协同执行应用的原因。

应用开发者选择处理器时，速度也不是唯一的决策因素。其他因素甚至可能更加重要。首先，所选处理器必须在市场上拥有非常广泛的存在，这称为处理器的装机量

(installed base)。原因很简单：只有面对庞大的客户群体，软件开发成本才有充分的经济合理性。运行在市场占有率很小的处理器上的应用不会拥有广泛的客户群体。与通用微处理器相比，传统并行计算系统的市场存在极其有限，这一直是一个主要问题。只有政府和大公司资助的少数精英应用，成功地在这类传统并行计算系统上实现了开发。众线程 GPU 改变了这一点。由于 GPU 在个人计算机市场上非常流行，其出货量已经达到数亿。几乎所有台式 PC 和高端笔记本电脑都配有 GPU。如今已有超过 10 亿个支持 CUDA 的 GPU 在使用。如此庞大的市场存在，使这些 GPU 成为对应用开发者具有经济吸引力的目标。

另一个重要决策因素是实用的形态和易于获得的运行环境。2006 年以前，并行软件应用主要运行在数据中心服务器或部门级集群上，但这类执行环境往往限制了应用的使用。例如，在医学成像应用中，基于 64 节点集群机器发表论文没有问题；但实际临床应用中的 MRI 设备通常由 PC 与专用硬件加速器组合而成。原因很简单：GE 和 Siemens 等制造商不可能向临床场所销售必须配套几机架计算服务器的 MRI 系统，而这样的部署在高校院系环境中却很常见。事实上，美国国立卫生研究院（NIH）曾有一段时间拒绝资助并行编程项目，因为他们认为巨型集群机器无法用于临床环境，并行软件的影响会受到限制。如今，许多公司已经推出配备 GPU 的 MRI 产品，NIH 也在资助 GPU 计算研究。

在 2006 年以前，图形芯片很难使用，因为程序员必须通过相当于图形 API（application programming interface，应用程序接口）的函数访问处理单元，也就是需要使用 OpenGL 或 Direct3D 技术来编程这些芯片。更直白地说，计算必须表达成某种“绘制像素”的函数，才能在早期 GPU 上执行。这种技术称为 GPGPU，即使用图形处理器进行通用编程（General Purpose Programming using a Graphics Processing Unit）。业界更常见的展开是“General-Purpose Computing on Graphics Processing Units”；这里保留原书的表述，并不把两种展开混为一谈。即使有更高层的编程环境，底层代码仍然需要适配为面向像素绘制的 API。这些 API 限制了早期 GPU 实际能够编写的应用类型，因此 GPGPU 没有成为普遍的编程现象。尽管如此，这项技术仍然足够令人兴奋，激励人们完成了一些富有开创性的工作并取得了优秀的研究成果。

2007 年 CUDA 发布后，一切发生了变化 [4]。CUDA 带来的并不只是软件变化，芯片中还增加了额外硬件。NVIDIA 实际上专门划出硅片面积来简化并行编程。在用于并行计算的 G80 及其后继芯片中，GPGPU 程序不再需要经过图形接口，而是由芯片上的新型通用并行编程接口直接处理 CUDA 程序的请求。这个通用编程接口极大扩展了可以在 GPU 上轻松开发的应用类型。此外，其他软件层也全部重新设计，使程序员能够使用熟悉的 C/C++ 编程工具。

虽然 GPU 是异构并行计算中的重要计算设备类型，但异构计算系统还会把其他重要设备用作加速器。例如，现场可编程门阵列（field programmable gate array, FPGA）被广泛用于加速网络应用。本书以 GPU 为学习载体所介绍的技术，同样适用于这类加速器的编程任务。

1.2 为什么需要更高速度或更多并行性？

正如第 1.1 节所述，大规模并行编程的主要动机，是让应用在未来几代硬件上继续获得速度提升。在“并行模式”和“高级模式与应用”两部分（第 2、3 部分，即第 7 至第 23 章）中我们会看到，对于适合并行执行的应用，优秀的 GPU 实现相对于单个 CPU 核心上的顺序执行，可以达到超过 $100\times$ 的加速比。如果应用包含我们所称的“数据并行性”，仅用几个小时的工作往往就可以获得 $10\times$ 的加速。

有人可能会问，为什么应用会继续要求更高的速度？今天的许多应用似乎已经运行得足够快了。尽管当今世界有无数计算应用，未来许多令人兴奋的大众市场应用，实际上就是我们过去称为“超级计算应用”或“超级应用”的东西。

例如，生物学研究界正越来越深入分子层面。显微镜可以说是分子生物学最重要的仪器，过去主要依靠光学或电子仪器。然而，我们可以借助这些仪器进行的分子级观测存在限制。把计算模型纳入进来，模拟底层分子活动，并用传统仪器设置边界条件，可以有效地解决这些限制。借助仿真，我们能够测量比单靠传统仪器所能想象的更多细节，也能检验更多假设。在可预见的未来，仿真会继续受益于计算速度的提升：在可接受的响应时间内，能够建模的生物系统规模会扩大，能够模拟的反应时间也会延长。这些增强将对科学和医学产生巨大影响。

对于视频、音频编码和处理等应用，可以想想我们对数字高清电视（HDTV）相对于旧式 NTSC 电视的满意程度。体验过 HDTV 的细节水平之后，人们很难再回到旧技术。但请考虑 HDTV 所需的全部处理工作：它是高度并行的过程，三维成像和可视化也一样。未来，视图合成、把低分辨率视频以高分辨率显示等新功能，将需要电视设备提供更多计算能力。在消费级产品中，我们正看到越来越多的视频和图像处理应用，用来改善照片和视频的对焦、光照以及其他关键方面。

更高的计算速度还可以带来更好的用户界面。现代智能手机用户享受着更加自然的交互界面，高分辨率触摸屏的显示效果足以媲美大屏电视。毫无疑问，未来版本的这类设备将加入具有三维视角的传感器和显示器、把虚拟空间与物理空间信息结合起来以提高可用性的应用，以及基于语音和计算机视觉的交互界面；这些功能都需要更高的计算速度。

消费电子游戏中也在发生类似的发展。过去，游戏中的驾驶实际上只是预先安排好的一组场景：汽车撞上障碍物后，车辆行驶路线并不会改变，只有游戏分数发生变化；车轮不会弯曲或损坏，无论撞到车轮还是失去一个车轮，驾驶难度都不会改变。随着计算速度提升，游戏可以基于动态仿真，而不再基于预先安排的场景。未来我们可以体验更多这样的真实效果：事故会损坏车轮，在线驾驶体验也会逼真得多。准确建模物理现象的能力已经催生了“数字孪生”概念：物理对象在仿真空间中拥有准确模型，可以以低得多的成本进行充分的压力测试和劣化预测。对物理效应进行逼真的建模和仿真，通常需要极其巨大的计算能力。

计算吞吐量大幅提升所带来的新应用中，一个重要例子是基于人工神经网络的深度学习。尽管神经网络从 20 世纪 70 年代起就一直受到积极研究，但由于训练需要太多带标签数据和太多计算，它们在实际应用中一度效果不佳。互联网的兴起带来了海量带标签的图片，GPU 的兴起则带来了计算吞吐量的跃升。结果，自 2012 年以来，基于神经网络的应用在计算机视觉和自然语言处理领域迅速普及。这种普及彻底改变了计算机视觉和自然语言处理应用，也推动了自动驾驶汽车和家庭助手设备的快速发展。

上述所有新应用都以不同方式、不同层次模拟和/或表示一个物理的、并发的世界，并且需要处理极大量数据。面对如此巨大的数据量，大部分计算都可以在数据的不同部分上并行完成，尽管这些结果最终需要在某个时刻汇合。多数情况下，有效管理数据传送会对并行应用能够达到的速度产生重大影响。少数每天处理这类应用的专家往往熟悉相关技术，但绝大多数应用开发者都能从对这些技术更直观的理解和实践性工作知识中获益。

我们的目标是以直观的方式向应用开发者介绍数据管理技术，而他们的正规教育未必来自计算机科学或计算机工程。我们还希望提供大量实用代码示例和动手练习，帮助读者获得真正可工作的知识；这要求有一种既便于并行实现、又支持恰当管理数据传送的实用编程模型。CUDA 提供了这样的编程模型，并且已经得到庞大开发者社区的充分检验。

1.3 加速真实应用

把应用并行化后，能够期待多大的加速比？计算系统 A 相对于计算系统 B 的应用加速比，定义为应用在 B 上的执行时间与在 A 上执行同一应用所需时间之比。例如，某应用在系统 A 上需要 10 秒，在系统 B 上需要 200 秒，那么系统 A 相对于系统 B 的加速比为

$$\frac{200 \text{ s}}{10 \text{ s}} = 20,$$

即 $20\times$ （20 倍）加速。

并行计算系统相对于串行计算系统能够达到的加速比，取决于应用中可以并行化的部分。例如，如果应用执行时间中可并行部分占 30%，即使把这部分加速 $100\times$ ，应用总执行时间也最多只能减少 29.7%。换言之，整个应用的加速比只有

$$\frac{1}{1 - 0.297} \approx 1.42 \times .$$

事实上，即使并行部分获得无限大的加速，也只能削减 30% 的执行时间，最高加速比不超过约 $1.43\times$ 。另一方面，如果执行时间的 99% 都属于并行部分，那么将并行部分加速 $100\times$ 后，应用执行时间会降至原来的 1.99%，整个应用可以获得 $50\times$ 加速。因此，要让大规模并行处理器有效加速应用，应用执行时间的绝大部分必须位于并行

部分。并行执行所能达到的加速水平会受到应用可并行部分大小的严重限制，这一事实称为阿姆达尔定律（Amdahl's Law）[5]。

研究人员已经在某些应用上取得超过 $100\times$ 的加速。然而，这通常只有在进行大量优化和调优之后才可能实现，而且往往还需要改进算法，使应用工作量的 99.9% 以上处于并行部分。

应用能够达到的加速水平还取决于从内存读取数据以及向内存写入数据的速度。在实践中，直接对应用进行并行化往往会使内存（DRAM）带宽达到饱和，结果只能获得约 $10\times$ 加速。诀窍在于设法绕过内存带宽限制：通过某种变换利用 GPU 上的专用片上内存，大幅减少对 DRAM 的访问。不过，代码还需要继续优化，以应对片上内存容量有限等限制。本书的一个重要目标，就是帮助读者充分理解这些优化方法并熟练掌握它们。

请记住，相对于单核 CPU 执行所得到的加速水平，也可能反映 CPU 是否适合该应用。在某些应用中，CPU 的表现非常好，因此很难再用 GPU 提升性能。大多数应用都有一些部分更适合由 CPU 执行。必须给 CPU 一个公平的机会，并确保代码编写方式能够让 GPU 与 CPU 执行互补，从而正确利用 CPU/GPU 组合的异构并行计算能力。如今，结合多核 CPU 和众核 GPU 的大众市场计算系统，已经把万亿次级（terascale）计算带到了桌面，把百亿亿次级（exascale）计算带到了集群。

图 1.2 展示了典型应用的主要组成部分。真实应用的大量代码往往是顺序的。图中将这些顺序部分画成桃子的“果核区域”：试图对这些部分应用并行计算技术，就像咬桃核一样——感觉并不好！这些部分很难并行化。CPU 往往能够很好地处理它们。好消息是，尽管这些部分可能占据大量代码，却往往只占超级应用执行时间的一小部分。

接下来是我们所谓的“桃肉区域”。这些部分容易并行化，早期图形应用也是如此。在异构计算系统中进行并行编程，可以大幅提升这类应用的速度。如图 1.2 所示，早期 GPGPU 编程接口只覆盖桃肉区域的一小部分，这类似于只能覆盖最令人兴奋的应用中的一小部分。我们将看到，CUDA 编程接口的设计目标是覆盖更多令人兴奋的应用“桃肉”。并行编程模型及其底层硬件仍在快速演进，以便高效并行化应用中越来越大的部分。

1.4 并行编程中的挑战

并行编程为什么困难？有人曾说过：如果你不在乎性能，并行编程非常容易；你甚至可以在一小时内写出并行程序。但如果不在乎性能，又何必编写并行程序呢？

本书讨论如何在并行编程中实现高性能所面临的几个挑战。首先，设计出与顺序算法具有相同算法（计算）复杂度的并行算法，可能相当困难。许多并行算法与对应的顺序算法执行相同数量的工作。然而，有些并行算法会做更多工作；事实上，有时

应用结构或覆盖范围	中文概念示意
顺序部分 (“桃核”)	难以并行化；传统单核 CPU 通常能够很好地处理这部分。它可能占据大量代码，但往往只占超级应用执行时间的一小部分。
数据并行部分 (“桃肉”)	容易在数据的不同部分上并行执行，是 GPU 加速最有价值的区域。
传统 CPU 覆盖	与顺序部分有较大重叠，但对大规模数据并行部分的覆盖有限。
早期 GPGPU 覆盖	只能覆盖可表达为“绘制像素”的数据并行部分中的一小部分。
CUDA/GPU 覆盖	面向吞吐量的 GPU 和 CUDA 旨在覆盖更大范围的“桃肉”区域。
障碍	功耗约束使得单核 CPU 难以继续扩展，以覆盖更多数据并行工作。

图 1.2: 顺序部分与数据并行部分的应用结构（依据原书图 1.2 重绘的概念示意）

它们会多做如此多工作，以至于在大规模输入数据集上最终运行得更慢。这尤其令人担忧，因为快速处理大规模输入数据集正是并行编程的重要动机。

例如，许多现实问题最自然的描述方式是数学递推式。把这些问题并行化，通常需要以反直觉的方式思考问题，执行时也可能需要重复工作。有一些重要的算法原语，例如前缀和（prefix sum）或扫描（scan），可以帮助我们问题的顺序递归形式转换为更适合并行的形式。第 11 章将更正式地介绍工作效率（work efficiency）概念，并借助前缀和等重要并行模式，说明如何设计具有与对应顺序算法相同计算复杂度的并行算法，以及其中涉及的方法和权衡。

第二，许多应用的执行速度受内存访问延迟和/或吞吐量限制。我们把这类应用称为内存受限（memory-bound）应用，以区别于计算受限（compute-bound）应用；后者受硬件执行算术运算的速率限制。要让内存受限应用实现高性能并行执行，通常需提高内存访问速度的方法。第 5 章和第 6 章将介绍内存访问优化技术，并在多章并行模式与应用中应用这些技术。

第三，相比对应的顺序程序，并程序的执行速度通常对输入数据特征更加敏感。许多现实应用必须处理特征变化很大的输入，例如不规则或不可预测的数据规模，以及不均匀的数据分布。这些规模和分布上的变化，可能导致分配给并行线程的工作量不均，并显著降低并行执行的有效性。并程序的性能有时会随这些特征发生剧烈变化。我们将在介绍并行模式和应用的章节中，介绍使数据分布更加规整，或针对数据特征调整线程工作分配的技术，以应对这些挑战。

第四，有些应用可以并行化，而且不同线程之间几乎不需要协作。这些应用通常称为易并行应用（embarrassingly parallel）。另一方面，另一些应用要求线程彼此协作，这就需要使用屏障或原子操作等同步操作。同步操作会给应用带来开销，因为线

程常常需要等待其他线程，而不是执行有用工作。本书将始终讨论降低这种同步开销的各种策略。

幸运的是，研究人员过去已经解决了其中许多挑战。不同应用领域之间还存在共同模式，使我们能够把一个领域中得到的解决方案应用到其他领域。这正是本书选择在重要并行计算模式和应用的语境中介绍应对这些挑战的关键技术的主要原因。

1.5 相关并行编程接口

过去几十年间，人们已经开发出许多并行编程语言和模型 [6]。应用最广泛的包括：用于共享内存多处理器系统的 OpenMP [7]，以及用于可扩展集群计算的消息传递接口（Message Passing Interface, MPI）[8]。二者都已经成为由主要计算机厂商支持的标准化编程接口。

一个 OpenMP 实现由编译器和运行时系统组成。程序员向 OpenMP 编译器指定有关循环的指令（directive）和编译提示（pragma）。借助这些指令和提示，OpenMP 编译器生成并行代码。运行时系统通过管理并行线程和资源，支持并行代码的执行。OpenMP 最初为 CPU 执行设计，后来扩展到支持 GPU 执行。OpenMP 的主要优点是，它提供编译器自动化和运行时支持，从而把许多并行编程细节抽象掉。这样的自动化和抽象，有助于让应用代码在不同厂商生产的系统之间，以及同一厂商不同代际的系统之间，实现更好的可移植性。我们把这一性质称为性能可移植性（performance portability）。不过，要有效地使用 OpenMP，程序员仍然需要理解涉及的全部并行编程概念。由于 CUDA 让程序员能够显式控制这些并行编程细节，因此即使读者打算把 OpenMP 作为主要编程接口，CUDA 也是很好的学习载体。此外，根据我们的经验，OpenMP 编译器仍在不断发展和改进。许多程序员可能仍需在 OpenMP 编译器能力不足的部分使用 CUDA 风格的接口。

MPI 是一种编程接口：集群中的计算节点不共享内存，所有数据共享和交互都必须通过显式消息传递完成。MPI 已广泛用于高性能计算（high-performance computing, HPC）。使用 MPI 编写的应用已经在超过 100,000 个节点的集群计算系统上成功运行。如今，许多 HPC 集群采用异构 CPU/GPU 节点。由于计算节点之间没有共享内存，把应用移植到 MPI 可能需要相当大的工作量。程序员需要进行域分解，把输入和输出数据划分到各个节点上；还需要根据域分解结果调用消息发送和接收函数，管理节点之间的数据交换。

另一方面，CUDA 为 GPU 内部的并行执行提供了共享内存，可以解决这一困难。CUDA 适合节点内部的编程，但大多数应用开发者仍需要使用 MPI 一类的接口来编程集群层级。随着 MPI 本身逐渐具备 CUDA 感知能力，以及 NCCL 和 NVSHMEM 等其他 API 不断提供支持，CUDA 的多 GPU 编程能力也在增强。因此，在使用多 GPU 节点的现代计算集群中，并程序员必须理解如何在 MPI 等多 GPU 编程模型

的语境下进行 CUDA 编程；第 23 章将介绍这一内容。

2009 年，包括 Apple、Intel、AMD/ATI 和 NVIDIA 在内的多个产业参与者共同开发了一种标准化编程模型，称为开放计算语言（Open Computing Language, OpenCL）[9]。原书正文将其写作“Open Compute Language”；译文采用 OpenCL 的官方展开。与 CUDA 类似，OpenCL 编程模型定义了语言扩展和运行时 API，允许程序员管理大规模并行处理器中的并行性和数据传送。与 CUDA 相比，OpenCL 更多依赖 API，而较少依赖语言扩展。这样，厂商就可以快速调整现有编译器和工具，以处理 OpenCL 程序。

OpenCL 是一种标准化编程模型：使用 OpenCL 开发的应用，无需修改，就能在所有支持 OpenCL 语言扩展和 API 的处理器上正确运行。不过，要在新的处理器上获得高性能，通常仍然需要修改应用。熟悉 OpenCL 和 CUDA 的读者会知道，二者的关键概念和特性非常相似。也就是说，CUDA 程序员只需很少努力就能学会 OpenCL 编程。更重要的是，使用 CUDA 学到的几乎所有技术，都可以轻松应用于 OpenCL 编程。

1.6 总体目标

我们的首要目标是教会读者如何编程大规模并行处理器，以实现高性能。因此，本书很大一部分内容都致力于开发高性能并行代码的技术。我们的方法不要求读者具备大量硬件专业知识；不过，读者需要对并行硬件架构有良好的概念性理解，才能推理代码的性能行为。因此，我们会用一些篇幅帮助读者直观理解基本硬件架构特征，并用大量篇幅介绍开发高性能并程序的技术。特别是，我们将重点讨论计算思维（computational thinking）[10] 技术，使读者能够用适合在大规模并行处理器上高性能执行的方式思考问题。

在大多数处理器上进行高性能并行编程，都需要了解硬件的工作方式。要构建工具和机器，使程序员不必了解这些知识就能开发高性能代码，可能还需要很多年。即使有了这样的工具，我们也怀疑了解硬件的程序员会比不了解硬件的人更有效地使用它们。因此，第 4 章将介绍 GPU 架构的基础；在讨论高性能并行编程技术时，我们还会讨论更加专门的架构概念。

我们的第二个目标是教授如何编写功能正确且可靠的并程序；这是并行计算中一个微妙的问题。曾经开发过并行系统的人都知道，实现初始性能还不够，真正的挑战是以一种能够调试代码并支持用户的方式实现性能。CUDA 编程模型鼓励使用简单形式的屏障同步、原子性和内存一致性来管理并行性。此外，它还提供了一组强大的工具，不仅可以调试功能问题，也可以定位性能瓶颈。

我们的第三个目标是让程序能够跨越未来的硬件代际进行扩展：探索并行编程方法，使未来更加并行的机器能够比今天的机器更快地运行代码。我们希望帮助读者掌

握并行编程，使程序能够扩展到新一代机器的性能水平。实现这种可扩展性的关键，是使内存数据访问更加规整和局部化，从而尽量减少关键资源的消耗以及更新数据结构时发生的冲突。因此，开发高性能并行代码的技术，对保证应用在未来的可扩展性同样重要。

要实现这些目标，需要大量技术知识，因此本书会介绍不少并行编程原则和模式[6]。我们不会孤立地教授这些原则和模式，而是把它们放在有用应用的并行化语境中来讲授。当然，我们无法覆盖全部原则和模式，所以选择了最有用且经过充分验证的技术进行详细介绍。事实上，本版显著增加了关于并行模式和应用的章节。现在，让我们快速概览本书其余内容。

1.7 本书的组织结构

本书分为三部分。第 1 部分介绍并行编程、数据并行、GPU 以及性能优化的基本概念。这些基础章节为读者成为 GPU 程序员提供必要的知识和技能。第 2 部分介绍基本的并行模式，第 3 部分介绍更高级的并行模式和应用。这两部分会应用第一部分所学的知识和技能，也会在需要时引入其他 GPU 架构特性和优化技术。

第 1 部分“基础概念”包括第 2 至第 6 章。第 2 章介绍数据并行和 CUDA 编程模型，并假设读者此前有 C++ 编程经验。它首先把 CUDA 介绍为 C++ 的一个简单而小型的扩展，用于支持异构 CPU/GPU 计算以及广泛使用的单程序多数据（Single Program Multiple Data, SPMD）并行编程模型。随后，本章介绍以下思考过程：

1. 识别应用程序中需要并行化的部分；
2. 隔离并行代码将使用的数据，并通过 API 函数在并行计算设备上分配内存；
3. 通过 API 函数把数据传送到并行计算设备；
4. 把并行部分开发为由并行线程执行的内核函数；
5. 启动内核函数，让并行线程执行它；
6. 最终通过 API 函数调用把数据传回主机处理器。

我们使用向量加法这一贯穿示例来说明这些概念。第 2 章的目标，是让读者掌握足够的 CUDA 编程模型概念，以便编写简单的并行 CUDA 程序；同时，它也涵盖了基于任何并行编程接口开发并行应用所需的若干基本技能。

第 3 章进一步介绍 CUDA 的并行执行模型，尤其是它如何处理多维数据以及多维线程组织方式。本章提供关于线程创建、组织、资源绑定和数据绑定的足够洞见，使读者能够使用 CUDA 实现复杂计算。

第 4 章介绍 GPU 计算架构，重点讨论计算核心如何组织，以及线程如何被调

度到这些核心上执行。本章还讨论各种架构因素及其对 GPU 上运行代码性能的影响，包括透明可扩展性、SIMD 执行与控制分歧、多线程与延迟容忍度，以及占用率 (occupancy) 等概念。

第 5 章在第 4 章基础上讨论 GPU 的内存架构，以及可用于保存 CUDA 变量、管理数据传送和提高程序执行速度的特殊内存。本章介绍分配和使用这些内存的 CUDA 语言特性。恰当地使用这些内存，可以大幅提高数据访问吞吐量，并帮助缓解内存系统中的流量拥塞。

第 6 章介绍当前 CUDA 硬件中的若干重要性能因素，尤其是理想的线程执行模式和内存访问模式。这些细节构成了程序员推理计算和数据组织决策后果的概念基础。本章最后给出 GPU 程序员经常使用的常见优化策略检查清单，用于优化各种计算模式。在本书接下来的两部分中，我们会反复使用这份检查清单来优化不同的并行模式和应用。

第 2 部分“基本并行模式”包括第 7 至第 15 章。第 7 章介绍卷积，这是一种常用的并行计算模式，源于数字信号处理和计算机视觉，并且需要仔细管理数据访问局部性。我们还借助该模式介绍现代 GPU 中的常量内存和缓存。第 8 章介绍与卷积相似的模板计算，但它源于求解微分方程，并具有为数据访问局部性进一步优化提供独特机会的特征。本章还用来介绍线程与数据的三维组织，并展示第 6 章中针对线程粒度的优化方法。

第 9 章讨论直方图，这是一种广泛用于统计数据分析 and 大规模数据集模式识别的模式。我们还借助该模式介绍原子操作，用于协调对共享数据的并发更新；并介绍私有化 (privatization) 优化，以降低这些操作的开销。第 10 章介绍归约树模式，用于概括一组输入数据；同时用它展示控制分歧和过多屏障同步对性能的影响，以及减轻这些影响的技术。第 11 章介绍前缀和或扫描，这是一种重要的并行计算模式，可以把本质上顺序的计算转换为并行计算；我们也借此介绍并行算法中的工作效率概念。第 12 章以两种形式介绍过滤模式：不稳定过滤（允许改变被过滤元素的原始顺序）和稳定过滤（必须保留原始顺序）。不稳定过滤用于展示线程如何协作以减少原子操作，稳定过滤则展示第 11 章介绍的扫描模式的一个重要应用。第 13 章介绍并行归并，这是分治式工作划分策略中广泛使用的模式；本章还介绍动态输入数据识别与组织。最后，第 14 章介绍 GPU 上的排序，涵盖三种并行排序算法：奇偶排序、归并排序（依赖第 13 章的归并模式）以及基数排序（依赖第 11 章介绍的扫描模式）。第 15 章更深入地重新讨论矩阵乘法，应用第 6 章检查清单中的大量高级优化方法。

1

第 3 部分“高级并行模式与应用”在精神上与第 2 部分类似，但所涵盖的模式更加复杂，通常也包含更多应用背景。因此，这些章节较少侧重介绍新技术或新特性，

¹原书此处把基数排序所依赖的扫描模式标为 Chapter 14；按本书目录和上下文，应为 Chapter 11。译文按语境更正，并保留此译者注。

而更多关注特定应用的考量。对于每个应用，我们首先识别构造并行执行的不同方式，然后分析每种方式的优点和缺点，接着逐步进行实现高性能所需的代码变换。这些章节帮助读者把前面章节的全部材料结合起来，并为读者承担自己的应用开发项目提供支持。

第 3 部分包括第 16 至第 23 章。第 16 章以图处理中的经典 Floyd-Warshall 算法和基因组序列比对中的 Smith-Waterman 算法为贯穿示例，展示动态规划和波前并行。第 17 章介绍稀疏矩阵计算，这一技术广泛用于处理极大规模数据集。本章向读者介绍重新排列数据以实现更高效并行访问的概念，包括数据压缩、填充、排序、转置和规整化。第 18 章介绍图算法，以及如何在 GPU 编程中高效实现图搜索。本章给出并行化图算法的多种策略，并讨论图结构如何影响最佳算法的选择；这些策略建立在直方图和归并等更基本的模式之上。

第 19 章和第 20 章讨论深度学习，它已经成为 GPU 计算中极其重要的领域。第 19 章介绍卷积神经网络的高效实现，利用分块等技术和卷积等模式。第 20 章介绍大语言模型，并讨论在 GPU 上训练和推理这些模型的先进优化方法，例如 KV 缓存和批处理。第 21 章讨论静电势图计算，以及如何利用从散布到聚集的变换来增强并行性、降低同步开销。

第 22 章介绍计算思维，即以更适合高性能计算的方式构造和求解计算问题的艺术。本章讨论如何组织程序的计算任务，使其能够并行执行。我们首先讨论把抽象的、特定于科学问题的概念转化为计算任务的过程；无论开发串行还是并行应用，这是生成高质量应用软件的重要第一步。随后，我们讨论并行算法结构及其对应用性能的影响，并以 CUDA 性能调优经验为基础。虽然我们不深入介绍这些并行编程风格的实现细节，但希望读者能够凭借本书打下的基础，学会使用其中任何一种风格编程。本章最后区分并行化计算的两种广泛策略：以输入为中心和以输出为中心，并回到书中介绍的模式与应用，回顾性地研究每种模式和应用所采用的策略。

第 3 部分的最后一章，即第 23 章，介绍异构集群上的 CUDA 编程，其中每个计算节点都包含 CPU 和 GPU。我们讨论如何在 CUDA 的配合下使用 MPI、NCCL 和 NVSHMEM 三种不同编程模型，把计算分配到集群中的多个 GPU，同时降低通信开销。

虽然本书各章都以 CUDA 为基础，但它们帮助读者建立的是一般并行编程的基础。我们相信，人类从具体示例中学习效果最好：也就是说，必须首先在某个特定编程模型的语境中学习概念；该模型为我们把知识推广到其他编程模型提供坚实立足点。在推广过程中，我们可以借助从 CUDA 示例中获得的具体经验。深入体验 CUDA 也能帮助我们形成成熟的理解，从而学习那些甚至可能与 CUDA 模型无关的概念。

第 24 章将对大规模并行编程作总结并展望未来。我们首先重新审视本书目标，总结各章如何共同帮助实现这些目标；然后预测快速发展的大规模并行计算会使其成为未来十年最令人兴奋的领域之一。

参考文献

- [1] J. von Neumann. First draft of a report on the EDVAC. *IEEE Annals of the History of Computing*, 15(4), 1993, 27–75.
- [2] H. Sutter and J. Larus. Software and the concurrency revolution: leveraging the full power of multicore processors demands new tools and new thinking from the software industry. *ACM Queue*, 3(7), 2005, 54–62.
- [3] W.-M. Hwu, K. Keutzer, and T. G. Mattson. The concurrency challenge. *IEEE Design & Test of Computers*, 25(4), 2008, 312–320.
- [4] NVIDIA. *CUDA C++ Programming Guide*, Release 13.0, 2025. <https://docs.nvidia.com/cuda/cuda-c-programming-guide>.
- [5] G. M. Amdahl. Computer architecture and Amdahl’s law. *Computer*, 46(12), 2013, 38–46.
- [6] T. G. Mattson, B. Sanders, and B. Massingill. *Patterns for Parallel Programming*. Pearson Education, 2004.
- [7] OpenMP.org. *OpenMP: The OpenMP API Specification for Parallel Programming*, 2025. <https://www.openmp.org>.
- [8] MPI Forum. *MPI: A Message-Passing Interface Standard*, 1994.
- [9] Khronos OpenCL Working Group. *The OpenCL Specification*, Version 1.0, 2008.
- [10] J. M. Wing. Computational thinking. *Communications of the ACM*, 49(3), 2006, 33–35.

第一部分

基础概念

第二章 异构数据并行计算

数据并行性是这样一种现象：要在数据集的不同部分上执行的计算彼此独立，因此可以并行执行。许多应用都具有丰富的数据并行性，适合进行可扩展的并行执行；在这种执行方式下，执行速度会随着系统中执行资源数量的增加而近似成比例地提高。因此，并行程序员有必要熟悉数据并行性的概念，以及用于编写能够利用数据并行性的代码的并行编程语言构造。本章将使用 CUDA C++ 语言构造开发一个简单的数据并程序。

2.1 数据并行性

现代软件应用运行缓慢时，问题通常来自数据——需要处理的数据太多。图像处理应用会处理包含数百万乃至数万亿像素的图像或视频；科学应用会用数十亿个网格点模拟流体动力学；分子动力学应用会模拟数千到数十亿个原子之间的相互作用；航空公司排班则必须处理数千个航班、机组和机场登机口。通常，这些像素、粒子、网格点、相互作用、航班等对象在很大程度上可以独立处理。

例如，在图像处理中，把彩色像素转换为灰度只需要该像素的数据；模糊图像时，把每个像素的颜色与附近像素的颜色求平均，只需要一个很小的像素邻域。即使是看起来需要全局信息的操作，例如求图像中所有像素的平均亮度，也可以拆分成许多能够独立执行的较小计算。对不同数据片段进行这种相互独立的求值，就是数据并行性的基础。编写数据并行代码，意味着围绕数据重新组织计算，使得到的独立计算能够并行执行，从而更快——通常是快得多——地完成整体任务。

下面用一个彩色图像转灰度图像的例子说明数据并行性。图 2.1 中的左图是由许多像素组成的彩色图像，每个像素包含红、绿、蓝三个分数值 (r, g, b) ，取值从 0（黑色）到 1（最大强度）。要把彩色图像转换为灰度图像，需要对每个像素应用下面的加权和公式，计算亮度值 L ：

$$L = r \times 0.299 + g \times 0.587 + b \times 0.114. \quad (2.1)$$

RGB 彩色图像表示

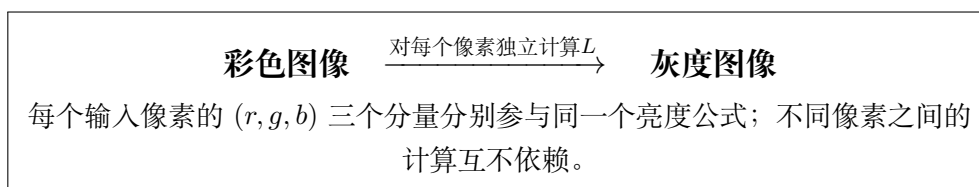


图 2.1: 彩色图像转换为灰度图像的概念示意（依据原书图 2.1 重绘）

在 RGB 表示中，图像的每个像素都表示为一个 (r, g, b) 值元组。一行图像的格式可以写成

$$(r \ g \ b) \ (r \ g \ b) \ \cdots \ (r \ g \ b).$$

每个元组表示红光 (R)、绿光 (G) 和蓝光 (B) 的混合。也就是说，渲染像素时，每个像素的 r 、 g 和 b 值分别表示红、绿、蓝光源的强度，其中 0 表示暗，1 表示满强度。

如果把输入看作由 RGB 值组成的数组 I ，把输出看作对应的亮度值数组 O ，就能得到图 2.2 所示的简单计算结构。例如， $O[0]$ 是按照上面的公式对 $I[0]$ 中的 RGB 值求加权和得到的； $O[1]$ 由 $I[1]$ 中的 RGB 值得到； $O[2]$ 由 $I[2]$ 中的 RGB 值得到，以此类推。这些逐像素计算彼此没有依赖，全部可以独立执行。显然，彩色转灰度体现了丰富的数据并行性，因为现代图像通常包含数百万个像素。当然，完整应用中的数据并行性可能更加复杂；本书很大一部分内容都致力于教授发现并利用数据并行性所需的“并行思维”。

$$\begin{array}{ccc}
 I[0] & I[1] & I[2] \\
 \downarrow & \downarrow & \downarrow \\
 O[0] & O[1] & O[2] \\
 \cdots; & \text{每一列都可独立执行}
 \end{array}$$

图 2.2: 彩色转灰度中的数据并行性：像素可以独立计算（依据原书图 2.2 重绘）

数据并行性与任务并行性

数据并行性并不是并行编程中唯一使用的并行性类型。任务并行性也被广泛用于并行编程。任务并行性通常通过对应用进行任务分解来暴露。例如，一个简单应用可能需要执行向量加法和矩阵-向量乘法；这两项工作各自就是一个任务。如果两个任务可以独立执行，就存在任务并行性。I/O 和数据传送也是常见的任务来源。

大型应用通常包含更多相互独立的任务，因此具有更多任务并行性。例如，在分子动力学模拟器中，自然任务可能包括振动力计算、转动力计算、非键合力的邻居识别、非键合力计算，以及速度和位置计算等。

一般而言，数据并行性是并程序获得性能提升的主要来源。面对大规模数据集，通常可以找到充足的数据并行性，从而利用大规模并行处理器，并让应用性能随着每一代拥有更多执行资源的硬件而提升；这种理想性质称为可扩展性（scalability）。不过，任务并行性同样可能在实现性能目标时发挥重要作用。

2.2 CUDA C++ 程序结构

下一步是学习如何编写程序，在异构计算系统中利用数据并行性以获得更快的执行速度。为了一般性地说明本书中的算法和技术，我们将使用 CUDA C++ 程序的代码示例。根据我们的经验，基于 CUDA C++ 学到的概念很容易扩展到其他编程语言和平台。

CUDA C++¹ 在流行的 C++ 编程语言基础上，以少量新语法和库函数扩展出面向异构计算系统的能力，使程序员能够针对同时包含 CPU 和 GPU 的系统编程。顾名思义，CUDA C++ 建立在 NVIDIA 的 CUDA 平台之上。CUDA 目前是大规模并行计算中最成熟的框架，已广泛用于高性能计算产业；在最常见的操作系统上，都能找到编译器、调试器和性能分析器等重要工具。

CUDA C++ 程序的结构反映了计算机中主机（host，即 CPU）与一个或多个设备（device，即 GPU）共存的情况。每个 CUDA C++ 源文件都可以混合包含主机代码和设备代码。默认情况下，任何只包含主机代码的传统 C++ 程序也是一个 CUDA C++ 程序；可以在任意源文件中加入设备代码。设备代码由特殊的 CUDA C++ 关键字明确标记，其中包括函数（或称内核，kernel），其代码以数据并行方式执行。

图 2.3 展示了 CUDA C++ 程序的执行过程。执行从主机代码（CPU 串行代码）开始，图中用一条代表线程的弯曲箭头表示。当调用内核——也就是设备代码的一部分（图中的设备并行内核）——时，设备上会启动大量线程执行该内核。一次内核调用所启动的全部线程统称为一个网格（grid）。这些线程是 CUDA 平台中并行执行的主要载体。图 2.3 展示了两个线程网络的执行，每个网格又由多个线程块组成；稍后会讨论网格和线程块的组织方式。当网格中的所有线程执行完毕后，网格终止，执行回到主机，直到再次启动另一个网格。正如图中所暗示的，异构计算应用可以管理 CPU 和 GPU 的重叠执行，从而同时利用二者。

线程

线程是现代计算机中处理器执行顺序程序方式的一种简化视图。线程由程序代码、当前正在执行的代码位置，以及变量和数据结构的值组成。从用户角度看，线程的执行是顺序的。程序员可以使用源代码级调试器，逐条

¹本书的代码示例为简洁起见使用 C 风格编程，并在需要时介绍 C++ 特性。不过，读者应当注意，CUDA 编程已经逐渐转向 C++，以支持现代泛型编程和其他面向对象的编程实践。



图 2.3: CUDA 程序的执行过程 (依据原书图 2.3 重绘)

执行语句来监控线程进展，查看下一条将要执行的语句，并在执行推进时检查变量和数据结构的值。

线程已经在编程中使用多年。如果程序员希望在应用中启动并行执行，就可以借助线程库或专用语言创建并管理多个线程。在 CUDA 中，每个设备线程的执行同样是顺序的；CUDA 程序通过调用内核函数启动并行执行，底层运行时机制则负责在设备上启动一个线程网格，让线程并行处理数据。

启动一个网格通常会生成大量线程，以利用数据并行性。在彩色转灰度的例子中，可以让每个线程计算输出数组中的一个像素；此时，网格启动所生成的线程数应等于图像中像素的数量。对于大图像，可能会生成数百万个线程。实践中，也可以让每个线程计算多个输出像素，以提高执行和数据访问效率；本书后文会讨论这一点。由于硬件支持高效的线程生成和调度，CUDA 程序员可以假设这些线程只需很少的时钟周期就能生成和调度；传统 CPU 线程的生成和调度通常则需要数千个时钟周期。第 3 章将展示如何实现彩色转灰度和图像模糊内核，本章为简洁起见使用向量加法作为贯穿示例。

2.3 向量加法示例

向量加法可以说是最简单的数据并行计算，是顺序编程中“Hello World”的并行对应物。在展示向量加法的内核代码之前，先回顾传统向量加法（主机代码）函数的工作方式会很有帮助。图 2.4 展示了一个由 main 函数和向量加法函数组成的简单 C++ 程序。

在本书的所有示例中，只要需要区分主机数据和设备数据，就会在主机使用的变量名后加上 `_h`，在设备使用的变量名后加上 `_d`，以提醒自己这些变量的预期用途。由于图 2.4 只有主机代码，因此其中只出现带 `_h` 后缀的变量。

假设要相加的向量存放在数组 *A* 和 *B* 中，它们在 main 函数中分配并初始化；输出向量存放在数组 *C* 中，该数组也在 main 函数中分配。为简洁起见，图中没有展示 main 函数如何分配和初始化 *A*、*B*、*C* 的细节。这些数组的指针，以及表示向量长度

的变量 N ，会传给 `vecAdd` 函数。注意，`vecAdd` 的参数带有 `_h` 后缀，以强调它们由主机使用。

```
1 // Compute vector sum C_h = A_h + B_h
2 void vecAdd(float* A_h, float* B_h, float* C_h, int n) {
3     for (int i = 0; i < n; ++i) {
4         C_h[i] = A_h[i] + B_h[i];
5     }
6 }
7 int main() {
8     // Memory allocation for arrays A, B, and C
9     // I/O to read A, B, and N elements each
10    ...
11    vecAdd(A, B, C, N);
12 }
13
```

图 2.4: 一个简单的传统向量加法 C++ 代码示例

C++ 语言中的指针

图 2.4 中的函数参数 A 、 B 、 C 都是指针。在 C 和 C++ 语言中，可以使用指针访问变量和数据结构。浮点变量 V 可以声明为：

```
float V;
```

指针变量 P 可以声明为：

```
float *P;
```

通过语句 $P = \&V$ 把 V 的地址赋给 P ，就可以让 P “指向” V 。 $*P$ 成为 V 的同义表示。例如， $U = *P$ 把 V 的值赋给 U ；又如， $*P = 3$ 会把 V 的值改为 3。

C 或 C++ 程序中的数组可以通过指向其第 0 个元素的指针访问。例如，语句 $P = \&(A[0])$ 让 P 指向数组 A 的第 0 个元素，于是 $P[i]$ 就成为 $A[i]$ 的同义表示。事实上，数组名 A 本身就是指向其第 0 个元素的指针。

在图 2.4 中，调用 `vecAdd` 时把数组名 A 作为第一个参数，会让函数的第一个参数 `A_h` 指向 A 的第 0 个元素。因此，函数体中的 `A_h[i]` 可以用来访问 `main` 函数中定义的数组 A 的 $A[i]$ 。关于 C 语言中指针详细用法的易读解释，可参见文献 [1]。

`vecAdd` 函数使用 `for` 循环遍历向量元素。在第 i 次迭代中，输出元素 `C_h[i]` 接收 `A_h[i]` 和 `B_h[i]` 的和。向量长度参数 `n` 用于控制循环，使迭代次数与向量长

度相匹配。函数分别通过指针 `A_h`、`B_h` 和 `C_h` 读取 `A`、`B` 的元素并写入 `C` 的元素。`vecAdd` 返回后，`main` 中的后续语句就可以访问 `C` 的新内容。

在并行执行向量加法的一种直接方法，是修改 `vecAdd` 函数，把计算移到设备上。这种修改后的 `vecAdd` 函数结构如图 2.5 所示。函数的第 1 部分在设备（GPU）内存中分配空间，保存 `A`、`B`、`C` 向量的副本，并把 `A` 和 `B` 向量从主机内存复制到设备内存。第 2 部分调用向量加法内核，在设备上启动一个线程网格。第 3 部分把和向量 `C` 从设备内存复制到主机内存，并释放设备内存中的三个数组。

```

1 void vecAdd(float* A_h, float* B_h, float* C_h, int n) {
2     int size = n * sizeof(float);
3     float *A_d, *B_d, *C_d;
4
5     // Part 1: allocate device memory for A, B, and C and copy the inputs
6     ...
7
8     // Part 2: invoke the kernel to launch a thread grid on the device
9     // and perform the actual vector addition
10    ...
11
12    // Part 3: copy C back from the device and free the device vectors
13    ...
14 }
15

```

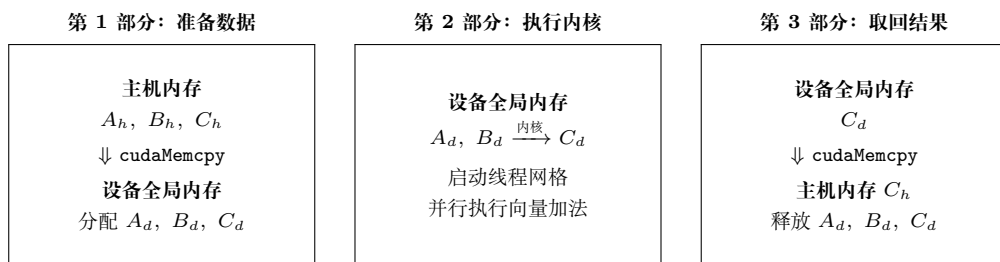


图 2.5: 把工作移到设备上的 `vecAdd`（主机代码）函数概要及数据流（依据原书图 2.5 重绘）

修改后的 `vecAdd` 函数本质上是一个外包代理：它把输入数据发送到设备，在设备上启动计算，再从设备收集结果，而且主程序甚至不需要知道向量加法实际上已经在设备上完成。实践中，这种“透明”的外包模型可能非常低效，因为数据需要来回复制。通常会把大型而重要的数据结构保留在设备上，只从主机代码调用操作这些数据结构的设备函数。不过，目前为了介绍 CUDA C++ 程序的基本结构，我们先使用这个简化的透明模型。修改后函数的细节，以及如何组成内核函数，将是本章后续内容。

2.4 设备全局内存与数据传送

在当前 CUDA 系统中，设备通常是配有自身动态随机存取存储器 (Dynamic Random-Access Memory, DRAM) 的硬件卡；这块 DRAM 称为设备全局内存 (device global memory)，通常简称全局内存。例如，NVIDIA Hopper H100 GPU 配备 80 GB 或 94 GB 全局内存。称其为“全局”内存，是为了区别于设备上其他同样可以由程序员访问的内存类型。CUDA 内存模型和不同设备内存类型的细节将在第 5 章讨论。

对于向量加法内核，调用内核之前，程序员需要在设备全局内存中分配空间，并把数据从主机内存传送到设备全局内存中的已分配空间。这些活动对应图 2.5 的第 1 部分。设备执行结束后，程序员还需要把输出数据从设备全局内存传回主机内存，并释放不再需要的设备全局内存空间；这些活动对应第 3 部分。CUDA 运行时系统（通常运行在主机上）提供 API 函数，代表程序员完成这些活动。下文简称“从主机向设备传送数据”，指把数据从主机内存复制到设备全局内存；反方向同理。

图 2.5 的第 1 和第 3 部分需要使用 CUDA API 函数，为 *A*、*B*、*C* 分配设备全局内存，把 *A* 和 *B* 从主机传到设备，在向量加法之后把 *C* 从设备传回主机，并释放 *A*、*B*、*C* 所占的设备全局内存。先介绍内存分配和释放函数。

<code>cudaMalloc()</code>	在设备全局内存中分配对象；需要两个参数：指向所分配对象的指针地址，以及以字节数表示的分配大小。
<code>cudaFree()</code>	从设备全局内存释放对象；需要一个参数，即待释放对象的指针。

图 2.6: 管理设备全局内存的 CUDA API 函数（依据原书图 2.6 重绘）

图 2.6 展示了分配和释放设备全局内存的两个 API 函数。主机代码可以调用 `cudaMalloc`，为对象分配一段设备全局内存。读者应注意，`cudaMalloc` 与标准 C 运行时库（它也是 C++ 的一部分）中的 `malloc` 函数非常相似。这种相似是有意的：CUDA C++ 只是带有少量扩展的 C++。CUDA C++ 使用标准 C/C++ 运行时库的 `malloc` 函数管理主机内存，并添加 `cudaMalloc` 作为 C++ 运行时库的扩展。尽量保持与原始 C/C++ 运行时库接近的接口，可以减少 C 或 C++ 程序员重新学习这些扩展的时间。

`cudaMalloc` 的第一个参数，是将被设置为指向所分配对象的指针变量的地址。这个地址本身是“指向指针的指针”；由于 API 的形参类型为 `void**`，调用者需要把该地址转换为 `(void**)`。这个内存分配函数是通用函数，不局限于某种对象类型。² 无论指针变量是什么类型，这个参数都允许 `cudaMalloc` 把所分配内存的地址写入提供的指针变量。调用内核的主机代码会把这个指针值传给需要访问所分配内存对象的

²`cudaMalloc` 通过通用指针交回所分配对象的地址，这会使动态分配的多维数组使用起来更加复杂；第 3 章将讨论这一问题。

内核。第二个参数给出要分配的数据大小，单位是字节；它与 C malloc 函数的大小参数一致。

注意，cudaMalloc 的格式与 C malloc 略有不同。C malloc 返回指向所分配对象的指针，只接收一个指定对象大小的参数；cudaMalloc 则把结果写入第一个参数所指定地址的指针变量。因此，cudaMalloc 接收两个参数。双参数形式使 cudaMalloc 能像其他 CUDA API 函数一样，使用返回值报告错误。

cudaFree 函数与 C free 函数非常相似。它接收一个输入参数，即待释放内存的指针，并从设备内存中释放相应空间。

CUDA C++ 还有更高级的库函数，用于在主机内存和设备内存中分配与释放空间。这些替代方法可以更快地复制数据，或者消除显式复制的需要，让数据按需自动复制。管理设备内存时，这些方法可以为程序提供更好的性能或便利性；第 23 章和附录 C 会讨论其他内存分配形式。

下面用一个简单代码例子说明 cudaMalloc 和 cudaFree 的使用：

```
1 float *A_d;  
2 int size = n * sizeof(float);  
3 cudaMalloc((void**)&A_d, size);  
4 ...  
5 cudaFree(A_d);
```

这是图 2.5 示例的延续。为清晰起见，我们在指针变量名后加 _d，表示它指向设备全局内存中的对象。A_d 的类型是 float*，所以它的地址 &A_d 的类型是 float**；cudaMalloc 的第一个形参类型是 void**，因此代码中的 (void**)&A_d 把 float** 转换为 void**。cudaMalloc 返回时，A_d 将指向为向量 A 分配的设备全局内存区域。第二个参数是要分配的区域大小。由于 size 的单位是字节，程序员在确定 size 的值时，需要把数组元素数量换算成字节数。

例如，为包含 n 个单精度浮点元素的数组分配空间时，size 等于 n 乘以单精度浮点数的大小；当前计算机中该大小通常是 4 字节。因此，size 的值为 $n*4$ 。计算完成后，调用 cudaFree 并把 A_d 作为参数传入，即可从设备全局内存中释放向量 A 的存储空间。注意，cudaFree 不需要改变 A_d 的值；它只需使用 A_d 的值，把已分配内存归还到可用内存池。因此，作为参数传入的只是 A_d 的值，而不是其地址。

A_d、B_d 和 C_d 中的地址都指向设备全局内存中的位置。这些地址不应在主机代码中解引用，只能用于调用 API 函数和内核函数。在主机代码中解引用设备全局内存指针可能导致异常或其他运行时错误，因为这些地址位于设备全局内存，用户主机代码无法访问。

讨论完 A_d 指针变量的声明，以及它在 cudaMalloc 和 cudaFree 调用中的用法后，读者应当能够用类似声明为 B_d、C_d 的指针变量及相应的 cudaMalloc 调用，补全图 2.5 中 vecAdd 示例的第 1 部分；第 3 部分则可以用针对 B_d 和 C_d 的 cudaFree

调用补全。

主机代码为数据对象分配设备全局内存后，就可以请求在主机与设备之间传送数据。这通过调用 CUDA API 函数完成。图 2.7 展示了这样的 API 函数 `cudaMemcpy`。它接收四个参数：第一个参数是待复制数据对象的目标地址；第二个参数是源地址；第三个参数指定复制的字节数；第四个参数表示复制涉及的内存位置：主机到主机、主机到设备、设备到主机或设备到设备。例如，内存复制函数也可以把设备全局内存中的数据从一个位置复制到另一个位置。

<code>cudaMemcpy()</code>	执行内存数据传送，需要四个参数：目标指针、源指针、要复制的字节数，以及传送类型/方向。
---------------------------	---

图 2.7: 在主机与设备之间传送数据的 CUDA API 函数（依据原书图 2.7 重绘）

`vecAdd` 函数在相加之前调用 `cudaMemcpy`，把 `A_h` 和 `B_h` 所指向的主机数组复制到 `A_d` 和 `B_d` 所指向的设备数组；向量加法完成后，再把 `C_d` 所指向的设备数组复制到 `C_h` 所指向的主机数组。假设 `A_h`、`B_h`、`A_d`、`B_d` 和 `size` 的值都已按前文设置好，三个 `cudaMemcpy` 调用如下：

```
1 cudaMemcpy(A_d, A_h, size, cudaMemcpyHostToDevice);
2 cudaMemcpy(B_d, B_h, size, cudaMemcpyHostToDevice);
3 ...
4 cudaMemcpy(C_h, C_d, size, cudaMemcpyDeviceToHost);
```

`cudaMemcpyHostToDevice` 和 `cudaMemcpyDeviceToHost` 是 CUDA 编程环境能够识别的预定义符号常量。通过正确安排源指针和目标指针，并使用适当的传送类型，同一个 `cudaMemcpy` 函数可以用于两个方向的数据传送。

总的来说，图 2.4 中的 `main` 函数调用同样在主机上执行的 `vecAdd`；图 2.5 概要展示的 `vecAdd` 在设备全局内存中分配空间、请求数据传送，并调用真正执行向量加法的内核。我们把这种用于调用内核的主机代码称为内核调用存根（stub）。图 2.8 给出了更完整版本的 `vecAdd` 函数。

与图 2.5 相比，图 2.8 中的 `vecAdd` 已经完成第 1 和第 3 部分。第 1 部分为 `A_d`、`B_d`、`C_d` 分配设备全局内存，并把 `A_h` 传到 `A_d`、把 `B_h` 传到 `B_d`；这是通过调用 `cudaMalloc` 和 `cudaMemcpy` 完成的。读者可以尝试使用适当的参数值自行编写这些函数调用，并与图 2.8 中的代码比较。第 2 部分调用内核，将在下一小节介绍。第 3 部分把向量和数据从设备传回主机，使 `main` 函数能够使用这些值；这通过调用 `cudaMemcpy` 完成。随后通过调用 `cudaFree` 释放设备全局内存中 `A_d`、`B_d` 和 `C_d` 所占的空间。

CUDA C++ 程序中的错误检查与处理

```
1 void vecAdd(float* A_h, float* B_h, float* C_h, int n) {  
2     int size = n * sizeof(float);  
3     float *A_d, *B_d, *C_d;  
4  
5     cudaMalloc((void**)&A_d, size);  
6     cudaMalloc((void**)&B_d, size);  
7     cudaMalloc((void**)&C_d, size);  
8  
9     cudaMemcpy(A_d, A_h, size, cudaMemcpyHostToDevice);  
10    cudaMemcpy(B_d, B_h, size, cudaMemcpyHostToDevice);  
11  
12    // The kernel invocation is introduced later  
13    ...  
14  
15    cudaMemcpy(C_h, C_d, size, cudaMemcpyDeviceToHost);  
16  
17    cudaFree(A_d);  
18    cudaFree(B_d);  
19    cudaFree(C_d);  
20 }  
21
```

图 2.8: 更完整版本的 vecAdd 函数

程序检查并处理错误非常重要。良好的错误检查和处理能够在错误发生时捕获它们，并在靠近问题根因的位置发现问题。CUDA API 函数会返回标志，指示其处理请求时是否发生错误；大多数错误来自调用时使用了不恰当的参数值。为简洁起见，本书的示例不展示错误检查代码。例如，图 2.8 中有如下 `cudaMalloc` 调用：

```
cudaMalloc((void **) &A_d, size);
```

实践中，CUDA 程序员会用检查返回错误条件的代码包围该调用，并打印相关错误消息，让用户知道发生了错误。一个简单版本如下：

```
1 cudaError_t err =
2     cudaMalloc((void **) &A_d, size);
3 if (err != cudaSuccess) {
4     printf("%s in %s at line %d \n",
5           cudaGetErrorString(err), __FILE__,
6           __LINE__);
7     exit(EXIT_FAILURE);
8 }
```

这样，如果系统设备内存不足，用户就会得到通知。恰当的错误检查可以节省数小时的调试时间；还可以定义 C++ 宏，使源代码中的检查代码更加简洁。原书示例的条件判断写作 `error`，但前一行声明的是 `err`；此处按上下文改为 `err`。

2.5 内核函数与线程

现在可以进一步讨论 CUDA C++ 内核函数，以及调用这些内核函数所产生的效果。在 CUDA 中，内核函数规定了 GPU 线程在并行阶段执行的代码。由于所有线程执行相同的代码，CUDA C++ 编程是著名的单程序多数据（Single-Program Multiple-Data, SPMD）[2] 并行编程风格的一个实例；SPMD 是并行计算系统中常见的编程风格。

当程序的主机代码调用内核时，运行在主机上的 CUDA 运行时系统会启动一个 GPU 线程网格，线程网格按两级层次组织。每个网格都组织为线程块（thread block）数组；为简洁起见，后文也简称为块（block）。同一网格中的所有块大小相同。在当前系统中，每个块最多可以包含 1024 个线程。图 2.9 展示了每个块包含 256 个线程的例子；每个线程用一条弯曲箭头表示，箭头来自一个标有该线程在块内索引编号的方框。

网格中的块数量以及每个块中的线程数量，由主机代码在调用内核时指定。同一个内核可以在主机代码的不同位置以不同的块数和线程数调用。对于给定的线程网

格，每个块中的线程数保存在名为 `blockDim` 的内置变量中。`blockDim` 的类型是一个 C++ 结构体，包含三个无符号整数字段 `x`、`y` 和 `z`，帮助程序员把线程组织成一维、二维或三维数组。一维组织只使用 `x` 字段；二维组织使用 `x` 和 `y` 字段；三维结构则使用全部三个字段。线程组织的维度通常反映数据的维度。由于线程是为并行处理数据而创建的，让线程组织方式反映数据组织方式是自然的。

```

          块 0              块 1              块 N - 1
0, 1, 2, ..., 254, 255  0, 1, 2, ..., 254, 255  0, 1, 2, ..., 254, 255
每个线程执行相同的内核代码：
i = blockDim.x * blockIdx.x + threadIdx.x
C[i] = A[i] + B[i]

```

图 2.9: 网格中的所有线程执行相同的内核代码（依据原书图 2.9 重绘）

内置变量

许多编程语言都有内置变量。这些变量具有特殊的含义和用途，通常由运行时系统预先初始化，并且在程序中一般是只读的。程序员不应为了其他目的重新定义这些变量。

在图 2.9 中，由于数据是一维向量，每个线程块都组织为一维线程数组。

每个块中的线程总数由内置变量 `blockDim.x` 给出。在图中，该值为 256。通常建议线程块中的线程总数是 32 的倍数，这是出于硬件效率的考虑；本书后文会再次讨论这一点。

CUDA 内核还可以访问另外两个内置变量 `threadIdx` 和 `blockIdx`，让线程彼此区分，并确定每个线程需要处理的数据区域。`threadIdx` 为每个线程提供块内的唯一坐标。在一维线程组织中只使用 `threadIdx.x`；图 2.9 中每个线程的小阴影方框显示了它的 `threadIdx.x` 值。每个块中的第一个线程的值为 0，第二个线程的值为 1，第三个线程的值为 2，以此类推。

`blockIdx` 为一个块中的所有线程提供共同的块坐标。在图 2.9 中，第一个块的所有线程的 `blockIdx.x` 值为 0，第二个块的值为 1，以此类推。可以用电话系统作类比：`threadIdx.x` 类似本地电话号码，`blockIdx.x` 类似区号。两者结合起来，就能为全国范围内的每条电话线形成唯一的电话号码。同样，每个线程也可以组合自己的 `threadIdx` 和 `blockIdx` 值，在整个网格中生成唯一的全局索引。

层次化组织

像 CUDA 线程一样，现实世界中的许多系统也按层次组织。美国电话系统就是一个好例子。最高层由许多“区域”组成，每个区域对应一个地理范围；同一区域内的所有电话线共享一个三位数区号。一个电话区域有时会大于一座城市。例如，美国伊利诺伊州中部的许多县和城市都属于同一电话区域，共享区号 217。

在一个区域内，每条电话线都有一个七位数的本地号码，因此每个区域最多约有一千万个号码。可以把每条电话线看作一个 CUDA 线程，把区号看作 `blockIdx.x`，把七位本地号码看作 `threadIdx.x`。这种层次化组织在保持局部性的同时，支持非常大的电话线路数量；拨打同一区域的号码时只需拨本地号码，偶尔拨打其他区域的号码时才需要拨 1、区号和本地号码。CUDA 线程的层次化组织也提供了一种局部性形式，本书很快就会研究这种局部性。

在图 2.9 中，使用下面的 C++ 语句计算唯一的全局索引 i ：

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}.$$

本例中 `blockDim.x` 为 256。块 0 中线程的 i 值范围为 0 到 255；块 1 中的范围为 256 到 511；块 2 中的范围为 512 到 767。也就是说，这三个块中线程的 i 值连续覆盖 0 到 767。由于每个线程使用 i 访问 A 、 B 和 C ，这些线程覆盖原始循环的前 768 次迭代。启动包含更多块的网格，就可以处理更长的向量；启动包含 n 个或更多线程的网格，就可以处理长度为 n 的向量。

图 2.10 展示了向量加法的内核函数。注意，内核代码不采用 `_h` 和 `_d` 后缀约定，因为这里不存在混淆的可能；本章示例的内核代码不会访问任何主机内存。³

```

1 // Compute vector sum C = A + B
2 // Each thread performs one pair-wise addition
3 __global__
4 void vecAddKernel(float* A, float* B, float* C, int n) {
5     int i = threadIdx.x + blockDim.x * blockIdx.x;
6     if (i < n) {
7         C[i] = A[i] + B[i];
8     }
9 }
10

```

图 2.10: 一个简单的向量加法内核函数

内核的语法遵循 C++ 标准，但有一些值得注意的扩展。首先，在 `vecAddKernel` 声明前有 CUDA 专用关键字 `__global__`。该关键字表示这是一个内核，可以被调用在设备上生成线程网格。

一般而言，CUDA C++ 用三个限定符关键字扩展 C++ 语言，可以在函数声明中使用。图 2.11 总结了这些关键字的含义。`__global__` 表示所声明的函数是 CUDA

³一般来说，内核代码可以通过称为统一虚拟地址 (Unified Virtual Address, UVA) 的机制访问主机内存。该机制允许 GPU 访问主机内存，但带宽低于访问自身设备全局内存；附录 C 会讨论这一机制。

C++ 内核函数。注意，单词 `global` 两侧各有两个下划线。这种内核函数在设备上执行，可以从主机调用；在支持动态并行性的 CUDA 系统中（从 Kepler 架构开始），也可以从设备调用。重要特征是，调用这种内核函数会在设备，即 GPU 上启动一个新的线程网格。

限定符关键字	可从哪里调用	在哪里执行	由谁执行
<code>__host__</code> （默认）	主机	主机	调用者主机线程
<code>__global__</code>	主机（或设备）	设备	新的设备线程网格
<code>__device__</code>	设备	设备	调用者设备线程

图 2.11: CUDA C++ 函数声明关键字（依据原书图 2.11 重绘）

`__device__` 关键字表示所声明的是 CUDA 设备函数。设备函数在 GPU 设备上执行，只能从内核函数或另一个设备函数中调用；它由调用它的设备线程执行，不会启动新的设备线程。

`__host__` 关键字表示所声明的是 CUDA 主机函数。主机函数就是在主机上执行的传统 C++ 函数，只能从另一个主机函数调用。如果声明中没有 CUDA 关键字，CUDA 程序中的所有函数默认都是主机函数。这很合理，因为许多 CUDA 应用是从只运行在 CPU 上的环境移植而来的；移植时，程序员会加入内核函数和设备函数，而原有函数保持为主机函数。让函数默认成为主机函数，可以免去程序员修改所有原有函数声明的繁琐工作。

既然所有函数默认都是主机函数，为什么还需要 `__host__` 关键字？有时，程序员希望一个函数同时能够在主机和设备上使用。标记 `__device__` 会使函数只在设备上可用；要让它两者上都可用，可以在函数声明中同时使用 `__host__` 和 `__device__`。这种组合会告诉编译系统，为同一个函数生成两份目标代码：一份在主机上执行，只能从主机函数调用；另一份在设备上执行，只能从设备函数或内核函数调用。当同一份函数源代码需要重新编译以生成设备版本时，这种能力非常有用，许多用户库函数可能属于这一类。

图 2.10 中 `vecAdd` 内核使用的第二个值得注意的 CUDA C++ 扩展，是内置变量 `threadIdx`、`blockIdx` 和 `blockDim`。回顾一下，所有线程执行相同的内核代码，因此必须有办法让线程彼此区分，并把每个线程导向数据的特定部分。这些内置变量让线程能够访问硬件寄存器，从而获得标识自身的坐标。不同线程会看到其 `threadIdx.x`、`blockIdx.x` 和 `blockDim.x` 变量的不同值。

利用内置变量计算出的全局索引（图 2.10 第 5 行）被写入一个类型为 `int` 的自动（局部）变量 `i`。⁴ 在 CUDA 内核函数中，自动变量对每个线程都是私有的。也就

⁴不同的 C 和 C++ 程序员在声明用于表示数组索引、数组大小、循环迭代计数器和循环边界的变量时，可能选择 `int` 或 `unsigned int`。使用 `unsigned int` 能明确表明这些变量绝不应为负数，并可能触发有助于防止错误的编译器警告。另一方面，在执行减法，或在算术运算中混用 `unsigned int` 与其他 `int` 变量时，使用 `int` 可

是说，每个线程都会生成一份 i ；如果网格启动了 10,000 个线程，就会有 10,000 份 i ，每个线程一份。一个线程赋给自己 i 的值，其他线程不可见。本书第 5 章会更详细地讨论这些自动变量。

比较图 2.4 和图 2.10，可以发现 CUDA 内核有一个重要特点：图 2.10 中没有对应原始循环的 `for` 循环。读者可能会问，循环去哪儿了？答案是，循环现在被线程网格替代。整个网格构成循环的等价物，网格中的每个线程对应原始循环的一次迭代。这有时称为循环并行性，即把原来顺序代码的迭代交给线程并行执行。

注意，`vecAddKernel` 中有 `if (i < n)`。这是因为并非所有向量长度都能表示为块大小的整数倍。例如，向量长度为 100，最小的高效线程块维度为 32，选择块大小 32 时，需要启动 4 个线程块才能处理全部 100 个向量元素。然而，4 个块共包含 128 个线程；必须禁止最后 28 个线程执行原程序没有要求的工作。由于所有线程执行相同代码，它们都会把自己的 i 值与 $n = 100$ 比较。借助 `if (i < n)`，前 100 个线程执行加法，最后 28 个线程不执行，从而允许内核处理任意长度的向量。

2.6 调用内核函数

实现内核函数后，下一步是在主机代码中调用它，以启动线程网格。图 2.12 展示了调用向量加法内核的主机代码。当主机代码调用内核时，会通过执行配置参数设置网格和线程块的维度。配置参数放在传统 C++ 函数参数之前的 `<<<` 和 `>>>` 括号之间。第一个配置参数给出网格中的块数，第二个参数指定每个块中的线程数；本例中每个块有 256 个线程。

```
1 int vecAdd(float* A, float* B, float* C, int n) {  
2     // Allocation and copying of A_d, B_d, and C_d omitted  
3     ...  
4     // Launch ceil(n/256) blocks with 256 threads per block  
5     vecAddKernel<<<ceil(n/256.0), 256>>>(A_d, B_d, C_d, n);  
6 }  
7
```

图 2.12: 包含向量加法内核调用语句的主机代码示例

译者注：图 2.12 按原书保留了 `int` 返回类型，但这段概要代码没有 `return` 语句；图 2.13 的完整版本使用 `void`，与该函数的行为一致。

为了保证网格中的线程足以覆盖所有向量元素，需要把网格块数设置为所需线程数（这里是 n ）除以线程块大小（这里是 256）后的向上取整，即天花板除法。实现

以避免错误。此外，由于 `int` 溢出的行为未定义，编译器可以在优化时利用这一事实。本书在上述声明中有时使用 `int`，有时使用 `unsigned int`；读者在自己的实现中可采用任一种风格。

天花板除法有很多方法；一种方法是对 $n/256.0$ 应用 C++ 的 `ceil` 函数。使用浮点数 256.0 可以确保除法结果为浮点值，使 `ceil` 能正确向上取整。例如，要生成 1000 个线程，就启动

$$\text{ceil}(1000/256.0) = 4$$

个线程块，结果是 $4 \times 256 = 1024$ 个线程。结合内核中的 `if (i < n)`，前 1000 个线程会对 1000 个向量元素执行加法，其余 24 个线程不执行。

图 2.13 展示了 `vecAdd` 函数中的最终主机代码；它补全了图 2.5 中的骨架代码。图 2.10 和图 2.13 共同展示了一个同时包含主机代码和设备内核的简单 CUDA 程序。

```

1 void vecAdd(float* A, float* B, float* C, int n) {
2     float *A_d, *B_d, *C_d;
3     int size = n * sizeof(float);
4
5     cudaMalloc((void**)&A_d, size);
6     cudaMalloc((void**)&B_d, size);
7     cudaMalloc((void**)&C_d, size);
8
9     cudaMemcpy(A_d, A, size, cudaMemcpyHostToDevice);
10    cudaMemcpy(B_d, B, size, cudaMemcpyHostToDevice);
11
12    vecAddKernel<<<ceil(n/256.0), 256>>>(A_d, B_d, C_d, n);
13
14    cudaMemcpy(C, C_d, size, cudaMemcpyDeviceToHost);
15
16    cudaFree(A_d);
17    cudaFree(B_d);
18    cudaFree(C_d);
19 }
20

```

图 2.13: 向量加法内核调用语句

这份源代码把线程块大小硬编码为每块 256 个线程，但使用的线程块数量取决于向量长度 n 。如果 $n = 750$ ，就使用 3 个线程块；如果 $n = 4000$ ，就使用 16 个线程块；如果 $n = 2,000,000$ ，就使用 7813 个线程块。注意，所有线程块都操作向量的不同部分，可以按任意顺序执行。程序员不能对执行顺序作任何假设。

执行资源较少的小型 GPU 可能一次只并行执行其中一两个线程块；较大的 GPU 可能同时执行 128 或 256 个线程块。这使 CUDA 内核的执行速度能够随硬件扩展：同一份代码在小型 GPU 上以较低速度运行，在大型 GPU 上以较高速度运行。第 4 章会再次讨论这一点。

2.7 编译

我们已经看到，实现 CUDA C++ 内核需要使用并非 C++ 标准组成部分的各种扩展。一旦代码中使用了这些扩展，就不能再交给传统 C++ 编译器；必须使用能够识别和理解这些扩展的编译器，例如 NVCC (NVIDIA CUDA Compiler)。

如图 2.14 顶部所示，NVCC 处理 CUDA C++ 程序，并利用 CUDA 关键字把主机代码和设备代码分离。主机代码是普通 C++ 代码，由标准 C/C++ 编译器编译，并作为传统 CPU 进程运行。标记了 CUDA 关键字、用于声明 CUDA 内核及其相关设备函数和数据结构的设备代码，则由 NVCC 编译为称为 PTX 的虚拟二进制文件。随后，NVCC 的运行时组件会进一步把这些 PTX 文件编译为真正的目标文件，并在支持 CUDA 的 GPU 设备上执行。

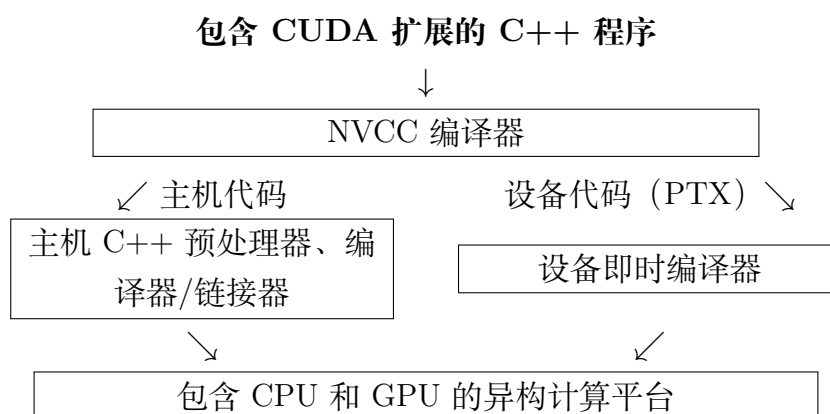


图 2.14: CUDA C++ 程序的编译过程概览 (依据原书图 2.14 重绘)

2.8 小结

本章快速、简化地概览了 CUDA C++ 编程模型。CUDA C++ 扩展 C++ 语言以支持并行计算，本章讨论了这些扩展中的一个基本子集。为方便读者，现将这些扩展总结如下。

函数声明。 CUDA C++ 扩展 C++ 函数声明语法，以支持异构并行计算。图 2.11 总结了这些扩展。使用 `__global__`、`__device__` 或 `__host__` 中的一个，程序员可以指示编译器生成内核函数、设备函数或主机函数。没有这些关键字的函数声明默认是主机函数。如果函数声明同时使用 `__host__` 和 `__device__`，编译器会生成两个版本，一个用于设备，一个用于主机。

内核调用与网格启动。 CUDA C++ 用被 `<<<` 和 `>>>` 括号包围的内核执行配置参数扩展 C++ 函数调用语法。这些执行配置参数只在调用内核函数、启动线程网格时使

用。本章讨论了定义网格维度和每个块维度的执行配置参数；关于内核启动扩展以及其他类型执行配置参数的更多细节，请参阅 CUDA 编程指南 [3]。

内置（预定义）变量。 CUDA 内核可以访问一组内置只读变量，让每个线程区分自己与其他线程，并确定自己需要处理的数据区域。本章讨论了 `threadIdx`、`blockDim` 和 `blockIdx`；第 3 章将进一步讨论如何使用这些变量，在处理多维数据时管理多维块和网格。

运行时 API。 CUDA 支持一组应用程序接口（Application Programming Interface, API）函数，为 CUDA C++ 程序提供服务。本章讨论的服务包括 `cudaMalloc`、`cudaFree` 和 `cudaMemcpy`；这些函数由主机代码调用，分别代表调用程序分配设备全局内存、释放设备全局内存，以及在主机和设备之间传送数据。关于其他 CUDA API 函数，请参阅 CUDA C++ 编程指南 [3]。

本章的目标是介绍 CUDA C++ 的核心概念，以及编写简单 CUDA 程序所需的基本 CUDA 到 C++ 的扩展。本章绝不是 CUDA 全部特性的完整说明；本书后续部分还会介绍更多特性。不过，我们强调的是这些特性所支持的关键并行计算概念。对于并行编程技术的代码示例，我们只会介绍足够使用的 CUDA C++ 特性。一般而言，我们始终建议读者查阅 CUDA C++ 编程指南 [3]，了解最新特性的更多细节。

2.9 练习

1. 如果希望网格中的每个线程计算向量加法的一个输出元素，那么在内核函数中，把线程/块索引映射到数据索引 i 的表达式是什么？

- a. `i = threadIdx.x + threadIdx.y;`
- b. `i = blockIdx.x + threadIdx.x;`
- c. `i = blockIdx.x * blockDim.x + threadIdx.x;`
- d. `i = blockIdx.x * threadIdx.x;`

2. 假设希望每个线程计算向量加法中相邻的两个元素。把线程/块索引映射到线程要处理的第一个元素的数据索引 i 的表达式是什么？

- a. `i = blockIdx.x * blockDim.x + threadIdx.x + 2;`
- b. `i = blockIdx.x * threadIdx.x * 2;`
- c. `i = (blockIdx.x * blockDim.x + threadIdx.x) * 2;`
- d. `i = blockIdx.x * blockDim.x * 2 + threadIdx.x;`

3. 希望每个线程计算两个元素。每个线程块处理由两个区段组成的、连续的 $2 \times \text{blockDim.x}$ 个元素。块中的所有线程先处理第一个区段，每个线程处理一个元素；然后它们一起移动到下一个区段，每个线程再处理一个元素。假设变量 i 是线程要处理的第一个元素的索引，那么把线程/块索引映射到第一个元素数据索引的表达式是什么？
- a. $i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x} + 2;$
 - b. $i = \text{blockIdx.x} * \text{threadIdx.x} * 2;$
 - c. $i = (\text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}) * 2;$
 - d. $i = \text{blockIdx.x} * \text{blockDim.x} * 2 + \text{threadIdx.x};$
4. 对于一次向量加法，向量长度为 8000，每个线程计算一个输出元素，线程块大小为 1024。程序员把内核调用配置为覆盖所有输出元素所需的最少线程块数。网格中有多少个线程？
- a. 8000
 - b. 8196
 - c. 8192
 - d. 8200
5. 要为包含 v 个整数元素的数组分配设备全局内存。cudaMalloc 的第二个参数应使用哪个表达式？
- a. n
 - b. v
 - c. $n * \text{sizeof(int)}$
 - d. $v * \text{sizeof(int)}$
6. 如果希望分配一个包含 n 个浮点元素的数组，并让浮点指针变量 A_d 指向所分配的内存，cudaMalloc 调用的第一个参数应使用什么表达式？
- a. n
 - b. $(\text{void} *) A_d$
 - c. $*A_d$
 - d. $(\text{void} **) \&A_d$
7. 如果希望把 3000 字节数据从主机数组 A_h (A_h 指向源数组的第 0 个元素) 复制到设备数组 A_d (A_d 指向目标数组的第 0 个元素)，CUDA 中适合进行该复制的 API 调用是什么？

- a. `cudaMemcpy(3000, A_h, A_d, cudaMemcpyHostToDevice);`
- b. `cudaMemcpy(A_h, A_d, 3000, cudaMemcpyDeviceToHost);`
- c. `cudaMemcpy(A_d, A_h, 3000, cudaMemcpyHostToDevice);`
- d. `cudaMemcpy(3000, A_d, A_h, cudaMemcpyHostToDevice);`

8. 应如何声明变量 `err`, 才能适当地接收 CUDA API 调用返回的值?

- a. `int err;`
- b. `cudaError err;`
- c. `cudaError_t err;`
- d. `cudaSuccess_t err;`

9. 考虑图 2.15 中的 CUDA 内核及调用它的主机函数。

```

1 __global__ void foo_kernel(float* a, float* b, unsigned int N) {
2     unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
3     if (i < N) {
4         b[i] = 2.7f * a[i] - 4.3f;
5     }
6 }
7 void foo(float* a_d, float* b_d) {
8     unsigned int N = 200000;
9     foo_kernel<<<(N + 128 - 1)/128, 128>>>(a_d, b_d, N);
10 }
11

```

图 2.15: 练习题 9 的 CUDA 代码

- a. 每个块中的线程数是多少?
 - b. 网格中的线程总数是多少?
 - c. 网格中的块数是多少?
 - d. 执行第 02 行代码的线程数是多少?
 - e. 执行第 04 行代码的线程数是多少?
10. 一位新的 CUDA 程序员对 CUDA 感到沮丧。他抱怨 CUDA 太繁琐: 自己计划在主机和设备上都执行的许多函数, 必须分别声明两次, 一次声明为主机函数, 一次声明为设备函数。你会如何帮助这位新程序员?

参考文献

- [1] Y. N. Patt and S. J. Patel. *Introduction to Computing Systems: From Bits and Gates to C and Beyond*. McGraw Hill Publisher, 2020, 2000, 2004.
- [2] M. J. E. Atallah. *Algorithms and Theory of Computation Handbook*. CRC Press, 1998.
- [3] NVIDIA. *CUDA C++ Programming Guide*, Release 13.0, 2025. <https://docs.nvidia.com/cuda/cuda-c-programming-guide>.

第三章 多维网格与数据

第二章中，我们学习了如何编写一个简单的 CUDA C++ 程序：调用内核函数，启动一个处理一维数组元素的一维线程网格。内核规定网格中每个线程要执行的语句。本章将更一般地考察线程的组织方式，并学习如何使用线程和线程块处理多维数组。全章贯穿多个例子，包括把彩色图像转换为灰度图像、模糊图像以及矩阵乘法。这些例子也将帮助读者熟悉数据并行推理，为后续讨论 GPU 架构、内存组织和性能优化做好准备。

3.1 多维网格组织

在 CUDA 中，网格中的所有线程都执行同一个内核函数，并依靠自己的坐标（即线程索引）彼此区分，同时确定要处理的数据区域。正如第二章所见，这些线程按两级层次组织：一个网格由一个或多个线程块组成，每个线程块又由一个或多个线程组成。同一线程块中的所有线程共享相同的块索引，该索引可以通过内置变量 `blockIdx` 访问。每个线程还有自己的线程索引，可以通过内置变量 `threadIdx` 访问。线程执行内核函数时，引用 `blockIdx` 和 `threadIdx` 会得到该线程的坐标。内核调用语句中的执行配置参数指定网格和每个线程块的维度；这些维度可以通过内置变量 `gridDim` 和 `blockDim` 访问。

一般而言，网格是一个三维线程块数组，而每个线程块又是一个三维线程数组。调用内核时，程序需要指定网格大小以及每个线程块在各个维度上的大小。这些大小通过内核调用语句中的执行配置参数（位于 `<<< ... >>>` 之间）指定。第一个执行配置参数以线程块数量为单位指定网格的维度，第二个以线程数量为单位指定每个线程块的维度。每个参数的类型都是 `dim3`，这是一种包含三个整数元素 `x`、`y` 和 `z` 的向量类型；三个元素分别指定三个维度的大小。如果程序使用的维度少于三个，只需把未使用维度的大小设为 1。

例如，下面的主机代码调用 `vecAddKernel`，生成一个一维网格；该网格包含 32 个线程块，每个线程块包含 128 个线程：

```
1 dim3 dimGrid(32, 1, 1);
2 dim3 dimBlock(128, 1, 1);
3 vecAddKernel<<<dimGrid, dimBlock>>>>(...);
```

在这个例子中，网格中的线程总数为

$$128 \times 32 = 4096.$$

注意，`dimBlock` 和 `dimGrid` 是由程序员在主机代码中定义的变量。只要类型为 `dim3`，它们可以使用任何合法的 C++ 变量名。例如，下面的语句与前面的语句实现相同的效果：

```
1 dim3 dog(32, 1, 1);
2 dim3 cat(128, 1, 1);
3 vecAddKernel<<<dog, cat>>>(...);
```

网格和线程块的维度也可以根据其他变量计算。例如，图 2.12 中的内核调用可以改写为：

```
1 dim3 dimGrid(ceil(n/256.0), 1, 1);
2 dim3 dimBlock(256, 1, 1);
3 vecAddKernel<<<dimGrid, dimBlock>>>(...);
```

这样，线程块数量就可以随向量长度变化，使网格拥有足够的线程覆盖所有向量元素。在本例中，程序员选择把线程块大小固定为 256；调用内核时变量 `n` 的值决定网格的维度。如果 `n` 等于 1000，网格包含 4 个线程块；如果 `n` 等于 4000，网格包含 16 个线程块。在这两种情况下，线程数都足以覆盖所有向量元素。网格启动后，网格和线程块的维度在整个网格执行完毕之前保持不变。

为了方便一维网格和一维线程块的调用，CUDA 提供了一种特殊的简写形式。不使用 `dim3` 变量时，可以直接使用算术表达式指定一维网格和线程块的配置。在这种情况下，CUDA 编译器把算术表达式作为 `x` 维度，并假定 `y` 和 `z` 维度均为 1。因此，图 2.12 中的内核调用可以写成：

```
1 vecAddKernel<<<ceil(n/256.0), 256>>>(...);
```

熟悉 C++ 的读者会发现，这种一维配置的“简写”利用了 C++ 构造函数和默认参数的工作方式。`dim3` 构造函数的参数默认值为 1。当只传入一个值而期望 `dim3` 时，该值传给构造函数的第一个参数，第二、第三个参数使用默认值 1。因此得到一个一维网格或线程块，其 `x` 维度大小就是传入的值，`y` 和 `z` 维度大小为 1。

在内核函数中，变量 `gridDim` 和 `blockDim` 的 `x` 字段会根据执行配置参数预先初始化。例如，如果 `n` 等于 4000，那么在 `vecAddKernel` 内引用 `gridDim.x` 和 `blockDim.x`，得到的值分别为 16 和 256。注意，与主机代码中的 `dim3` 变量不同，内核函数中的这些变量名属于 CUDA C++ 规范，不能更改。也就是说，`gridDim` 和 `blockDim` 是内置变量，始终反映网格和线程块的维度。

在 CUDA C++ 中, `gridDim.x` 的允许取值范围为 1 到 $2^{31} - 1$; `gridDim.y` 和 `gridDim.z` 的允许取值范围为 1 到 $2^{16} - 1 = 65,535$ 。同一线程块中的所有线程共享 `blockIdx.x`、`blockIdx.y` 和 `blockIdx.z` 的值。在所有线程块之间, `blockIdx.x` 的取值范围为 0 到 `gridDim.x-1`, `blockIdx.y` 的取值范围为 0 到 `gridDim.y-1`, `blockIdx.z` 的取值范围为 0 到 `gridDim.z-1`。

下面转而讨论线程块的配置。每个线程块组织成一个三维线程数组。

把 `blockDim.z` 设为 1, 就可以创建二维线程块。像向量加法示例那样, 一维线程块满足

$$\text{blockDim.y} = 1, \quad \text{blockDim.z} = 1.$$

前文已经提到, 同一网格中的所有线程块具有相同的维度和大小。线程块每个维度的线程数量由内核调用的第二个执行配置参数指定。在内核中, 可以通过 `blockDim` 的 `x`、`y` 和 `z` 字段访问这一配置。

在当前 CUDA 系统中, 一个线程块的总大小限制为 1024 个线程。只要线程总数不超过 1024, 这些线程可以以任意方式分布在三个维度中。例如, 下面的 `blockDim` 值都是允许的:

$$(512, 1, 1), \quad (8, 16, 4), \quad (32, 16, 2).$$

另一方面, `blockDim` 为 (32, 32, 2) 时线程总数超过 1024, 因此不允许。

网格及其线程块不需要具有相同的维度。网格可以比线程块维度更高, 反之亦然。例如, 图 3.1 展示了一个小型示例, 其中 `gridDim` 为 (2, 2, 1), `blockDim` 为 (4, 2, 2)。可以使用下面的主机代码创建该网格:

```
1 dim3 dimGrid(2, 2, 1);
2 dim3 dimBlock(4, 2, 2);
3 KernelFunction<<<dimGrid, dimBlock>>>>(...);
```

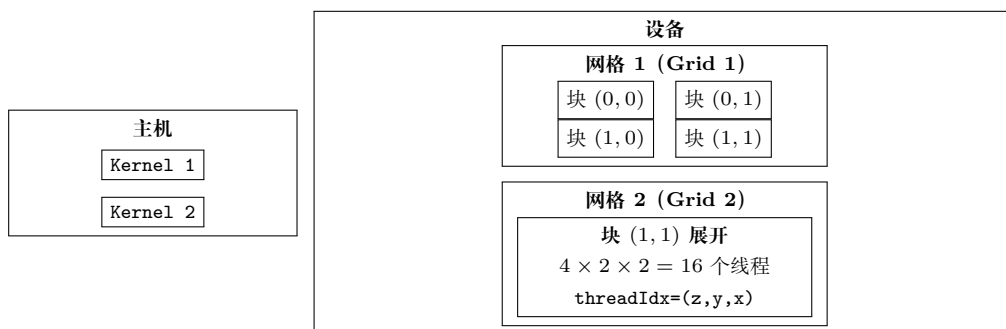


图 3.1: CUDA 多维网格组织示例 (依据原书图 3.1 重绘的概念示意)

图 3.1 中的网格由 4 个线程块组成, 排列成一个 2×2 数组。每个线程块使用 (`blockIdx.y`, `blockIdx.x`) 标记。例如, 块 (1,0) 的 `blockIdx.y=1`, `blockIdx.x=0`。需要注意, 块和线程标签的排列顺序是最高维度在前。这与设置执行配置参数时代码

使用的顺序相反；代码中最低维度在前。用这种相反的顺序标记块，更适合说明线程坐标如何映射到多维数据的索引。

每个 `threadIdx` 也包含三个字段：x 坐标 `threadIdx.x`、y 坐标 `threadIdx.y` 和 z 坐标 `threadIdx.z`。图 3.1 展示了线程块内部的组织方式。在这个例子中，每个线程块组织成一个 $4 \times 2 \times 2$ 的线程数组。由于网格中的所有线程块具有相同的维度，图中只展开其中一个线程块。图 3.1 展开块 (1,1)，展示其中的 16 个线程。例如，线程 (1,0,2) 具有 `threadIdx.z=1`、`threadIdx.y=0` 和 `threadIdx.x=2`。在这个例子中，网格共有 4 个线程块，每个线程块 16 个线程，总计 64 个线程。

译者注：原书相应说明中有一处疑似排印错误，写作 `threadId.x`；CUDA C++ 的内置变量实际是 `threadIdx.x`，本译稿按正确标识符规范化。这里使用很小的数字只是为了让图示保持简单；典型 CUDA 网格包含数千到数百万个线程。

3.2 将线程映射到多维数据

选择一维、二维还是三维线程组织，通常取决于数据的性质。例如，图片是由像素组成的二维数组，使用由二维线程块组成的二维网格，通常很适合处理图片中的像素。图 3.2 展示了这种处理方式，输入图片 `Pin` 的大小为 62×76 （垂直方向或 y 方向有 62 个像素，水平方向或 x 方向有 76 个像素）。假设我们选择 16×16 的线程块，即 x 方向和 y 方向各有 16 个线程。需要在 y 方向使用 4 个线程块，在 x 方向使用 5 个线程块，因此总共需要 $4 \times 5 = 20$ 个线程块，如图 3.2 所示。粗线表示线程块边界；本概念重绘用文字标出实际像素区和线程覆盖区，具体的有效线程分类见图 3.5。每个线程负责处理一个像素，其 y、x 坐标由 `blockIdx`、`blockDim` 和 `threadIdx` 的值推导得到。

1

垂直（行）坐标为：

$$\text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}.$$

水平（列）坐标为：

$$\text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}.$$

¹本书始终按多维数据的降序维度书写其尺寸：先写 z 维，再写 y 维，最后写 x 维。例如，对于垂直方向（或 y 方向）有 n 个像素、水平方向（或 x 方向）有 m 个像素的图片，我们将其写作 $n \times m$ 。这遵循 C++ 多维数组的索引约定。例如，正文和图中可以简写 $P[y][x]$ 为 $P_{y,x}$ 。遗憾的是，这与 `gridDim` 和 `blockDim` 中数据维度的排列顺序相反。当根据要处理的多维数组定义线程网格的维度时，这种差异尤其容易造成混淆。

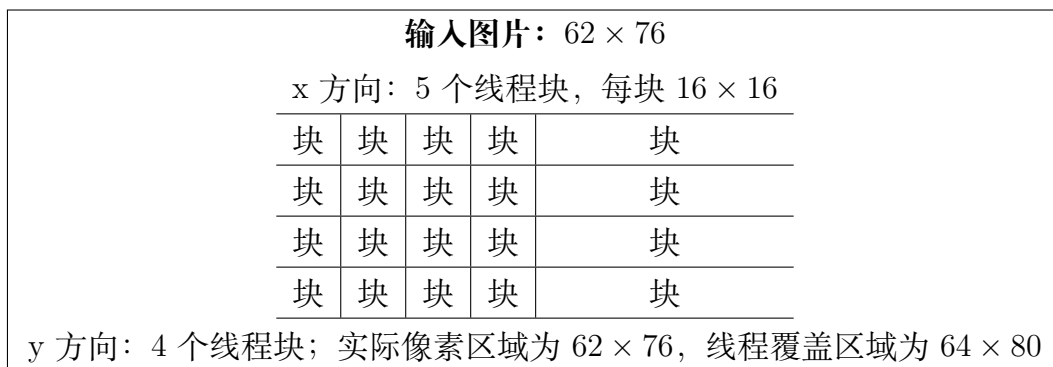


图 3.2: 使用二维线程网格处理输入图片 P_{in} (62×76 , 依据原书图 3.2 重绘的概念示意)

例如, 块 (1,0) 中线程 (0,0) 要处理的 P_{in} 元素可由下面的索引得到:

$$\begin{aligned}
 \text{行索引} &= \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y} \\
 &= 1 \times 16 + 0 = 16, \\
 \text{列索引} &= \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x} \\
 &= 0 \times 16 + 0 = 0.
 \end{aligned}$$

因此, 该线程处理的元素是 $P_{in}[16, 0]$ 。

注意, 图 3.2 中 y 方向多生成了 2 个线程, x 方向多生成了 4 个线程。也就是说, 为了处理 62×76 个像素, 我们总共生成了 64×80 个线程。这与一维内核 `vecAddKernel` 使用 4 个 256 线程块处理 1000 个元素时的情况类似。回顾第二章图 2.10, 需要用 `if` 语句防止多出的 24 个线程产生影响。同样, 图片处理内核也应检查线程的垂直索引和水平索引是否落在有效像素范围内。

假设主机代码使用整数变量 n 记录 y 方向的像素数, 使用另一个整数变量 m 记录 x 方向的像素数。进一步假设输入图片数据已经复制到设备全局内存, 可以通过指针变量 P_{in_d} 访问; 输出图片已经分配在设备内存中, 可以通过 P_{out_d} 访问。下面的主机代码调用二维内核 `colorToGrayscaleConversion` 处理图片:

```

1 dim3 dimGrid(ceil(m/16.0), ceil(n/16.0), 1);
2 dim3 dimBlock(16, 16, 1);
3 colorToGrayscaleConversion<<<dimGrid, dimBlock>>>(<
4     Pout_d, Pin_d, m, n);

```

译者注: 原书此处的主机调用顺序与图 3.4 中的内核形参声明 P_{out} , P_{in} 相反。为使示例按该声明正确工作, 本译稿将调用顺序校正为 P_{out_d} , P_{in_d} ; 这一处改动特此注明。

为简单起见, 本例把线程块维度固定为 16×16 。另一方面, 网格维度取决于图片的尺寸。处理一张 1500×2000 (300 万像素) 的图片时, 需要生成 11,750 个线程块: y 方向 94 个, x 方向 125 个。在内核函数中引用 `gridDim.x`、`gridDim.y`、`blockDim.x`

和 `blockDim.y`，得到的值分别为 125、94、16 和 16。

在展示内核代码之前，需要先理解 C 和 C++ 语句如何访问动态分配的多维数组。理想情况下，我们希望把 `Pin_d` 当作二维数组访问，使第 j 行、第 i 列的元素可以写成 `Pin_d[j][i]`。但是，CUDA C++ 所基于的 C++ 标准要求，在把 `Pin` 当作二维数组访问时，列数必须在编译期已知。对于动态分配的数组，这个信息在编译期并不知道。事实上，使用动态数组的部分原因正是允许数组大小和维度根据运行时的数据大小变化。因此，动态分配二维数组的列数在编译期未知，是设计所决定的。结果是，在当前 CUDA C++ 中，程序员需要显式地把动态分配的二维数组线性化（或“展平”）为等价的一维数组。

实际上，C 和 C++ 中的所有多维数组最终都会被线性化。这是因为现代计算机使用“扁平”的内存空间（见“内存空间”侧栏）。对于静态分配的数组，编译器允许程序员使用 `Pin_d[j][i]` 这样的高维索引语法访问元素；在底层，编译器会把它们线性化为等价的一维数组，并把多维索引语法转换为一维偏移量。对于动态分配的数组，由于编译期缺少执行这种转换所需的维度信息，这项转换就留给程序员完成。

内存空间

内存空间是对现代计算机中处理器如何访问内存的一种简化视图。通常，每个正在运行的应用程序都有自己的内存空间。应用要处理的数据以及为应用执行的指令，都存放在该内存空间的位置中。每个位置通常可以容纳一个字节，并具有一个地址。需要多个字节的变量——例如占 4 字节的 `float` 和占 8 字节的 `double`——存放在连续的字节位置中。访问内存空间中的数据值时，处理器给出起始地址（起始字节位置的地址）以及所需的字节数。

大多数现代计算机至少有 4G 个按字节编址的位置。这里，每个 G 表示的位置数为

$$1,073,741,824 = 2^{30}.$$

所有位置都带有从 0 到最大编号的地址。由于每个位置只有一个地址，我们称这种内存空间具有“扁平”组织。因此，所有多维数组最终都会被“展平”为等价的一维数组。C++ 程序员虽然可以使用多维数组语法访问元素，但编译器会把这种访问转换为一个指向数组起始元素的基指针，以及根据多维索引计算得到的一维偏移量。

二维数组至少有两种线性化方式。一种方式是把同一行的所有元素放在连续位置中，再把各行依次放入内存空间。这种布局称为行主序（row-major layout），如图 3.3 所示。为提高可读性，我们用 $M_{j,i}$ 表示矩阵 M 第 j 行、第 i 列的元素。它等价于 C++ 中的 `M[j][i]`，但在正文和图中更易读。图 3.3 展示了一个 4×4 矩阵 M 被线

性化为包含 16 个元素的一维数组：先放第 0 行的 4 个元素，再放第 1 行的 4 个元素，以此类推。因此，矩阵第 j 行、第 i 列元素的一维等价索引为

$$j \times 4 + i.$$

$j \times 4$ 项跳过第 j 行之前所有行的元素， i 项再选择第 j 行对应区段中的元素。例如， $M_{2,1}$ 的一维索引为 $2 \times 4 + 1 = 9$ ，图中一维数组的 M_9 就是它的等价元素。这正是 C 和 C++ 编译器线性化二维数组的方式。

$M_{0,0}$	$M_{0,1}$	$M_{0,2}$	$M_{0,3}$
$M_{1,0}$	$M_{1,1}$	$M_{1,2}$	$M_{1,3}$
$M_{2,0}$	$M_{2,1}$	$M_{2,2}$	$M_{2,3}$
$M_{3,0}$	$M_{3,1}$	$M_{3,2}$	$M_{3,3}$

↓ 按行展开

$M_{0,0}$	$M_{0,1}$	$M_{0,2}$	$M_{0,3}$	$M_{1,0}$	$M_{1,1}$	$M_{1,2}$	$M_{1,3}$
$M_{2,0}$	$M_{2,1}$	$M_{2,2}$	$M_{2,3}$	$M_{3,0}$	$M_{3,1}$	$M_{3,2}$	$M_{3,3}$

$M_{2,1} \longrightarrow M_{2 \times 4 + 1} = M_9$

图 3.3: 二维 C++ 数组的行主序布局（索引为 $j \times \text{Width} + i$ ）及其一维索引（依据原书图 3.3 重绘）

另一种二维数组线性化方式是把同一列的所有元素放在连续位置中，再把各列依次放入内存空间。这种布局称为列主序（column-major layout），由 FORTRAN 编译器使用。二维数组的列主序布局等价于其转置形式的行主序布局。这里不再详细讨论列主序，只提醒主要编程经验来自 FORTRAN 的读者：CUDA C++ 使用行主序，而不是列主序。此外，许多为 FORTRAN 程序设计的 C 库使用列主序，以匹配 FORTRAN 编译器的布局。因此，如果从 C 或 C++ 程序调用这些库，库的手册通常会要求用户转置输入数组。

现在可以研究图 3.4 所示的 `colorToGrayscaleConversion` 内核。该内核使用下面的公式把每个彩色像素转换为对应的灰度值：

$$L = 0.299r + 0.587g + 0.114b.$$

译者注：原书正文在这里给出的灰度公式与图 3.4 代码中的权重 0.21f、0.71f 和 0.07f 并不相同。本译稿保留了两处底本内容，并将这一差异明确标出。

水平方向总共有 `blockDim.x*gridDim.x` 个线程。与 `vecAddKernel` 示例类似，下面的表达式生成从 0 到 `blockDim.x*gridDim.x-1` 的每个整数值：

$$\text{col} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}.$$

`gridDim.x*blockDim.x` 大于或等于图片宽度（从主机代码传入的 `m`）。换言之，水平方向的线程数至少等于像素数；同样，垂直方向的线程数也至少等于像素数。

因此，只要测试并确保 `row` 和 `col` 都在有效范围内，即满足 `(col < width) && (row < height)`，就能覆盖图片中的每个像素。由于每一行有 `width` 个像素，行

```
1 // The input image is encoded as unsigned chars [0, 255]
2 // Each pixel is 3 consecutive chars for the 3 channels (RGB)
3 __global__
4 void colorToGrayscaleConversion(unsigned char* Pout,
5                                 unsigned char* Pin, int width, int height) {
6     int col = blockIdx.x*blockDim.x + threadIdx.x;
7     int row = blockIdx.y*blockDim.y + threadIdx.y;
8     if (col < width && row < height) {
9         // Get 1D offset for the grayscale image
10        int grayOffset = row*width + col;
11        // One can think of the RGB image having CHANNEL
12        // times more columns than the grayscale image
13        int rgbOffset = grayOffset*CHANNELS;
14        unsigned char r = Pin[rgbOffset];    // Red value
15        unsigned char g = Pin[rgbOffset + 1]; // Green value
16        unsigned char b = Pin[rgbOffset + 2]; // Blue value
17        // Perform the rescaling and store it
18        // We multiply by floating-point constants
19        Pout[grayOffset] = (unsigned char)(0.21f*r + 0.71f*g + 0.07f*b);
20    }
21 }
22
```

图 3.4: colorToGrayscaleConversion 内核及其二维线程到数据的映射（依据原书图 3.4 重绘）

row、列 col 处像素的一维索引为 $\text{row} \times \text{width} + \text{col}$ (图中第 10 行)。这个一维索引 grayOffset 是 Pout 的像素索引, 因为输出灰度图像中的每个像素占一个字节 (unsigned char)。

以 62×76 的图片为例, 块 (1,0) 中线程 (0,0) 计算得到的 Pout 像素的一维索引为:

$$\begin{aligned}\text{行索引} &= \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y} \\ &= 1 \times 16 + 0 = 16, \\ \text{列索引} &= \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x} \\ &= 0 \times 16 + 0 = 0.\end{aligned}$$

因此,

$$\begin{aligned}P_{\text{out}}[16, 0] &= \text{Pout}[16 \times 76 + 0] \\ &= \text{Pout}[1216].\end{aligned}$$

对于 Pin, 需要把灰度像素索引乘以 3 (图中第 13 行), 因为每个彩色像素由三个元素存储: 分别是 r、g、b, 每个元素占一个字节。所得的 rgbOffset 是 Pin 中该彩色像素的起始位置。程序从 Pin 的三个连续字节位置读取 r、g 和 b 的值, 计算灰度值, 再使用 grayOffset 把结果写入 Pout。²

以同一个 62×76 图片为例, 块 (1,0) 中线程 (0,0) 所处理像素的 Pin 第一个分量的一维索引为:

$$\begin{aligned}\text{行索引} &= \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y} \\ &= 1 \times 16 + 0 = 16, \\ \text{列索引} &= \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x} \\ &= 0 \times 16 + 0 = 0.\end{aligned}$$

因此,

$$\begin{aligned}P_{\text{in}}[16, 0] &= \text{Pin}[16 \times 76 \times 3 + 0] \\ &= \text{Pin}[3648].\end{aligned}$$

访问的数据是从字节偏移 3648 开始的三个字节。

图 3.5 展示了处理 62×76 图片时 colorToGrayscaleConversion 的执行。使用 16×16 线程块时, 该内核生成 64×80 个线程; 网格包含 $4 \times 5 = 20$ 个线程块, 垂直方向 4 个, 水平方向 5 个。线程块的执行行为分为图中四个阴影区域。

图中的第 1 区域包含覆盖图片大部分像素的 12 个线程块。这些线程的 col 和 row 值都在有效范围内, 全部通过 if 测试并处理深色区域中的像素; 每个线程块的 $16 \times 16 = 256$ 个线程都会处理像素。

²这里假设 CHANNELS 是值为 3 的常量, 其定义位于内核函数之外。

覆盖区域: 62×76 像素, 线程覆盖 64×80				
区域 1 12 个块, 256 个 线程有效	区域 1	区域 1	区域 1	区域 2 192/256 有效
区域 1	区域 1	区域 1	区域 1	区域 2
区域 1	区域 1	区域 1	区域 1	区域 2
区域 3 224/256 有效	区域 3	区域 3	区域 3	区域 4 168/256 有效

图 3.5: 用 16×16 线程块覆盖 76×62 (宽 $\times$ 高, 本章正文记作 62×76) 图片时的四类区域 (依据原书图 3.5 重绘)

第 2 区域包含覆盖图片右上方中等阴影区域的 3 个线程块。这些线程的 `row` 值始终在有效范围内, 但其中一些 `col` 值超过图片宽度 $m=76$ 。这是因为水平方向的线程数总是程序员选择的 `blockDim.x` (此处为 16) 的倍数。覆盖 76 个像素所需的最小 16 的倍数是 80。因此, 每行有 12 个线程的 `col` 值在有效范围内并处理像素; 其余 4 个线程的 `col` 值超出范围, 无法通过 `if` 条件。每个线程块中总共有 $12 \times 16 = 192$ 个线程处理像素。

第 3 区域对应覆盖图片左下方中等阴影区域的 4 个线程块。这些线程的 `col` 值始终在有效范围内, 但其中一些 `row` 值超过图片高度 $n=62$ 。这是因为垂直方向的线程数总是程序员选择的 `blockDim.y` (此处为 16) 的倍数。覆盖 62 个像素所需的最小 16 的倍数是 64。因此, 每列有 14 个线程的 `row` 值在有效范围内并处理像素; 其余 2 个线程无法通过 `if` 条件。总共有 $16 \times 14 = 224$ 个线程处理像素。

第 4 区域包含覆盖右下方浅色区域的线程。与第 2 区域一样, 最上方 14 行中每行有 4 个线程的 `col` 值超出范围; 与第 3 区域一样, 最下方两行的 `row` 值全部超出范围。因此, 只有 $14 \times 12 = 168$ 个线程处理像素。

前面对二维数组的讨论可以很容易扩展到三维数组, 只需在线性化数组时加入另一个维度。具体做法是把数组的每个“平面”依次放入地址空间。假设程序员使用变量 `m` 和 `n` 分别记录三维数组的列数和行数, 还需要在调用内核时确定 `blockDim.z` 和 `gridDim.z` 的值。在内核中, 数组索引还要包含另一个全局索引:

```
1 int plane = blockIdx.z*blockDim.z + threadIdx.z;
```

三维数组 `P` 的线性化访问形式为:

$$P[\text{plane} * m * n + \text{row} * m + \text{col}].$$

处理三维 `P` 数组的内核需要检查三个全局索引 `plane`、`row` 和 `col` 是否都落在数组的有效范围内。CUDA 内核中三维数组的使用方式, 将在第 8 章的模板 (stencil) 计算

模式中进一步研究。

3.3 图像模糊——一个更复杂的内核

到目前为止，我们研究了两个内核。第一个是 `vecAddKernel`。第二个内核的名称为 `colorToGrayscaleConversion`。在这两个内核中，每个线程只对一个数组元素执行少量算术运算。这些内核很好地说明了 CUDA 的基本程序结构和数据并行执行概念。

此时，读者可能会提出一个显而易见的问题：CUDA 程序中的所有线程是否都只彼此独立地执行如此简单的操作？答案是否定的。在真实的 CUDA 程序中，线程经常会对自己的数据执行复杂操作，并且需要彼此协作。接下来的几章将逐步研究更复杂、具有这些特征的例子。我们先从图像模糊函数开始。

图像模糊会平滑图像中像素值的突变。图 3.6 展示了图像模糊的效果。简单地说，模糊就是让图像变得不清晰。对人眼而言，模糊图像往往会隐藏细节，呈现“整体图景”，也就是图中主要主题对象的印象。在计算机图像处理算法中，图像模糊的一种常见用途，是用周围干净的像素值修正有问题的像素值，从而减小噪声和颗粒状渲染效果的影响。在计算机视觉中，图像模糊可以让边缘检测和目标识别算法关注主题对象，而不会被大量细粒度对象拖累。在显示系统中，有时会把图像的其余部分模糊掉，以突出某个特定区域。

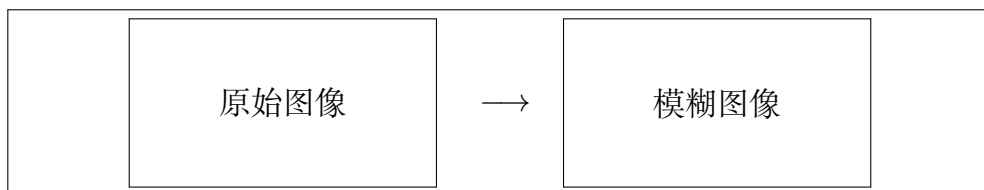


图 3.6: 原始图像与模糊图像的概念对比（依据原书图 3.6 重绘）

从数学上说，图像模糊函数把输入图像中包含原始像素的一个像素块进行加权求和，用结果计算输出图像像素的值。正如第 7 章将要学习的，这类加权求和属于卷积模式。本章采用一种简化方法：取围绕目标像素并包含目标像素在内的 $N \times N$ 像素块的简单平均值。为保持算法简单，我们不根据像素到目标像素的距离为不同像素赋予不同权重。在实际的卷积模糊方法中，使用这种权重很常见，例如高斯模糊（Gaussian Blur）。

图 3.7 展示了使用 3×3 像素块进行图像模糊的例子。计算位置为 (row, col) 的输出像素值时，像素块以输入图像中同一位置的像素为中心。 3×3 像素块覆盖三行 $(row-1, row, row+1)$ 和三列 $(col-1, col, col+1)$ 。例如，计算位置 $(25, 50)$ 处的

输出像素时，要使用下面 9 个像素：

(24, 49) (24, 50) (24, 51)
 (25, 49) (25, 50) (25, 51)
 (26, 49) (26, 50) (26, 51)

(row-1,col-1)	(row-1,col)	(row-1,col+1)
(row,col-1)	目标像素	(row,col+1)
(row+1,col-1)	(row+1,col)	(row+1,col+1)

图 3.7: 输出像素由输入图像中以自身为中心的邻域像素块求平均得到（依据原书图 3.7 重绘）

图 3.8 展示了图像模糊内核。与 `colorToGrayscaleConversion` 内核采用的策略类似，我们让每个线程计算一个输出像素，也就是说，线程到输出数据的映射保持不变。在内核开始处，可以看到熟悉的 `col` 和 `row` 索引计算（第 3–4 行），以及检查两个索引是否在图像高度 `h` 和宽度 `w` 的有效范围内的 `if` 语句（第 5 行）。只有 `col` 和 `row` 都在有效范围内的线程，才允许参与执行。

如图 3.7 所示，`col` 和 `row` 的值也给出了该线程负责计算的输出像素中心位置，以及用于计算该输出像素的输入像素块中心位置。图 3.8 中的嵌套 `for` 循环（第 10–11 行）遍历像素块中的所有像素。我们假设程序定义了常量 `BLUR_SIZE`。`BLUR_SIZE` 的值表示像素块每一侧包含的像素数，也称为半径。像素块一条边上的像素总数为

$$2 \times \text{BLUR_SIZE} + 1.$$

例如，对于 3×3 像素块，`BLUR_SIZE` 设为 1；对于 7×7 像素块，`BLUR_SIZE` 设为 3。外层循环遍历像素块的各行，内层循环对每一行遍历像素块的各列。

在 3×3 像素块的例子中，`BLUR_SIZE` 为 1。对于负责计算输出像素 (25, 50) 的线程，在外层循环第一次迭代中，`curRow = row - BLUR_SIZE = (25 - 1) = 24`。因此，内层循环遍历输入图像的第 24 行，`curCol` 从 `col - BLUR_SIZE = 50 - 1 = 49` 变化到 `col + BLUR_SIZE = 51`。第一次外层循环处理的像素就是 (24, 49)、(24, 50) 和 (24, 51)。读者可以验证，第二次外层循环处理 (25, 49)、(25, 50) 和 (25, 51)；第三次处理 (26, 49)、(26, 50) 和 (26, 51)。

第 16 行使用 `curRow` 和 `curCol` 的线性化索引访问当前迭代所经过的输入像素值，并把像素值累加到运行中的求和变量 `pixVal` 中。第 17 行通过递增 `pixels` 记录又加入了一个像素值。处理完像素块中的所有像素后，第 22 行把 `pixVal` 除以 `pixels`，计算像素块的平均值，并使用 `row` 和 `col` 的线性化索引把结果写入输出像素。

第 15 行的条件语句保护第 16、17 行的执行。例如，计算靠近图像边缘的输出像素时，像素块可能延伸到输入图像的有效范围之外。假设使用 3×3 像素块，图 3.9

```
1 __global__
2 void blurKernel(unsigned char *in, unsigned char *out, int w, int h) {
3     int col = blockIdx.x*blockDim.x + threadIdx.x;
4     int row = blockIdx.y*blockDim.y + threadIdx.y;
5     if (col < w && row < h) {
6         int pixVal = 0;
7         int pixels = 0;
8
9         // Get average of the surrounding BLUR_SIZE x BLUR_SIZE box
10        for (int blurRow = -BLUR_SIZE; blurRow < BLUR_SIZE + 1; ++blurRow) {
11            for (int blurCol = -BLUR_SIZE; blurCol < BLUR_SIZE + 1; ++
12            blurCol) {
13                int curRow = row + blurRow;
14                int curCol = col + blurCol;
15                // Verify we have a valid image pixel
16                if (curRow >= 0 && curRow < h && curCol >= 0 && curCol < w)
17                {
18                    pixVal += in[curRow*w + curCol];
19                    ++pixels; // Keep track of number of pixels in the avg
20                }
21            }
22            // Write our new pixel value out
23            out[row*w + col] = (unsigned char)((float)pixVal/pixels);
24        }
25    }
```

图 3.8: 图像模糊内核 (依据原书图 3.8 重绘)

展示了这种情况。在情况 1 中，正在模糊左上角像素；预期像素块中的 9 个像素有 5 个并不存在于输入图像中。此时输出像素的 row 和 col 值分别为 0 和 0。嵌套循环的 9 次迭代中，curRow 和 curCol 的取值为：

$$(-1,-1), (-1,0), (-1,1), (0,-1), (0,0), (0,1), (1,-1), (1,0), (1,1).$$

对于超出图片范围的 5 个像素，至少有一个索引小于 0。curRow>=0 和 curCol>=0 条件会捕获这些值，并跳过第 16、17 行的执行。因此，只有 4 个有效像素会被累加到 pixVal，pixels 也只递增 4 次，所以第 22 行可以正确计算平均值。

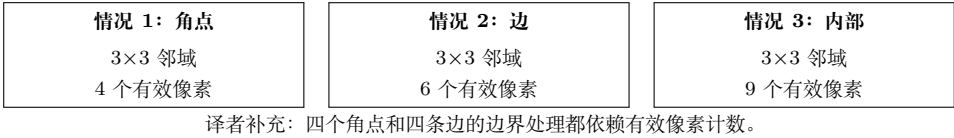


图 3.9: 图像边缘附近像素的边界条件（依据原书图 3.9 重绘的概念示意）

读者应自行分析图 3.9 中其他情况，并研究 blurKernel 中嵌套循环的执行行为。需要注意，大多数线程会发现其负责的 3×3 像素块中的所有像素都位于输入图像内，因此会累加全部 9 个像素。但是，四个角点的线程只累加 4 个像素；四条边上其他像素的线程累加 6 个像素。这些差异正是必须用 pixels 变量记录实际累加像素数量的原因。

3.4 矩阵乘法

矩阵-矩阵乘法（简称矩阵乘法）是基础线性代数子程序（Basic Linear Algebra Subprograms, BLAS）标准的重要组成部分（见“线性代数函数”侧栏）。它是许多线性代数求解器（例如 LU 分解）的基础，也是深度学习中的重要计算；第 19 章讨论卷积神经网络、第 20 章讨论大语言模型时还会看到这一点。

线性代数函数

线性代数运算广泛用于科学和工程应用。基础线性代数子程序 (BLAS) 是发布基础代数运算库的事实标准。BLAS 函数分为三个等级；等级越高，函数执行的运算数量越多。

一级函数执行如下形式的向量运算：

$$\mathbf{y} = \alpha \mathbf{x} + \mathbf{y},$$

其中 $\mathbf{x}$ 和 $\mathbf{y}$ 是向量， α 是标量。第二级函数执行如下形式的矩阵-向量运算：

$$\mathbf{y} = \alpha A\mathbf{x} + \beta \mathbf{y},$$

其中 A 是矩阵， $\mathbf{x}$ 和 $\mathbf{y}$ 是向量， α 和 β 是标量。第 17 章讨论稀疏线性代数中的一种二级函数。第三级函数执行如下形式的矩阵-矩阵运算：

$$C = \alpha AB + \beta C,$$

其中 A 、 B 和 C 是矩阵， α 和 β 是标量。

第二章的向量加法例子是一级 BLAS 函数的一个特例，其中 $\alpha = 1$ 。本章的矩阵-矩阵乘法例子是三级函数的一个特例，其中 $\alpha = 1$ 、 $\beta = 0$ 。这些 BLAS 函数很重要，因为它们是更高层代数函数（例如线性方程组求解器和特征值分析）的基本构件。正如后文将要讨论的，不同 BLAS 函数实现的性能，在顺序和并行计算机上都可能存在数量级上的差异。

一个 $i \times j$ (i 行、 j 列) 矩阵 M 与一个 $j \times k$ (j 行、 k 列) 矩阵 N 相乘，会产生一个 $i \times k$ 矩阵 P 。矩阵乘法中，输出矩阵 P 的每个元素都是 M 的一行与 N 的一列的内积。我们继续采用如下约定： $P_{\text{row}, \text{col}}$ 表示垂直方向第 row 行、水平方向第 col 列的元素。如图 3.10 所示， $P_{\text{row}, \text{col}}$ (矩阵 P 中的小方格) 是由 M 的第 row 行 (M 中的水平条带) 与 N 的第 col 列 (N 中的垂直条带) 形成的向量的内积。内积也称点积，是两个向量对应元素乘积之和，即：

$$P_{\text{row}, \text{col}} = \sum_{k=0}^{\text{Width}-1} M_{\text{row}, k} N_{k, \text{col}}.$$

例如，在图 3.10 中，假设 $\text{row}=1$ 、 $\text{col}=5$ ，则：

$$\begin{aligned} P_{1,5} &= M_{1,0}N_{0,5} + M_{1,1}N_{1,5} + M_{1,2}N_{2,5} + \cdots \\ &\quad + M_{1,\text{Width}-1}N_{\text{Width}-1,5}. \end{aligned}$$

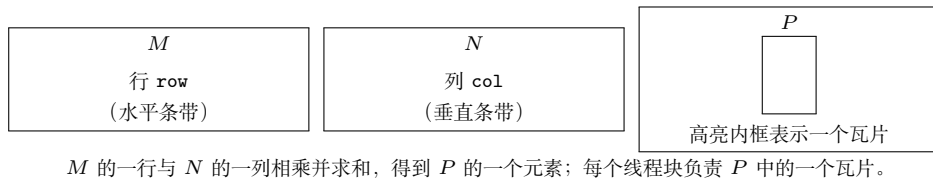


图 3.10: 使用多个线程块分块计算矩阵 P (依据原书图 3.10 重绘)

为了使用 CUDA 实现矩阵乘法，可以沿用 `colorToGrayscaleConversion` 内核的线程映射方法，把网格中的线程映射到输出矩阵 P 的元素。也就是说，每个线程负责计算一个 P 元素。每个线程要计算的 P 元素的行、列索引仍然是：

$$\text{row} = \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y},$$

$$\text{col} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}.$$

采用这种一一对应的映射后，线程的 `row` 和 `col` 索引也就是其输出元素的行、列索引。图 3.11 展示了基于这种线程到数据映射的内核源代码。读者应当立即看到熟悉的 `row`、`col` 计算模式（第 3–4 行），以及检查两个索引是否都在有效范围内的 `if` 语句（第 5 行）。这些语句与 `colorToGrayscaleConversion` 内核中的对应语句几乎相同。

```

1  __global__ void MatrixMulKernel(float* M, float* N,
2                                float* P, int Width) {
3      int row = blockIdx.y*blockDim.y + threadIdx.y;
4      int col = blockIdx.x*blockDim.x + threadIdx.x;
5      if ((row < Width) && (col < Width)) {
6          float Pvalue = 0;
7          for (int k = 0; k < Width; ++k) {
8              Pvalue += M[row*Width+k]*N[k*Width+col];
9          }
10         P[row*Width+col] = Pvalue;
11     }
12 }
13

```

图 3.11: 每个线程计算一个 P 元素的矩阵乘法内核（依据原书图 3.11 重绘）

唯一重要的区别是，我们作了一个简化假设：`MatrixMulKernel` 只需要处理方阵，因此把 `width` 和 `height` 都替换为单个 `Width`。这种线程到数据的映射实际上把 P 划分成多个瓦片 (tile)，图 3.10 中的大型浅色方格就是其中一个瓦片。每个线程块负责计算其中一个瓦片。

下面关注每个线程执行的工作。回顾一下， $P_{\text{row},\text{col}}$ 是 M 的第 `row` 行与 N 的第 `col` 列的内积。在图 3.11 中，我们使用 `for` 循环执行内积运算。进入循环前，先把局部变量 `Pvalue` 初始化为 0（第 6 行）。循环每次从 M 的第 `row` 行和 N 的第 `col` 列各访问一个元素，把两个元素相乘，并把乘积累加到 `Pvalue`（第 8 行）。

先看循环中如何访问 M 的元素。 M 按行主序线性化为等价的一维数组：从第 0 行开始，各行依次放在内存空间中。因此，第 1 行的起始元素是 `M[1*Width]`，因为需要跳过第 0 行的所有元素。一般而言，第 `row` 行的起始元素为 `M[row*Width]`。由于同一行的所有元素位于连续位置，第 `row` 行第 `k` 个元素位于 `M[row*Width+k]`。这就是图 3.11 第 8 行使用的一维数组偏移量。

再看如何访问 N 。如图 3.11 所示，第 `col` 列的起始元素是第 0 行的第 `col` 个元素，即 `N[col]`。访问该列的下一个元素时，需要跳过一整行，因为同一列的下一个元素位于下一行。因此，第 `col` 列的第 `k` 个元素位于 `N[k*Width+col]`（第 8 行）。

退出 `for` 循环后，所有线程都把其 P 元素的值保存在 `Pvalue` 变量中。随后，每

个线程使用一维等价索引 $\text{row} \times \text{Width} + \text{col}$ 把自己的输出元素写入 P (第 10 行)。再次说明, 这种索引模式与 `colorToGrayscaleConversion` 内核使用的模式相似。

下面用一个小例子说明矩阵乘法内核的执行。图 3.12 展示了 `BLOCK_WIDTH=2` 时的 4×4 矩阵 P 。虽然这样小的矩阵和线程块大小并不现实, 但可以把整个例子放在一幅图中。矩阵 P 被分为四个瓦片, 每个线程块计算一个瓦片。具体做法是创建 2×2 的线程块, 每个线程计算一个 P 元素。在这个例子中, 块 $(0,0)$ 的线程 $(0,0)$ 计算 $P_{0,0}$, 而块 $(1,0)$ 的线程 $(0,0)$ 计算 $P_{2,0}$ 。

$P_{0,0}$	$P_{0,1}$	$P_{0,2}$	$P_{0,3}$
$P_{1,0}$	$P_{1,1}$	$P_{1,2}$	$P_{1,3}$
$P_{2,0}$	$P_{2,1}$	$P_{2,2}$	$P_{2,3}$
$P_{3,0}$	$P_{3,1}$	$P_{3,2}$	$P_{3,3}$

每个 2×2 方格是一个线程块
 块 $(0,0)$ 的线程 $(0,0) \rightarrow P_{0,0}$
 块 $(1,0)$ 的线程 $(0,0) \rightarrow P_{2,0}$

图 3.12: `MatrixMulKernel` 的一个小型执行示例 (依据原书图 3.12 重绘)

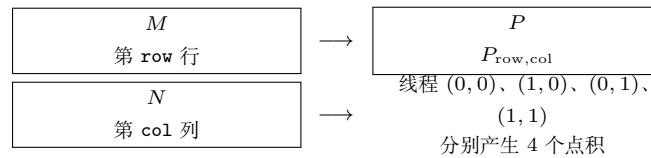


图 3.13: 一个线程块执行矩阵乘法时的动作 (依据原书图 3.13 重绘)

在 `MatrixMulKernel` 中, `row` 和 `col` 索引确定线程要计算的 P 元素。`row` 索引同时确定 M 的行, `col` 索引确定 N 的列, 它们构成该线程的输入。图 3.13 展示了每个线程块中的乘法动作。对于小型矩阵乘法例子, 块 $(0,0)$ 中的线程产生 4 个点积。块 $(0,0)$ 中线程 $(1,0)$ 的行、列索引分别为 $0 \times 2 + 1 = 1$ 和 $0 \times 2 + 0 = 0$, 因此该线程映射到 $P_{1,0}$, 计算 M 第 1 行与 N 第 0 列的点积。**译者注:** 原书此处将两个乘数排印为 0; 由于本例的线程块为 2×2 , 这里按前文的索引公式校正为 2。

现在手动跟踪图 3.11 中块 $(0,0)$ 的线程 $(0,0)$ 执行 `for` 循环的过程。第 0 次迭代 ($k=0$) 中, $\text{row} \times \text{Width} + k = 0 \times 4 + 0 = 0$, $k \times \text{Width} + \text{col} = 0 \times 4 + 0 = 0$ 。因此访问的输入元素是 $M[0]$ 和 $N[0]$, 它们分别是 $M_{0,0}$ 和 $N_{0,0}$ 的一维等价元素; 它们确实是 M 第 0 行和 N 第 0 列中的第 0 个元素。

第 1 次迭代 ($k=1$) 中, $\text{row} \times \text{Width} + k = 0 \times 4 + 1 = 1$, $k \times \text{Width} + \text{col} = 1 \times 4 + 0 = 4$ 。因此访问 $M[1]$ 和 $N[4]$, 它们分别是 $M_{0,1}$ 和 $N_{1,0}$ 的一维等价元素; 它们是 M 第 0 行和 N 第 0 列中的第 1 个元素。

第 2 次迭代 ($k=2$) 中, $\text{row} \times \text{Width} + k = 0 \times 4 + 2 = 2$, $k \times \text{Width} + \text{col} = 2 \times 4 + 0 = 8$, 因此访问 $M[2]$ 和 $N[8]$, 也就是 $M_{0,2}$ 和 $N_{2,0}$ 的一维等价元素。最后, 第 3 次迭代 ($k=3$) 中, $\text{row} \times \text{Width} + k = 0 \times 4 + 3 = 3$, $k \times \text{Width} + \text{col} = 3 \times 4 + 0 = 12$, 访问 $M[3]$ 和 $N[12]$, 也就是 $M_{0,3}$ 和 $N_{3,0}$ 的一维等价元素。

由此可以验证，对于块 (0,0) 中的线程 (0,0)，这个 for 循环确实计算了 M 第 0 行与 N 第 0 列的内积。循环结束后，线程写入 $P[\text{row} \times \text{Width} + \text{col}]$ ，即 $P[0]$ 。这是 $P_{0,0}$ 的一维等价元素，因此该线程成功计算了所需内积，并把结果写入 $P_{0,0}$ 。其他线程或其他线程块中的线程也可以用同样方式手动执行并验证这个循环。

由于网格的大小受每个网格允许的最大线程块数量以及每个线程块允许的最大线程数限制，`MatrixMulKernel` 能处理的最大输出矩阵 P 也受这些约束限制。如果要计算大于这个限制的输出矩阵，可以把输出矩阵分成网格能够覆盖的子矩阵，并让主机代码为每个子矩阵启动一个不同的网格。另一种方法是修改内核代码，让每个线程计算 P 的多个元素。我们将在第 5 章进一步优化矩阵乘法实现，并在第 15 章深入讨论这一重要计算的高级优化。

3.5 小结

CUDA 网格和线程块都是最多包含三个维度的多维结构。网格和线程块的多维性有助于组织线程，使其映射到多维数据。内核执行配置参数定义网格及其线程块的维度。`blockIdx` 和 `threadIdx` 中的唯一坐标让网格中的线程能够识别自己以及自己的数据区域。程序员有责任在内核函数中使用这些变量，使线程正确识别要处理的数据部分。访问多维数据时，程序员通常需要把多维索引线性化为一维偏移。原因是，C++ 中动态分配的多维数组通常以行主序的一维数组形式存储。本章使用复杂度逐步增加的例子，帮助读者熟悉多维网格处理多维数组的机制。这些技能是理解本书后续并行模式及其相关优化技术的基础。

练习

1. 本章实现了一个矩阵乘法内核，每个线程生成一个输出矩阵元素。本题要求实现不同的矩阵-矩阵乘法内核并比较它们。
 - a. 编写一个让每个线程生成一个输出矩阵行的内核，并为该设计填写执行配置参数。
 - b. 编写一个让每个线程生成一个输出矩阵列的内核，并为该设计填写执行配置参数。
 - c. 分析上述两种内核设计各自的优点和缺点。
2. 矩阵-向量乘法接收输入矩阵 B 和向量 $\mathbf{c}$ ，生成输出向量 $\mathbf{a}$ 。输出向量 $\mathbf{a}$ 的每个元素都是输入矩阵 B 的一行与 $\mathbf{c}$ 的点积，即

$$a_i = \sum_j B_{i,j} \times c_j.$$

为简单起见，只处理元素为单精度浮点数的方阵。编写矩阵-向量乘法内核以及可以调用它的主机存根函数。该存根函数接收四个参数：输出向量 **a** 的指针、输入矩阵 **B** 的指针、输入向量 **c** 的指针，以及每个维度的元素数量 N 。使用一个线程计算输出向量的一个元素。

3. 考虑下面的 CUDA 内核以及调用它的主机函数：

练习题 3 的 CUDA 代码：

```
1 __global__ void foo_kernel(float* a, float* b, unsigned int M,
    unsigned int N) {
2     unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
3     unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;
4     if (row < M && col < N) {
5         b[row*N + col] = a[row*N + col]/2.1f + 4.8f;
6     }
7 }
8 void foo(float* a_d, float* b_d) {
9     unsigned int M = 150;
10    unsigned int N = 300;
11    dim3 bd(16, 32);
12    dim3 gd((N - 1)/16 + 1, (M - 1)/32 + 1);
13    foo_kernel<<<gd, bd>>>(a_d, b_d, M, N);
14 }
```

- a. 每个线程块中的线程数是多少？
 - b. 网格中的线程数是多少？
 - c. 网格中的线程块数是多少？
 - d. 执行第 05 行代码的线程数是多少？
4. 考虑一个宽度为 400、高度为 500 的二维矩阵。该矩阵以一维数组形式存储。请分别指定矩阵第 20 行、第 10 列元素的数组索引：
- a. 矩阵按行主序存储时；
 - b. 矩阵按列主序存储时。
5. 考虑一个宽度为 400、高度为 500、深度为 300 的三维张量。该张量按行主序存储为一维数组。请指定 $x = 10$ 、 $y = 20$ 、 $z = 5$ 的张量元素的数组索引。

第四章 计算架构与调度

第一章介绍过，CPU 的设计目标是尽量降低指令执行延迟，而 GPU 的设计目标是尽量提高指令执行吞吐量。第二章和第三章介绍了 CUDA 编程的核心特征：编写和调用内核，启动并执行由大量线程组成的网格。在接下来的三章中，我们将讨论现代 GPU 的架构，包括计算架构、内存架构，以及由理解这些架构而产生的性能优化技术。

本章介绍 CUDA C++ 程序员需要理解、并据此分析内核代码性能行为的若干 GPU 计算架构特征。我们首先给出计算架构的高层次简化视图，并研究灵活的资源分配、线程块调度和占用率 (occupancy) 等概念；随后进一步讨论线程调度、延迟容忍、控制流分歧和同步。最后，我们介绍可用于查询 GPU 可用资源的 API 函数，以及帮助估计 GPU 执行内核时占用率的工具。

接下来的两章将介绍 GPU 内存架构的核心概念和编程注意事项。第五章重点讨论片上内存架构；第六章将简要介绍片外内存架构，然后详细讨论整个 GPU 架构的各种性能考量。掌握这些概念的 CUDA C++ 程序员，将能够编写和理解高性能并行内核。

4.1 现代 GPU 的架构

图 4.1 展示了 CUDA C++ 程序员所看到的、典型 CUDA-capable GPU 架构的高层次视图。GPU 由一组高度多线程化的流式多处理器 (Streaming Multiprocessor, SM) 组成。每个 SM 都有多个称为流式处理器 (streaming processor) 的处理单元¹，这些处理单元共享控制逻辑和内存资源。图 4.1 中，SM 内的小方格表示流式处理器。例如，Hopper H100 GPU 有 132 个 SM，每个 SM 有 128 个流式处理器，因此整个 GPU 共有

$$132 \times 128 = 16,896$$

个流式处理器。从 Hopper 架构开始，GPU 中的 SM 被分组为 GPU 处理集群 (GPU Processing Cluster, GPC)。例如，Hopper H100 的 132 个 SM 分布在 8 个 GPC 中。

SM 的内存资源由不同的片上内存结构组成，在图 4.1 中统称为 **Memory**。第五

¹流式处理器有时也称为 CUDA core。为避免与 CPU core 混淆，本书统一使用“流式处理器”这一术语。

章将讨论这些片上内存结构。GPU 还配有若干 GB 的片外设备内存，在图中称为**全局内存 (Global Memory)**。全局内存通过多个内存控制器或内存通道访问；这些控制器或通道在整体上提供较高的数据访问带宽。

较早的 GPU 使用 GDDR (Graphics Double Data Rate) SDRAM (Synchronous Dynamic Random Access Memory)。从 NVIDIA Pascal 架构开始，GPU 还可能使用 HBM (High-Bandwidth Memory)、HBM2、HBM2E 或 HBM3。这些内存由与 GPU 集成在同一封装中的 DRAM 模块构成。为简洁起见，本书后文将这些内存类型统称为 DRAM。第六章将讨论访问 GPU DRAM 时涉及的最重要概念。

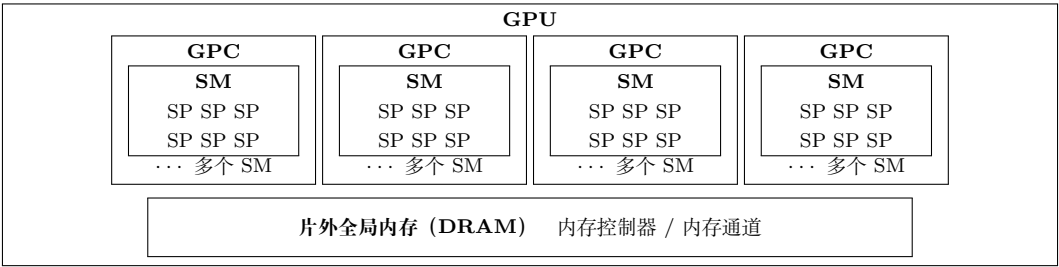


图 4.1: CUDA-capable GPU 的架构（依据原书图 4.1 重绘的概念示意；SP 表示流式处理器）

4.2 线程块调度

调用内核时，CUDA 运行时系统会启动一个执行内核代码的线程网格。虽然 GPU 拥有大量执行资源，但一个内核所启动的线程网格很容易需要远超 GPU 当前可用的执行资源。因此，需要在线程块执行资源变得可用时，将线程分配给这些资源的线程块调度机制。

在 CUDA 中，线程以线程块为单位分配给 SM。也就是说，一个线程块中的所有线程会同时分配到同一个 SM。图 4.2 展示了这种分配方式。同一个 SM 可以同时分配多个线程块，具体数量取决于 SM 的可用资源。例如，图中每个 SM 分配了三个线程块。不过，线程块需要预留执行所需的硬件资源，所以一个 SM 能同时分配的线程块数量是有限的；这一数量取决于多种因素，第 4.7 节将讨论这些因素。

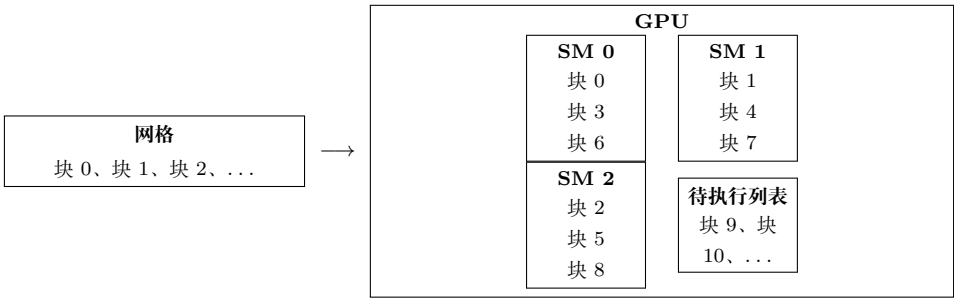


图 4.2: 线程块按块分配给流式多处理器（依据原书图 4.2 重绘）

由于 SM 数量有限，而且每个 SM 能同时分配的线程块数量也有限，因此一个 CUDA 设备能够同时执行的线程块总数存在上限。但是，网格通常包含远多于该上限的线程块。为了确保网格中的所有线程块都得到执行，运行时系统会维护待执行线程块列表，并在已分配线程块执行完毕后，把新的线程块分配给 SM。

以线程块为单位把线程分配给 SM，可以保证同一个线程块中的线程在同一个 SM 上、并且在相近的时间执行。这一保证使同一线程块中的线程能够以不同线程块之间无法实现的方式交互。例如，它们可以进行线程块范围的屏障同步，也可以通过位于 SM 中的低延迟 SRAM (Static Random Access Memory) 共享内存交换数据。第五章将讨论共享内存。

从 Hopper 架构开始，CUDA 引入了一个可选的层次结构：线程块集群 (thread block cluster)，即一组线程块。类似于同一线程块中的线程会被分配到同一个 SM，同一线程块集群中的线程块会共同调度到同一个 GPC。集群中的线程可以通过相应 API 进行同步，并且可以访问由集群中所有线程块共享内存组成的分布式共享内存 (distributed shared memory)。第五章将进一步讨论分布式共享内存。

4.3 同步与透明可扩展性

CUDA 允许同一线程块中的线程使用线程块范围的屏障同步函数协调活动。最常用的函数是 `__syncthreads()`。注意，该函数名中的两个下划线分别由两个连续的 `_` 字符组成；这是 CUDA 中表示内建函数 (intrinsic function) 的约定，见“内建函数”侧栏。

当一个线程调用 `__syncthreads()` 时，它会停留在调用所在的程序位置，直到同一线程块中的每个线程都到达该位置。调用同步函数的线程被称为到达了与该程序位置对应的屏障。屏障确保线程块中的所有线程都完成当前执行阶段，然后任何线程才能进入下一阶段。只有当线程块中的所有线程都到达屏障后，它们才会被释放，继续执行屏障之后的语句。

内建函数

现代处理器通常提供能够执行关键功能或显著提高性能的特殊指令。这些指令通常以内建函数 (intrinsic，简称 intrinsic) 的形式提供给程序员。从程序员的角度看，它们像库函数；但编译器会特殊处理它们，把每次调用转换为相应的特殊指令。最终代码中通常没有普通的函数调用，只有与用户代码内联的特殊指令。GNU Compiler Collection (gcc)、Intel C Compiler 和 Clang/LLVM C Compiler 等主流现代编译器都支持内建函数。

在线程块集群层次，CUDA 通过 Cooperative Groups API [1] 提供集群范围同步机制。线程块集群中的所有线程可以在某个程序位置同步，以确保同一集群中的所有

线程都到达屏障，并且它们对分布式共享内存的操作都已完成，然后再从屏障中释放。参与某个屏障的线程集合称为该屏障的**范围（scope）**。

在网格层次，整个网格中的线程也可以通过 Cooperative Groups API [1] 进行网格范围的屏障同步。不过，网格范围同步的开销高于线程块范围和集群范围同步，并且必须满足若干重要限制，以确保网格中的所有线程确实同时在执行并能够到达屏障。下面主要讨论线程块范围的屏障同步。

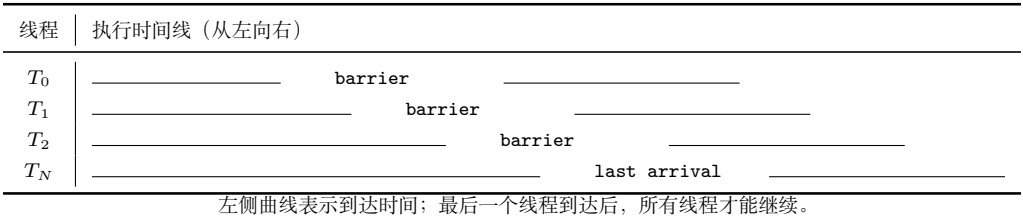


图 4.3: 屏障同步的执行时序示例（依据原书图 4.3 重绘）

屏障同步是协调并行活动的简单而常用的方法。现实生活中也经常需要屏障同步。例如，四位朋友乘车去购物中心。他们可以分别去不同商店购买自己的物品，这比四人始终聚在一起、依次访问所有商店高效得多。但是，在离开购物中心之前需要进行屏障同步：必须等四位朋友都回到车上才能出发。提前完成购物的人必须等待尚未完成的人；如果没有屏障，汽车可能在某人仍在商场时离开。

图 4.3 中有 N 个参与同步的线程，时间从左向右推进。有些线程较早到达屏障，有些线程较晚到达。早到达的线程等待晚到达的线程；当最后一个线程到达屏障时，所有线程都可以被释放并继续执行。屏障同步保证“没有人掉队”。

在 CUDA 中，如果存在 `__syncthreads()` 语句，则线程块中的所有线程都必须执行它。当 `__syncthreads()` 放在 `if` 语句中时，线程块中的所有线程要么都执行包含该调用的路径，要么都不执行。对于 `if-else` 语句，如果两个路径各有一个 `__syncthreads()`，线程块中的所有线程也必须全部执行 `then` 路径，或全部执行 `else` 路径。两个调用是不同的屏障同步点。

图 4.4 展示了一个错误用法：从第 04 行开始的 `if` 语句中有两个屏障同步调用。偶数 `threadIdx.x` 的线程执行 `then` 路径，其余线程执行 `else` 路径。第 06 行和第 10 行的调用定义了两个不同的屏障。由于不能保证线程块中的所有线程会到达其中任一屏障，这段代码违反了屏障同步函数的使用规则，执行行为未定义。错误的屏障同步用法可能导致错误结果，也可能让线程永远相互等待，即死锁。程序员必须避免这种不恰当的用法。

屏障同步会对其范围内的线程施加执行约束。这些线程应当在相近的时间完成，以免等待时间过长。更重要的是，系统必须确保参与屏障的所有线程都能获得最终到达屏障所需的资源，否则永远无法到达的线程可能导致死锁。CUDA 运行时通过以线程块为单位分配执行资源来满足这一约束：线程块中的所有线程不仅必须被分配到同一个 SM，还必须同时被分配到该 SM。只有当运行时为线程块中的所有线程取得所

```
1 __global__ void foo_kernel(...) {  
2     ...  
3     ...  
4     if (threadIdx.x % 2 == 0) {  
5         ...  
6         __syncthreads();  
7     } else {  
8         ...  
9         ...  
10        __syncthreads();  
11    }  
12 }  
13
```

图 4.4: 错误使用 `__syncthreads()` 的示例；第 06 行和第 10 行是不同的屏障（依据原书图 4.4 重绘）

需资源后，线程块才能开始执行。

这说明 CUDA 屏障同步应尽量限制在较小范围内。如果屏障同步只限于线程块范围，CUDA 运行时就可以按照任意相对顺序执行不同线程块。这种灵活性使 CUDA 程序具有可扩展性，如图 4.5 所示。图中时间从上向下推进。在只有少量执行资源的低成本系统中，同时执行的线程块较少，左侧示例每次执行两个线程块；在执行资源更多的高端系统中，右侧示例每次执行四个线程块。如今的高端 GPU 可以同时执行数百乃至数千个线程块。

同时执行的一组线程块通常称为一个**波次 (wave)**。网格中的线程块总数与能够同时执行的线程块数之比，称为波次数。例如，图 4.5 左侧示例有四个波次，右侧示例有两个波次。当线程块之间负载不均衡时，较多波次更理想，因为硬件有更多机会在 SM 之间平衡负载；当线程块负载均衡时，较少波次通常可以接受，有时还更理想。第六章讨论线程粗化时会再次看到这一点。

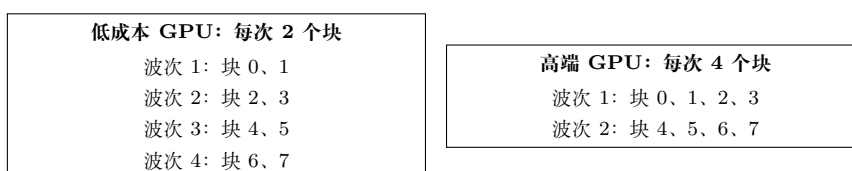


图 4.5: 线程块之间没有同步约束时，CUDA 程序具有透明可扩展性（依据原书图 4.5 重绘）

使用较少波次时，需要避免最后一个波次不完整、只包含少量线程块而无法充分利用硬件的情况。例如，一个有 660 个线程块的网格运行在能够同时执行 264 个线程

块的 GPU 上时, 会产生

$$660/264 = 2.5$$

个波次。如果线程块负载较均衡, 先执行 264 个块, 再执行 264 个块, 最后执行只有 132 个块的部分波次。这个部分波次会使 GPU 资源利用不足, 这种现象有时称为 **尾部效应 (tail effect)**。为避免尾部效应, 最好把网格线程块数量配置为同时可运行线程块数量的整数倍, 从而得到整数个波次; 该数量取决于 GPU 所拥有的硬件资源。

能够在资源数量不同的各种 GPU 上执行同一份应用代码, 使厂商可以按照不同市场对成本、功耗和性能的要求生产多种 GPU。例如, 移动处理器可以用极低功耗较慢地执行应用, 桌面处理器则可以用更高功耗更快地执行同一应用; 两者不需要修改应用代码。这种让同一应用代码在具有不同执行资源的硬件上运行的能力称为 **透明可扩展性 (transparent scalability)**, 它减轻了应用开发者的负担, 也提高了应用的可用性。

4.4 Warp 与 SIMD 硬件

前面看到, 当线程块处于不同的屏障同步范围时, 它们可以按照任意相对顺序执行, 从而允许 CUDA 应用在不同设备之间透明扩展。但是, 还没有讨论同一线程块内部线程的执行时序。概念上, 应假设线程块中的线程也可以按照任意相对顺序执行。在具有多个阶段的算法中, 只要希望屏障范围内的所有线程完成上一阶段后再进入下一阶段, 就应使用屏障同步。内核正确性不应依赖线程在没有屏障同步时恰好同步执行的假设。

CUDA GPU 中的线程调度是硬件实现概念, 必须结合具体硬件讨论。在迄今为止的大多数实现中, 线程块被分配到流式多处理器后, 会进一步划分为每组 32 个线程的单元, 称为 **warp**。warp 的大小由实现决定, 未来 GPU 代际可能发生变化。了解 warp 有助于理解和优化特定 CUDA 设备代际上的应用性能。

warp 是 SM 中的线程调度单位。图 4.6 展示了一个实现中线程块划分为 warp 的方式。示例中, 块 1、块 2 和块 3 都分配给同一个 SM; 每个块又被划分为多个 warp。每个 warp 由连续的 `threadIdx` 值组成: 线程 0–31 构成第一个 warp, 线程 32–63 构成第二个 warp, 以此类推。如果每个线程块有 256 个线程, 则每个块有

$$256/32 = 8$$

个 warp; SM 中有三个这样的块, 因此共有 $8 \times 3 = 24$ 个 warp。

如果线程块是一维数组, 即只使用 `threadIdx.x`, 划分很直接。一个 warp 中的 `threadIdx.x` 连续递增。warp 大小为 32 时, warp 0 从线程 0 到线程 31, warp 1 从线程 32 到线程 63。一般而言, warp n 从线程 $32n$ 开始, 到线程 $32(n+1)-1$ 结束。如果线程块大小不是 32 的倍数, 最后一个 warp 会用非活动线程填充到 32 个位置。

线程块	线程范围	warp 数量
Block 1	0-31, 32-63, ..., 224-255	8
Block 2	0-31, 32-63, ..., 224-255	8
Block 3	0-31, 32-63, ..., 224-255	8
SM 中总计		24

图 4.6: 线程块被划分为 warp 以进行线程调度 (依据原书图 4.6 重绘)

例如, 一个有 48 个线程的块会划分为两个 warp, 第二个 warp 用 16 个非活动线程填充。

对于包含多个线程维度的线程块, 在划分 warp 之前, 线程维度会投影为线性化的行主序布局。线性布局中, 较大的 y、z 坐标排在较小坐标之后。二维线程块的情况是: 先放置 `threadIdx.y=0` 的所有线程, 再放置 `threadIdx.y=1` 的所有线程, 以此类推; 相同 y 值的线程按照递增的 `threadIdx.x` 连续排列。

$T_{0,0}$	$T_{0,1}$	$T_{0,2}$	$T_{0,3}$
$T_{1,0}$	$T_{1,1}$	$T_{1,2}$	$T_{1,3}$
$T_{2,0}$	$T_{2,1}$	$T_{2,2}$	$T_{2,3}$
$T_{3,0}$	$T_{3,1}$	$T_{3,2}$	$T_{3,3}$

→ $T_{0,0}, T_{0,1}, T_{0,2}, T_{0,3}, T_{1,0}, T_{1,1}, \dots, T_{3,3}$ (行主序线性布局)

图 4.7: 将二维线程放入线性布局 (依据原书图 4.7 重绘)

图 4.7 上半部分是二维线程块, 下半部分是线性化视图。前四个线程的 y 坐标为 0, 按 x 坐标递增排列; 接下来四个线程的 y 坐标为 1, 也按 x 坐标递增排列。这里的坐标分别对应 CUDA 字段 `threadIdx.y` 和 `threadIdx.x`。示例中的 16 个线程构成半个 warp, 另用 16 个非活动线程补足一个 32 线程 warp。

如果二维线程块为 8×8 , 64 个线程会构成两个 warp。第一个 warp 从 $T_{0,0}$ 到 $T_{3,7}$, 第二个 warp 从 $T_{4,0}$ 到 $T_{7,7}$ 。读者可以自行画出该图验证这一点。

对于三维线程块, 首先把 `threadIdx.z=0` 的所有线程放入线性顺序, 其中线程按二维线程块的方式排列; 然后放入 `threadIdx.z=1` 的线程, 以此类推。例如, 一个三维 $2 \times 8 \times 4$ 线程块 (x 维为 4、y 维为 8、z 维为 2) 共有 64 个线程, 会划分为两个 warp: 第一个 warp 从 $T_{0,0,0}$ 到 $T_{0,7,3}$, 第二个 warp 从 $T_{1,0,0}$ 到 $T_{1,7,3}$ 。

SM 设计为按照单指令多数据 (Single Instruction, Multiple Data, SIMD) 模型执行 warp 中的所有线程。也就是说, 在任意时刻, 一条指令会被取出并由 warp 中所有线程执行。为此, 一个 SM 被组织为多个处理分区, SM 上的 warp 分布到这些处理分区中。图 4.8 给出了一个玩具示例: SM 由两个处理分区组成, 每个处理分区包含 8 个共享一条指令取出/分发单元的流式处理器。作为真实例子, 拥有 128 个流式处理器的 Hopper H100 SM 被组织为四个处理分区, 每个分区有 32 个流式处理器。

同一 warp 中的线程会被分配到同一个处理分区, 该分区为 warp 取出指令, 并让 warp 中所有线程执行这条指令。由于 CUDA 线程调度机制把 warp 映射到处理分区, 实际上限制了 warp 中所有线程在任意时刻执行相同指令, 因此 warp 的执行行



图 4.8: SM 由处理分区组成，以执行 SIMD 指令（依据原书图 4.8 重绘）

为通常称为单指令多线程（Single Instruction, Multiple Thread, SIMT）。

SIMT 的一个重要优点是，程序员可以用标量线程编写内核代码，之后由 SM 把线程分组为 warp，透明地使用 SIMD 硬件。这不同于传统 CPU SIMD 编程：在 CPU 中，程序员或编译器通常需要使用 API 在每个线程内部显式利用 SIMD 硬件，增加了另一层编程复杂度。²

SIMD 的优势在于，指令取出/分发单元等控制硬件的成本由多个执行单元共享。这样设计可以让较小比例的硬件用于控制，把更大比例的芯片面积用于提高算术吞吐量。在可预见的未来，warp 划分仍会是常见的实现技术。不过，warp 大小可能因实现而异；截至目前，所有 CUDA 设备都使用了类似的 32 线程 warp 配置。

由于同一 warp 中线程的调度方式具有特殊关系，CUDA 提供了一组 warp 级原语，即在 warp 内进行高效数据交换和同步的 API 函数。程序员可以使用它们提高涉及线程协作的计算速度和效率。这些原语支持以 warp 为中心的编程风格，把 warp 作为软件层次结构中（线程块与线程之间）的额外层次。第十、十一和十二章将介绍相关 warp 级原语。

Warp 与 SIMD 硬件

1945 年，John von Neumann 在报告中描述了构建电子计算机的模型，该模型以先驱 EDVAC 计算机的设计为基础。今天通常称为“冯·诺依曼模型”，它几乎是所有现代计算机的基础蓝图。

冯·诺依曼模型中的计算机有输入/输出（I/O），可以向系统提供程序和数据，也可以从系统产生结果。执行程序时，计算机首先把程序及其数据输入内存。程序由一组指令组成；控制单元维护程序计数器（Program Counter, PC），其中保存下一条待执行指令的内存地址。在每个“指令周期”中，控制单元使用 PC 将指令取入指令寄存器（Instruction Register, IR），再检查指令位以确定计算机各组件要执行的动作。这也是该模型被称为“存储程序”模型的原因：用户只需把不同程序存入内存，就能改变计算机行为。

将线程作为 warp 执行的动机，可以用适合 GPU 的修改版冯·诺依曼模型表示。对应图 4.8 中处理分区的处理器只有一个负责取出和分发指令的控制单元；相同的控制信号会发送到多个处理单元，每个处理单元对应 SM

²虽然 SIMT 比 SIMD 更易编程，但程序员仍应避免控制流分歧，因为分歧会对性能产生负面影响，见第 4.5 节。

中的一个流式处理器，并执行 warp 中的一个线程。所有处理单元都由指令寄存器中的同一条指令控制，它们的执行差异来自寄存器文件中不同的数据操作数。这就是处理器设计中的 SIMD。

例如，所有处理单元都可以执行 `add r1, r2, r3`，但不同处理单元中的 `r2` 和 `r3` 内容不同。SIMD 是 Flynn 分类法 [2] 中四类计算机架构之一，另外三类是单指令单数据 (SISD)、多指令单数据 (MISD) 和多指令多数据 (MIMD)。现代处理器的控制单元相当复杂，包括指令取出逻辑和指令缓存访问端口；让多个处理单元共享一个控制单元，可以显著降低硬件制造成本和功耗。

4.5 控制流分歧

当 warp 中所有线程在处理数据时遵循相同的执行路径，更正式地说，遵循相同的控制流时，SIMD 执行效果很好。例如，对于 `if-else` 结构，如果 warp 中所有线程都执行 `then` 路径，或者所有线程都执行 `else` 路径，执行就很高效。

但是，当同一 warp 中的线程采取不同控制流路径时，SIMD 硬件会分别遍历这些路径，每条路径执行一次。例如，部分线程走 `then` 路径、其余线程走 `else` 路径时，硬件会执行两次：一次执行 `then` 路径，另一次执行 `else` 路径。每次遍历其中一条路径时，走另一条路径的线程都不会产生有效作用。

同一 warp 中的线程采取不同控制流路径时，称这些线程发生了**控制流分歧 (control divergence)**。这种多遍执行方式使 SIMD 硬件能够实现 CUDA 线程的完整语义：硬件仍对 warp 中所有线程执行相同指令，但只让选择了当前路径的线程在该遍中生效，从而既保持线程独立性，又利用 SIMD 硬件的低成本。分歧的代价是额外路径遍历，以及每次遍历中非活动线程仍占用的执行资源。

线程	then 遍	else 遍
0-23	执行 A	非活动
24-31	非活动	执行 B

两条路径执行完后，warp 中线程可能重新汇合并执行 C。

图 4.9: warp 在 `if-else` 语句处分歧的示例（依据原书图 4.9 重绘）

图 4.9 中，包含线程 0-31 的 warp 到达 `if-else` 语句时，线程 0-23 走 `then` 路径，线程 24-31 走 `else` 路径。warp 首先执行一遍，其中线程 0-23 执行 A、线程 24-31 非活动；随后再执行一遍，其中线程 24-31 执行 B、线程 0-23 非活动。之后线程可能重新汇合并执行 C。

在 Pascal 及更早架构中，这些遍按顺序执行，即一遍执行完后再执行另一遍。从 Volta 架构开始，不同遍可以并发执行，也就是一条路径的执行可以与另一条路径交

错。这一特性称为独立线程调度 (independent thread scheduling)。因此，不能假定 warp 中线程在分歧路径执行后一定会重新汇合。如果 warp 中所有线程必须完成某个阶段后才能继续，应使用 `__syncwarp()` 等 warp 级屏障同步原语保证正确性。

分歧也可能出现在其他控制流结构中。图 4.10 展示了分歧 for 循环：每个线程执行不同数量的迭代，次数从 4 到 8 不等。前四次迭代中所有线程都活动并执行 A；第五次迭代中，部分线程执行 A，另一些线程（例如线程 4）已经完成迭代而处于非活动状态；第八次迭代中，图示线程中只有线程 1 和线程 7 执行 A，其他线程均非活动。

迭代	1	2	3	4	5	6	7	8
线程 1	A	A	A	A	A	A	A	A
线程 4	A	A	A	A	—	—	—	—
线程 7	A	A	A	A	A	A	A	A

A 表示执行，— 表示非活动；不同线程的循环次数不同。

图 4.10: warp 在 for 循环处分歧的示例（依据原书图 4.10 重绘）

检查控制结构的判定条件，可以判断它是否可能造成线程分歧。如果判定条件基于 `threadIdx.x` 值，控制语句就可能导致分歧。例如，条件 `threadIdx.x > 2` 会让块中第一个 warp 的线程走两条不同路径：线程 0、1、2 走一条路径，线程 3、4、5 等走另一条路径。类似地，如果循环条件依赖线程索引，循环也可能造成分歧。

使用带线程控制分歧的控制结构，一个常见原因是处理线程映射到数据时的边界条件。这是因为线程总数通常必须是线程块大小的倍数，而数据大小可以是任意数值。从第二章的向量加法内核开始，我们就使用了 `if (i < n)`，因为并非所有向量长度都是线程块大小的倍数。

例如，假设向量长度为 1003，线程块大小为 64。需要启动 16 个线程块处理全部 1003 个元素，但 16 个线程块共有 1024 个线程，因此必须禁止第 15 个线程块中最后 21 个线程执行原程序不允许的工作。这 16 个线程块被划分为 32 个 warp，只有最后一个线程块中的第二个 warp 会产生控制流分歧。

随着处理向量长度增大，边界条件造成的分歧对性能的影响会降低。长度为 100 时，四个 warp 中有一个分歧，影响可能很明显；长度为 1000 时，32 个 warp 中只有一个分歧，分歧只影响约 3% 的执行时间；即使使该 warp 执行时间加倍，对总时间的影响也约为 3%。长度达到 10,000 或更大时，只有 313 个 warp 中的一个分歧，影响会远低于 1%。

二维数据（例如第三章的彩色转灰度示例）也需要用 `if` 处理数据边缘线程的边界条件。对 62×76 图片，使用 $4 \times 5 = 20$ 个 16×16 线程块，每个块划分成 8 个 warp，每个 warp 包含两行线程，总计 160 个 warp。参照第三章图 3.5：

- 区域 1 的 12 个块中没有 warp 分歧，共 $12 \times 8 = 96$ 个 warp；

- 区域 2 的 24 个 warp 全部产生分歧；
- 区域 3 中，底部 warp 完全映射到图像外，因此不会通过 if，不产生分歧；
- 区域 4 中，前 7 个 warp 产生分歧，最后一个 warp 不产生分歧。

总计 160 个 warp 中有 31 个产生控制流分歧。如果处理 200×150 的图片，使用 16×16 线程块，则共有 $13 \times 10 = 130$ 个线程块，即 1040 个 warp。区域 1-4 的 warp 数分别为

$$864 = 12 \times 9 \times 8, \quad 72 = 9 \times 8, \quad 96 = 12 \times 8, \quad 8 = 1 \times 8.$$

其中只有 80 个 warp 产生分歧，性能影响小于 8%。如果处理水平尺寸超过 1000 像素的真实图片，影响会低于 2%。

4.6 Warp 调度与延迟容忍

线程被分配到 SM 时，通常会有比 SM 中流式处理器更多的线程分配给它。也就是说，每个 SM 的执行单元只能在任意时刻执行已分配线程的一部分。在较早的 GPU 设计中，一个 SM 在任意时刻只能为一个 warp 执行一条指令；较新的设计中，一个 SM 可以同时为少量 warp 执行指令，因为 SM 包含多个处理分区。无论哪种情况，硬件在任意时刻都只能为分配给 SM 的 warp 子集执行指令。

为什么要给 SM 分配这么多 warp？答案是为了容忍全局内存访问等长延迟操作。当 warp 要执行的指令需要等待先前发起的长延迟操作结果时，该 warp 不会被选中执行；硬件会选择另一个不再等待结果的驻留 warp。没有等待前序指令结果的 warp 已经 **准备好执行**。若有多个 warp 准备好执行，优先级逻辑会选择其中一个。

SM 在每个时刻选择不同准备好执行的 warp 的能力称为**细粒度多线程 (fine-grained multithreading)**。它让 SM 可以利用某些线程的指令等待时间，执行其他线程的指令。因此，即使指令延迟很长，warp 的细粒度多线程执行仍能保持执行单元高利用率。这通常称为**延迟容忍 (latency tolerance)** 或 **延迟隐藏 (latency hiding)**。

延迟容忍

日常生活中经常需要容忍延迟。例如，在邮局寄包裹的人理想情况下应在到达服务柜台前填好所有表格和标签。但现实中，有些人会等到柜员告诉他们填哪张表、如何填写。柜台前排着长队时，应最大化柜员的工作效率。让一个人在柜台前填写表格、让所有其他人等待，并不是好办法；柜员应在此人填写表格时服务下一位顾客。其他顾客已经“准备好”，不应被需要更多时间填写表格的人阻塞。

因此，好的柜员会礼貌地请第一位顾客先到一旁填写表格，同时服务其他顾客。通常，第一位顾客填完表格、柜员也服务完当前顾客后，就会立即得到服务，而不必重新排到队尾。可以把这些顾客看作 warp，把柜员看作硬件执行单元；需要填写表格的顾客对应于其后续执行依赖长延迟操作的 warp。

warp 调度也可以容忍流水线浮点运算、条件分支等其他类型的操作延迟。只要有足够多的 warp，硬件就很可能在任意时刻找到一个可执行 warp，使执行硬件保持充分利用，同时让其他 warp 等待长延迟操作的结果。选择准备好的 warp 不会给执行时间线引入空闲或浪费时间，这称为**零开销线程调度 (zero-overhead thread scheduling)**。

线程、上下文切换与零开销调度

根据冯·诺依曼模型，现代计算机中的线程是一个程序及其在冯·诺依曼处理器上的执行状态。线程包括程序代码、当前执行的指令，以及变量和数据结构的值。程序代码保存在内存中，PC 跟踪正在执行的程序指令地址，IR 保存当前指令，寄存器和内存保存变量与数据结构的值。

现代处理器支持上下文切换，让多个线程轮流推进。通过保存和恢复 PC、寄存器及内存中的状态，可以暂停一个线程并在稍后正确恢复。然而，保存和恢复寄存器内容会产生显著的执行时间开销。

零开销调度指 GPU 能够让等待长延迟指令结果的 warp 休眠，同时激活已经准备好的 warp，而不在处理单元中引入额外空闲周期。传统 CPU 的上下文切换会产生这种空闲周期，因为切换线程需要把离开线程的寄存器内容保存到内存，再从内存加载进入线程的执行状态。GPU SM 通过把已分配 warp 的所有执行状态保存在硬件寄存器中实现零开销调度，因此在 warp 之间切换时不需要保存和恢复状态。

warp 调度会把 warp 指令的长等待时间“隐藏”起来：硬件在等待期间执行其他 warp 的指令。容忍长操作延迟是 GPU 不像 CPU 那样把大量芯片面积用于缓存和分支预测机制的主要原因。GPU 因而可以把更多芯片面积用于浮点执行单元和内存访问通道。

要使延迟容忍有效，最好给 SM 分配远多于其执行单元同时支持数量的线程。例如，Hopper H100 GPU 的每个 SM 有 128 个流式处理器，但同时最多可以分配 2048 个线程。因此，一个 SM 可以拥有多达

$$2048/128 = 16$$

倍于当前一个时钟周期可执行数量的线程。这种线程相对于 SM 的过量配置 (over-subscription) 对于延迟容忍非常重要，因为当前 warp 遇到长延迟操作时，它提高了找到另一个 warp 执行的概率。

4.7 资源划分与占用率

为了容忍较高的操作延迟，给 SM 分配很多 warp 是有利的；但 SM 不一定总能分配它所支持的最大 warp 数。分配给 SM 的 warp 数与 SM 支持的最大 warp 数之比，称为**占用率 (occupancy)**。

除了流式处理器，SM 的执行资源还包括寄存器、共享内存（第五章讨论）、线程块槽位和线程槽位。这些资源会在线程之间动态划分。例如，Hopper H100 GPU 每个 SM 最多支持 32 个线程块、64 个 warp（即 2048 个线程），每个线程块最多 1024 个线程。如果线程块大小为 1024，SM 的 2048 个线程槽位会分给 2 个线程块；如果线程块大小分别为 512、256、128 或 64，则可分给 4、8、16 或 32 个线程块。

线程块大小	每个 SM 可同时容纳的块数	线程总数
1024	2	2048
512	4	2048
256	8	2048
128	16	2048
64	32	2048

线程槽位可以在不同线程块之间动态划分，使 SM 既能执行许多小线程块，也能执行少量大线程块。这与固定划分形成对比：固定划分会在线程块需求小于固定资源时浪费槽位，也无法支持需要更多槽位的线程块。

动态资源划分可能使不同资源限制之间产生微妙交互，造成资源利用不足。以 H100 为例，线程块大小从 1024 变化到 64 时，每个 SM 的线程块数从 2 变化到 32，这些情况下分配给 SM 的线程总数都是 2048，占用率最大。但如果每个线程块只有 32 个线程，填满 2048 个线程槽位需要 64 个块；每个 H100 SM 只有 32 个块槽位，最多只能容纳 32 个块，即

$$32 \times 32 = 1024$$

个线程。此时占用率为

$$1024/2048 = 50\%.$$

因此，要充分利用线程槽位并达到最大占用率，每个线程块至少需要 64 个线程。

另一种降低占用率的情况是线程槽位数不能被线程块大小整除。H100 每个 SM 最多支持 2048 个线程；如果线程块大小为 768，SM 只能容纳 2 个线程块，即 1536 个线程，剩余 512 个线程槽位未使用。此时占用率为

$$1536/2048 = 75\%.$$

前面的讨论没有考虑寄存器和共享内存等资源约束。CUDA 内核中的自动变量会放入寄存器；有些内核使用很多自动变量，有些只使用少量变量。因此，有些内核

每线程需要很多寄存器，有些需要很少寄存器。SM 动态划分寄存器后，可以在每线程寄存器需求较少时容纳更多线程块，在需求较多时容纳更少线程块。

H100 GPU 每个 SM 最多有 65,536 个寄存器。若要达到满占用率，2048 个线程需要的寄存器数意味着每线程不能超过

$$65,536/2,048 = 32$$

个寄存器。如果内核每线程使用 64 个寄存器，65,536 个寄存器最多支持 1024 个线程，因此无论线程块大小如何设置，占用率最多为 50%。编译器有时可以通过寄存器溢出减少每线程寄存器需求并提高占用率，但线程从内存访问溢出寄存器值通常会增加执行时间，甚至使整个网格执行时间变长。

假设一个内核每线程使用 31 个寄存器，并配置为每块 512 个线程。SM 同时运行

$$2048/512 = 4$$

个块，共使用

$$2048 \times 31 = 63,488$$

个寄存器，低于 65,536 的上限。若再声明两个自动变量，使每线程使用 33 个寄存器，则 2048 个线程需要 67,584 个寄存器，超过上限。CUDA 运行时可能只给每个 SM 分配 3 个块，使寄存器需求降为

$$3 \times 512 \times 33 = 50,688.$$

这样，SM 上的线程数从 2048 降为 1536，占用率从 100% 降到 75%。这种资源使用略微增加却造成并行度和性能显著下降的现象，有时称为**性能悬崖 (performance cliff)** [4]。

所有动态划分资源之间会相互作用，因此准确确定每个 SM 上运行的线程数可能很难。Occupancy Calculator [5] 可以根据特定设备和内核的资源使用情况计算每个 SM 上实际运行的线程数；它是 NVIDIA Nsight Compute profiler [6] 的一部分。主机代码也可以调用 CUDA Occupancy API。一个常用的函数是

```
cudaOccupancyMaxActiveBlocksPerMultiprocessor()
```

它可以在给定 GPU 和网格配置时确定内核占用率。

4.8 查询设备属性

前面对 SM 资源划分的讨论引出一个重要问题：如何知道某个设备可用的资源数量？CUDA 应用运行时，如何知道设备中的 SM 数量，以及每个 SM 可以分配的线程块和线程数量？应用通常需要在多种硬件系统上运行，因此需要查询底层硬件可用的资源和能力，以便利用更强的系统，同时适配资源较少的系统。

资源与能力查询

日常生活中，我们也会查询环境的资源和能力。例如预订酒店时，可以查看房间配套设施。若房间配有吹风机，就不必自己携带；若没有游泳池但有健身房，就可以准备跑鞋和运动服。不同地区的酒店提供的牙膏、牙刷、洗发水、护发素、微波炉、冰箱和泳池也可能不同。酒店设施就是酒店的属性，或者说资源与能力。经验丰富的旅行者会在酒店网站上查询这些属性，选择更符合需求的酒店，并更有效地准备行李。

每个 CUDA 设备 SM 的资源数量由设备的计算能力 (compute capability) 指定。一般而言，计算能力级别越高，每个 SM 可用的资源越多，而且 GPU 的计算能力通常会随代际增加。A100 GPU 的计算能力为 8.0，H100 GPU 的计算能力为 9.0。

CUDA C++ 提供了让主机代码查询系统中设备属性的内置机制。CUDA 运行时系统（设备驱动）提供 `cudaGetDeviceCount` API，返回系统中 CUDA-capable 设备的数量。主机代码可以使用：

```
1 int devCount;
2 cudaGetDeviceCount(&devCount);
```

现代 PC 往往有两个或更多 CUDA-capable 设备，因为许多 PC 带有一个或多个集成 GPU。集成 GPU 是默认图形单元，提供现代窗口用户界面所需的基本图形能力和硬件资源，但大多数 CUDA 应用在这些设备上的性能并不理想。因此，主机代码可以遍历所有可用设备，查询其资源和能力，并选择拥有足够资源、能够提供满意性能的设备。

CUDA 运行时把系统中的设备编号为 0 到 `devCount-1`。函数

`cudaGetDeviceProperties`

根据设备编号返回设备属性。例如，主机代码可以遍历设备：

```
1 cudaDeviceProp devProp;
2 for (unsigned int i = 0; i < devCount; i++) {
3     cudaGetDeviceProperties(&devProp, i);
4     // Decide if device has sufficient resources/capabilities
5 }
```

`cudaDeviceProp` 是一个 C 结构体类型，其字段表示 CUDA 设备属性。CUDA C++ Programming Guide 列出了该类型的全部字段。调用完成后，属性会写入变量 `devProp`。

`devProp.maxThreadsPerBlock` 表示查询设备允许的每个线程块最大线程数。有些设备允许每块最多 1024 个线程，有些设备允许更少；未来设备也可能允许更多。因

此，程序最好查询可用设备，判断它们是否允许应用所需的线程块大小。使用线程块集群的 CUDA 程序还应查询最大集群大小。可以调用

```
cudaOccupancyMaxPotentialClusterSize()
```

获取。

设备中的 SM 总数由 `devProp.multiProcessorCount` 给出。如果应用需要大量 SM 才能获得满意性能，应检查该属性。设备时钟频率保存在 `devProp.clockRate` 中。时钟频率与 SM 数量的组合，可以较好地表示设备的最大硬件执行吞吐量。

主机代码可以通过以下字段查询线程块各维度允许的最大线程数：

```
devProp.maxThreadsDim[0]    x 维,
devProp.maxThreadsDim[1]    y 维,
devProp.maxThreadsDim[2]    z 维.
```

例如，自动调优系统可以使用这些信息设置线程块维度的搜索范围。类似地，以下三个字段

```
devProp.maxGridSize[0], devProp.maxGridSize[1],
devProp.maxGridSize[2]
```

分别给出网格 *x*、*y*、*z* 维允许的最大线程块数。它们可以用于判断一个网格是否有足够线程覆盖整个数据集，或者是否需要采用迭代方法。

`devProp.regsPerBlock` 给出单个线程块可使用的寄存器上限，可用于判断特定设备上的内核是否能达到最大占用率，或者是否受寄存器使用限制。这个字段名称略有误导：对于多数计算能力级别，一个线程块可使用的最大寄存器数确实等于 SM 中的寄存器总数；但对于某些计算能力级别，一个线程块的寄存器上限小于 SM 的总量。

前文还讨论过 warp 大小取决于硬件。warp 大小可以通过 `devProp.warpSize` 获取。`cudaDeviceProp` 还有许多其他字段；随着本书介绍相应概念和特征，我们会在后文讨论它们。

4.9 小结

CUDA-capable GPU 由流式多处理器 (SM) 组成，每个 SM 包含多个由流式处理器构成的处理分区，并共享控制逻辑和内存资源。启动网格时，网格中的线程块以任意顺序分配给 SM，从而使 CUDA 应用具有透明可扩展性。透明可扩展性带来一个约束：不同线程块中的线程不应相互同步。从 Hopper 架构开始，SM 被分组为 GPC，线程块还可选择分组为线程块集群。同一集群中的线程块在同一 GPC 上执行，可以相互同步并访问彼此的分布式共享内存。

线程以线程块为单位分配给 SM。线程块分配到 SM 后，进一步划分为 warp。warp 中的线程遵循单指令多数据（SIMD）模型执行。如果同一 warp 中的线程采取不同执行路径，处理分区会分遍执行这些路径，每个线程只在与自己所选路径对应的遍中活动。

分配给 SM 的线程数可能远多于 SM 能同时执行的线程数。在任意时刻，SM 只执行驻留 warp 子集中的指令，这让其他 warp 可以等待长延迟操作，而不影响大量执行单元的总吞吐量。分配给 SM 的线程数与 SM 支持的最大线程数之比称为占用率。更高的占用率通常有助于隐藏长操作延迟并获得较高执行吞吐量。

不同 CUDA-capable 设备对每个 SM 可用资源的限制可能不同。例如，设备会限制每个 SM 可容纳的线程块数、线程数、寄存器数和其他资源数量。对于每个内核，一个或多个资源限制可能成为占用率的决定因素。CUDA C++ 提供占用率计算器和 API，帮助开发者为不同 GPU 选择合适的内核与启动配置；它也支持程序员在运行时查询 GPU 可用资源。

练习

1. 考虑下面的 CUDA 内核及调用它的主机函数：

```
1 __global__ void foo_kernel(int* a, int* b) {
2     unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
3     if (threadIdx.x < 40 || threadIdx.x >= 104) {
4         b[i] = a[i] + 1;
5     }
6     if (i % 2 == 0) {
7         a[i] = b[i] * 2;
8     }
9     for (unsigned int j = 0; j < 5 - (i % 3); ++j) {
10        b[i] += j;
11    }
12 }
13 void foo(int* a_d, int* b_d) {
14     unsigned int N = 1024;
15     foo_kernel<<<(N + 128 - 1)/128, 128>>>(a_d, b_d);
16 }
```

- a. 每个线程块中有多少个 warp?
- b. 网格中有多少个 warp?
- c. 对第 04 行语句：
 - i. 网格中有多少个活动 warp?

- ii. 网格中有多少个分歧 warp?
 - iii. 块 0 的 warp 0 的 SIMD 效率 (百分比) 是多少?
 - iv. 块 0 的 warp 1 的 SIMD 效率 (百分比) 是多少?
 - v. 块 0 的 warp 3 的 SIMD 效率 (百分比) 是多少?
- d. 对第 07 行语句:
- i. 网格中有多少个活动 warp?
 - ii. 网格中有多少个分歧 warp?
 - iii. 块 0 的 warp 0 的 SIMD 效率 (百分比) 是多少?
- e. 对第 09 行循环:
- i. 有多少次迭代没有分歧?
 - ii. 有多少次迭代产生分歧?
2. 对向量加法, 假设向量长度为 2000, 每个线程计算一个输出元素, 线程块大小为 512。网格中有多少个线程?
3. 对上一题, 考虑长度为 2000 的边界检查, 预计有多少个 warp 会产生分歧?
4. 假设一个有 8 个线程的线程块在到达屏障前执行一段代码。8 个线程执行这段代码所需时间 (微秒) 分别为 2.0、2.3、3.0、2.8、2.4、1.9、2.6 和 2.9, 其余时间都在等待屏障。线程总执行时间中有百分之多少用于等待屏障?
5. CUDA 程序员认为, 如果每个线程块只启动 32 个线程, 那么在需要屏障同步的地方可以省略 `__syncthreads()`。你认为这是好主意吗? 请解释原因。
6. 如果 CUDA 设备的 SM (流式多处理器) 最多容纳 1536 个线程和 4 个线程块, 下列哪一种线程块配置会使 SM 中的线程数最多?
- a. 每块 128 个线程;
 - b. 每块 256 个线程;
 - c. 每块 512 个线程;
 - d. 每块 1024 个线程。
7. 假设设备每个 SM 最多允许 64 个线程块和 2048 个线程。指出下列每种每个 SM 的分配是否可行; 若可行, 给出占用率:
- a. 8 个块, 每块 128 个线程;
 - b. 16 个块, 每块 64 个线程;
 - c. 32 个块, 每块 32 个线程;

- d. 64 个块，每块 32 个线程；
 - e. 32 个块，每块 64 个线程。
8. 某 GPU 的硬件限制为：每个 SM 2048 个线程、32 个线程块和 64 K (65,536) 个寄存器。对下列内核特征，判断内核能否达到满占用率；如果不能，指出限制因素：
- a. 每块 128 个线程，每线程 30 个寄存器；
 - b. 每块 32 个线程，每线程 29 个寄存器；
 - c. 每块 256 个线程，每线程 34 个寄存器。
9. 一名学生说，他用每块 32×32 个线程的矩阵乘法内核成功计算了两个 1024×1024 矩阵的乘法。该学生使用的 CUDA 设备每块最多允许 512 个线程、每个 SM 最多允许 8 个线程块；并且每个线程计算结果矩阵的一个元素。你对此有什么反应？为什么？

参考文献

- [1] NVIDIA, *CUDA C++ Programming Guide*, Release 12.3, 2023. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [2] M. Flynn, "Very high-speed computing systems," *Proceedings of the IEEE*, 54(12), 1966, pp. 1901–1909.
- [3] NVIDIA, *NVIDIA Tesla V100 GPU Architecture*, Version WP-08608-001_v1.1, 2017.
- [4] S. Ryoo, C. I. Rodrigues, S. S. Stone, S. S. Bagsorkhi, S.-Z. Ueng, J. A. Stratton, W.-m. W. Hwu, "Program optimization space pruning for a multithreaded GPU," in *Proceedings of the 6th Annual IEEE/ACM International Symposium on Code Generation and Optimization*, 2008, pp. 195–204.
- [5] NVIDIA, *Nsight Compute: Occupancy Calculator*, 2023. <https://docs.nvidia.com/nsight-compute/NsightCompute/index.html#occupancy-calculator>.
- [6] NVIDIA, *Nsight Compute*, 2023. <https://docs.nvidia.com/nsight-compute/index.html>.

第五章 内存架构与数据局部性

到目前为止，我们已经学习了如何编写 CUDA 内核函数，以及如何配置和协调由大量线程组成的内核执行。第四章还介绍了 GPU 硬件的计算架构，以及线程如何被调度执行。本章重点讨论 GPU 的片上内存架构，并开始研究如何组织和放置数据，使大量线程能够高效访问它们。

到目前为止学到的 CUDA 内核，很可能只达到底层硬件潜在速度的一小部分。这是因为通常由片外 DRAM 实现的全局内存具有很长的访问延迟（数百个时钟周期）和有限的访问带宽。虽然大量可执行线程在理论上能够容忍较长的内存访问延迟，但全局内存访问路径上的流量拥塞很容易导致只有极少数线程取得进展，从而使流式多处理器（SM）中的一部分核心处于空闲状态。为避免这种拥塞，GPU 提供了多种额外的片上内存资源，用来访问数据，并消除往返全局内存的大部分冗余流量。本章将研究如何使用不同内存类型提高 CUDA 内核的执行性能。

5.1 内存带宽作为性能限制因素

GPU 的硬件资源是有限的，因此在给定时间内能够完成的工作量也有限。最常被引用的两种上限是峰值计算吞吐量和峰值内存带宽。峰值计算吞吐量规定硬件单位时间能够执行的算术运算数量。例如，Hopper H100 GPU 的峰值计算吞吐量为 66.9 万亿次浮点运算每秒（TFLOPS）。对于 32 位整数、64 位整数、双精度浮点数等不同数据类型，硬件通常有不同的上限。本章主要关注单精度浮点运算。

峰值内存带宽规定硬件单位时间能够从某种内存结构访问的数据字节数。例如，Hopper H100 GPU 的峰值全局内存带宽为 3.35 TB/s。不同内存结构通常具有不同的带宽上限；本章主要关注全局内存带宽。

不同内核的性能可能受不同硬件资源限制。如果内核性能受硬件计算吞吐量限制，则称其为**计算受限 (compute-bound)**。这类内核的大部分时间都在执行单元中忙于执行指令，性能取决于核心执行这些指令的速度。如果内核性能受硬件内存带宽限制，则称其为**内存受限 (memory-bound)**。这类内核的大部分时间都在使用内存通道，计算核心经常处于等待数据到达的空闲状态，性能取决于内存向核心提供数据的速度。

要判断一个内核是计算受限还是内存受限，可以考察它从全局内存访问的每字节数据所执行的浮点运算数量，即 FLOP/B。我们把这个比值称为 **计算量与全局内存访问量之比 (compute-to-global-memory-access ratio)**。文献中也常称其为算术强度 (arithmetic intensity) 或计算强度 (computational intensity)。计算受限内核通常具有较高比值：相对于访问的内存量，执行了很多算术运算；内存受限内核的比值通常较低。

区分两类内核的阈值可以由峰值计算吞吐量除以峰值内存带宽得到。对 H100 GPU,

$$\frac{66.9 \text{ TFLOPS}}{3.35 \text{ TB/s}} = 20.0 \text{ FLOP/B.}$$

因此，在 H100 上，计算量与全局内存访问量之比高于 20.0 FLOP/B 的内核很可能是计算受限的，低于该比值的内核很可能是内存受限的。

峰值计算吞吐量和峰值内存带宽是硬件上限，而不是性能保证。计算受限内核不一定能达到 66.9 TFLOPS，内存受限内核也不一定能达到 3.35 TB/s。只有高效利用硬件资源、经过充分优化的内核，才能接近这些上限。将实际内核性能与硬件上限比较，可以评估内核的优化效率。Roofline 模型提供了一种分析方法，用于判断内核是计算受限还是内存受限，并评估其相对于硬件峰值的性能。

Roofline 模型

Roofline 模型是一种可视化模型，用于评估应用相对于运行硬件限制所达到的性能。基本模型如图 5.1 所示。横轴是以 FLOP/B 表示的算术强度或计算强度，反映应用每加载一个字节数据所完成的工作量；纵轴是以 GFLOPS 表示的计算吞吐量。图中的两条线表示硬件限制：水平线由硬件能够维持的峰值计算吞吐量决定；从原点出发、具有正斜率的直线由硬件能够维持的峰值内存带宽决定。两条线的交点表示应用从内存受限转为计算受限的计算强度阈值。

计算强度较低的应用属于内存受限，受内存带宽限制，不能达到峰值计算吞吐量。在这个区域中，提高计算强度会提高潜在计算吞吐量，正如内存带宽限制线的斜率所表示的那样。当计算强度超过阈值后，应用变为计算受限，不再受峰值内存带宽限制，而是受峰值计算吞吐量限制。

图中的一个点表示一个应用：横坐标是其计算强度，纵坐标是其达到的计算吞吐量。这些点必然位于两条限制线下方，因为应用不可能超过硬件峰值。点相对于两条线的位置反映应用效率：靠近限制线的点说明内存带宽或计算单元利用较好，远低于限制线的点说明资源使用效率较低。

例如，点 A1 和 A2 都表示内存受限应用，A3 表示计算受限应用。A1 高效使用资源，接近峰值内存带宽；A2 没有做到这一点，还有机会通过提

高内存带宽利用率来改善吞吐量。对 A1 而言，改善吞吐量的唯一方法是提高应用的计算强度。

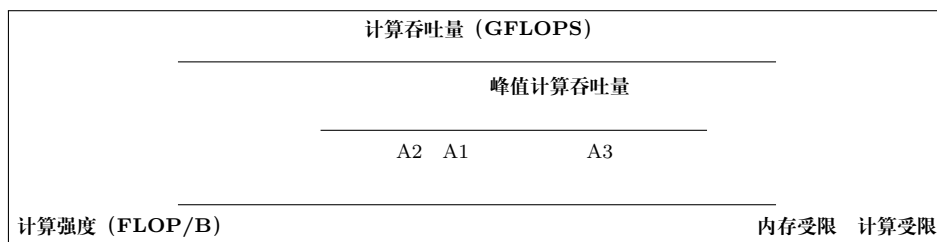


图 5.1: Roofline 模型的概念示意：两条硬件限制线及应用点（依据原书 Roofline 模型侧栏重绘）

下面以第二章图 2.10 中的向量加法内核为例。每个线程的主要计算可以写成：

$$C[i] = A[i] + B[i].$$

每个线程执行一次浮点加法，并从全局内存访问 12 B 数据：读取 $A[i]$ 需要 4 B，读取 $B[i]$ 需要 4 B，写入 $C[i]$ 需要 4 B。因此，内核的计算量与全局内存访问量之比为

$$\frac{1 \text{ FLOP}}{12 \text{ B}} = 0.083 \text{ FLOP/B},$$

远低于 H100 的 20.0 FLOP/B 阈值。如此低的比值说明，在 H100 上执行的向量加法内核深处于内存受限区域。

假设内核把两个各含 10 亿个元素的向量相加。它将访问 12 GB 数据：读取第一个向量 4 GB，读取第二个向量 4 GB，写入结果向量 4 GB。如果内核执行时间为 4 ms，则数据平均访问速率为

$$\frac{12 \text{ GB}}{4 \text{ ms}} = 3 \text{ TB/s}.$$

与硬件峰值内存带宽相比：

$$\frac{3 \text{ TB/s}}{3.35 \text{ TB/s}} = 90\%.$$

这表示内核利用了 90% 的硬件内存带宽资源。这样的分析称为 **光速分析 (speed-of-light analysis)**：可以说内核以硬件限制的“光速”的 90% 执行，只有约 10% 的性能优化空间。对 H100 而言，完成该向量加法的最快可能时间约为

$$\frac{12 \text{ GB}}{3.35 \text{ TB/s}} = 3.6 \text{ ms}.$$

再看矩阵乘法。假设两个 $N \times N$ 矩阵相乘，得到一个 $N \times N$ 结果矩阵。每个输出元素需要对两个长度为 N 的向量求点积，包含 N 次乘法和 N 次加法。因此，整个矩阵乘法执行

$$N^2(N + N) = 2N^3 \text{ FLOP}.$$

它需要读取两个各含 N^2 个元素的矩阵，并写入一个同样大小的矩阵；若每个元素占 4 B，访问数据总量为 $12N^2$ B。假设理想实现每个数据元素只访问一次，则计算量与全局内存访问量之比为

$$\frac{2N^3}{12N^2} = 0.167N \text{ FLOP/B.}$$

当 $N = 1024$ 时，比值为 167 FLOP/B，显著高于 H100 的 20.0 FLOP/B 阈值，因此矩阵乘法具有成为高度计算受限计算的潜力。

译者注：底本此处同时出现 25.1 FLOP/B 和 20.0 FLOP/B 两个阈值；本译稿按同一节前文由 66.9 TFLOPS 和 3.35 TB/s 得出的 20.0 FLOP/B 保留，并将底本的数值差异显式标出。

现在考察第三章图 3.11 中的具体矩阵乘法内核。内核执行时间中最主要的部分是对 M 的一行和 N 的一列求点积的 for 循环：

```
1 for (int k = 0; k < Width; ++k) {
2     Pvalue += M[row*Width+k] * N[k*Width+col];
3 }
```

循环每次执行两次全局内存访问、一次浮点乘法 and 一次浮点加法。两次内存访问读取 M 、 N 中的元素；乘法将两个元素相乘，加法把乘积累加到 Pvalue。因此，计算量与全局内存访问量之比为

$$\frac{2 \text{ FLOP}}{8 \text{ B}} = 0.25 \text{ FLOP/B.}$$

由于线程退出循环后还需要把 Pvalue 写回全局内存，实际比值略低于此值。每个输入元素会被多次访问，所以该比值也低于理想实现的 $0.167N$ ，并低于 H100 的 20.0 FLOP/B 阈值。因此，第三章的这个具体实现是内存受限而非计算受限。

由于该矩阵乘法内核受内存带宽限制，其单精度浮点运算吞吐量上限为

$$(3.35 \text{ TB/s})(0.25 \text{ FLOP/B}) = 0.84 \text{ TFLOPS.}$$

这只有 H100 峰值单精度吞吐量 66.9 TFLOPS 的约 1%。H100 还配有可加速矩阵乘法的专用张量核心，其峰值吞吐量为 989 TFLOPS；0.84 TFLOPS 只有该峰值的约 0.08%。实际计算量与全局内存访问量之比会高于 0.25 FLOP/B，因为一部分全局内存访问可能由片上缓存满足，而不是访问 DRAM。不过，我们仍然可以在内核实现层面主动提高该比值。

提高比值需要减少内核执行的全局内存访问次数。矩阵乘法为减少访问提供了很好的机会，且可以用相对简单的技术捕获。矩阵乘法函数的执行速度可能随着全局内存访问减少程度的不同而相差几个数量级，因此它是介绍这些技术的优秀初始例子。本章将介绍一种常用的减少全局内存访问的方法，并用矩阵乘法展示该方法。

5.2 CUDA 内存类型

CUDA 设备包含多种内存，可以帮助程序员提高内核的计算量与全局内存访问量之比。通过把 CUDA 变量声明为某种内存类型，程序员可以指定变量的可见范围和访问速度。图 5.2 展示了 CUDA 设备内存。图底部是全局内存和常量内存。这两种内存都可以由主机写入 (W) 和读取 (R)；全局内存还可以由设备读写，而常量内存支持设备端低延迟、高带宽的只读访问。第二章已经介绍全局内存，第七章将详细讨论常量内存。

局部内存也可以读写。声明为局部内存的 CUDA 变量实际上放在全局内存中，具有类似的访问延迟，但不会在线程之间共享。每个线程拥有全局内存中的一段私有局部内存，用来保存局部变量和无法分配到寄存器的数据，例如静态分配的数组、溢出的寄存器值以及线程调用栈中的其他元素。不过，规模较小、大小固定并且只使用常量索引访问的静态数组，可能被分配到寄存器中；第六章将进一步讨论。

图中的寄存器是片上内存的一种。寄存器中的变量可以以极高速度、高并行度访问。寄存器分配给单独线程，每个线程只能访问自己的寄存器。内核通常使用寄存器保存线程私有且频繁访问的变量。

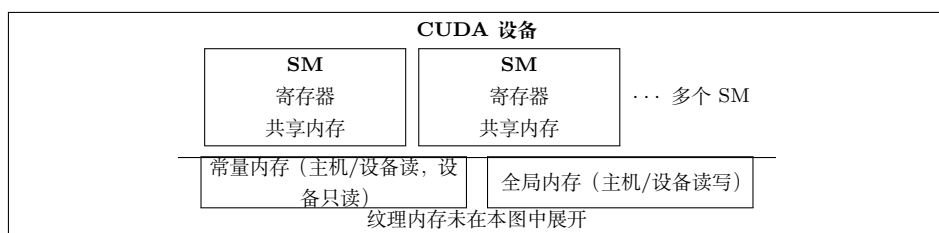


图 5.2: CUDA 设备内存模型的不完整概览（依据原书图 5.1 重绘）

CPU 与 GPU 的寄存器架构

CPU 和 GPU 的不同设计目标导致它们采用不同的寄存器架构。CPU 在不同线程之间进行上下文切换时，会把离开线程的寄存器保存到内存，再从内存恢复进入线程的寄存器。GPU 则通过把分配给处理分区的所有线程寄存器保存在处理分区的寄存器文件中，实现零开销调度。因此，在线程 warp 之间切换是瞬时的，因为进入线程的寄存器已经在寄存器文件中。

GPU 寄存器文件需要明显大于 CPU 寄存器文件。第四章还介绍过 GPU 支持动态资源划分：SM 可以给每线程配置较少寄存器并执行大量线程，也可以给每线程配置较多寄存器并执行较少线程。因此，GPU 寄存器文件必须支持寄存器的动态划分；CPU 寄存器架构则为每个线程预留固定寄存器集合，不考虑线程实际需求。

共享内存是另一种片上内存。共享内存中的数据访问速度相对于全局内存很高，

但不如寄存器。共享内存分配给线程块；块中所有线程都能访问为该块声明的共享内存变量。共享内存是同一线程块中的线程共享输入数据和中间结果的高效手段。

要充分理解寄存器、共享内存和全局内存之间的差异，需要进一步了解它们在现代处理器中的实现和使用方式。几乎所有现代处理器都可以追溯到 John von Neumann 在 1945 年提出的模型（图 5.3）。CUDA 设备也不例外。CUDA 设备的全局内存对应冯·诺依曼模型中的 Memory；处理器框对应今天通常看到的处理器芯片边界。全局内存在芯片外，由 DRAM 实现，因此访问延迟长、带宽相对低。寄存器对应冯·诺依曼模型中的 Register File，位于处理器芯片上，访问延迟很短，带宽远高于全局内存。在典型设备中，所有 SM 寄存器文件的聚合访问带宽至少比全局内存高两个数量级。变量放入寄存器后，其访问不再消耗片外全局内存带宽，也会提高计算量与全局内存访问量之比。

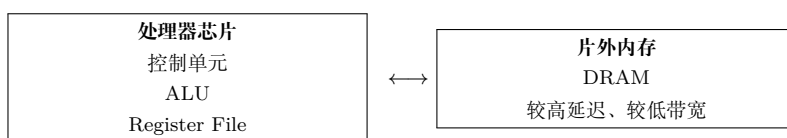


图 5.3: 基于冯·诺依曼模型的现代计算机中内存与寄存器的关系（依据原书图 5.2 重绘）

寄存器访问还涉及较少指令。多数现代处理器的算术指令具有内建寄存器操作数。例如，浮点加法指令可能写成：

```
1 fadd r1, r2, r3
```

r2 和 r3 是寄存器编号，指定输入操作数在寄存器文件中的位置；结果保存在 r1。当算术指令的操作数位于寄存器中时，不需要额外指令把操作数提供给执行算术计算的算术逻辑单元（ALU）。

如果操作数位于全局内存，处理器必须执行内存加载操作，把操作数提供给 ALU。例如，若浮点加法的第一个操作数在全局内存中，相关指令可能是：

```
1 load r2, r4, offset
2 fadd r1, r2, r3
```

load 指令把 r4 内容与偏移相加形成地址，访问全局内存，并把值放入 r2；随后 fadd 使用 r2 和 r3 中的值完成加法并把结果写入 r1。由于处理器每个时钟周期只能取出和执行有限数量的指令，增加 load 通常会增加处理时间，这也是把操作数放入寄存器可以提高执行速度的原因。

把操作数放入寄存器还有能源效率方面的优势。现代计算机中，从寄存器文件访问一个值所消耗的能量至少比从全局内存访问低一个数量级。不过，今天的 GPU 中每个线程可用的寄存器数量相当有限；如果寄存器使用量超过满占用率场景的限制，应用占用率可能降低。因此，在可能的情况下，也要避免过度使用这一有限资源。

共享内存和寄存器都在芯片上，但功能和访问成本不同。访问共享内存中的数据仍然需要执行内存加载指令，和访问全局内存类似；但共享内存位于芯片上，因此延迟低得多、吞吐量高得多。由于需要加载指令，共享内存的延迟高于寄存器、带宽低于寄存器。在计算机体系结构术语中，共享内存是一种暂存存储器（scratchpad memory）。

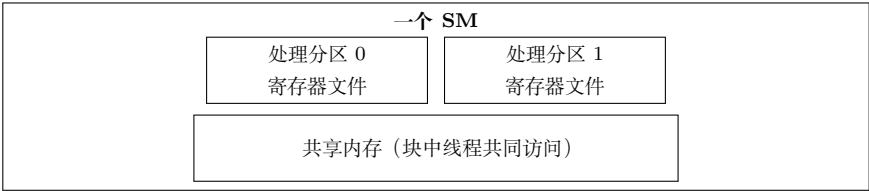


图 5.4: CUDA 设备 SM 中的共享内存与寄存器（依据原书图 5.3 重绘）

CUDA 中共享内存和寄存器的一个重要区别是，共享内存中的变量可由线程块中的所有线程访问，而寄存器数据对单个线程私有。共享内存专门用于支持同一线程块中线程之间高效、高带宽的数据共享。由于一个线程块中的线程可以分布在多个处理单元上，共享内存硬件通常被设计为允许多个处理单元同时访问其内容。

从 Hopper 架构开始，同一线程块集群中的线程可以访问集群中任意线程块的共享内存，这一特性称为**分布式共享内存**。它扩大了线程无需访问全局内存即可协作的范围，并提供了更大的快速内存池。

寄存器、局部内存、共享内存和全局内存具有不同功能、延迟和带宽。因此，必须理解如何声明变量，使变量位于预期的内存类型中。图 5.5 总结了 CUDA 变量声明语法及其属性。

声明形式	内存位置	作用域	生命周期
自动标量变量	寄存器	单个线程	内核执行期间
自动数组变量	局部内存（可能优化到寄存器）	单个线程	内核执行期间
<code>--shared--</code>	共享内存	一个线程块	内核执行期间
<code>--constant--</code>	全局内存/常量缓存	所有网格	整个应用
<code>--device--</code>	全局内存	所有网格	整个应用

图 5.5: CUDA 变量声明限定符及其属性（依据原书图 5.4 重绘）

变量的作用域表示可以访问该变量的线程集合：单个线程、一个线程块中的所有线程，或者所有网格中的所有线程。如果作用域是单个线程，则会为每个线程创建私有版本，每个线程只能访问自己的版本。例如，一个内核用 100 万个线程启动，并声明了线程作用域变量，就会创建该变量的 100 万个版本。

生命周期表示变量在程序执行期间可用的时间范围：网格执行期间，或整个应用执行期间。若生命周期在网格执行期间，变量必须在内核函数体内声明，只能由内核代码使用；多次调用内核时，变量值不会在调用之间保持，每次调用都必须初始化。若生命周期贯穿整个应用，变量必须在函数体外声明，其内容会在整个应用执行期间保持，并可供所有内核访问。

不属于数组的变量称为标量变量。内核函数和设备函数中声明的所有自动标量变量都会放入寄存器，作用域是单个线程。内核声明自动变量时，执行该内核的每个线程都会获得一份私有副本；线程终止时，其自动变量也随之消失。第三章图 3.8 中的 `blurRow`、`blurCol`、`curRow`、`curCol`、`pixels` 和 `pixVal` 都是自动变量。这些变量访问极快且高度并行，但必须小心不要超过硬件寄存器容量；寄存器使用过多会降低 SM 占用率。

自动数组变量默认不会放入寄存器，而是放入线程局部内存，可能产生较长访问延迟和访问拥塞。如果自动数组的所有访问都使用常量索引，编译器可能决定把它放入寄存器。自动数组的作用域与自动标量相同，为单个线程；每个线程拥有一份私有数组，线程终止时数组内容也消失。

在声明前加 `__shared__` 会声明 CUDA 共享内存变量，也可以在 `__shared__` 前加可选的 `__device__`，效果相同。此类声明通常位于内核函数或设备函数中。共享内存变量的作用域是线程块：块中所有线程看到同一版本；内核执行期间每个线程块拥有一份副本。内核结束时，共享内存变量的内容消失。共享内存常用于保存内核某一阶段频繁使用和重复使用的全局内存数据；第五章的矩阵乘法将展示这一用法。

在声明前加 `__constant__` 会声明 CUDA 常量变量，也可以在它前面加可选的 `__device__`。常量变量必须在函数体外声明，作用域是所有网格，生命周期是整个应用执行期间。常量变量通常用于向内核提供输入值，内核代码不能修改其值。常量变量存放在全局内存中，但会被缓存以便高效访问；访问模式合适时，常量内存访问可以非常快速且高度并行。目前一个应用中常量变量总大小限制为 65,536 B；必要时应把输入数据拆分以适应该限制。第七章将展示常量内存的使用。

只使用 `__device__` 限定符的变量是全局变量，会放入全局内存。访问全局变量较慢；全局变量对所有内核的所有线程可见，内容在整个应用执行期间保持。全局变量常用于在多次内核调用之间传递信息，而无需把信息作为内核参数传入。通常认为全局变量是不好的编程风格，应谨慎使用，因为它们可能导致错误并降低代码模块化程度。

5.3 通过分块减少内存流量

CUDA 设备内存的使用存在内在权衡：全局内存容量大但速度慢，共享内存容量小但速度快。常见策略是把数据划分为称为**瓦片 (tile)**的子集，使每个子集能够放入共享内存。“瓦片”一词来自铺墙类比：大型墙面对应全局内存数据，小瓦片对应能够放入共享内存的子集。一个重要条件是，内核对这些瓦片的计算能够彼此独立；并非所有数据结构和任意内核都能任意分块。

分块概念可以用第三章的矩阵乘法说明。第三章图 3.13 对应图 3.11 的内核。为方便读者，图 5.6 重新给出一个小型例子。为简洁起见，我们把 $P[y*Width+x]$ 、

$M[y*Width+x]$ 和 $N[y*Width+x]$ 分别记作 $P_{y,x}$ 、 $M_{y,x}$ 和 $N_{y,x}$ 。例子使用四个 2×2 线程块计算矩阵 P ； P 矩阵中的粗框表示每个块负责计算的输出瓦片。图中突出显示块 $(0,0)$ 的四个线程，它们计算 $P_{0,0}$ 、 $P_{0,1}$ 、 $P_{1,0}$ 和 $P_{1,1}$ 。

$P_{0,0}$	$P_{0,1}$	$P_{0,2}$	$P_{0,3}$
$P_{1,0}$	$P_{1,1}$	$P_{1,2}$	$P_{1,3}$
$P_{2,0}$	$P_{2,1}$	$P_{2,2}$	$P_{2,3}$
$P_{3,0}$	$P_{3,1}$	$P_{3,2}$	$P_{3,3}$

粗框：块 $(0,0)$ 的输出瓦片；每个线程计算一个 P 元素。

图 5.6: 矩阵乘法的小型示例（为简洁起见使用线性化索引记号，依据原书图 5.5 重绘）

图 5.6 中，块 $(0,0)$ 的线程访问 M 和 N 元素。线程 $(0,0)$ 依次读取 $M_{0,0}$ 和 $N_{0,0}$ 、 $M_{0,1}$ 和 $N_{1,0}$ 、 $M_{0,2}$ 和 $N_{2,0}$ 、 $M_{0,3}$ 和 $N_{3,0}$ 。图 5.7 展示块 $(0,0)$ 中所有线程的全局内存访问：线程沿垂直方向排列，访问时间从左向右推进。每个线程在执行期间访问 M 的四个元素和 N 的四个元素；不同线程访问的 M 、 N 元素存在显著重叠。

线程	随时间发生的全局内存访问			
$(0,0)$	$M_{0,0}, N_{0,0}$	$M_{0,1}, N_{1,0}$	$M_{0,2}, N_{2,0}$	$M_{0,3}, N_{3,0}$
$(0,1)$	$M_{0,0}, N_{0,1}$	$M_{0,1}, N_{1,1}$	$M_{0,2}, N_{2,1}$	$M_{0,3}, N_{3,1}$
$(1,0)$	$M_{1,0}, N_{0,0}$	$M_{1,1}, N_{1,0}$	$M_{1,2}, N_{2,0}$	$M_{1,3}, N_{3,0}$
$(1,1)$	$M_{1,0}, N_{0,1}$	$M_{1,1}, N_{1,1}$	$M_{1,2}, N_{2,1}$	$M_{1,3}, N_{3,1}$

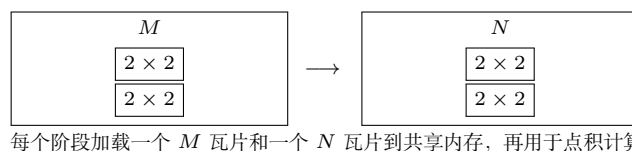
图 5.7: 块 $(0,0)$ 中所有线程执行的全局内存访问（依据原书图 5.6 重绘）

例如，线程 $(0,0)$ 和 $(0,1)$ 都访问 $M_{0,0}$ 以及 M 第 0 行的其余元素；线程 $(0,1)$ 和 $(1,1)$ 都访问 $N_{0,1}$ 以及 N 第 1 列的其余元素。第三章图 3.13 的内核让这两个线程分别从全局内存读取 M 第 0 行。如果能让它们协作，使这些 M 元素只从全局内存加载一次，就能把访问次数减半。事实上，块 $(0,0)$ 中每个 M 和 N 元素都被访问恰好两次；若四个线程协作访问全局内存，整体流量可以减半。

读者可以验证，矩阵乘法中全局内存流量的潜在减少量与线程块负责的输出瓦片大小成正比。如果瓦片大小为 $TILE_WIDTH \times TILE_WIDTH$ ，潜在减少因子就是 $TILE_WIDTH$ 。例如，使用 32×32 输出瓦片，通过线程协作，理论上可以把全局内存流量减少到原来的 $1/32$ 。

现在介绍分块矩阵乘法算法。基本思想是：在线程分别使用输入元素计算点积之前，由线程协作把 M 和 N 的子集加载到共享内存。共享内存容量很小，把输入元素加载到其中时必须小心不要超过容量。解决方法是把 M 和 N 矩阵划分成更小输入瓦片，并选择能够放入共享内存的瓦片大小。最简单的形式是，输入瓦片和输出瓦片的维度都等于线程块维度，如图 5.8 所示。

图 5.8 中，粗线把 M 、 N 划分为 2×2 输入瓦片。每个线程的点积计算被分为多个阶段；每个阶段中，线程块协作把一个 M 瓦片和一个 N 瓦片加载到共享内存。

图 5.8: 对 M 和 N 分块以利用共享内存 (依据原书图 5.7 重绘)

可以让块中每个线程各加载一个 M 元素和一个 N 元素。存放 M 元素的共享内存数组称为 Mds ，存放 N 元素的数组称为 Nds 。

在阶段 1 开始时，块 $(0,0)$ 的四个线程协作加载 M 瓦片：`thread(0,0)` 把 $M_{0,0}$ 加载到 $Mds[0][0]$ ，`thread(0,1)` 把 $M_{0,1}$ 加载到 $Mds[0][1]$ ，`thread(1,0)` 把 $M_{1,0}$ 加载到 $Mds[1][0]$ ，`thread(1,1)` 把 $M_{1,1}$ 加载到 $Mds[1][1]$ 。 N 瓦片以相同方式加载。

两个瓦片加载到共享内存后，就可以用于点积计算。共享内存中的每个值会被使用两次。例如，线程 $(1,1)$ 加载到 $Mds[1][1]$ 的 $M_{1,1}$ ，会被线程 $(1,0)$ 和线程 $(1,1)$ 各使用一次。让每个全局内存值加载到共享内存并被重复使用，可以把全局内存访问次数减少两倍；若瓦片为 $N \times N$ ，减少因子就是 N 。

阶段	活动	活动	活动	计算	活动	计算
Phase 1	加载 M 瓦片	加载 N 瓦片	<code>--syncthreads()</code>	使用 Mds/Nds	<code>--syncthreads()</code>	
Phase 2	加载下一 M 瓦片	加载下一 N 瓦片	<code>--syncthreads()</code>	使用 Mds/Nds	<code>--syncthreads()</code>	

图 5.9: 分块矩阵乘法的执行阶段 (依据原书图 5.8 重绘)

每个点积现在分为两个阶段。每个阶段中，每个线程把两对输入矩阵元素的乘积累加到 $Pvalue$ 。 $Pvalue$ 是自动变量，因此每个线程都有自己的私有版本。一般而言，如果输入矩阵维度为 $Width$ 、瓦片大小为 $TILE_WIDTH$ ，点积会分为

$$\frac{Width}{TILE_WIDTH}$$

个阶段。创建这些阶段是减少全局内存访问的关键：每个阶段只关注输入矩阵的一小部分，线程可以协作把该子集加载到共享内存，再用共享内存满足重叠的输入需求。

Mds 和 Nds 会在不同阶段重复使用。每个阶段都用同一组共享内存数组保存当前阶段的 M 、 N 子集，因此小容量共享内存能够服务大部分全局内存访问。这样的集中访问行为称为**局部性 (locality)**。当算法具有局部性时，就有机会使用小而快速的内存服务大部分访问，并把这些访问从全局内存中移除。局部性对于多核 CPU 和多线程 GPU 的高性能都很重要；第六章将再次讨论。

5.4 分块矩阵乘法内核

现在可以给出使用共享内存减少全局内存流量的分块矩阵乘法内核。图 5.10 中的内核实现了图 5.9 所示的阶段。

```

1 #define TILE_WIDTH 16
2 __global__ void matrixMulKernel(float* M, float* N, float* P, int Width) {
3
4     __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
5     __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];
6
7     int bx = blockIdx.x; int by = blockIdx.y;
8     int tx = threadIdx.x; int ty = threadIdx.y;
9
10    // Determine the row and column of the P element to work on
11    int Row = by * TILE_WIDTH + ty;
12    int Col = bx * TILE_WIDTH + tx;
13
14    // Loop over the M and N tiles required to compute P element
15    float Pvalue = 0;
16    for (int ph = 0; ph < Width/TILE_WIDTH; ++ph) {
17
18        // Collaborative loading of M and N tiles into shared memory
19        Mds[ty][tx] = M[Row*Width + ph*TILE_WIDTH + tx];
20        Nds[ty][tx] = N[(ph*TILE_WIDTH + ty)*Width + Col];
21        __syncthreads();
22
23        for (int k = 0; k < TILE_WIDTH; ++k) {
24            Pvalue += Mds[ty][k] * Nds[k][tx];
25        }
26        __syncthreads();
27
28    }
29    P[Row*Width + Col] = Pvalue;
30 }
31

```

图 5.10: 使用共享内存的分块矩阵乘法内核（依据原书图 5.9 重绘）

第 04、05 行把 `Mds` 和 `Nds` 声明为共享内存数组。共享内存变量的作用域是线程块，所以每个线程块都有一份 `Mds` 和 `Nds`，块中所有线程都可以访问同一份数组。这一点很重要，因为块中每个线程都必须能够访问由其他线程加载到共享内存中的 M 、 N 元素。

第 07、08 行把 `threadIdx` 和 `blockIdx` 保存到较短的自动变量中，使代码更简洁。自动标量变量位于寄存器，作用域是单个线程；每个线程都有私有的 `tx`、`ty`、`bx` 和 `by`，存放在该线程可访问的寄存器中。线程结束时，这些变量也随之消失。

第 11、12 行确定线程负责生成的 P 元素的行列索引。每个线程负责一个 P 元素。水平方向（ x 方向）的列索引为

$$\text{Col} = \text{bx} \times \text{TILE_WIDTH} + \text{tx}.$$

每个块在水平方向覆盖 `TILE_WIDTH` 个 P 元素；位于块 `bx` 中的线程前面有 `bx` 个块，即 `bx*TILE_WIDTH` 个线程，它们覆盖同样多的 P 元素；当前块中再向前数 `tx` 个线程，因此得到上述索引。图 5.11 展示了这一映射。

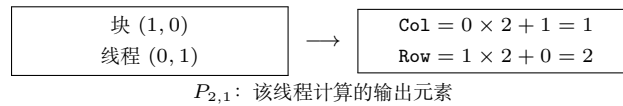


图 5.11: 分块乘法中的矩阵索引计算（依据原书图 5.10 重绘）

同理，垂直方向（ y 方向）的行索引为

$$\text{Row} = \text{by} \times \text{TILE_WIDTH} + \text{ty}.$$

在图 5.11 的例子中，块 (1, 0) 的线程 (0, 1) 负责的 x 索引为 $0 \times 2 + 1 = 1$ ， y 索引为 $1 \times 2 + 0 = 2$ ，因此它负责 $P_{2,1}$ 。

图 5.10 第 16 行开始的循环遍历计算 P 元素所需的所有阶段。循环变量 `ph` 表示已经完成的阶段数量；每个阶段使用一个 M 瓦片和一个 N 瓦片，因此在阶段开始时，前面阶段已经处理了 `ph*TILE_WIDTH` 对 M 、 N 元素。

每个阶段中，第 19、20 行把适当的 M 、 N 元素加载到共享内存。线程已经知道自己要处理的 M 行和 N 列，现在需要确定要加载的 M 列和 N 行。每个块有 `TILE_WIDTH^2` 个线程，它们共同加载同样数量的 M 、 N 元素；让每个线程各加载一个 M 元素和一个 N 元素即可。要加载的 M 区域起始列为 `ph*TILE_WIDTH`，因此每个线程加载距离起点 `tx` 的元素； N 区域起始行为 `ph*TILE_WIDTH`，每个线程加载距离起点 `ty` 的元素。

第 19 行的访问是

$$M[\text{Row} \times \text{Width} + \text{ph} \times \text{TILE_WIDTH} + \text{tx}],$$

线性化索引由行 `Row` 和列 `ph*TILE_WIDTH+tx` 构成。由于 `Row` 是 `ty` 的线性函数, `TILE_WIDTH^2` 个线程凭借不同的 `tx`、`ty` 组合各加载一个不同的 `M` 元素。第 20 行以类似方式使用

$$(\text{ph} * \text{TILE_WIDTH} + \text{ty}) * \text{Width} + \text{Col}$$

访问 `N` 元素。读者可以用图 5.8 和图 5.9 的小例子验证单个线程的地址计算。

第 21 行的 `__syncthreads()` 确保所有线程都完成把 `M`、`N` 瓦片加载到共享内存, 然后任何线程才可以继续。因为一个线程要使用的元素可能由其他线程加载, 所以必须确保共享内存中的所有元素都已准备好。第 23 行的循环使用瓦片元素执行一个点积阶段。图 5.11 中的箭头表示 `M`、`N` 元素的访问方向, 循环变量 `k` 对应第 23 行; 这些访问发生在保存当前瓦片的 `Mds`、`Nds` 中。第 26 行的第二个 `__syncthreads()` 确保所有线程都用完共享内存中的元素后, 才进入下一次迭代并加载下一组瓦片, 避免某些线程过早覆盖其他线程仍在使用的输入值。

两个 `__syncthreads()` 展示了并程序员经常需要分析的两种数据依赖。第一种是读后写依赖 (read-after-write dependence): 线程必须等待其他线程把数据写入正确位置后才能读取。第二种是写后读依赖 (write-after-read dependence): 线程必须等待所有需要数据的线程读完后才能覆盖它。二者也分别称为真实依赖和伪依赖; 后者是因为复用同一内存位置产生的, 如果使用不同位置就不会存在。

第 16–28 行的循环嵌套展示了**条带化 (strip-mining)** 技术: 把一个长循环拆成多个阶段。每个阶段是一个只执行原循环少量连续迭代的内层循环; 原循环变成外层循环, 负责按原有顺序反复调用内层循环, 从而完成所有迭代。在内层循环前后加入屏障同步, 可以强制同一线程块中的线程在每个阶段集中处理输入数据的一个片段。条带化是数据并程序创建分块阶段的重要方法。

1

所有点积阶段完成后, 线程退出外层循环, 在第 29 行使用由 `Row` 和 `Col` 计算出的线性化索引, 把结果写入自己的 `P` 元素。

分块算法的收益很大。矩阵乘法的全局内存访问可以减少 `TILE_WIDTH` 倍; 使用 32×32 瓦片时, 访问量可减少 32 倍, 计算量与全局内存访问量之比从 0.25 FLOP/B 提高到 8 FLOP/B。虽然仍低于 H100 的 20 FLOP/B 阈值, 内存带宽现在可以支持更高的计算速率。H100 的全局内存带宽为 3.35 TB/s, 因此理论吞吐量为

$$(3.35 \text{ TB/s})(8 \text{ FLOP/B}) = 26.8 \text{ TFLOPS},$$

显著高于未分块内核的 0.84 TFLOPS。实际比值和计算吞吐量可能更高, 因为片上缓存也会提供帮助。

26.8 TFLOPS 仍只有设备峰值 66.9 TFLOPS 的 40%。

¹条带化长期用于 CPU 编程。为了在顺序程序中改善局部性, 条带化之后常常还会进行循环交换; 它也是向量化编译器为 CPU 程序生成向量或 SIMD 指令的主要手段。

要把比值提高到 20 FLOP/B 以上、进入计算受限区域，还需要进一步减少全局内存访问。第十五章将介绍其中一些优化。矩阵乘法在很多领域都很重要，因此已经有高度优化的库可供直接使用。

例如，cuBLAS [1] 包含许多高级优化，可以帮助数值线性代数应用获得接近峰值的性能。CUTLASS [2] 也提供了类似的优化能力。

分块并不只表示把频繁访问的数据放入共享内存，它是一种更一般的优化：把频繁访问的数据子集放入比原位置更快的内存。在分块矩阵乘法中， M 、 N 因为被同一线程块中的不同线程重复访问而放入共享内存； P 也进行了分块，因为同一个线程会重复访问同一个元素，所以把它放入寄存器 `Pvalue`。这种方式称为 **寄存器分块 (register tiling)**。本例中的寄存器分块看起来很简单，第八、十一和十五章会介绍更复杂的例子。

分块提高矩阵乘法性能并非 GPU 独有。CPU 也长期使用分块（或 blocking）技术，确保 CPU 线程在特定时间窗口内复用的数据位于片上缓存。区别在于，CPU 分块依赖 CPU 缓存隐式地把复用数据保留在片上，而 GPU 分块显式使用共享内存和寄存器。CPU 核心通常一次运行一两个线程，线程可以依赖缓存保留最近访问的数据；GPU SM 同时运行大量线程以隐藏延迟，这些线程可能争用缓存槽位，使 GPU 缓存不够可靠。因此，对于需要复用的重要数据，GPU 往往必须使用共享内存。

前面的分块矩阵乘法内核作了两个简化假设：矩阵宽度是线程块宽度的倍数，因此不能正确处理任意宽度；矩阵是方阵，而实际应用不一定如此。下一节通过边界检查去除这两个假设中的第一个，并为处理一般矩阵奠定基础。

5.5 边界检查

现在扩展分块矩阵乘法内核，使其能够处理不是瓦片宽度倍数的矩阵宽度。把图 5.8 的小例子改为 3×3 的 M 、 N 、 P 矩阵，瓦片宽度仍为 2，得到图 5.12。图中展示块 (0,0) 在第二阶段的内存访问模式：`thread(0,1)` 和 `thread(1,1)` 试图加载不存在的 M 元素；`thread(1,0)` 和 `thread(1,1)` 试图访问不存在的 N 元素。

	M 的第二阶段访问	N 的第二阶段访问
(0, 0)	$M_{0,2}$	$N_{2,0}$
(0, 1)	$M_{0,3}$ (越界)	$N_{2,1}$
(1, 0)	$M_{1,2}$	$N_{3,0}$ (越界)
(1, 1)	$M_{1,3}$ (越界)	$N_{3,1}$ (越界)

图 5.12: 加载靠近边缘的输入矩阵元素：块 (0,0) 的一个阶段（依据原书图 5.11 重绘）

访问不存在的元素有两类问题。访问行末之后的元素（图中线程 (0,1) 和 (1,1) 对 M 的访问）会访问错误元素。在线性化二维矩阵中， $M_{0,2}$ 后面是 $M_{1,0}$ ；因此线程试图访问 $M_{0,3}$ 时，实际上会得到 $M_{1,0}$ ，随后用于点积计算会破坏输出结果。

访问列末之后的 N 元素则会访问数组分配区域之外的内存。在某些系统中，这些访问会返回其他数据结构中的随机值；在另一些系统中，系统会拒绝访问并终止程序。无论哪种结果都不可接受。

问题并不只出现在执行的最后阶段。图 5.13 展示块 (1,1) 在阶段 0 的访问模式：线程 (1,0) 和 (1,1) 试图加载不存在的 $M_{3,0}$ 、 $M_{3,1}$ ；线程 (0,1) 和 (1,1) 试图访问不存在的 $N_{0,3}$ 、 $N_{1,3}$ 。

	M 的阶段 0 访问	N 的阶段 0 访问
(0, 0)	$M_{2,0}$	$N_{0,2}$
(0, 1)	$M_{2,1}$	$N_{0,3}$ (越界)
(1, 0)	$M_{3,0}$ (越界)	$N_{1,2}$
(1, 1)	$M_{3,1}$ (越界)	$N_{1,3}$ (越界)

图 5.13: 块 (1,1) 在阶段 0 加载输入元素 (依据原书图 5.12 重绘)

不能简单地禁止不负责有效 P 元素的线程，因为这些线程仍可能需要为块中其他线程加载数据。例如，块 (1,1) 中的线程 (1,0) 不计算有效 P 元素，但在阶段 0 仍需加载 $M_{2,1}$ ，供块中其他线程使用。反过来，负责有效 P 元素的线程也可能访问不存在的 M 或 N 元素，例如图 5.12 中负责 $P_{0,1}$ 的线程 (0,1) 仍会访问不存在的 $M_{0,3}$ 。

因此，加载 M 瓦片、加载 N 瓦片以及计算/存储 P 元素需要不同的边界条件。一个实用规则是：每次内存访问都应有对应的检查，确保访问所使用的索引在被访问数组的范围内。

加载输入瓦片时，线程应检查要加载的输入元素是否有效。图 5.10 第 19 行的线性化索引由 y 索引 `Row` 和 x 索引 `ph*TILE_WIDTH+tx` 构成，因此边界条件是

$$\text{Row} < \text{Width} \ \&\& \ (\text{ph} * \text{TILE_WIDTH} + \text{tx}) < \text{Width}.$$

条件为真时，线程才加载 M 元素；加载 N 元素的条件是

$$(\text{ph} * \text{TILE_WIDTH} + \text{ty}) < \text{Width} \ \&\& \ \text{Col} < \text{Width}.$$

若条件为假，线程不加载元素，而是把共享内存位置设为 0.0f。这个值用于点积时不会改变结果。最后，线程只有在负责有效 P 元素时才存储最终点积，条件是

$$(\text{Row} < \text{Width}) \ \&\& \ (\text{Col} < \text{Width}).$$

加入边界检查后，分块矩阵乘法内核只需再修改一步，就能处理一般矩阵乘法。一般矩阵乘法定义为： $j \times k$ 矩阵 M 与 $k \times l$ 矩阵 N 相乘，得到 $j \times l$ 矩阵 P 。目前的内核只能处理方阵。要扩展为一般矩阵乘法，只需把 `Width` 参数替换为三个无符号整数参数 j 、 k 、 l ：

- `Width` 表示 M 高度或 P 高度时，替换为 j ；

```

1 #define TILE_WIDTH 16
2 __global__ void matrixMulKernel(float* M, float* N, float* P, int Width) {
3
4     __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
5     __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];
6
7     int bx = blockIdx.x; int by = blockIdx.y;
8     int tx = threadIdx.x; int ty = threadIdx.y;
9
10    int Row = by * TILE_WIDTH + ty;
11    int Col = bx * TILE_WIDTH + tx;
12    float Pvalue = 0;
13    for (int ph = 0; ph < ceil(Width/(float)TILE_WIDTH); ++ph) {
14        if ((Row < Width) && (ph*TILE_WIDTH+tx) < Width)
15            Mds[ty][tx] = M[Row*Width + ph*TILE_WIDTH + tx];
16        else Mds[ty][tx] = 0.0f;
17        if ((ph*TILE_WIDTH+ty) < Width && Col < Width)
18            Nds[ty][tx] = N[(ph*TILE_WIDTH + ty)*Width + Col];
19        else Nds[ty][tx] = 0.0f;
20        __syncthreads();
21        for (int k = 0; k < TILE_WIDTH; ++k) {
22            Pvalue += Mds[ty][k] * Nds[k][tx];
23        }
24        __syncthreads();
25    }
26    if (Row < Width && Col < Width)
27        P[Row*Width + Col] = Pvalue;
28 }
29

```

图 5.14: 带边界条件检查的分块矩阵乘法内核 (依据原书图 5.13 重绘)

- Width 表示 M 宽度或 N 高度时, 替换为 k ;
- Width 表示 N 宽度或 P 宽度时, 替换为 1。

这三个修改留给读者作为练习。

5.6 内存使用对占用率的影响

第四章讨论过, 为容忍长延迟操作, 需要尽量提高 SM 上线程的占用率。内核的内存使用在占用率调优中发挥重要作用。CUDA 寄存器和共享内存可以有效减少全局内存访问, 但必须注意 SM 对这些内存的容量限制。每个 CUDA 设备都提供有限的内存资源, 这些资源限制了给定应用中能够同时驻留在 SM 上的线程数。一般而言, 每个线程需要的资源越多, 每个 SM 能驻留的线程就越少。程序员必须在两种收益之间谨慎平衡: 使用片上内存提高计算强度, 以及保持高占用率以改善延迟容忍。

共享内存也会限制分配给每个 SM 的线程数。H100 GPU 每个 SM 最多可配置 228 KB 共享内存, 同时每个 SM 最多支持 2048 个线程。若要使用全部 2048 个线程槽位, 每线程平均共享内存使用量不应超过

$$\frac{228 \text{ KB}}{2048 \text{ threads}} = 114 \text{ B/thread.}$$

在分块矩阵乘法例子中, 每个块有 TILE_WIDTH^2 个线程, 并为 Mds 、 Nds 各使用 $\text{TILE_WIDTH}^2 \times 4 \text{ B}$ 共享内存。因此, 每线程平均使用

$$\frac{\text{TILE_WIDTH}^2 \times 4 \text{ B} + \text{TILE_WIDTH}^2 \times 4 \text{ B}}{\text{TILE_WIDTH}^2 \text{ threads}} = 8 \text{ B/thread.}$$

所以, 分块矩阵乘法内核的占用率不会受共享内存限制。

另一方面, 假设一个内核的线程块使用 38 KB 共享内存, 每个块有 256 个线程。平均每线程使用

$$\frac{38 \text{ KB}}{256 \text{ threads}} = 152 \text{ B/thread.}$$

在这种使用量下, 内核无法达到满占用率; 每个 SM 最多容纳

$$\frac{228 \text{ KB}}{152 \text{ B/thread}} = 1536 \text{ threads.}$$

因此, 该内核可达到的最大占用率为

$$\frac{1536}{2048} = 75\%.$$

对于需要大量共享内存的内核, CUDA 允许重新配置 SM, 把更多片上资源用于共享内存, 代价是减少其他片上内存资源, 尤其是缓存。这种可配置性让可预测和不太可预测的访问模式都能利用片上内存, 程序员可以根据应用需求分配片上资源。

每个设备允许线程块使用的共享内存上限也可能不同。通常希望内核根据硬件可用共享内存或程序需求动态确定使用量。主机代码可以调用

`cudaGetDeviceProperties`

查询设备属性。假设属性写入变量 `devProp`，则字段 `devProp.sharedMemPerBlock` 给出单个线程块可使用的共享内存上限，程序员可以据此确定每个线程块应使用的共享内存。这个字段不等同于 SM 的共享内存总量；后者还会受到具体架构和运行时配置的影响。

图 5.10 和图 5.14 的内核不支持主机代码动态调整共享内存使用量。图 5.9 的声明把大小固定为编译期常量：

```
1 __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
2 __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];
```

若代码包含：

```
1 #define TILE_WIDTH 32
```

则 `Mds` 和 `Nds` 都有 $32^2 = 1024$ 个元素。若要改变大小，必须修改 `TILE_WIDTH` 并重新编译，内核不能轻易在运行时调整共享内存用量。

CUDA 可以用另一种声明方式动态调整共享内存：在共享内存声明前加 `extern`，并省略数组大小。这样，`Mds` 和 `Nds` 必须合并为一个动态分配的一维数组：

```
1 extern __shared__ char Mds_Nds[];
```

因为只有一个合并数组，还需要手动定义 `Mds` 区域和 `Nds` 区域的起始位置，并根据行列索引使用线性化索引访问它们。

调用内核时，可以把每个块使用的动态共享内存字节数作为内核调用的第三个执行配置参数：

```
1 size_t size = ...;
2 matrixMulKernel<<<dimGrid, dimBlock, size>>>
3     (Md, Nd, Pd, Width, size/2, size/2);
```

`size_t` 是用于保存动态分配数据结构大小信息的内置类型，`size` 以字节数表示。对 32×32 瓦片，需要

$$2 \times 32 \times 32 \times 4 = 8192 \text{ B}$$

来容纳 `Mds` 和 `Nds`。底本将运行时设置 `size` 的细节留作练习。

图 5.15 展示了如何修改图 5.9 和图 5.13 的内核，使 `Mds`、`Nds` 使用动态大小共享内存。也可以把数组各区域大小作为内核参数传入；本例增加两个参数，分别表示 `Mds` 区域和 `Nds` 区域的大小，单位为字节。主机代码传入 `size/2` 作为两个参数。


```
1 #define TILE_WIDTH 32
2 __global__ void matrixMulKernel(float* M, float* N, float* P, int Width,
3                                 unsigned Mds_sz, unsigned Nds_sz) {
4
5     extern __shared__ char Mds_Nds[];
6
7     float *Mds = (float *) Mds_Nds;
8     float *Nds = (float *) Mds_Nds + Mds_sz;
9
```

图 5.15: 使用动态大小共享内存的分块矩阵乘法内核关键片段（依据原书图 5.14 重绘；区域大小参数按底本以字节传递）

译者注：上图保留底本的代码写法，但这里必须明确单位。若 `Mds_sz` 表示字节数，`Nds` 的起始地址应按字节偏移计算，例如 `(float *) (Mds_Nds + Mds_sz)`；若保留代码中的指针加法形式，`Mds_sz` 就必须表示 `float` 元素数。启动参数 `size/2` 也必须与所采用的单位保持一致。

5.7 小结

现代处理器中，程序执行速度取决于内核代码的计算量与全局内存访问量之比。比值较高时，内核是计算受限的，执行速度可以接近硬件峰值计算吞吐量；比值较低时，内核是内存受限的，执行速度受操作数从内存取得的速率限制。

CUDA 提供寄存器、共享内存和常量内存。这些内存比全局内存小得多，但访问速度高得多。把数据放入这些内存，可以在不消耗全局内存带宽的情况下访问数据，从而提高内核的计算量与全局内存访问量之比。不过，有效使用这些内存通常需要重新设计算法。本章以矩阵乘法为例介绍了分块这一提高数据访问局部性、有效使用共享内存的常用策略。在并行编程中，分块使用屏障同步，让多个线程在执行的每个阶段共同关注输入数据的一个子集，并把该子集放入特殊内存以获得更高访问速度。

CUDA 程序员必须注意这些特殊内存的容量有限，且容量取决于硬件实现。一旦超过容量，内存限制会减少每个 SM 中能够同时执行的线程数，降低 GPU 计算吞吐量和延迟容忍能力。开发应用时分析硬件限制，是并行编程的重要能力。

虽然本章在 GPU 的 CUDA 编程背景下介绍了分块算法，但分块同样适用于几乎所有并行计算系统。应用必须具有数据访问局部性，才能有效利用这些系统中的高速内存。例如，在多核 CPU 系统中，数据局部性可以帮助应用有效使用片上数据缓存，降低内存访问延迟并获得高性能。片上缓存容量同样有限，也要求计算具有局部性。因此，在其他编程模型下开发其他并行计算系统的应用时，分块算法同样有用。

本章的目标是介绍局部性、分块和不同 CUDA 内存类型。我们用共享内存实现了一个分块矩阵乘法内核；还研究了在使用分块技术时为什么需要边界条件检查，以支持任意数据维度；最后简要讨论了动态大小共享内存，使内核可以根据硬件能力调整每个线程块使用的共享内存大小。

练习

1. 考虑矩阵加法。能否使用共享内存减少全局内存带宽消耗？提示：分析每个线程访问的元素，看看不同线程之间是否存在共同访问。
2. 分别为使用 2×2 瓦片和 4×4 瓦片的 8×8 矩阵乘法，画出图 5.7 的等价图。验证全局内存带宽的减少量确实与瓦片的维度成正比。
3. 如果忘记在图 5.9 的内核中使用一个或两个 `__syncthreads()`，可能发生什么类型的错误执行行为？
4. 假设寄存器和共享内存容量都不是问题，使用共享内存而不是寄存器保存从全局内存取出的值，一个重要原因是什么？请解释。
5. 对分块矩阵乘法内核，如果使用 32×32 瓦片，输入矩阵 M 和 N 的内存带宽使用量减少多少？
6. 假设启动一个 CUDA 内核，网格有 1000 个线程块，每块有 512 个线程。如果在内核中声明一个局部变量，在该内核的整个执行生命周期中会创建多少个该变量的版本？
7. 对上一题，如果声明一个共享内存变量，在该内核的整个执行生命周期中会创建多少个该变量的版本？
8. 考虑把两个 $N \times N$ 输入矩阵相乘。输入矩阵中的每个元素从全局内存请求多少次？
 - a. 不使用分块时；
 - b. 使用大小为 $T \times T$ 的瓦片时。
9. 一个内核每线程执行 36 次浮点运算，并进行 7 次 32 位全局内存访问。对下面每组设备属性，判断该内核是计算受限还是内存受限：
 - a. 峰值 FLOPS = 200 GFLOPS，峰值内存带宽 = 100 GB/s；
 - b. 峰值 FLOPS = 300 GFLOPS，峰值内存带宽 = 250 GB/s。
10. 为操作瓦片，一名 CUDA 新手编写了下面的设备内核，用于转置矩阵中的每个瓦片。瓦片大小为 `BLOCK_WIDTH`，矩阵各维度都是 `BLOCK_WIDTH` 的倍数。内核

调用和代码如下；BLOCK_WIDTH 在编译时已知，可以取 1 到 20 之间的任意值。

```

1 dim3 blockDim(BLOCK_WIDTH, BLOCK_WIDTH);
2 dim3 gridDim(A_width/blockDim.x, A_height/blockDim.y);
3 BlockTranspose<<<gridDim, blockDim>>>(A, A_width, A_height);
4 __global__ void
5 BlockTranspose(float* A_elements, int A_width, int A_height)
6 {
7     __shared__ float blockA[BLOCK_WIDTH][BLOCK_WIDTH];
8     int baseIdx = blockDim.x * BLOCK_SIZE + threadIdx.x;
9     baseIdx += (blockDim.y * BLOCK_SIZE + threadIdx.y) * A_width;
10    blockA[threadIdx.y][threadIdx.x] = A_elements[baseIdx];
11    A_elements[baseIdx] = blockA[threadIdx.x][threadIdx.y];
12 }

```

- 在 BLOCK_SIZE 的可能取值范围内，哪些值能让该内核在设备上正确执行？
- 如果不是所有值都能正确执行，错误执行行为的根本原因是什么？如何修改代码，使其对所有 BLOCK_SIZE 值都能工作？

译者注：

底本的声明使用 BLOCK_WIDTH。索引表达式和小题文字则使用 BLOCK_SIZE。

这是底本内部命名不一致。本译稿保留这一差异。

正确实现应统一使用同一个宏名。

11. 考虑下面的 CUDA 内核及调用它的主机函数：

```

1 __global__ void foo_kernel(float* a, float* b) {
2     unsigned int i = blockDim.x*blockDim.x + threadIdx.x;
3     float x[4];
4     __shared__ float y_s;
5     __shared__ float b_s[128];
6     for (unsigned int j = 0; j < 4; ++j) {
7         x[j] = a[j*blockDim.x*gridDim.x + i];
8     }
9     if (threadIdx.x == 0) {
10        y_s = 7.4f;
11    }
12    b_s[threadIdx.x] = b[i];
13    __syncthreads();
14    b[i] = 2.5f*x[0] + 3.7f*x[1] + 6.3f*x[2] + 8.5f*x[3]
15        + y_s*b_s[threadIdx.x] + b_s[(threadIdx.x + 3)%128];
16 }
17 void foo(int* a_d, int* b_d) {

```

```
18     unsigned int N = 1024;  
19     foo_kernel<<<(N + 128 - 1)/128, 128>>>(a_d, b_d);  
20 }
```

- a. 变量 `i` 有多少个版本?
- b. 数组 `x[]` 有多少个版本?
- c. 变量 `y_s` 有多少个版本?
- d. 数组 `b_s[]` 有多少个版本?
- e. 每个线程块使用多少字节共享内存?
- f. 该内核的浮点运算量与全局内存访问量之比是多少 (OP/B)?

译者注：底本的主机函数参数类型写成 `int*`，而内核形参为 `float*`；本译稿保留题目代码并提醒读者，实际编译时应统一类型。

12. 某 GPU 的硬件限制为：每个 SM 2048 个线程、32 个线程块、65,536 个寄存器和 96 KB 共享内存。对下列内核特征，判断内核能否达到满占用率；如果不能，指出限制因素：
 - a. 每块 64 个线程，每线程 27 个寄存器，每个 SM 4 KB 共享内存；
 - b. 每块 256 个线程，每线程 31 个寄存器，每个 SM 8 KB 共享内存。

参考文献

- [1] NVIDIA, *cuBLAS*, 2025. <https://docs.nvidia.com/cuda/cublas/>.
- [2] NVIDIA, *CUTLASS*, 2025. <https://docs.nvidia.com/cutlass/>.

第六章 性能考量

并行程序的执行速度会随着程序的资源需求与硬件资源限制之间的相互作用而发生很大变化。要在几乎所有并行编程模型中获得高性能，就必须管理并行代码与硬件资源限制之间的相互作用。这是一项实用技能，需要深入理解硬件架构；而且，最好的学习方式是在面向高性能设计的并行编程模型中亲自完成练习。

到目前为止，我们已经学习了 GPU 架构的基本方面，以及这些方面对性能的影响。第四章介绍了 GPU 的计算架构和相关性能考量，例如控制流分歧与占用率。第五章介绍了 GPU 的片上内存架构，以及使用共享内存分块和寄存器分块来缓解全局内存带宽限制。本章将进一步介绍片外内存（DRAM）架构，并讨论与之相关的性能考量，例如内存访问合并、隐藏内存延迟以及向量加载和存储。我们还将介绍其他重要优化，包括避免共享内存 bank 冲突、线程粒度粗化、循环展开和双缓冲。最后，我们用一份常见性能优化检查清单结束本书的第一部分；这份清单将指导后文第二、第三部分中并行模式的性能优化。

不同应用对架构资源的需求不同，因此不同的架构限制可能成为主导因素和性能限制，通常称为**瓶颈 (bottleneck)**。在某个 CUDA 设备上，通过用一种资源的使用换取另一种资源的使用，往往可以显著改善应用性能。如果被缓解的资源限制原本确实是主导限制，并且被加重的另一种资源不会对并行执行造成负面影响，这种策略通常很有效。缺少这种理解时，性能调优就会变成猜测：看似合理的策略可能带来性能提升，也可能完全没有效果。

6.1 全局内存访问合并

CUDA 内核性能最重要的因素之一，是如何访问全局内存；全局内存的有限带宽可能成为性能瓶颈。CUDA 应用利用大规模数据并行性，自然会在很短时间内从全局内存处理大量数据。第五章研究了利用共享内存的分块技术：线程块中的线程协作，从而减少必须从全局内存访问的数据总量。本节进一步讨论如何高效地在全局内存与共享内存或寄存器之间搬运数据，这一技术称为**内存访问合并 (memory coalescing)**。内存访问合并通常与分块技术结合使用，使 CUDA 设备能够高效利用全局内存带宽，达到其潜在性能。

CUDA 设备的全局内存由 DRAM 实现，这一点我们在第五章已经看到。DRAM 中的数据位存储在 DRAM 单元中；DRAM 单元是很小的电容，电荷的有无分别表示 1 和 0。从 DRAM 单元读取数据时，小电容必须用其中极少的电荷驱动一条通向传感器的高电容线路，并触发传感器的检测机制，由后者判断电容中的电荷是否足以表示“1”。现代 DRAM 芯片完成这一过程需要几十纳秒（见“为什么 DRAM 很慢？”侧栏）。这与现代计算设备亚纳秒级的时钟周期形成鲜明对比。由于这一过程相对于理想的数据访问速度（每字节亚纳秒级访问）非常慢，现代 DRAM 设计使用并行性来提高数据访问速率，通常称为 **内存访问吞吐量 (memory access throughput)**。

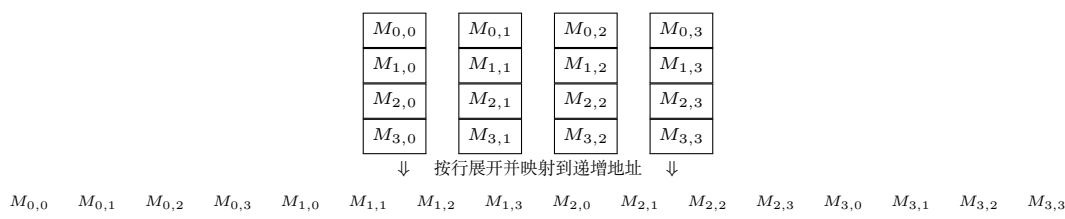


图 6.1: 按照行主序把矩阵元素放入线性数组（依据原书图 6.1 重绘）

为什么 DRAM 很慢？ DRAM 单元中的电容很小，而用于访问它的位线却连接着大量单元，具有很大的电容。译码器必须驱动连接到成千上万个单元输出晶体管栅极的水平线；水平线完全充电或放电到足以打开输出栅极的电平，可能需要较长时间。更困难的是，输出栅极打开后，单元还要驱动通向传感放大器的垂直线，使传感放大器能够检测其内容。这一过程依赖电荷共享：栅极释放单元中存储的极少电荷，而这点电荷必须足以改变长位线的大电容上的电势，触发传感放大器的检测机制。可以把它类比为：有人在连接许多办公室的走廊一端拿着一小杯咖啡，另一端的人要根据沿走廊传播的香味判断咖啡的味道。

增大每个单元中的电容、使用更强的电容，可以加快这一过程。然而，DRAM 的发展方向恰恰相反。为了让每个芯片存储更多比特，每个单元中的电容一直在缩小、强度也一直在降低。这就是 DRAM 访问延迟长期没有下降的原因。

每次访问一个 DRAM 位置时，还会同时访问包含该位置在内的一段连续位置。每个 DRAM 芯片都配有许多传感器，它们并行工作，每个传感器感测这些连续位置中的一个数据位。传感器完成检测后，这些连续位置中的数据可以高速传输到处理器。这些被同时访问并交付的连续位置称为 **DRAM 突发 (DRAM burst)**。如果应用集中使用一个突发中的数据，DRAM 就能以远高于真正随机访问的速率提供数据；随机访问会把位置分散在许多突发中。

现代 GPU 还使用基于 SRAM 的片上 L1 和 L2 缓存，以降低全局内存访问延迟并提高吞吐量。由于这些缓存不是 CUDA 编程模型的主要组成部分，我们没有专门介绍它们；不过，它们的存在和行为会影响 CUDA 应用的实际性能。就全局内存访问模式而言，即使数据已经从 DRAM 取出并放入 SRAM 缓存，集中访问连续内存

位置仍然有益。当访问 L2 缓存中的数据时，一段连续位置会从 L2 缓存移动到 L1 缓存。数据在不同缓存层级之间移动时所采用的这种粒度称为**缓存块 (cache block)**或**缓存行 (cache line)**。如果应用集中使用一个缓存行中的数据，缓存就能以更高的速率提供数据，因为需要移动的缓存行更少。

认识到现代 DRAM 的突发组织方式以及现代缓存层次的缓存行组织方式后，当前 CUDA 设备采用了一种让程序员能够高效访问全局内存的技术：把线程的内存访问组织成有利的模式。这一技术利用了 warp 中线程在任意时刻执行同一条指令这一事实。当 warp 中所有线程执行加载指令时，硬件会检测它们是否访问连续的全局内存位置。最有利的访问模式是：warp 中所有线程访问同一个 DRAM 突发和/或缓存行中的连续全局内存位置。此时，硬件会把这些访问合并为一个较大的访问请求。例如，对 warp 的一条加载指令，如果线程 0 访问全局内存位置 X ，线程 1 访问位置 $X + 1$ ，线程 2 访问位置 $X + 2$ ，依此类推，那么访问 DRAM 或缓存时，这些访问就会被合并为一个针对连续位置的请求。

除了保证 warp 中线程访问连续的全局内存位置，还要注意这些访问在全局内存中的对齐方式。当这段连续位置的起始地址与一个突发或缓存行的起始位置对齐时，合并访问只需最少数量的内存事务。相反，如果起始地址未对齐，访问的数据可能跨越更多突发或缓存行，从而产生额外的内存流量。

为了理解如何有效利用合并硬件，我们需要回顾 C 多维数组元素的内存地址是如何形成的。回忆第三章（图 3.3，此处为方便起见复制为图 6.1）：C 和 CUDA 中的多维数组元素按照行主序约定放入线性寻址的内存空间。行主序是指，数据的放置保留了行的结构：同一行中相邻元素被放入地址空间中连续的位置。在图 6.1 中，第 0 行的四个元素先按出现顺序放置，接着是第 1 行、第 2 行和第 3 行的元素。显然，虽然 $M_{0,0}$ 和 $M_{1,0}$ 在二维矩阵中看起来相邻，但在线性寻址的内存中，它们相隔四个位置。

假设与图 6.1 中的矩阵 M 类似的多维数组 N 被用作矩阵乘法的第二个输入矩阵。在这种情况下，被分配来计算连续输出元素的 warp 中相邻线程，会遍历该输入矩阵中连续的列。图 6.2 左上部分给出了这种计算的代码，右上部分给出了访问模式的逻辑视图：相邻线程遍历连续的列。检查代码可以看出，对 N 的访问能够合并。数组 N 的索引是 $k * \text{Width} + \text{col}$ 。所有线程中的变量 k 和 Width 都相同；变量 col 定义为 $\text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ ，因此相邻线程（具有连续的 threadIdx.x 值）拥有连续的 col 值，也就会访问 N 中连续的元素。

图 6.2 下半部分展示了这种访问模式的物理视图。在第 0 次迭代中，相邻线程访问第 0 行中相邻的元素；这些元素在内存中相邻，对应图中的“第 0 次迭代的加载”。在第 1 次迭代中，相邻线程访问第 1 行中同样相邻的元素，对应“第 1 次迭代的加载”。这个过程继续遍历所有行。可以看出，线程形成的内存访问模式是有利于合并的。事实上，到目前为止本书实现的所有内核都具有自然合并的内存访问。

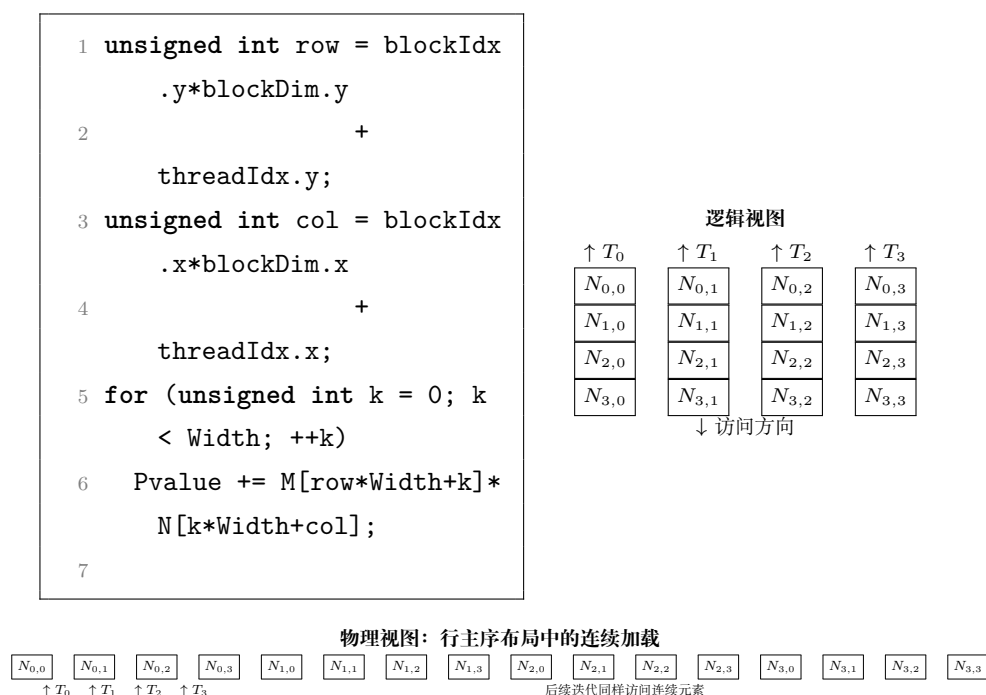


图 6.2: 合并的全局内存访问模式（依据原书图 6.2 重绘）

现在假设矩阵不是按行主序存储，而是按列主序存储。出现这种情况可能有多种原因。例如，我们可能需要访问一个按行主序存储矩阵的转置形式。在线性代数中，经常需要同时使用矩阵的原始形式和转置形式；最好避免同时创建并存储这两种形式。常见做法是先以一种形式（例如原始形式）创建矩阵；需要转置形式时，通过访问原始矩阵并交换行索引和列索引的角色来访问其元素。这等价于把转置矩阵视为按列主序布局。无论原因是什么，下面观察当矩阵乘法示例的第二个输入矩阵按列主序存储时得到的内存访问模式。

图 6.3 展示了矩阵按列主序存储时，相邻线程遍历连续列的情况。图的左上部分是代码，右上部分是内存访问的逻辑视图。程序仍然试图让每个线程访问矩阵 N 的一列；不过，for 循环中访问 N 的索引计算已经调整，以反映假设的列主序布局。检查代码可以看出，对 N 的访问不利于合并。数组 N 的索引是 $\text{col} * \text{Width} + k$ 。与之前一样， col 定义为 $\text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ ，因此相邻线程拥有连续的 col 值。但是，索引中的 col 乘以 Width ，这意味着相邻线程访问的 N 元素相隔 Width 个位置，所以这种访问不利于合并。

图 6.3 下半部分展示了与图 6.2 很不相同的内存访问物理视图。在第 0 次迭代中，相邻线程在逻辑上访问第 0 行中的连续元素，但由于列主序布局，这些元素在内存中并不相邻；图中将这些加载标为“第 0 次迭代的加载”。类似地，在第 1 次迭代中，相邻线程访问第 1 行中的连续元素，但它们在内存中同样不相邻。对于实际矩阵，每个维度通常有数百甚至数千个元素。相邻线程在每次迭代访问的 N 元素可能相隔数百甚至数千个位置。硬件会判断这些访问彼此相距很远，因而无法合并。

当计算本身不自然地适合合并时，有多种策略可以优化代码以实现内存访问合

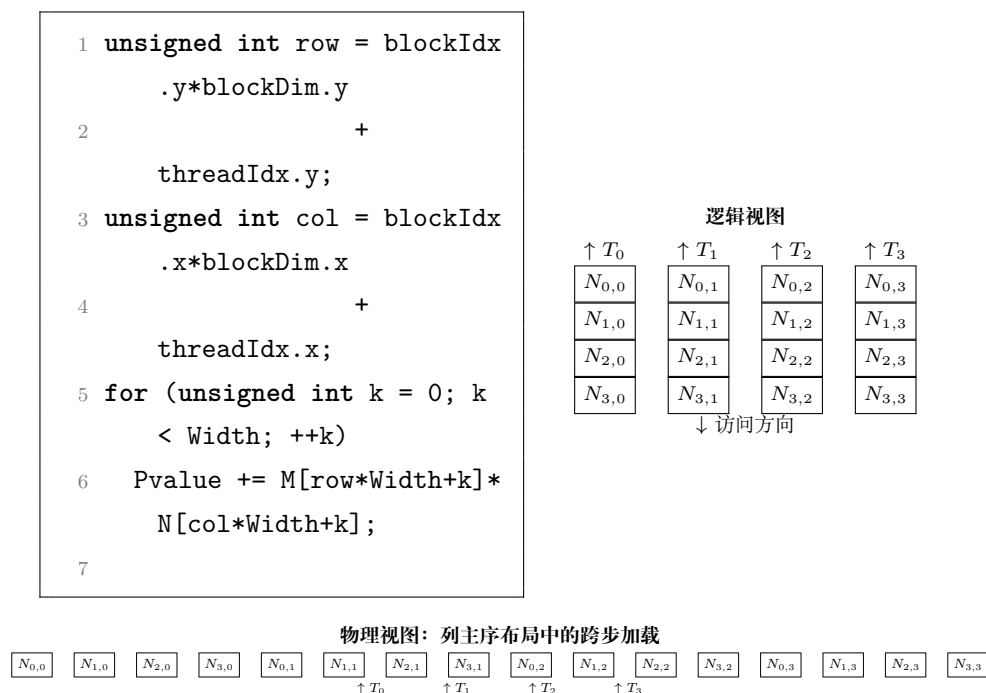


图 6.3: 未合并的全局内存访问模式（依据原书图 6.3 重绘）

并。一种策略是重新安排线程到数据的映射方式，另一种策略是重新安排数据本身的布局。第 6.8 节会讨论这些策略，并在全书中看到它们的应用。还有一种策略，是以合并方式在全局内存和共享内存之间传输数据，然后在对数据布局不太敏感、访问延迟更低的共享内存中执行不利的访问模式。本书后续会反复看到这种策略，包括现在要应用于矩阵-矩阵乘法的一个优化：第二个输入矩阵按列主序布局。该优化称为**转角 (corner turning)**。

图 6.4 展示了转角的一个应用示例。例子中，输入矩阵 M 在全局内存中按行主序存储，输入矩阵 N 在全局内存中按列主序存储；二者相乘得到输出矩阵 P ，输出矩阵在全局内存中按行主序存储。图中展示了负责输出瓦片顶边四个连续元素的四个线程如何加载输入瓦片元素。

对矩阵 M 输入瓦片的访问方式与第五章相同。四个线程加载输入瓦片顶边的四个元素。每个线程加载的输入元素，在输入瓦片中的局部行、列索引与该线程负责的输出元素在输出瓦片中的局部行、列索引相同。由于相邻线程访问 M 同一行中的连续元素，而按行主序布局的这些元素在内存中相邻，因此访问是合并的。

另一方面，对矩阵 N 输入瓦片的访问必须不同于第五章。图 6.4(a) 展示了如果采用第五章的相同安排会得到什么访问模式。尽管线程块中的前四个线程在逻辑上加载输入瓦片顶边的四个连续元素，但由于 N 采用列主序布局，相邻线程加载的元素在内存中相距很远。换句话说，负责输出瓦片同一行中连续元素的相邻线程，会加载内存中不连续的位置，导致未合并的内存访问。

解决这个问题的方法，是让线程块中前四个连续线程加载输入瓦片左边（同一列）的四个连续元素，如图 6.4(b) 所示。直观地说，就是在线程计算加载 N 输入瓦

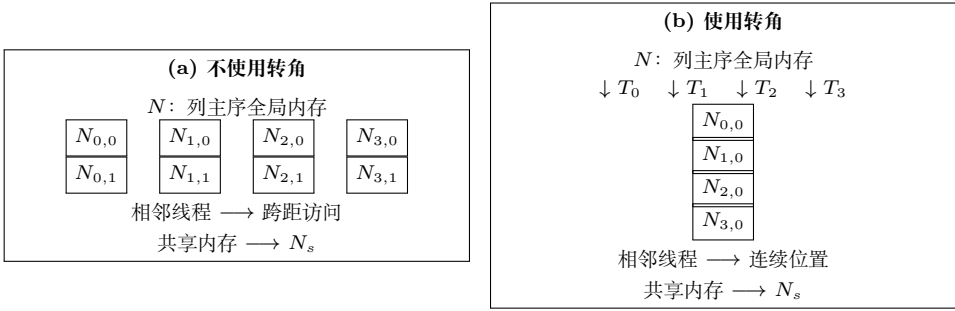


图 6.4: 通过转角合并对列主序矩阵 N 的访问 (依据原书图 6.4 重绘)

片的线性化索引时, 交换 `threadIdx.x` 和 `threadIdx.y` 的角色。由于 N 按列主序布局, 同一列中的连续元素在内存中相邻。因此, 相邻线程会加载内存中相邻的输入元素, 从而保证内存访问合并。可以把 N 的元素瓦片按列主序或行主序放入共享内存; 无论采用哪种方式, 输入瓦片加载完成后, 每个线程都能以很小的性能代价访问自己的输入。这是因为共享内存由 SRAM 技术实现, 并不要求访问合并。注意, 这里用的是访问 4×4 瓦片时前四个线程的玩具示例; 实际的转角瓦片大小将是 warp 大小 32, 因此线程块中的前 32 个线程会访问瓦片左边的 32 个连续元素。

虽然上面的例子关注于合并全局内存加载操作, 但合并对于全局内存存储操作同样重要。事实上, 未合并存储的影响可能比未合并加载更严重, 因为未合并存储会产生部分存储操作, 执行读-改-写访问; 在启用了纠错码 (Error Correcting Code, ECC) 的 GPU 上, 这可能需要更多内存带宽。

合并内存访问的主要优点, 是把多个内存访问合并为一次访问, 从而减少全局内存流量。只有在同一时刻发生、并且访问相邻内存位置的访问, 才有可能被合并。

计算机系统之外也会出现交通拥堵, 如图 6.5 所示的高速公路交通拥堵。拥堵的根本原因是, 太多汽车挤过一条原本只为更少车辆设计的道路。发生拥堵后, 每辆车的行驶时间都会大幅增加; 上下班通勤时间很容易变成原来的两三倍。

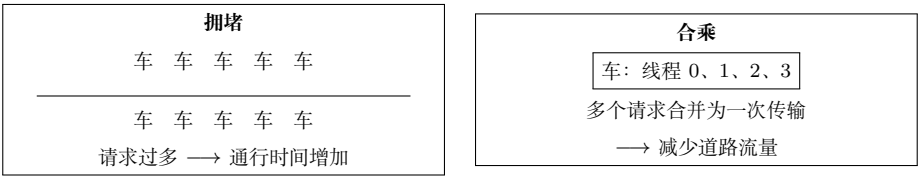


图 6.5: 降低高速公路系统中的交通拥堵 (依据原书图 6.5 重绘的概念示意)

减少交通拥堵的大多数方案, 都是减少道路上的汽车数量。在通勤人数不变的情况下, 人们需要共享车辆。常见方式是拼车: 一组通勤者轮流驾驶一辆车送大家上班。在一些国家, 政府会通过政策鼓励拼车; 例如, 规定某些车辆类别在特定日期不能上路, 促使不同日期允许上路的车主组成拼车小组。在另一些国家, 政府会为减少道路车辆的行为提供激励, 例如把拥堵高速公路的部分车道指定为合乘车道, 只允许载有两三名以上乘客的车辆使用。还有一些国家把汽油价格设置得很高, 使人们通过拼车节省费用。这些鼓励拼车的措施, 都是为了克服拼车需要额外协调这一事实, 如图 6.6

所示。

希望拼车的工作人员必须妥协，并就共同的通勤时间表达成一致。图 6.6 上半部分展示了适合拼车的时间表。时间从左向右流逝；工作人员 A 和工作人员 B 拥有相似的睡眠、工作和晚餐安排，因此很容易共乘一辆车上下班。相似的时间表使他们更容易同意共同的出发时间和返程时间。图的下半部分则不同：工作人员 A 和 B 的生活习惯非常不同。工作人员 A 通宵聚会、白天睡觉、晚上工作；工作人员 B 夜间睡觉、早上上班、下午 6 点回家吃晚餐。两人的时间表差异太大，无法协调出共同的时间乘坐同一辆车上下班。

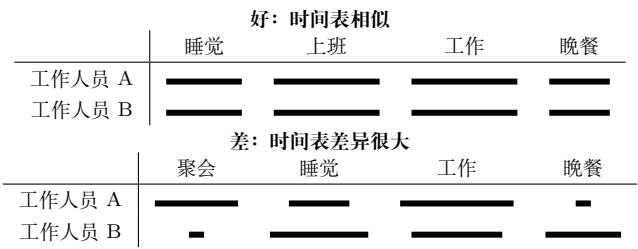


图 6.6: 拼车需要参与者之间进行时间同步（依据原书图 6.6 重绘）

内存访问合并与拼车安排非常相似。可以把数据看作通勤者，把 DRAM 访问请求看作车辆。当 DRAM 请求速率超过 DRAM 系统提供的访问吞吐量时，就会出现交通拥堵，算术单元随之空闲。如果多个线程访问同一个 DRAM 位置，它们有可能组成“合乘”，把多个访问合并为一个 DRAM 请求。不过，这要求线程具有相似的执行时间表，使它们的数据访问能够合并。warp 中的线程是理想候选者，因为得益于 SIMD 执行，它们会同时执行加载指令。

现代 CPU 也会在缓存设计中识别 DRAM 的突发组织方式。CPU 缓存行通常对应 DRAM 突发的一部分、一个突发，甚至多个突发。充分使用所触及的每个缓存行中的字节的应用，通常比随机访问内存位置的应用性能高得多。本章介绍的技术同样可以帮助 CPU 程序通过减少 DRAM 访问拥堵来提高速度。

6.2 隐藏内存延迟

如第 6.1 节所述，DRAM 突发是一种并行组织方式：DRAM 核心阵列中的多个位置可以并行访问。不过，仅靠突发还不足以实现现代处理器所需要的 DRAM 访问带宽。DRAM 系统通常还采用两种并行组织方式，即 bank 和通道 (channel)。在最高层次上，一个处理器包含一个或多个通道。每个通道都是一个内存控制器，其总线把一组 DRAM bank 连接到处理器。图 6.7 展示了一个包含四个通道的处理器，每个通道的总线连接四个 DRAM bank。在实际系统中，处理器通常有一到八个通道，而每个通道会连接更多 bank。

总线的数据传输带宽由总线宽度和时钟频率决定。现代双倍数据速率 (Double

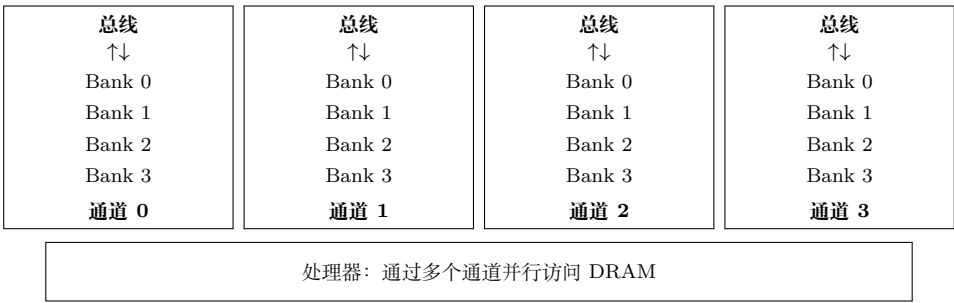


图 6.7: DRAM 系统中的通道与 bank（依据原书图 6.7 重绘）

Data Rate, DDR) 总线在每个时钟周期传输两次数据，分别发生在时钟周期的上升沿和下降沿。例如，一个时钟频率为 3.2 GHz 的 64 位 DDR5-6400 总线，其带宽为

$$8\text{ B} \times 2 \times 3.2\text{ GHz} = 51.2\text{ GB/s}.$$

这个数值看起来很大，但对于现代 CPU 和 GPU 往往仍然太小。例如，现代 CPU 可能至少需要 200 GB/s 的内存带宽，而 GPU 可能需要超过 1000 GB/s，这相当于 CPU 需要四条总线、GPU 需要 20 条总线。传统电路板上的处理器驱动如此多的总线会消耗过多功耗，这促使人们采用高带宽内存（High Bandwidth Memory, HBM）技术。HBM 把处理器与 DRAM 封装在一起，大幅缩小二者之间连接的尺寸和容量，并提高时钟频率以及连接处理器和 DRAM 的总线数量。例如，HBM3E 支持 4.9 GHz/s 的总线时钟频率和 16 条 64 位总线，可提供 1 TB/s 的内存带宽。

译者注：底本在 HBM3E 的例子中写作 4.9 GHz/s。频率的常用单位通常是 GHz，而不是 GHz/s；这里保留底本的数值和单位写法，以便与原文对照。

对每个通道而言，与之连接的 bank 数量由充分利用总线数据传输带宽所需的 bank 数量决定，如图 6.8 所示。每个 bank 都包含 DRAM 单元阵列、用于访问这些单元的传感放大器，以及把数据突发交付到总线的接口（见第 6.1 节）。

图 6.8(a) 展示了单个 bank 连接到一个通道时的数据传输时序，其中有两处对该 bank 中 DRAM 单元的连续内存读取。回顾第 6.1 节，每次访问都需要较长延迟：译码器要启用单元，单元还要把存储的电荷共享给传感放大器。图中的浅灰部分表示这种延迟。传感放大器完成工作后，突发数据通过总线传输；图 6.8(a) 左侧的深色部分表示突发数据传输时间。第二次内存读取同样会在传输突发数据之前经历很长的访问延迟（时间帧中两个深色部分之间的浅色部分）。



图 6.8: 通过多个 bank 提高通道数据传输带宽利用率（依据原书图 6.8 重绘）

实际上,访问延迟(浅灰部分)比数据传输时间(深色部分)长得多。显然,单 bank 组织方式会严重浪费通道总线的数据传输带宽。例如,如果 DRAM 单元阵列访问延迟与数据传输时间之比为 20 : 1,通道总线的最大利用率只有

$$\frac{1}{21} = 4.8\%.$$

也就是说,一个 16 GB/s 的通道向处理器交付数据的速率不会超过 0.76 GB/s,这完全不可接受。把多个 bank 连接到通道总线可以解决这个问题。

当两个 bank 连接到同一条通道总线时,第一个 bank 正在服务另一个访问的同时,可以在第二个 bank 中启动一次访问。因此,可以重叠访问 DRAM 单元阵列的延迟。图 6.8(b) 展示了双 bank 组织方式的时序。假设 bank 0 在图示时间窗口之前已经开始工作;第一个 bank 开始访问其单元阵列后不久,第二个 bank 也开始访问其单元阵列。当 bank 0 的访问完成时,它传输突发数据(时间帧最左侧的深色部分)。bank 0 完成数据传输后,bank 1 就可以传输自己的突发数据(第二个深色部分)。接着这一模式会在后续访问中重复。

从图 6.8(b) 可以看出,拥有两个 bank 可能使通道总线的数据传输带宽利用率翻倍。一般而言,如果单元阵列访问延迟与数据传输时间之比为 R ,而我们希望完全利用通道总线的数据传输带宽,那么至少需要 $R + 1$ 个 bank。例如,当比值为 20 时,每条通道总线至少需要连接 21 个 bank。通常,每条通道总线连接的 bank 数量还需要大于 R ,原因有两个。

第一,更多 bank 可以降低多个同时访问指向同一 bank 的概率,这种情况称为 **bank 冲突 (bank conflict)**。每个 bank 一次只能服务一个访问;发生冲突时,这些访问的单元阵列访问延迟就不能再重叠。bank 数量越多,这些访问分散到多个 bank 的概率越高。第二,每个单元阵列的大小需要在访问延迟和可制造性之间取得合理平衡,这限制了每个 bank 能提供的单元数量。仅为了支持所需的内存容量,系统就可能需要很多 bank。

图 6.9 展示了把数组 M 的元素分布到通道和 bank 中的玩具示例。假设突发大小很小,只包含两个元素(8 字节)。这种分布由硬件设计决定:数组的前 8 字节 ($M[0]$ 和 $M[1]$) 存储在通道 0 的 bank 0 中;接下来的 8 字节 ($M[2]$ 和 $M[3]$) 存储在通道 1 的 bank 0 中;再接下来的 8 字节 ($M[4]$ 和 $M[5]$) 存储在通道 2 的 bank 0 中;最后 8 字节 ($M[6]$ 和 $M[7]$) 存储在通道 3 的 bank 0 中。

通道 0	通道 1	通道 2	通道 3
Bank 0: $M[0], M[1]$	Bank 0: $M[2], M[3]$	Bank 0: $M[4], M[5]$	Bank 0: $M[6], M[7]$
Bank 1: $M[8], M[9]$	Bank 1: $M[10], M[11]$	Bank 1: $M[12], M[13]$	Bank 1: $M[14], M[15]$
Bank 0: $M[16], M[17]$	Bank 0: $M[18], M[19]$	...	...

图 6.9: 把数组元素交错分布到通道和 bank (依据原书图 6.9 重绘)

此时分布回绕到通道 0,但下一个 8 字节会使用 bank 1。这样, $M[10]$ 和 $M[11]$ 位于通道 1 的 bank 1, $M[12]$ 和 $M[13]$ 位于通道 2 的 bank 1, $M[14]$ 和 $M[15]$ 位于

通道 3 的 bank 1。虽然图中没有画出，后续元素也会回绕，并从通道 0 的 bank 0 开始。例如， $M[16]$ 和 $M[17]$ 会存储在通道 0 的 bank 0， $M[18]$ 和 $M[19]$ 会存储在通道 1 的 bank 0，依此类推。

图 6.9 所示的分布方案通常称为**交错数据分布(interleaved data distribution)**，它把元素分散到系统中的多个 bank 和通道。即使数组比较小，这种方案也能很好地把元素分散开：在转移到通道 1 的 bank 0 之前，只向通道 0 的 bank 0 分配足以填满其 DRAM 突发的元素。在这个玩具示例中，只要数组至少有 16 个元素，存储这些元素时就会涉及所有通道和 bank。

线程的并行执行与 DRAM 系统的并行组织之间存在重要联系。为了达到设备规定的内存访问带宽，必须有足够多的线程同时发起内存访问。这一观察体现了最大化占用率的另一个好处。回顾第四章：最大化占用率可以保证 SM 上驻留足够多的线程来隐藏核心流水线延迟，从而高效利用指令吞吐量。现在可以看到，最大化占用率还可以保证有足够多的内存访问请求来隐藏 DRAM 访问延迟，从而高效利用内存带宽。当然，要获得最佳带宽利用率，这些内存访问必须均匀分布到多个通道和 bank 中，而且对每个 bank 的访问也必须是合并的。

下面用一个例子说明并行线程执行与并行内存组织之间的相互作用。我们使用图 5.5 中的例子，并将其复制为图 6.10。假设矩阵乘法使用 2×2 线程块和 2×2 瓦片完成。

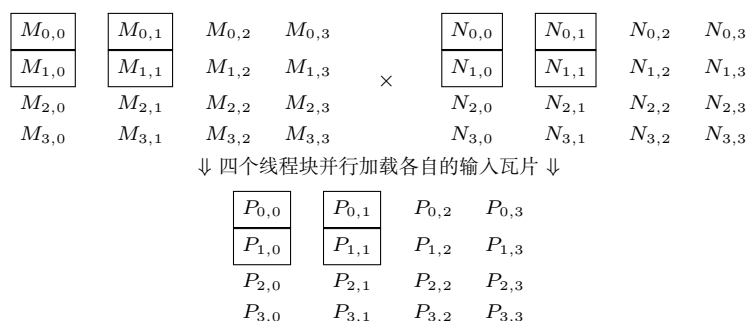


图 6.10: 矩阵乘法中线程块并行加载输入瓦片的示例（依据原书图 6.10 重绘）

在内核执行的第 0 阶段，四个线程块都加载它们的第一个瓦片。每个瓦片涉及的 M 元素如图 6.11 所示。表的第 2 行给出了第 0 阶段访问的 M 元素及其二维索引；第 3 行给出了相同的 M 元素及其线性化索引。假设所有线程块并行执行，可以看到每个线程块会发起两次合并访问。根据图 6.9 中的分布，第 0 阶段所有线程块的这些合并访问会发送到通道 0 和通道 2 中的 bank。访问可以并行执行，从而同时利用两个通道，并提高每条通道的数据传输带宽利用率。

还可以看到， $\text{Block}_{\{0,0\}}$ 和 $\text{Block}_{\{0,1\}}$ 会加载相同的 M 元素。只要这些线程块的执行时间足够接近，现代设备中的大多数缓存就会把这些访问合并为一次。事实上，GPU 设备中的缓存主要就是为了合并这种访问、减少对 DRAM 系统的访问次数而设计的。

加载阶段	Block0,0	Block0,1	Block1,0	Block1,1
	$M_{0,0}, M_{0,1}$	$M_{0,0}, M_{0,1}$	$M_{2,0}, M_{2,1}$	$M_{2,0}, M_{2,1}$
阶段 0 (二维索引)	$M_{1,0}, M_{1,1}$	$M_{1,0}, M_{1,1}$	$M_{3,0}, M_{3,1}$	$M_{3,0}, M_{3,1}$
	$M[0], M[1]$	$M[0], M[1]$	$M[8], M[9]$	$M[8], M[9]$
阶段 0 (线性索引)	$M[4], M[5]$	$M[4], M[5]$	$M[12], M[13]$	$M[12], M[13]$
	$M_{0,2}, M_{0,3}$	$M_{0,2}, M_{0,3}$	$M_{2,2}, M_{2,3}$	$M_{2,2}, M_{2,3}$
阶段 1 (二维索引)	$M_{1,2}, M_{1,3}$	$M_{1,2}, M_{1,3}$	$M_{3,2}, M_{3,3}$	$M_{3,2}, M_{3,3}$
	$M[2], M[3]$	$M[2], M[3]$	$M[10], M[11]$	$M[10], M[11]$
阶段 1 (线性索引)	$M[6], M[7]$	$M[6], M[7]$	$M[14], M[15]$	$M[14], M[15]$

图 6.11: 各阶段由线程块加载的 M 元素 (依据原书图 6.11 重绘)

图 6.11 的第 4、5 行展示了内核执行第 1 阶段时加载的 M 元素。此时，访问会发送到通道 1 和通道 3 中的 bank；这些访问同样可以并行执行。读者应当能够看出，线程并行执行与 DRAM 系统的并行结构之间存在相互促进的关系：一方面，充分利用 DRAM 系统潜在的访问带宽，需要许多线程同时访问 DRAM 中的数据；另一方面，设备的执行吞吐量依赖于高效利用 DRAM 系统的并行结构，即 bank 和通道。例如，如果同时执行的线程都访问同一个通道中的数据，内存访问吞吐量和设备整体执行速度都会大幅降低。

读者可以自行验证：使用同样的 2×2 线程块配置，计算两个更大的矩阵（例如 8×8 矩阵）将使用图 6.9 中的全部四个通道。另一方面，如果 DRAM 突发大小增加，就需要计算更大的矩阵，才能充分利用所有通道的数据传输带宽。

6.3 向量加载与存储

第 6.2 节已经说明，最大化 GPU 上线程的占用率，可以让线程同时发起足够多的内存访问，从而高效利用内存带宽。还可以让每个线程在一条指令中访问更大块的连续数据，进一步增加同时访问的数据量。每个线程在一条指令中访问大量连续数据的加载和存储指令，分别称为**向量加载 (vector load)**和**向量存储 (vector store)**。

例如，考虑第二章图 2.10 中的向量加法内核。该内核中，每个线程加载两个 4 B（即 4 字节）的浮点值，并存储一个 4 B 的浮点值。由于相邻线程在每次访问中访问内存中的相邻元素，所以这些内存访问是合并的。此外，由于该内核没有过度使用任何资源，它达到了最大占用率，并能同时发起足够多的内存加载和存储，以高效利用内存带宽。不过，该内核的内存带宽利用率仍然可以进一步提高。原实现中，每访问 4 B 内存，线程就执行一条加载或存储指令；相反，可以使用向量加载和存储，让每个线程每条指令访问 16 B 内存。

图 6.12 给出了实现这一优化的代码。在该例中，每个线程像以前一样计算自己的全局索引（第 02 行）。不过，启动的线程数只有待相加元素数量的四分之一，索引 i 表示四个浮点值组成的数据块，而不是单个值。为了加载包含四个浮点值的数据块，先把 x 和 y 数组的指针转换为 `float4` 类型，再对其解引用（第 03–04 行）。`float4`

类型把四个浮点值表示为一个组合的 16 B 对象。对 float4 类型指针解引用时，编译器会把访问转换为向量加载或存储，一次访问整个 16 B 对象。加载得到的 x4 和 y4 各包含四个浮点值，可以通过字段 .x、.y、.z 和 .w 访问。代码分别相加这些字段，并把结果放入另一个 float4 对象 z4（第 05–09 行）。最后，在存储 z4 时把 z 数组的指针转换为 float4 类型（第 10 行），从而确保该存储操作作为向量存储执行。

```

1 __global__ void vecAdd_kernel(float* x, float* y, float* z, int N) {
2     int i = blockDim.x*blockIdx.x + threadIdx.x;
3     float4 x4 = ((float4*)x)[i];
4     float4 y4 = ((float4*)y)[i];
5     float4 z4;
6     z4.x = x4.x + y4.x;
7     z4.y = x4.y + y4.y;
8     z4.z = x4.z + y4.z;
9     z4.w = x4.w + y4.w;
10    ((float4*)z)[i] = z4;
11 }
12

```

图 6.12: 使用向量加载的向量加法内核（依据原书图 6.12 重绘）

如果 x 和 y 数组中的元素总数不能被 4 整除，就需要边界条件，确保边界线程使用标量加载和存储，而不是向量加载和存储，以免访问越界数据。还需要边界条件，确保最后一个线程块中的越界线程不处于活动状态。这些边界条件留给读者作为练习。

向量加载和存储减少了访问全局内存相关的指令执行开销，使内存带宽能够得到更高效的利用。在图 6.12 中，两条向量加载指令（第 03–04 行）取代了八条标量加载指令，使加载相同数据量的指令执行开销降低了 75%。当内核无法以满占用率执行时，向量加载和存储也很重要：它们仍然允许内核发起足够多的并发内存访问，以容忍内存延迟并充分利用内存带宽。第 15 章将看到这种情况的一个例子。

6.4 共享内存 bank 冲突

全局内存由 DRAM 技术实现，而 GPU 中的共享内存由访问延迟短得多的 SRAM 技术实现。较短的访问延迟使共享内存不需要采用 DRAM 的突发技术，因此可以使用较窄的 bank。例如，GPU 中一种典型的共享内存结构有 32 个 bank，每个 bank 宽 32 位（4 字节）。因此，当一个 GPU 线程从共享内存访问一个 32 位整数或浮点值时，该线程就独占了一个 bank 的整个输出。

当同一 warp 中的线程访问共享内存中的不同值时，最好让每个线程访问不同的

内存 bank，使这些值能够并行提供。为此，共享内存把连续的 4 B 数据块以交错方式放入不同 bank，方式类似于第 6.2 节描述的 DRAM bank 和通道中的数据放置。例如，前 4 B 放在 bank 0，接下来的 4 B 放在 bank 1，再接下来的 4 B 放在 bank 2，依此类推；32 个 bank 用尽后，重新从 bank 0 开始。因此，如果 warp 中的 32 个线程访问 32 个连续的 4 B 整数或浮点值，每个线程都会由不同的 bank 并行服务。

遗憾的是，如果同一 warp 中线程的访问彼此不够相邻，这些访问可能在同一时刻命中同一个 bank，从而产生 **bank 冲突**。发生共享内存 bank 冲突时，一个 bank 无法同时服务多个线程；对同一 bank 的访问必须逐个服务。硬件会检测同一 warp 中线程的共享内存访问是否存在 bank 冲突，并把这些访问拆分为多个无冲突访问。

例如，考虑第 6.1 节图 6.4(b) 中的转角优化。虽然线程以合并方式从全局内存加载数据，但它们写入共享内存时采用的是跨步模式。写入共享内存的代码如下：

```
1 __shared__ float a[TILE_DIM][TILE_DIM]; // TILE_DIM = 32
2 a[threadIdx.x][threadIdx.y] = ...;
```

假设线程块维度为 32×32 ，同一 warp 中的线程具有相同的 `threadIdx.y` 值，而 `threadIdx.x` 值连续。因此，这些线程会写入同一列中的连续元素。线程 0 写入 `a[0][0]`，线程 1 写入 `a[1][0]`，线程 2 写入 `a[2][0]`，依此类推。由于数组 `a` 按行主序存储，`a[i][j]` 的线性索引为

$$i \times 32 + j.$$

所以 `a[0][0]`、`a[1][0]`、`a[2][0]` 等元素的线性索引是 0、32、64 等。也就是说，同一 warp 中的线程访问的元素相隔 32 个位置 (128 B)。

现在假设索引 0 处的元素位于 bank 0。索引 1 到 31 处的元素分别位于 bank 1 到 bank 31；到达最后一个 bank 后，顺序重新从 bank 0 开始，所以索引 32 处的元素位于 bank 0。于是，线程 0 和线程 1 都写入 bank 0；同样，线程 2 写入的索引 64 处元素也位于 bank 0 ($64 \bmod 32$)。实际上，同一 warp 中的所有线程都会写入同一个 bank，造成 32 路 bank 冲突！硬件会把这些写入串行化，对同一个 bank 执行 32 次连续访问，而其他 bank 都没有被使用。

为了避免 bank 冲突，需要重新安排数据，使同一 warp 中线程访问的元素位于不同 bank。可以在瓦片中增加一列，从而在连续行之间加入额外元素。这些为改变数据布局而加入、但不使用的额外元素称为**填充 (padding)**。下面的代码展示了如何在转角示例中加入填充：

```
1 __shared__ float a[TILE_DIM][TILE_DIM + 1]; // TILE_DIM = 32
2 a[threadIdx.x][threadIdx.y] = ...;
```

关键变化是，数组 `a` 的列数从 `TILE_DIM` 变成了 `TILE_DIM + 1`。

下面观察这一变化对共享内存 bank 访问的影响。现在数组 `a` 有 33 列，`a[i][j]`

的线性索引是

$$i \times 33 + j.$$

因此，线程 0 在线性索引 0 处访问 $a[0][0]$ ，它位于 bank 0。线程 1 访问 $a[1][0]$ ，线性索引为 $1 \times 33 + 0 = 33$ ，它位于 bank 1 ($33 \bmod 32$)。线程 2 访问 $a[2][0]$ ，线性索引为 $2 \times 33 + 0 = 66$ ，它位于 bank 2 ($66 \bmod 32$)。显然，bank 冲突被消除了：warp 中 32 个线程分别访问不同的 bank，bank 可以并行服务这些访问，不需要硬件串行化。

与许多性能优化技术一样，填充也有代价和限制。最明显的代价是会消耗额外的共享内存资源。另一个重要代价是：访问填充数组时，线性化索引的计算可能比原数组维度为 2 的幂时更昂贵。例如，当列数为 32（即 2^5 ）时，线性化索引中的 $i \times 32$ 可以简化为移位操作 $i \gg 5$ 。但是，当列数填充为 33 时，乘法必须实现为

$$i \times (32 + 1) = (i \gg 5) + i.$$

增加的操作数取决于填充引入的额外列数。

填充数组所带来的实际收益取决于访问模式和数组原始维度。例如，如果原数组的列数是 16 而不是 32，那么同一 warp 中线程之间的 bank 冲突会分散到两个 bank，每个 bank 上形成 16 路冲突。因此，填充带来的性能收益会更低。这是直观的：更小的步长意味着同一 warp 内的访问彼此更接近。

6.5 线程粗化

到目前为止，在我们看到的所有内核中，工作都以最细粒度在线程之间并行化。也就是说，每个线程被分配最小可能的工作单元。例如，第二章的向量加法内核中，每个线程负责一个输出元素；第三章的 RGB 转灰度和图像模糊内核中，每个线程负责输出图像中的一个像素；第三章和第五章的矩阵乘法内核中，每个线程负责输出矩阵中的一个元素。

以最细粒度在线程之间并行化的优点，是能够增强第四章讨论过的透明可扩展性。如果硬件拥有足够资源并行执行全部工作，应用就向硬件暴露了足够多的并行性，可以充分利用硬件。相反，如果硬件没有足够资源同时执行全部工作，硬件可以简单地把工作拆成多个 wave 串行执行：当前运行的线程块完成并释放足够资源后，再延迟执行其他线程块。

以最细粒度并行化的缺点，在于并行化工作本身可能存在开销。这种开销有多种形式，例如不同线程块重复加载数据、重复工作、同步开销和指令执行开销等。当硬件并行执行线程时，承担这些并行化开销通常是值得的。然而，如果由于资源不足，硬件最终不得不把工作串行化，那么这些开销就是不必要的。此时，更好的做法是由

程序员部分地串行化工作，降低并行化开销；方法是让每个线程负责多个工作单元，这通常称为**线程粗化 (thread coarsening)**。

例如，考虑下面的代码。内核中的每个线程负责对不同数据执行某种计算 `foo`，数据由索引 `i` 区分：

```
1 unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
2 foo(i);
```

这里，每个线程只对一个 `i` 值执行一次 `foo`。另一种方式是使用线程粗化，让每个线程对多个 `i` 值多次执行 `foo`。线程被分配执行的工作单元数量称为**粗化因子 (coarsening factor)**。下面的代码以粗化因子 4 为例，展示如何对上面的代码进行线程粗化：

```
1 unsigned int iStart = 4*(blockIdx.x*blockDim.x + threadIdx.x);
2 for(unsigned int c = 0; c < 4; ++c) {
3     unsigned int i = iStart + c;
4     foo(i);
5 }
```

这里，把全局线程索引乘以 4，使每个线程得到 4 个连续整数的起始值。例如，线程 0 的 `iStart` 为 0，负责整数 0、1、2、3；线程 1 的 `iStart` 为 4，负责整数 4、5、6、7，依此类推。然后，每个线程执行一个循环，遍历分配给自己的 4 个值。这个循环称为**粗化循环 (coarsening loop)**。在每次循环迭代中，线程计算该迭代负责的工作单元对应的索引 `i`，并调用 `foo`。

本章用玩具示例介绍线程粗化；在全书中还会看到把线程粗化应用到真实内核的例子。第 6.3 节已经给出一个线程粗化例子：向量加法内核中，每个线程负责相加四对元素，而不是一对元素，从而利用向量加载和存储。这里，把向量加法分配到更多线程上的开销，是加载和存储指令的指令执行开销。如果线程确实并行运行，承担这种开销可能值得；但是，如果线程过多而硬件必须把它们串行执行，那么最好像图 6.12 中那样，在每个线程内部自行串行执行多次加法。在单个线程中串行执行多次加法，可以把内存访问合并为向量加载和存储，从而减少访问内存的指令执行开销。

另一个适合线程粗化的例子，是第五章的分块矩阵乘法内核。在该例中，把计算分配给许多线程块意味着不同线程块会重复加载相同的输入瓦片。减少线程块数量、让每个线程块处理更大的输出瓦片，可以减少每个输入瓦片被加载的次数。第 15 章将深入讨论这个例子，并使用线程粗化改善矩阵乘法的数据重用。

线程粗化是一种强大的优化，对许多应用都可能带来显著性能提升。它是一种常用优化，在本书许多章节中都会反复出现。不过，应用线程粗化时需要避免几个陷阱。

第一个陷阱是在线程粗化没有必要时仍然使用它。回顾前文，只有并行化计算存在开销时，线程粗化才有益。并非所有计算都有这种开销；对于某些计算，线程完全独立运行，不需要协调，也不共享数据。对这类计算应用线程粗化，通常不会带来显

著性能差异。

第二个陷阱是粗化过度，导致硬件资源利用不足。回顾第四章，尽可能向硬件暴露并行性，可以实现透明可扩展性，让硬件根据执行资源数量灵活地并行或串行执行工作。程序员粗化线程时，会减少暴露给硬件的并行性。如果粗化因子太大，暴露的并行性不足，一些并行执行资源就会闲置。

例如，一个包含 1056 个线程块的内核运行在一块能够同时执行 264 个线程块的 GPU 上时，会有 4 个 wave，且每个 wave 都能充分利用 GPU。但如果该内核中的线程按因子 8 粗化，内核就只包含 132 个线程块，即 0.5 个 wave。此时 GPU 一半的资源会闲置，很可能导致性能下降。程序员还应选择不会产生部分 wave 的粗化因子。例如，一个未粗化、包含 1584 个线程块的内核运行在同一块 GPU 上时，有 6 个能够充分利用 GPU 的 wave；如果线程按因子 4 粗化，内核将有 396 个线程块，即 1.5 个 wave。因此，第一个完整 wave 完成后，第二个部分 wave 会降低 GPU 资源利用率，并产生第四章讨论过的尾部效应。

在实践中，不同设备拥有不同数量的执行资源，因此最佳粗化因子通常与设备和数据集都有关，需要针对不同设备和数据集重新调优。应用线程粗化后，可扩展性就不再那么透明：为了在不同代 GPU 上达到最佳性能，可能需要使用不同粗化因子重新编译内核。

第三个陷阱是增加资源消耗，以至于显著降低占用率并损害性能。根据内核的不同，线程粗化可能要求每线程使用更多寄存器，或每线程块使用更多共享内存，因为线程和线程块负责执行更多工作。如果程序员使用过多寄存器或共享内存，占用率就可能下降。程序员必须谨慎权衡：降低占用率带来的性能损失，不能大于线程粗化可能带来的性能收益。

6.6 循环展开

回顾第一章，GPU 采用吞吐量导向设计，而不是延迟导向设计；GPU 核心被优化为并发执行大量线程，代价是单个线程会经历较长延迟。第四章讨论过一种重要的延迟容忍优化，即最大化 SM 上的线程占用率。最大化占用率可以确保：当 warp 因等待长延迟指令的输出而停顿时，很可能有另一个已经准备好的 warp 可以执行，从而让 SM 核心保持忙碌。本节讨论另一种重要的延迟容忍优化，即**循环展开 (loop unrolling)**。

循环展开是一种转换循环的优化：按某个因子复制循环体，同时将循环迭代次数减少同样的因子。例如，考虑下面的循环：

```
1 for(unsigned int i = 0; i < 16; ++i) {  
2     A(i);  
3     B(i);  
4 }
```

其中，A(i) 和 B(i) 表示循环第 i 次迭代执行的某些操作或指令。如果以 4 为展开因子，循环会转换为：

```
1 for(unsigned int i = 0; i < 16; i += 4) {  
2     A(i);  
3     B(i);  
4     A(i + 1);  
5     B(i + 1);  
6     A(i + 2);  
7     B(i + 2);  
8     A(i + 3);  
9     B(i + 3);  
10 }
```

循环展开有两个与延迟容忍相关的优点。第一个优点是，减少循环迭代次数会减少分支指令数量。CPU 拥有复杂的分支预测机制来降低分支指令延迟，而 GPU 没有这样复杂的分支预测。当 GPU 线程遇到分支指令时，会停顿，直到分支结果确定。循环展开减少分支指令数量，可以降低这种停顿。

第二个优点是，循环展开暴露出更多可以重新排序的独立指令，从而避免停顿。在上面的例子中，如果 B(i) 依赖 A(i)，线程执行 A(i) 后必须停顿，等待 A(i) 完成，才能执行 B(i)。如果还有不必等待 A(i) 的其他操作，就可以把这些操作放到 A(i) 和 B(i) 之间，以避免停顿。例如，如果 A(i + 1) 独立于 A(i) 和 B(i)，则展开后的循环体可以进一步转换为：

```
1 for(unsigned int i = 0; i < 16; i += 4) {  
2     A(i);  
3     A(i + 1);  
4     A(i + 2);  
5     A(i + 3);  
6     B(i);  
7     B(i + 1);  
8     B(i + 2);  
9     B(i + 3);  
10 }
```

把 A(i + 1)、A(i + 2) 和 A(i + 3) 移到 A(i) 与 B(i) 之间，可以确保线程在等待 A(i) 完成、准备执行 B(i) 时仍有足够指令可执行。以这种方式重新排列指令来避免停顿，是一种重要优化，称为**指令调度 (instruction scheduling)**。

在实践中，循环展开和指令调度会由编译器自行决定，并且通常会积极执行，程序员不必为此操心。在上面的例子中，循环体很小，循环边界也是已知且很小的常量，编译器很可能完全展开该循环，消除所有分支，并暴露最多的可调度指令。如果程序员希望控制循环展开，可以在循环前放置 `\#pragma unroll N` 指令，要求编译器按因子 `N` 展开循环。例如：

```
1 #pragma unroll 4
2 for(unsigned int i = 0; i < 16; ++i) {
3     A(i);
4     B(i);
5 }
```

在这段代码中，循环只会按因子 4 展开。把展开因子设为 1，则要求编译器不执行循环展开。在全书中，除非特别需要强调某个循环应当展开，否则我们不会在循环上标注 `\#pragma unroll` 指令。

一类常能从循环展开中受益的循环，是应用线程粗化的循环（见第 6.5 节）。循环展开对于高效实现线程粗化至关重要。例如，考虑下面的代码：

```
1 int x;
2 foo(x);
```

在这段代码中，每个线程负责处理一个放在寄存器中的变量 `x`。如果对它应用线程粗化，每个线程就要负责多个 `x`。假设粗化因子为 4，粗化后的代码可以写成：

```
1 int x[4];
2 for(unsigned int c = 0; c < 4; ++c) {
3     foo(x[c]);
4 }
```

然而，回顾第五章，像 `x[4]` 这样的局部数组默认放在全局内存中。数组必须放在能够用变量索引访问的内存中，因为代码通过 `x[c]` 进行访问。不幸的是，把 `x[4]` 放在全局内存中可能引入内存访问开销，抵消线程粗化的收益。不过，线程粗化循环的循环边界很小且为常量（这对粗化循环很典型），因此可以完全展开。完全展开后，代码如下：

```
1 int x[4];
2 foo(x[0]);
3 foo(x[1]);
4 foo(x[2]);
5 foo(x[3]);
```

循环展开后，对局部数组 `x` 的所有访问都使用常量索引，而不是变量索引，因而编译器可以把局部数组 `x` 从内存提升到寄存器。因此，循环展开不仅减少分支、暴露

更多可调度指令，还暴露了传播常量以及把变量提升到寄存器以实现更高效访问的机会。

6.7 双缓冲

回顾第五章，同一线程块中的线程使用 `__syncthreads()` 执行屏障同步，以协调对共享内存的访问。考虑下面的代码：同一线程块中的线程在计算过程中彼此传递数据。

```
1 for(unsigned int i = 0; i < N; ++i) {  
2     ... = buffer[anotherThreadID];  
3     __syncthreads();  
4     buffer[myThreadID] = ...;  
5     __syncthreads();  
6 }
```

在每次循环迭代中，每个线程从一个先前由另一个线程写入的内存位置读取值，然后把一个新值写入与自己 ID 对应的内存位置。这个新值可能在后续迭代中被其他线程读取。上面的代码包含两次屏障同步，以保证正确执行。第二次屏障同步确保所有线程都完成向内存写值之后，其他线程才读取这些值。换句话说，该屏障强制执行 **读后写依赖 (read-after-write dependence)**。这种依赖也称为 **真依赖 (true dependence)**，因为它不可避免：必须先写入一个值，才能读取它。

另一方面，第一次屏障同步确保所有线程都完成当前所需的读操作，其他线程才能在下次迭代中覆盖这些值。换句话说，该屏障强制执行 **写后读依赖 (write-after-read dependence)**。这种依赖也称为 **假依赖 (false dependence)**，因为实际上不必等待读取一个值之后再写入另一个值。之所以被迫等待，只是因为被读取的值和要写入的值占据同一个内存位置。如果这两个值不占据同一个内存位置，写后读依赖（即假依赖）就会消失，第一次屏障同步也就不需要了。

基于这一观察，可以在不同循环迭代中使用不同的内存缓冲区，从而消除不必要的屏障同步。一种方法是每次迭代分配一个不同的缓冲区，但这需要大量内存资源，并不理想。更好的方法是只分配两个缓冲区，并在每次迭代之间交替使用它们。这种优化称为 **双缓冲 (double-buffering)**，因为总共只需分配两个不断交换角色的缓冲区。

下面的代码展示了如何把双缓冲应用到上面的例子：

```
1 for(unsigned int i = 0; i < N; ++i) {  
2     ... = inBuffer[anotherThreadID];  
3     outBuffer[myThreadID] = ...;  
4     __syncthreads();  
5     swap(inBuffer, outBuffer);  
6 }
```

在这个例子中，分配了两个不同的缓冲区，一个由 `inBuffer` 指向，另一个由 `outBuffer` 指向。由于从 `inBuffer` 读取、向 `outBuffer` 写入，写操作不会覆盖其他线程正在读取的值。因此，可以删除分隔这两次内存访问的 `__syncthreads()` 调用。不过，写操作完成后，最新数据位于 `outBuffer`，下一次迭代应当从这个缓冲区读取。在循环体末尾交换两个缓冲区，就能保证下一次迭代从正确的缓冲区读取。本例只是一个玩具示例；在全书中还会看到把双缓冲应用到真实内核的更具体例子。

译者注：底本在“消除哪一次屏障”的表述中写作“消除第二次屏障”，但给出的双缓冲代码实际删除的是读操作与写操作之间的第一次屏障，并保留写入后用于跨线程同步的第二次屏障。本译文按代码和后续解释的实际含义表述。

6.8 优化检查清单

在本书第一部分中，我们已经介绍了 CUDA 程序员为改善代码性能而采用的各种常见优化。图 6.13 把这些优化汇总为一份检查清单。这份清单并不穷尽所有可能的优化，但包含了不同应用中普遍存在、程序员应当优先考虑的许多通用优化。在本书第二、第三部分中，我们会把清单中的优化应用到各种并行模式和应用中，以理解它们在不同上下文中的具体表现。本节先简要概览每种优化，以及应用它们的策略。

图 6.13 中的优化按照其试图改善的性能方面分为三大类。计算利用率类优化旨在减少 GPU 核心空闲，让核心始终有有用工作可做。内存利用率类优化旨在提高内存子系统访问数据的效率。同步延迟类优化旨在减少线程等待屏障同步的需要。某些优化可能同时属于多个类别；这里把它们归入通常使用时的主要目标类别。最后一种优化“线程粗化”标为一般优化，因为它所属的类别取决于应用场景。

计算利用率类包含三种主要优化。第一种是调节 SM 上线程的占用率。第四章介绍过，让线程数远多于核心数，可以提供足够多的工作来隐藏核心流水线中的长延迟操作。为了最大化占用率，程序员可以调节内核的资源使用，确保每个 SM 允许的最大线程块数量或寄存器数量不会限制同时分配到 SM 的线程数量。第五章还介绍了共享内存，它也是一种需要谨慎调节使用量、以免限制占用率的资源。本章第 6.2 节讨论过，最大化占用率不仅可以隐藏核心流水线延迟，也有助于隐藏内存延迟。让更多线程同时执行，可以产生足够多的内存访问，充分利用内存带宽。

在全书中，我们会应用各种增加内核共享内存或寄存器使用量的优化，并假设程

类别	优化	对计算核心的收益	对内存的收益	主要策略
计算利用率	占用率调优	更多工作可隐藏流水线延迟	更多并行内存访问可隐藏 DRAM 延迟	调整每块线程数、每块共享内存和每线程寄存器的使用量
计算利用率	循环展开	减少分支指令，暴露更多可独立调度的指令序列	可能把局部数组索引转成常量，减少内存流量	优先使用常量边界的循环；通常由编译器完成
计算利用率	减少控制流分歧	提高 SIMD 执行效率，减少空闲核心	—	重新安排线程到工作和/或数据的分配；重新安排数据布局
内存利用率	使用合并的全局内存访问	减少等待全局内存访问的流水线停顿	减少全局内存流量，更充分利用突发和缓存行	重新安排线程到数据的映射；重新安排数据布局；以合并方式搬入共享内存后在共享内存中执行不规则访问（例如转角、打包）
内存利用率	共享内存分块	减少等待全局内存访问的流水线停顿	减少全局内存流量	把线程块内重用的数据放入共享内存，使其只在全局内存与 SM 之间传输一次
内存利用率	寄存器分块	减少等待内存访问的流水线停顿	减少共享内存流量	把 warp 或线程内重用的数据放入寄存器，使其只在共享内存与寄存器之间传输一次
内存利用率	向量加载和存储	减少管理内存访问的指令	更多并行内存访问可隐藏 DRAM 延迟	让每个线程一次加载 16 B 的连续数据块
内存利用率	避免共享内存 bank 冲突	减少等待共享内存访问的流水线停顿	减少共享内存流量并提高 bank 利用率	重新安排线程到数据的映射；重新安排数据布局（例如加入填充）
内存利用率	私有化（后文介绍）	减少等待原子更新的流水线停顿	减少竞争并串行化原子更新	先对数据的私有副本执行部分更新，完成后再更新公共副本
同步延迟	Warp 级原语	减少线程块范围的同步	减少全局和共享内存流量	先在 warp 级别执行需要屏障的操作，再在线程块级别汇总结果
同步延迟	双缓冲	消除强制假依赖的屏障	—	用不同缓冲区分别保存写入值和前一轮读取值
一般优化	线程粗化	取决于并行化开销	取决于并行化开销	给每个线程分配多个并行工作单元，避免不必要的并行化开销

图 6.13: 常见性能优化检查清单（依据原书图 6.13 重绘）

程序员能够适当调节这些资源的使用，使其不会不利地影响占用率。第 15 章将看到一个例子：使用更多资源所带来的收益可能超过占用率下降的代价；同时，还会用其他优化改善计算利用率，以补偿占用率下降。

提高计算利用率的第二种优化是循环展开。本章第 6.6 节说明，循环展开可以减少长延迟分支指令的数量，暴露更多可调度的独立指令以减少停顿，还可能把局部数组索引转化为常量，使数组能够提升到寄存器。全书中，我们会假设编译器能够在有利可图时恰当地展开循环。第 11 章介绍前缀和模式，第 15 章介绍矩阵乘法的高级优化；在这两章中，我们会强调循环展开对于在线程粗化后启用寄存器分块的重要性。第 15 章还会说明，内核以低占用率执行时，循环展开对于补偿性能损失至关重要。

提高计算利用率的第三种优化是减少控制流分歧。第四章介绍过控制流分歧，并强调同一 warp 中线程采取相同控制路径，才能保证 SIMD 执行期间所有核心都有效工作。到目前为止，本书第一部分中的内核除了边界条件处不可避免的分歧外，都没有表现出控制流分歧。不过，在第二、第三部分的许多例子中，控制流分歧会成为性能的显著损害因素。

有多种策略可以减少控制流分歧。一种策略是重新安排工作和/或数据在线程之间的分配，确保先使用一个 warp 中的所有线程，再使用其他 warp 中的线程。第 10 章介绍归约模式时会看到这种策略。重新安排工作和/或数据的分配方式，也可以确保同一 warp 中的线程具有相近的工作量。第 18 章介绍图遍历时，会讨论以顶点为中心和以边为中心的并行化方案之间的权衡。另一种减少控制流分歧的策略，是重新安排数据布局，确保处理相邻数据的同一 warp 线程具有相近工作量。第 17 章介绍稀疏矩阵计算和存储格式时，尤其是在讨论 JDS 格式时，会看到这种策略。

图 6.13 中的内存利用率类包含六种主要优化。第一种是使用合并的全局内存访问：确保同一 warp 中的线程在同一个缓存行和/或 DRAM 突发内访问相邻的内存位置。本章第 6.1 节介绍过这种优化，重点是硬件能够把对相邻内存位置的访问合并为单个内存请求，从而减少全局内存流量，提高 DRAM 突发和缓存行的利用率。到目前为止，本书第一部分中的内核都自然地产生合并访问；不过，第二、第三部分中会看到更多不规则内存访问模式，需要额外努力才能实现合并。

对于访问模式不规则的应用，可以采用多种策略实现合并。一种策略是以合并方式把数据从全局内存加载到共享内存，然后在共享内存中执行不规则访问。第 6.1 节已经通过转角看到了这种策略；第 12 章介绍过滤模式、第 14 章介绍排序模式时还会看到其他例子。在这些模式中，线程会以分散方式把结果写入数组；相反，线程可以协作在共享内存中执行分散访问，然后把共享内存中的结果写回全局内存，并让目标地址相近的元素以更合并的方式写出。这种优化有时称为**打包 (packing)**。第 13 章介绍归并模式时也会看到该策略：同一线程块中的线程需要在同一个数组中执行二分搜索，因此它们先以合并方式把数组加载到共享内存，然后每个线程在共享内存中执行二分搜索。

在访问模式不规则的应用中，实现合并的另一种策略，是重新安排线程到数据元素的映射方式。第 10 章介绍归约模式、第 15 章介绍矩阵乘法的高级优化时，会看到这种策略。还有一种策略是重新安排数据本身的布局。第 17 章介绍稀疏矩阵计算和存储格式时，尤其在讨论 ELL 和 JDS 格式时，会看到这种策略。重新安排数据布局的其他常见例子包括转置矩阵，或把结构体数组转换为数组结构体。

提高内存利用率的第二种优化，是把线程块内部会重用的数据分块放入共享内存，并反复从共享内存访问，使这些数据只需在全局内存与 SM 之间传输一次。第五章在矩阵乘法上下文中介绍过共享内存分块：处理同一个输出瓦片的线程协作，把相应的输入瓦片加载到共享内存，然后反复从共享内存访问这些输入瓦片。在第二、第三部分的大多数并行模式中，还会再次应用共享内存分块。我们会看到，当输入瓦片和输出瓦片尺寸不同时，分块会遇到挑战；第 7 章的卷积模式和第 8 章的模板模式都会遇到这种情况。第 15 章会把共享内存分块更复杂地应用于矩阵乘法，第 16 章会把它应用于动态规划中的序列比对。

提高内存利用率的第三种优化，是把 warp 或线程内部会重用的数据分块放入寄存器，并反复从寄存器访问，使这些数据只需在全局内存或共享内存与寄存器之间传输一次。第三章和第五章已经应用过寄存器分块：每个线程使用局部变量或寄存器累积结果，然后再写回全局内存。可以把所有线程的寄存器集合看作共同组成输出数据的一个瓦片。第 8 章会再次看到寄存器分块：共享内存输入瓦片暂时移动到寄存器，再从寄存器移回，以实现高效访问并节省共享内存容量。第 11 章介绍前缀和模式、第 15 章介绍矩阵乘法高级优化时，也会再次使用寄存器分块；在这些例子中，每个线程负责多个输出元素，并重用多个对应的输入元素。

提高内存利用率的第四种优化，是使用向量加载和存储，以较少的指令执行开销访问更大的内存块。本章第 6.3 节已经在向量加法中介绍了这种优化。它可以应用于本书讨论的大多数计算。由于这种优化很简单，我们不会在全书中逐章应用，只在它特别相关时简要提及。

提高内存利用率的第五种优化，是避免共享内存 bank 冲突。本章第 6.4 节已经在转角上下文中介绍了这种优化。避免 bank 冲突的主要策略，是重新安排线程到数据的映射，或重新安排数据布局，通常会加入填充元素。第 11 章介绍前缀和、第 15 章介绍矩阵乘法高级优化时，会再次看到这种优化；在这些例子中，线程加载输入数据各自负责的部分时，会以跨步方式访问共享内存。

提高内存利用率的第六种优化，是**私有化 (privatization)**。本章还没有正式介绍这种优化，这里为了完整性先提及。私有化用于多个线程或线程块需要更新同一个公共输出的情况。为避免并发更新同一数据的开销，可以创建数据的私有副本，先对私有副本执行部分更新，完成后再从私有副本对公共副本执行最终更新。第 9 章介绍直方图模式时，会看到多个线程更新同一组直方图计数器的私有化例子；第 12 章介绍过滤模式、第 18 章介绍图遍历时，也会看到多个线程向同一个输出列表插入条目

时的私有化例子。

图 6.13 中的同步延迟类包含两种主要优化。第一种是使用 warp 级原语来减少同步延迟。第四章曾简要提到，warp 级原语能够让同一 warp 中的线程彼此同步并共享数据，而不必执行开销更高的线程块范围屏障同步，也不必访问共享内存。使用 warp 级原语降低同步开销的主要策略，是把线程块范围的计算分解到多个 warp 中：先在 warp 级别执行需要屏障同步的操作，再在线程块级别汇总 warp 级结果。第 10、11、12 章分别介绍归约、前缀和和过滤模式时，会看到使用 warp 级原语优化频繁同步的计算。第 15 章介绍矩阵乘法高级优化时，也会使用 warp 级原语辅助寄存器分块。

降低同步延迟的第二种优化是双缓冲。双缓冲在本章第 6.7 节首次介绍；它通过为写入和之前的读取使用不同缓冲区，消除假（写后读）依赖，从而可以删除强制这些假依赖的屏障同步。第 11 章介绍前缀和、第 15 章介绍矩阵乘法高级优化时，会在真实上下文中应用双缓冲。

图 6.13 中的最后一种优化是线程粗化。这是一种一般优化，其性能目标取决于具体计算。本章第 6.5 节首次介绍线程粗化：如果硬件最终会把线程串行执行，就把多个并行工作单元分配给同一线程，以降低并行化开销。在本书第二、第三部分中，会在不同上下文中看到线程粗化，每次要降低的并行化开销也不同。

第 8 章介绍模板模式、第 15 章介绍矩阵乘法高级优化时，线程粗化用于增大输出瓦片尺寸，从而重用已从内存加载、用于计算更多输出值的输入数据。在这种情况下，使用更小输出瓦片、暴露更细粒度并行性所带来的开销，是更多线程块重复加载输入数据。第 9 章介绍直方图模式时，线程粗化有助于减少私有化优化中需要提交到公共副本的私有副本数量。第 10 章介绍归约模式、第 11 章介绍前缀和模式时，线程粗化用于减少同步和控制流分歧开销；第 11 章还会用线程粗化减少并行算法相对于顺序算法所执行的重复工作。第 12 章介绍过滤模式、第 14 章介绍排序模式时，线程粗化有助于改善内存访问合并。

再次强调，图 6.13 的检查清单并不穷尽所有优化；它包含了不同计算模式中常见的主要优化类型。这些优化会在第二、第三部分的多个章节中出现。我们还会看到只在特定章节出现的其他优化。例如，第 7 章介绍卷积模式时，会介绍常量内存的使用。

6.9 优化策略

第 6.8 节说明，不同优化针对性能的不同方面。因此，决定对某个内核应用哪种优化时，首先必须理解究竟是哪种资源限制了该内核的性能。限制内核性能的资源通常称为**性能瓶颈**（performance bottleneck）。优化通常会增加一种资源的使用，以减轻另一种资源的负担。如果应用的优化没有针对瓶颈资源，可能看不到任何收益；更糟的是，如果该优化增加了瓶颈资源的使用，性能甚至可能下降。

例如，共享内存分块会增加共享内存的使用，以减轻全局内存带宽压力。当内核的瓶颈资源是全局内存带宽时，这种优化非常有效。然而，如果内核性能受占用率限制，并且占用率已经受过多共享内存使用限制，那么应用共享内存分块很可能使情况更糟。

性能优化是一个迭代过程：识别内核的性能瓶颈，应用优化缓解该瓶颈；然后识别新的瓶颈，继续缓解它，如此反复。GPU 计算平台通常提供各种性能分析工具，用于识别内核的性能瓶颈。关于如何使用性能分析工具，读者可以参考 CUDA 文档 [1]。性能瓶颈可能与硬件有关，也就是说，同一个内核在不同设备上可能遇到不同瓶颈。因此，识别性能瓶颈并应用性能优化，需要深入理解 GPU 架构，以及不同 GPU 设备之间的架构差异。

一项重要技能，是知道何时停止迭代优化，因为内核性能可能已经无法进一步改善。第五章把 Roofline 模型作为评估内核距离硬件上限还有多远的工具。通过估计内核执行的计算量和从内存访问的数据量，可以得到内核的计算强度，并据此判断内核是计算受限还是内存受限。随后，可以使用 Roofline 模型，把性能分析器报告的内核实际资源利用率与硬件上限进行比较。接近硬件上限的内核，进一步优化的空间很小；尚未达到硬件上限的内核，则很可能仍然可以继续优化。

6.10 小结

本章介绍了 GPU 的片外内存 (DRAM) 架构，并讨论了相关性能考量，例如全局内存访问合并，以及利用内存并行性隐藏内存延迟。随后，我们介绍了几种重要优化：线程粒度粗化、循环展开和双缓冲。结合本章和前几章的知识，读者应当能够分析所遇到的任意内核代码的性能行为。最后，我们给出了一份广泛用于优化各种计算的常见性能优化检查清单。在本书接下来的两部分中，我们将继续研究这些优化在并行计算模式和应用案例中的实际运用。

练习

1. 考虑下面的 CUDA 内核：


```

1 __global__ void foo_kernel(float* a, float* b, float* c,
2                             float* d, float* e) {
3     unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
4     __shared__ float a_s[256];
5     __shared__ float bc_s[4*256];
6     a_s[threadIdx.x] = a[i];
7     for (unsigned int j = 0; j < 4; ++j) {
8         bc_s[j*256 + threadIdx.x] =
9             b[j*blockDim.x*gridDim.x + i] + c[i*4 + j];
10    }
11    __syncthreads();
12    d[i + 8] = a_s[threadIdx.x];
13    e[i*8] = bc_s[threadIdx.x*4];
14 }

```

对下面每个内存访问，说明它是合并的、未合并的，还是不适用合并概念：

- a. 第 05 行对数组 `a` 的访问；
 - b. 第 05 行对数组 `a_s` 的访问；
 - c. 第 07 行对数组 `b` 的访问；
 - d. 第 07 行对数组 `c` 的访问；
 - e. 第 07 行对数组 `bc_s` 的访问；
 - f. 第 10 行对数组 `a_s` 的访问；
 - g. 第 10 行对数组 `d` 的访问；
 - h. 第 11 行对数组 `bc_s` 的访问；
 - i. 第 11 行对数组 `e` 的访问。
2. 编写一个使用转角的矩阵乘法内核函数，设计应对应图 6.4。
 3. 对第五章的分块矩阵乘法，在 `BLOCK_SIZE` 的可能取值范围内，哪些 `BLOCK_SIZE` 值能让内核完全避免对全局内存的未合并访问？（只需考虑正方形线程块。）
 4. 实现一个使用向量加载的向量加法内核，并正确处理边界条件。
 5. 考虑下面的共享内存数组，以及一个包含一个 warp（32 个线程）的线程块对该数组的访问：

```

1 __shared__ float a[1024];
2 a[threadIdx.x*stride] = ...;

```

假设 `a[0]` 位于 bank 0。对于下面每个 `stride` 值，指出该 warp 访问了哪些 bank，以及每个 bank 上的 bank 冲突数量（如果有）：

- a. `stride = 32;`
- b. `stride = 31;`
- c. `stride = 24;`
- d. `stride = 16;`
- e. `stride = 12;`
- f. `stride = 8;`
- g. `stride = 7。`

参考文献

- [1] NVIDIA, *Nsight Compute*, 2023. <https://docs.nvidia.com/nsight-compute/index.html>.

第二部分

并行模式

第七章 卷积

接下来的几章将讨论一组重要的并行计算模式。这些模式是许多并行应用中大量并行算法的基础。我们从卷积开始。卷积是一种常见的数组运算，广泛用于信号处理、数字录音、图像处理、视频处理和计算机视觉。在这些应用领域中，卷积通常作为一种滤波器，把信号和像素转换为更理想的值。本书前面介绍过的图像模糊内核就是一种滤波器，它平滑信号值，使人能够看清整体趋势。另一个例子是高斯滤波器：它们是卷积滤波器，可用于增强图像中物体的边界和边缘。

卷积通常需要执行大量算术运算，才能生成一个输出元素。对于高分辨率图像和视频等大型数据集，输出元素（像素）数量很多，因此计算量可能非常大。一方面，卷积计算中的输出数据元素可以彼此独立地计算，这是并行计算所希望看到的特征。另一方面，在处理不同输出数据元素时，输入数据之间存在大量共享，而且边界条件相对棘手。这些性质使卷积成为应用复杂分块方法和输入数据暂存方法的重要场景，而这正是本章的重点。

7.1 背景

卷积是一种数组运算：每个输出元素都是相应输入元素与其周围一组输入元素的加权和。加权求和所使用的权重由一个滤波器数组定义，通常称为卷积核。由于 CUDA 内核函数和卷积核之间存在一个容易混淆的名称冲突，下面将这些滤波器数组统一称为卷积滤波器（convolution filter），以避免混淆。

卷积可以作用于不同维度的输入数据：一维数据（例如音频）、二维数据（例如照片）、三维数据（例如视频）等。在音频数字信号处理中，一维数组元素是随时间采样得到的信号幅度；也就是说，输入数据元素 x_i 是音频信号幅度的第 i 个样本。对一维数据执行的卷积称为一维卷积，其数学定义是：给定一个含有 n 个元素的输入数组

$$[x_0, x_1, \dots, x_{n-1}]$$

和一个含有 $2r + 1$ 个元素的滤波器数组

$$[f_0, f_1, \dots, f_{2r}],$$

返回输出数组 y ，其中

$$y_i = \sum_{j=-r}^r f_{j+r} \times x_{i+j}.$$

由于滤波器的大小是奇数 $2r + 1$ ，加权求和相对于正在计算的元素对称。也就是说，加权求和会包含计算位置两侧各 r 个输入元素；这也是把 r 称为滤波器半径 (filter radius) 的原因。

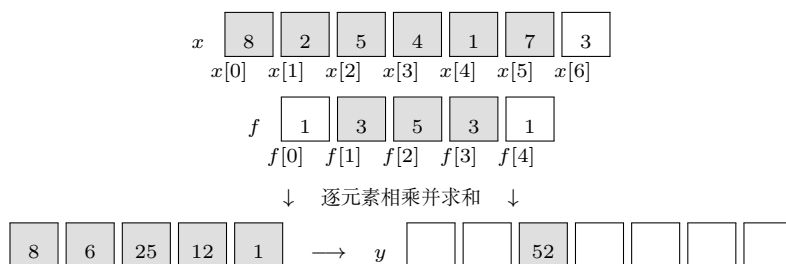


图 7.1: 一维卷积的内部元素示例 (依据原书图 7.1 重绘)

图 7.1 展示了一个一维卷积示例：把含有 5 个元素 ($r = 2$) 的卷积滤波器 f 应用于含有 7 个元素的输入数组 x 。下面遵循 C++ 语言约定： x 和 y 的元素索引从 0 到 6， f 的元素索引从 0 到 4。由于滤波器半径为 2，每个输出元素都是相应输入元素、左侧两个元素和右侧两个元素的加权和。

例如， $y[2]$ 的值是 $x[0]$ (即 $x[2 + (-2)]$) 到 $x[4]$ (即 $x[2 + 2]$) 的加权和。在这个例子中，输入数组元素的值任意取为

$$[8, 2, 5, 4, 1, 7, 3],$$

滤波器元素给出的权重为

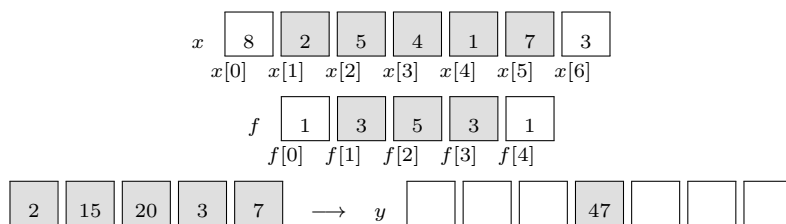
$$[1, 3, 5, 3, 1].$$

每个滤波器元素先与对应的 x 元素相乘，再把乘积相加。如图 7.1 所示， $y[2]$ 的计算为

$$\begin{aligned} y[2] &= f[0] \times x[0] + f[1] \times x[1] + f[2] \times x[2] \\ &\quad + f[3] \times x[3] + f[4] \times x[4] \\ &= 1 \times 8 + 3 \times 2 + 5 \times 5 + 3 \times 4 + 1 \times 1 \\ &= 52. \end{aligned}$$

在图 7.1 中， $y[i]$ 的计算可以看成从 $x[i - 2]$ 开始的 x 子数组与 f 数组之间的内积¹。图 7.2 展示了 $y[3]$ 的计算。与图 7.1 相比，这次计算沿 x 数组移动了一个元素，即 $y[3]$ 是 $x[1]$ (即 $x[3 + (-2)]$) 到 $x[5]$ (即 $x[3 + 2]$) 的加权和。可以把它写成如下

¹由于卷积的每个输出元素都可以看成 x 的一个子数组与 f 数组之间的内积，读者可能会思考能否用矩阵乘法表述卷积。确实存在这种表述，第 19 章 (卷积神经网络) 会对此进行讨论。

图 7.2: 一维卷积中 $y[3]$ 的计算 (依据原书图 7.2 重绘)

内积:

$$\begin{aligned}
 y[3] &= f[0] \times x[1] + f[1] \times x[2] + f[2] \times x[3] \\
 &\quad + f[3] \times x[4] + f[4] \times x[5] \\
 &= 1 \times 2 + 3 \times 5 + 5 \times 4 + 3 \times 1 + 1 \times 7 \\
 &= 47.
 \end{aligned}$$

由于卷积是根据相邻元素定义的, 在计算靠近数组两端的输出元素时, 自然会出现边界条件。如图 7.3 所示, 当计算 $y[1]$ 时, $x[1]$ 左侧只有一个 x 元素。按照卷积定义, 输入元素数量不足。处理这种边界条件的典型方法, 是为缺失的 x 元素定义一个默认值。在大多数应用中, 默认值是 0; 图 7.3 也采用这一约定。例如, 在音频信号处理中, 可以假设录音开始前和结束后的信号幅度为 0。此时

$$\begin{aligned}
 y[1] &= f[0] \times 0 + f[1] \times x[0] + f[2] \times x[1] \\
 &\quad + f[3] \times x[2] + f[4] \times x[3] \\
 &= 1 \times 0 + 3 \times 8 + 5 \times 2 + 3 \times 5 + 1 \times 4 \\
 &= 53.
 \end{aligned}$$

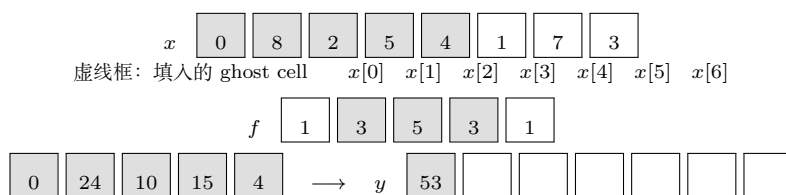


图 7.3: 一维卷积的边界条件 (依据原书图 7.3 重绘)

图 7.3 中不存在的 x 元素用虚线框表示。显然, 计算 $y[0]$ 时会涉及两个缺失的 x 元素, 在本例中二者都假设为 0; $y[0]$ 的计算留给读者作为练习。这些缺失元素在文献中通常称为“幽灵单元”(ghost cell)。并行计算中使用分块时, 还会因为其他原因产生 ghost cell。这些 ghost cell 可能显著影响分块的有效性和效率, 稍后还会回到这一点。

并非所有应用都假设 ghost cell 的值为 0。有些应用可能假设每个 ghost cell 都包含边缘最近的有效数据元素的值; 另一些应用可能通过把输入数组的副本与原数组首尾连接起来, 构造输入数组的循环视图, 从而推导 ghost cell 的值。

在图像处理和计算机视觉中，输入数据通常表示为 x - y 空间中的二维数组，像素位于其中。因此，图像卷积是二维卷积，如图 7.4 所示。二维卷积中的滤波器 f 也是二维数组。滤波器在 x 和 y 方向上的尺寸，决定了加权和中包含的邻居范围。假设滤波器在 x 方向的尺寸为 $2r_x + 1$ ，在 y 方向的尺寸为 $2r_y + 1$ ，则每个 P 元素的计算为

$$P_{y,x} = \sum_{j=-r_y}^{r_y} \sum_{k=-r_x}^{r_x} f_{j+r_y, k+r_x} \times N_{y+j, x+k}.$$

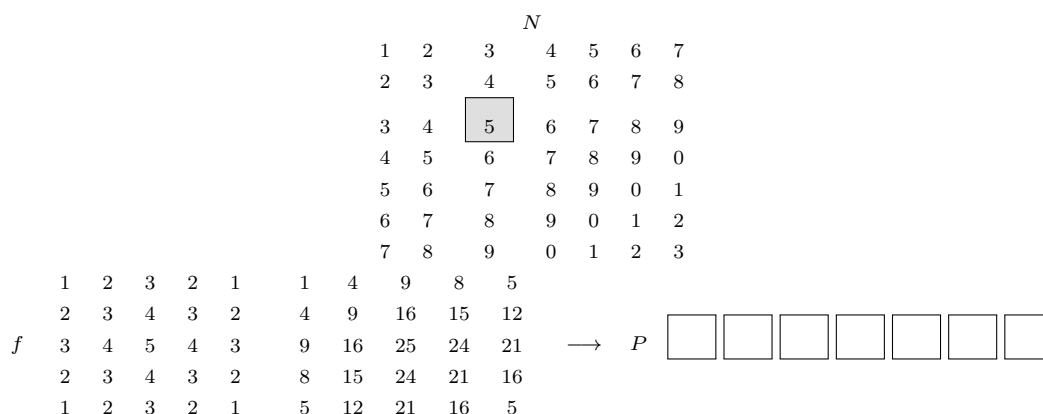


图 7.4: 二维卷积示例（依据原书图 7.4 重绘）

为简洁起见，下面用 $N_{y,x}$ 表示 C++ 数组寻址中的 $N[y][x]$ 。由于 N 和 P 很可能是动态分配的数组，实际代码示例将使用线性化索引。图 7.4 展示 $P_{2,2}$ 的计算。计算结果如下：

$$\begin{aligned} P_{2,2} &= N_{0,0} \times f_{0,0} + N_{0,1} \times f_{0,1} + N_{0,2} \times f_{0,2} + N_{0,3} \times f_{0,3} + N_{0,4} \times f_{0,4} \\ &\quad + N_{1,0} \times f_{1,0} + N_{1,1} \times f_{1,1} + N_{1,2} \times f_{1,2} + N_{1,3} \times f_{1,3} + N_{1,4} \times f_{1,4} \\ &\quad + N_{2,0} \times f_{2,0} + N_{2,1} \times f_{2,1} + N_{2,2} \times f_{2,2} + N_{2,3} \times f_{2,3} + N_{2,4} \times f_{2,4} \\ &\quad + N_{3,0} \times f_{3,0} + N_{3,1} \times f_{3,1} + N_{3,2} \times f_{3,2} + N_{3,3} \times f_{3,3} + N_{3,4} \times f_{3,4} \\ &\quad + N_{4,0} \times f_{4,0} + N_{4,1} \times f_{4,1} + N_{4,2} \times f_{4,2} + N_{4,3} \times f_{4,3} + N_{4,4} \times f_{4,4} \\ &= 1 \times 1 + 2 \times 2 + 3 \times 3 + 4 \times 2 + 5 \times 1 \\ &\quad + 2 \times 2 + 3 \times 3 + 4 \times 4 + 5 \times 3 + 6 \times 2 \\ &\quad + 3 \times 3 + 4 \times 4 + 5 \times 5 + 6 \times 4 + 7 \times 3 \\ &\quad + 4 \times 2 + 5 \times 3 + 6 \times 4 + 7 \times 3 + 8 \times 2 \\ &\quad + 5 \times 1 + 6 \times 2 + 7 \times 3 + 8 \times 2 + 5 \times 1 \\ &= 321. \end{aligned}$$

图 7.4 中的例子使用一个 5×5 滤波器，即 $r_y = 2$ 、 $r_x = 2$ 。一般而言，滤波器不一定是方阵，但通常是方阵。为了生成一个输出元素，我们取输入数组 N 中以对应位置为中心的子数组，然后把滤波器数组元素与图像数组元素逐一相乘。图 7.4 中 N 和 P 下方给出了 5×5 乘积数组；输出元素就是乘积数组中所有元素之和。

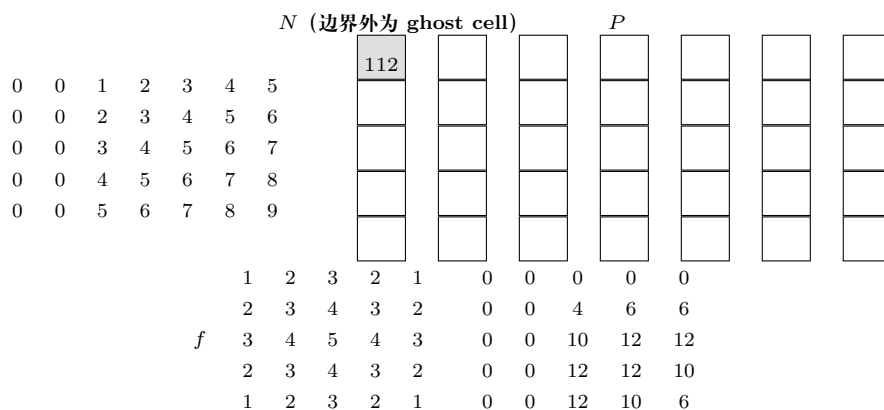


图 7.5: 二维卷积的边界条件 (依据原书图 7.5 重绘)

图 7.5 展示了二维卷积的边界条件。与一维卷积类似，二维卷积也必须处理边界；但由于 x 和 y 两个方向都有边界，情况更复杂：输出元素的计算可能涉及水平边界、垂直边界，或同时涉及二者。图 7.5 中 $P_{1,0}$ 的计算涉及输入子数组中缺失的两列和一行。与一维卷积一样，不同应用会对缺失的 N 元素采用不同默认值；本例假设默认值为 0。这些边界条件也会影响分块的效率，稍后还会回到这一点。

7.2 并行卷积——一个基本内核

卷积的所有输出元素都可以并行计算，因此卷积非常适合数据并行计算。借鉴矩阵乘法中的经验，我们可以快速写出一个简单的并行卷积内核。下面给出二维卷积的代码示例；读者可以把这些代码改写为一维和三维版本，作为练习。为简单起见，假设卷积滤波器是方阵。

第一步是定义内核的主要输入参数。我们假设二维卷积内核接收：输入数组 N 的指针、滤波器 F 的指针、输出数组 P 的指针、方形滤波器的半径 r ，以及输入和输出数组的宽度 `width` 和高度 `height`。因此，内核签名为：

```
1 __global__ void
2 convolution_2D_basic_kernel(float *N, float *F, float *P, int r,
3                             int width, int height) {
4     // kernel body
5 }
```

译者注：底本在这段话中说内核接收五个参数，但随后列出的形参实际为 N 、 F 、 P 、 r 、`width` 和 `height` 六项；译文按代码保留六个形参。

第二步是确定并实现线程到输出元素的映射。由于输出数组是二维的，一个简单而合理的方法是把线程组织成二维网格，让网格中的每个线程计算一个输出元素。每个线程块最多可以包含 1024 个线程，因此最多可以计算 1024 个输出元素，通常组织成 32×32 的方形。图 7.6 展示了一个玩具示例：输入和输出都是 16×16 图像，每

个线程块组织成 4×4 的线程数组，即 x 方向 4 个线程、 y 方向 4 个线程；整个网格组织成 4×4 的线程块数组。

在这个例子中，线程到输出元素（这里就是输出像素）的分配很简单：每个线程负责计算一个输出元素，其 x 和 y 索引分别与线程的 x 、 y 索引相同。

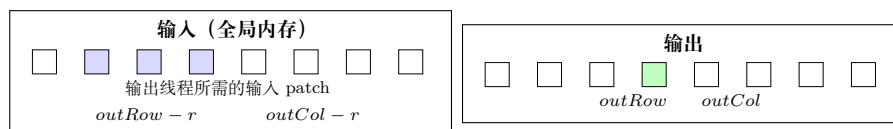


图 7.6: 二维卷积的并行化与线程组织（依据原书图 7.6 重绘）

读者应该能认出，图 7.6 中的并行化安排与第三章 ColorToGrayScaleConversion 示例相同。因此，可以使用图 7.7 内核第 2、3 行中的语句，根据每个线程的块索引、块维度和线程索引计算输出元素索引。例如，块 (1, 1) 中的线程 (1, 1) 被映射到输出元素

$$P_{1 \times 4 + 1, 1 \times 4 + 1} = P_{5,5},$$

该元素在图 7.6 右侧以阴影标出。

确定每个线程负责的输出元素索引后，就可以找出计算该输出元素所需的输入 N 元素。如图 7.6 所示，块 (1, 1) 中的线程 (1, 1) 计算 $P_{5,5}$ 时，使用的输入元素的 x 索引范围为

$$outCol - r = 5 - 2 = 3 \quad \text{到} \quad outCol + r = 5 + 2 = 7,$$

y 索引范围同样为 3 到 7。对所有线程而言， $outCol - r$ 和 $outRow - r$ 定义了计算 $P_{outRow, outCol}$ 所需输入元素 patch（图 7.6 左侧阴影方块）的左上角。因此，可以用双重嵌套循环遍历这些索引值并完成计算，即图 7.7 第 05–13 行的代码。

译者注：底本图 7.7 的参数 F 和 P 声明为一维指针，但代码体中使用了二维下标；本译稿按线性化存储约定改为一维索引。同时，为了适应宽度或高度不是线程块尺寸整数倍的输入，在图 7.7 和图 7.9 的输出写回前都增加了范围检查。

寄存器变量 `Pvalue` 累积所有中间结果，从而避免反复写回 DRAM。内层 `for` 循环中的 `if` 语句（第 09 行）检查所使用的输入 N 元素是否是 N 数组左、右、上、下边界之外的 ghost cell。由于假设 ghost cell 的值为 0，可以直接跳过该 ghost cell 元素及其对应滤波器元素的乘加操作。循环结束后，把 `Pvalue` 写入输出元素 P （第 15 行）。

从图 7.7 的内核可以得到两个观察。第一，第 09–10 行存在控制流分歧。负责计算靠近 P 数组四条边缘的输出元素的线程，需要处理 ghost cell；如第 7.1 节所示，这些线程遇到的 ghost cell 数量各不相同，因此会在 `if` 语句中作出不同决定。计算 $P_{0,0}$ 的线程大部分时间都会跳过乘加语句，而计算 $P_{0,1}$ 的线程跳过的次数较少，其他线程依此类推。控制流分歧的代价取决于输入数组的宽度和高度，以及滤波器半径。对于

```

1  __global__ void convolution_2D_basic_kernel(float *N, float *F, float *P,
2                                          int r, int width, int height) {
3      int outCol = blockIdx.x*blockDim.x + threadIdx.x;
4      int outRow = blockIdx.y*blockDim.y + threadIdx.y;
5      float Pvalue = 0.0f;
6      for (int fRow = 0; fRow < 2*r+1; fRow++) {
7          for (int fCol = 0; fCol < 2*r+1; fCol++) {
8              int inRow = outRow - r + fRow;
9              int inCol = outCol - r + fCol;
10             if (inRow >= 0 && inRow < height &&
11                 inCol >= 0 && inCol < width) {
12                 Pvalue += F[fRow*(2*r+1) + fCol]*
13                     N[inRow*width + inCol];
14             }
15         }
16     }
17     if (outRow < height && outCol < width) {
18         P[outRow*width + outCol] = Pvalue;
19     }
20 }
21

```

图 7.7: 带边界条件处理的二维卷积内核 (依据原书图 7.7 重绘)

大型输入数组和相对较小的滤波器，控制流分歧只发生在少量输出元素的计算中，因此影响较小。由于卷积经常用于大图像，我们预计控制流分歧的影响从中等到可以忽略不计。更重要的性能考量是内存带宽。

7.3 内存带宽考量

沿用第五章分析矩阵乘法时采用的方法，我们分析卷积的算术强度，并推理其对内存带宽的需求。为简单起见，假设输入数组是尺寸为 $n \times n$ 的方阵，滤波器是尺寸为 $m \times m$ 的方阵，其中

$$m = 2r + 1$$

且 r 是滤波器半径。假设数组和滤波器元素都是 32 位 (4 B) 单精度浮点值。

先计算卷积执行的算术运算数量。对于 $n \times n$ 个输出元素中的每一个，卷积操作都会遍历 $m \times m$ 滤波器，并对每个滤波器元素执行两次算术运算：一次乘法和一次加法。因此，卷积操作执行的算术运算总数为

$$2 \cdot n^2 \cdot m^2 \text{ FLOP.}$$

再计算卷积操作需要访问的内存数据量。至少需要加载卷积滤波器和输入数组，并存储输出数组。滤波器包含 m^2 个元素，输入和输出数组各包含 n^2 个元素。由于每个元素占 4 B，卷积需要访问的字节总数为

$$8 \cdot n^2 + 4 \cdot m^2.$$

由于滤波器通常远小于数组 ($m \ll n$), 可以忽略对滤波器的访问, 把访问字节数近似为 $8 \cdot n^2$ B。

用算术运算数量除以内存访问字节数, 可得卷积的算术强度:

$$\frac{2 \cdot n^2 \cdot m^2}{8 \cdot n^2} = \frac{1}{4} m^2 \text{ FLOP/B.}$$

换句话说, 一个完美的卷积实现只访问每个输入和输出元素一次; 它每访问一个字节内存, 就执行 $\frac{1}{4} m^2$ 次浮点运算。

一个有趣的观察是, 卷积的算术强度取决于卷积滤波器大小 m 。小滤波器的卷积算术强度低, 因此属于内存受限。例如, 使用 3×3 滤波器的卷积, 其算术强度为

$$\frac{1}{4} \cdot 3^2 = 2.25 \text{ FLOP/B,}$$

在 H100 GPU 上属于内存受限。由于性能受内存带宽限制, 这种计算不可能达到 GPU 的峰值计算吞吐量。另一方面, 大滤波器的卷积具有较高算术强度, 因此属于计算受限。例如, 使用 11×11 滤波器的卷积, 其算术强度为

$$\frac{1}{4} \cdot 11^2 = 30.25 \text{ FLOP/B,}$$

在 H100 GPU 上属于计算受限。如果采用合适的优化, 这种计算的性能可以接近 GPU 峰值计算吞吐量。

现在回到第 7.2 节图 7.7 中的基本卷积内核。在图中第 10 行, 每个线程在每次循环迭代中从全局内存加载两个元素 (8 B), 并对它们执行两次算术运算。忽略第 14 行的存储操作, 因为它位于循环嵌套之外, 在这个内核中影响很小。因此, 该实现的算术强度为

$$\frac{2 \text{ FLOP}}{8 \text{ B}} = 0.25 \text{ FLOP/B.}$$

无论滤波器大小如何, 这个算术强度都非常低。

实际算术强度会因为片上缓存而更高。相同的滤波器元素和输入元素会被多个线程访问, 很可能已经位于 L1 或 L2 缓存中。不过, 为了更可靠地获得较高算术强度、更接近设备峰值吞吐量, 程序员还可以使用共享内存或常量缓存等其他片上内存结构。第 7.4 节将讨论常量内存和常量缓存, 第 7.5 节将讨论共享内存分块。

7.4 常量内存与缓存

卷积中滤波器数组 F 的使用方式有三个值得注意的方面。第一, F 通常很小; 大多数卷积滤波器的半径不超过 7。即使在三维卷积中, 滤波器通常也只包含不超过

$$7^3 = 343$$

个元素。第二，在卷积内核执行期间， F 的内容不会改变。第三，所有线程都会访问滤波器元素；更好的是，所有线程都以相同顺序访问，从 $F_{0,0}$ 开始，在图 7.7 的双重嵌套 for 循环中每次向前移动一个元素。这三个方面使滤波器非常适合放入常量内存并使用缓存，如图 7.8 所示。

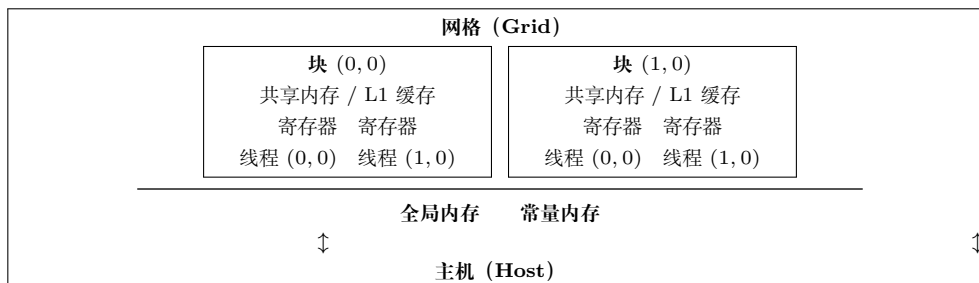


图 7.8: CUDA 内存模型回顾（依据原书图 7.8 重绘）

如第五章（图 5.4）所述，CUDA 允许程序员把变量声明为位于常量内存中。与全局内存变量一样，常量内存变量对所有线程块可见。主要区别是：在内核执行期间，线程不能修改常量内存变量的值。此外，常量内存的容量很小，目前为 64 KB。

使用常量内存时，主机代码需要以不同于全局内存变量的方式分配和复制常量内存变量。假设滤波器半径由编译期常量 `FILTER_RADIUS` 指定。要把滤波器数组 F 声明在常量内存中，主机代码把它声明为一个全局变量：

```
1 #define FILTER_RADIUS 2
2 __constant__ float F[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];
```

这个全局变量声明应当位于源文件中任何函数之外。关键字 `__constant__`（两侧各有两个下划线）告诉编译器，应当把数组 F 放入设备常量内存。

假设主机代码已经在主机内存数组 F_h 中分配并初始化了滤波器，其中包含

$$(2 \times \text{FILTER_RADIUS} + 1)^2$$

个元素。可以按如下方式把 F_h 的内容从主机内存传输到设备常量内存中的 F ：

```
1 cudaMemcpyToSymbol(F, F_h,
2     (2*FILTER_RADIUS + 1) *
3     (2*FILTER_RADIUS + 1) * sizeof(float));
```

这个 API 函数是一种特殊的内存复制函数，它告知 CUDA 运行时，复制到常量内存中的数据在内核执行期间不会改变。一般而言，它的用法是：

```
1 cudaMemcpyToSymbol(dest, src, size)
```

其中，`dest` 是常量内存中目标位置的指针，`src` 是主机内存中源数据的指针，`size` 是要复制的字节数。该函数还可以接受 `offset` 和 `kind` 两个参数，但它们很少使用，通常省略；详情请参阅 CUDA C++ 编程指南。

内核函数像访问全局变量一样访问常量内存变量，因此不需要把它们的指针作为参数传给内核。我们可以把内核修改为使用常量内存，如图 7.9 所示。注意，该内核看起来几乎与图 7.7 中的内核完全相同；唯一差别是， F 不再通过传入的指针访问，而是作为全局变量访问。这里仍然适用 C++ 语言关于全局变量作用域的规则。如果主机代码和内核代码位于不同文件中，内核代码文件必须包含相应的外部声明信息，以确保内核能够看见 F 的声明。

```

1  __global__ void convolution_2D_const_mem_kernel(float *N, float *P, int r,
2                                     int width, int height) {
3      int outCol = blockIdx.x*blockDim.x + threadIdx.x;
4      int outRow = blockIdx.y*blockDim.y + threadIdx.y;
5      float Pvalue = 0.0f;
6      for (int fRow = 0; fRow < 2*r+1; fRow++) {
7          for (int fCol = 0; fCol < 2*r+1; fCol++) {
8              int inRow = outRow - r + fRow;
9              int inCol = outCol - r + fCol;
10             if (inRow >= 0 && inRow < height &&
11                 inCol >= 0 && inCol < width) {
12                 Pvalue += F[fRow][fCol]*N[inRow*width + inCol];
13             }
14         }
15     }
16     if (outRow < height && outCol < width) {
17         P[outRow*width + outCol] = Pvalue;
18     }
19 }
20

```

图 7.9: 使用常量内存存放 F 的二维卷积内核（依据原书图 7.9 重绘）

与全局内存变量一样，常量内存变量实际上也位于 DRAM 中。不过，由于 CUDA 运行时知道常量内存变量不会在内核执行期间被修改，因此会指导硬件在内核执行期间积极缓存这些变量。要理解使用常量内存的好处，需要先更多了解现代处理器的内存和缓存层次。

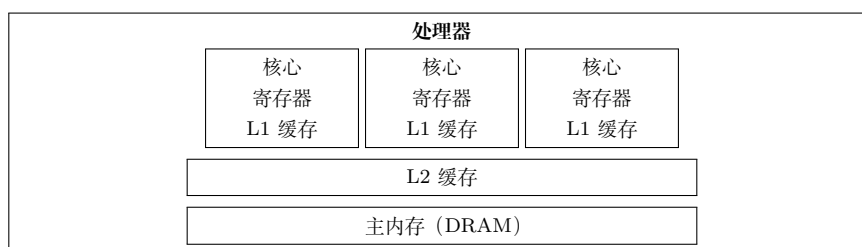


图 7.10: 现代处理器缓存层次结构的简化视图（依据原书图 7.10 重绘）

正如第五章和第六章所讨论的，DRAM 的长延迟和有限带宽几乎在所有现代处理器中都构成瓶颈。为了缓解这一内存瓶颈，现代处理器通常采用片上缓存 (cache)，减少需要从主内存 (DRAM) 访问的变量数量，如图 7.10 所示。

与 CUDA 共享内存，或者一般意义上的 scratchpad 内存不同，缓存对程序是“透明的”。要使用 CUDA 共享内存保存全局变量的值，程序需要把变量声明为

`__shared__`，并显式地把全局内存变量的值复制到共享内存变量中。使用缓存时，程序只需访问原始的全局内存变量。处理器硬件会自动把最近或最频繁使用的变量保留在缓存中，并记住它们原来的全局内存地址。之后再次使用某个保留的变量时，硬件会根据地址检测出缓存中存在该变量的副本，于是从缓存提供变量值，避免访问 DRAM。

内存的容量与速度之间存在权衡。因此，现代处理器通常采用多级缓存。缓存级别的编号反映了缓存与处理器之间的距离。最低级别，即一级缓存（L1），直接连接到处理器核心，如图 7.10 所示；它在延迟和带宽方面都以接近处理器的速度运行。不过，L1 缓存容量很小，通常不超过 128 KB。二级缓存（L2）更大，容量从几百 KB 到几十 MB 不等，但访问可能需要几十到几百个时钟周期。L2 通常由多个 CPU 核心或 CUDA 设备的多个 SM 共享，因此访问带宽也由多个 SM 共享。如今一些高端处理器甚至还有三级缓存（L3），容量可以达到数百 MB。

常量内存变量在大规模并行处理器的内存设计和使用中扮演着有趣的角色。由于常量内存变量在内核执行期间不会被修改，SM 在缓存它们时不需要支持线程写入。为通用缓存支持高吞吐量写入，需要复杂的硬件逻辑，并且会消耗大量芯片面积和功耗。不需要支持写入，就可以设计出在芯片面积和功耗方面都非常高效的常量专用缓存。此外，由于常量内存很小（64 KB），小型专用缓存就能有效捕获每个内核频繁使用的常量内存变量。这种专用缓存称为现代 GPU 中的常量缓存（constant cache）。

当一个 warp 中的所有线程访问同一个常量内存变量时，例如图 7.9 中对 F 的访问（索引与线程索引无关），常量缓存能够提供极高带宽来满足这些线程的数据需求。而且，由于 F 通常很小，可以假设所有 F 元素实际上始终来自常量缓存。因此，可以简单地认为访问 F 元素不消耗 DRAM 带宽。

基于这一点，计算图 7.9 内核的算术强度。图中第 10 行，每个线程在每次循环迭代中从全局内存加载一个元素（4 B），并对它们执行两次算术运算。常量内存中对 F 的加载没有计入，因为它几乎总能命中常量缓存。忽略循环嵌套之外的第 14 行存储操作。因此，该实现的算术强度为

$$\frac{2 \text{ FLOP}}{4 \text{ B}} = 0.5 \text{ FLOP/B.}$$

换句话说，这是图 7.7 基本内核的两倍。实际算术强度还会更高，因为多个线程访问的 N 元素有时也能在 L1 或 L2 缓存中找到。不过，仍然可以用共享内存分块更可靠地优化对 N 的访问，第 7.5 节将对此进行介绍。

还有一种把滤波器权重放入常量内存的方法，不需要声明全局数组变量。CUDA 编译器也会把内核参数放在常量内存中。因此，如果把滤波器权重作为内核参数传入，它们同样会被放在常量内存中。一种简洁的传参方式，是把滤波器权重放入 C++ 结构体，并按值传递该结构体。这样，内核可以用不同滤波器权重被多次调用，而不需要每次都使用 `cudaMemcpyToSymbol` 显式复制权重。CUDA 运行时仍然需要在内核启动过程中复制这些权重，但复制操作对程序员是透明的。

利用滤波器权重恒定性质的另一种方法，是避免从全局内存访问它们。如果滤波器权重在不同程序执行之间不变，并且在编译期已知，就可以把权重值直接硬编码到代码中。不过，这样做会使内核无法复用于其他滤波器权重。

7.5 带 halo 单元的分块卷积

可以通过共享内存分块优化来缓解卷积的内存带宽瓶颈。回顾第五章，共享内存分块让线程协作，把输入元素加载到片上内存，以便后续重复使用。首先需要定义输入瓦片和输出瓦片，因为理解这些定义对于理解优化实现非常重要。

把每个线程块处理的输出元素集合称为输出瓦片 (output tile)。回忆图 7.6 中 16×16 二维卷积的玩具示例：使用 $4 \times 4 = 16$ 个块，每个块含有 $4 \times 4 = 16$ 个线程。在该例中有 16 个输出瓦片。这里使用每块 16 个线程只是为了让示例足够小；实际应用中每块至少应有 32 个线程，即一个 warp，通常还要更多线程，才能获得较好的占用率和数据重用。

输入瓦片 (input tile) 是计算一个输出瓦片中的元素所需的输入元素集合。图 7.11 展示了块 (1,1) 处理的输出瓦片及其对应的输入瓦片 (左侧阴影区域)。注意，输入瓦片在每个方向上都需要扩展滤波器半径 (本例中为 2)，以确保包含计算输出瓦片边缘元素所需的所有 halo 输入元素。这个扩展会使输入瓦片显著大于输出瓦片。在本例中，每个输出瓦片包含

$$4^2 = 16$$

个元素，而每个输入瓦片包含

$$(4 + 2 + 2)^2 = 8^2 = 64$$

个元素；输入瓦片是输出瓦片的 4 倍。不过，这个比例很大，是因为为了便于可视化，示例使用了很小的输出瓦片。实际应用中，输出瓦片尺寸会大得多，输入瓦片与输出瓦片的比例也会更小。例如，若输出瓦片大小为 $16 \times 16 = 256$ ，使用同一个 5×5 滤波器，则输入瓦片大小为

$$(16 + 2 + 2)^2 = 400,$$

此时输入瓦片与输出瓦片的比例约为 1.6。虽然这个比例远小于 4，但仍说明即使采用实际输出瓦片尺寸，输入瓦片也可能显著大于输出瓦片。

本节介绍一种把共享内存分块应用于卷积的方法：线程块中的所有线程协作，把整个输入瓦片加载到共享内存，然后从共享内存访问输入元素，计算输出瓦片元素。这种策略应该让读者想起第五章讨论的分块矩阵乘法。主要区别在于，第五章的分块矩阵乘法假设输入瓦片与输出瓦片尺寸相同，而卷积的输入瓦片大于输出瓦片。输入瓦片尺寸和输出瓦片尺寸的差异，使分块卷积内核的设计更复杂。



图 7.11: 二维卷积中的输入瓦片与输出瓦片 (依据原书图 7.11 重绘)

有两种简单的线程组织方式可以处理这种尺寸差异。第一种方式启动尺寸与输入瓦片相匹配的线程块。这样加载输入瓦片很简单，因为每个线程只需加载一个输入元素。不过，由于块尺寸大于输出瓦片尺寸，计算输出元素时必须禁用一部分线程。第二种方式启动尺寸与输出瓦片相匹配的线程块。这一策略使输入瓦片加载更复杂，因为线程需要迭代，才能确保所有输入瓦片元素都被加载；但它简化了输出元素的计算，因为块尺寸与输出瓦片相同，不需要禁用线程。本节给出基于第一种线程组织方式的内核，第二种方式留给读者作为练习。

```

1 #define IN_TILE_DIM 32
2 #define OUT_TILE_DIM ((IN_TILE_DIM) - 2*(FILTER_RADIUS))
3 __constant__ float F[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];
4 __global__ void convolution_tiled_2D_const_mem_kernel(float *N, float *P,
5                                                     int width, int height) {
6     int col = blockIdx.x*OUT_TILE_DIM + threadIdx.x - FILTER_RADIUS;
7     int row = blockIdx.y*OUT_TILE_DIM + threadIdx.y - FILTER_RADIUS;
8     // loading input tile
9     __shared__ float N_s[IN_TILE_DIM][IN_TILE_DIM];
10    if(row>=0 && row<height && col>=0 && col<width) {
11        N_s[threadIdx.y][threadIdx.x] = N[row*width + col];
12    } else {
13        N_s[threadIdx.y][threadIdx.x] = 0.0;
14    }
15    __syncthreads();
16    // Calculating output elements
17    int tileCol = threadIdx.x - FILTER_RADIUS;
18    int tileRow = threadIdx.y - FILTER_RADIUS;
19    // turning off the threads at the edges of the block
20    if (col >= 0 && col < width && row >= 0 && row < height) {
21        if (tileCol >= 0 && tileCol < OUT_TILE_DIM &&
22            tileRow >= 0 && tileRow < OUT_TILE_DIM) {
23            float Pvalue = 0.0f;
24            for (int fRow = 0; fRow < 2*FILTER_RADIUS+1; fRow++) {
25                for (int fCol = 0; fCol < 2*FILTER_RADIUS+1; fCol++) {
26                    Pvalue += F[fRow][fCol]*N_s[tileRow+fRow][tileCol+fCol];
27                }
28            }
29            P[row*width+col] = Pvalue;
30        }
31    }
32 }
33

```

图 7.12: 使用常量内存的分块二维卷积内核 (依据原书图 7.12 重绘)

译者注：底本图 7.12 的共享内存数组声明缺少元素类型；本译稿按输入数据类型补为 float 数组。

图 7.12 所示内核采用第一种线程组织方式。每个线程首先计算自己负责加载/计

算的输入/输出元素的列索引 (`col`) 和行索引 (`row`) (第 06–07 行)。内核分配了一个与输入瓦片大小相同的共享内存数组 `N_s` (第 09 行), 并把输入瓦片加载到该数组 (第 10–15 行)。第 10 行的条件用于检查线程尝试加载的输入瓦片元素是否是 ghost cell。如果是, 线程不执行内存加载, 而是把 0 放入共享内存。所有线程都执行屏障同步 (第 15 行), 确保整个输入瓦片已经位于共享内存中, 之后任何线程才能继续计算输出元素。

现在, 所有输入瓦片元素都位于 `N_s` 数组中, 每个线程都可以使用 `N_s` 元素计算自己的输出 `P` 元素值。注意, 输出瓦片小于输入瓦片, 而线程块尺寸与输入瓦片相同。因此, 每个块中只有一部分线程用于计算输出瓦片元素。

选择块中线程子集有多种方式。我们的实现禁用位于边缘的一层线程, 如图 7.13 所示。图中给出了一个小型卷积示例: 使用 3×3 滤波器 (滤波器半径为 1)、 8×8 输入瓦片、 8×8 线程块和 6×6 输出瓦片。图 7.13 左侧展示输入瓦片和线程块; 由于二者尺寸相同, 它们重叠绘制。左侧中央的粗线框包围了用于计算输出瓦片元素的活动线程。本例中, 活动线程的 x 、 y 坐标都从 1 到 6。

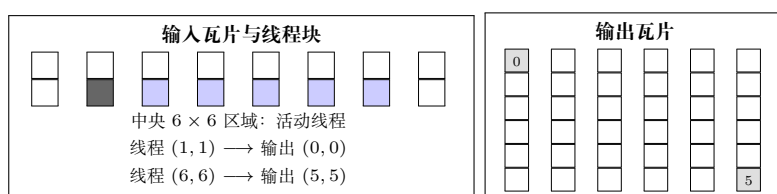


图 7.13: 利用共享内存输入瓦片计算输出瓦片时的线程组织示例 (依据原书图 7.13 重绘)

图 7.13 还展示了活动线程到输出瓦片元素的映射。活动线程 (t_x, t_y) 使用输入瓦片中的一个 patch, 计算输出元素

$$(t_x - \text{FILTER_RADIUS}, t_y - \text{FILTER_RADIUS}).$$

该 patch 的左上角是输入瓦片中的元素 $(t_x - \text{FILTER_RADIUS}, t_y - \text{FILTER_RADIUS})$ 。这对应图 7.12 第 17–18 行: 输入 patch 起始位置的列索引 `tileCol` 和行索引 `tileRow` 分别被设置为 `threadIdx.x - FILTER_RADIUS` 和 `threadIdx.y - FILTER_RADIUS`。

在图 7.13 的小例子中, 线程 (1,1) 的 `tileCol` 和 `tileRow` 分别为 0。因此, 线程 (1,1) 使用输入瓦片左上角虚线框标出的 3×3 输入 patch, 计算输出瓦片的 (0,0) 元素。图 7.12 第 24–28 行的循环嵌套遍历输入 patch 并生成输出元素。块中的线程 (1,1) 遍历左上角为 `N_s[0][0]` 的 patch, 而线程 (5,5) 遍历左上角为 `N_s[5][5]` 的 patch。

第 06–07 行先用块索引乘以输出瓦片尺寸, 得到块负责的输出瓦片在 `P` 数组中的水平和垂直起始位置; 再加上线程索引减去滤波器半径得到瓦片内偏移。因此, `row` 和 `col` 变量提供了活动线程负责的输出元素索引。每个线程使用这两个索引, 在第 29 行写入最终输出元素。

图 7.12 中的分块二维卷积内核明显比图 7.9 中的基本内核更长、更复杂。增加这些复杂性，是为了减少对 N 元素的 DRAM 访问次数。目标是提高内核算术强度，使最终性能不受内存带宽限制，或者降低其受内存带宽限制的程度。第 7.4 节中已经得到图 7.9 内核的算术强度为 0.5 FLOP/B。下面推导图 7.12 内核的算术强度。

处理数据边缘瓦片的块中，处理 ghost cell 的线程不会为这些 ghost cell 执行内存访问，因此这些块的内存访问次数会减少。可以通过枚举使用每个 ghost cell 的线程数量来计算减少的访问次数。不过，对于大型输入数组和小滤波器，ghost cell 的影响显然很小。因此，在计算分块卷积内核的算术强度时忽略 ghost cell 的影响，只考虑内部线程块，即其 halo cell 不是 ghost cell 的线程块。

令 t 表示方形输出瓦片沿一个维度的大小（即 OUT_TILE_DIM），令 m 表示滤波器维度（即 $2 \times \text{FILTER_RADIUS} + 1$ ）。分配给输出瓦片元素的每个线程，对滤波器的每个元素执行一次乘法和一次加法。因此，一个内部块中的线程总共执行

$$t^2 \cdot m^2 \cdot 2$$

次算术运算。

对于全局内存访问，所有全局内存加载都已从循环嵌套中移出，放入把输入元素加载到共享内存的代码中。每个分配给输入瓦片元素的线程加载一个 4 B 的输入值。由于输入瓦片维度 IN_TILE_DIM 为 $t + m - 1$ ，每个内部块加载的总字节数为

$$(t + m - 1)^2 \cdot 4.$$

由于加载已经移出循环嵌套，其数量不再显著多于存储，所以分析时还必须考虑存储。内部线程块中的线程总共存储一个 $t \times t$ 输出瓦片，每个元素占 4 B。因此，总存储字节数为

$$t^2 \cdot 4.$$

用内部块执行的算术运算量除以该块加载和存储的字节数，可以得到图 7.12 分块卷积内核的算术强度：

$$\frac{t^2 \cdot m^2 \cdot 2}{((t + m - 1)^2 + t^2) \cdot 4} = \frac{1}{4} \cdot m^2 \cdot \frac{1}{\frac{1}{2} \left(1 + \frac{m-1}{t}\right)^2 + \frac{1}{2}}.$$

值得注意的是，这个算术强度更接近第 7.3 节推导的理想算术强度 $\frac{1}{4}m^2$ 。随着输出瓦片维度 t 增大，衰减因子趋近于 1.0。直观地说，更大的输出瓦片使我们能够重用加载到共享内存中的输入瓦片元素，计算更多输出瓦片元素；因此，更大的 t 会带来更高的算术强度。当然，内核实现中的 t 受到多种硬件资源限制，例如线程块大小上限和共享内存容量上限。第 8 章将介绍克服这些挑战的优化技术。

图 7.14 展示了上面推导的分块卷积内核算术强度表达式，如何随不同滤波器尺寸和瓦片尺寸变化。可以清楚看到，小滤波器和小瓦片尺寸时计算属于内存受限，大

		算术强度 (FLOP/B)					
滤波器维度 m	输出瓦片维度 t	8	16	28	32	64	理想
3		1.76	1.99	2.10	2.11	2.18	2.25
5		3.85	4.88	5.42	5.52	5.87	6.25
7		6.03	8.48	9.90	10.17	11.16	12.25
9		8.10	12.46	15.27	15.80	17.88	20.25
11		9.98	16.62	21.29	22.22	25.89	30.25
13		11.66	20.80	27.79	29.23	35.06	42.25

左上区域通常更可能内存受限；右下区域通常更可能计算受限

图 7.14: 二维分块卷积中算术强度随瓦片尺寸和滤波器尺寸的变化（依据原书图 7.14 重绘）

滤波器和大瓦片尺寸时属于计算受限。以 5×5 滤波器和 28×28 输出瓦片（ 32×32 输入瓦片）为例，算术强度为 5.42 FLOP/B。实际值可能更高，因为不同线程块使用的某些值可以在缓存中找到。这个值已经相当接近 5×5 滤波器的理想算术强度 6.25 FLOP/B。

图 7.12 中实现的分块卷积内核存在一些低效之处。许多低效都源于：为了加载 halo cell，输入瓦片尺寸和块尺寸大于输出瓦片尺寸。第一个低效之处是，内核启动了一些线程，它们只把数据从全局内存加载到共享内存，并不参与计算阶段，因而浪费了计算资源。对于内存受限内核，在访问内存时拥有更多并行性可能有益；但对于计算受限内核，被排除线程浪费的资源可能更有价值，因此这种做法可能成为缺点。

第二个低效之处是，输出瓦片尺寸不是 2 的幂，这可能导致把最终结果存储到全局内存时出现未对齐的内存访问。由于线程块（以及输入瓦片）的最大尺寸为 32×32 ，对于 5×5 滤波器，输出瓦片为 28×28 。因此，每个线程块存储的输出瓦片在内存中并非完全对齐，写入时可能产生低效。第三个低效之处是，支持更大的滤波器尺寸需要缩小输出瓦片，因为输入瓦片尺寸无法继续扩大。正如前面看到的，缩小输出瓦片会降低内核算术强度，还会加剧前面提到的两个低效之处。

为克服这些问题，可以让块大小与输出瓦片尺寸相同，并让线程在加载输入瓦片时迭代，从而把更大的输入瓦片加载到共享内存。这就是本节前面提到、留作练习的第二种线程组织方式。另一种可能是完全不把 halo cell 加载到共享内存，而依赖缓存快速访问它们。第 7.6 节介绍后一种方法。

7.6 使用缓存处理 halo 单元的分块卷积

一个线程块输入瓦片中的 halo cell，同时也是相邻瓦片输入瓦片中的内部元素。例如，在图 7.11 中，一个输入瓦片浅色标出的 halo cell 也是相邻线程块输入瓦片的内部元素。线程块需要这些 halo cell 时，它们很可能已经因为相邻线程块的访问而位于 L2 缓存中。因此，对 halo cell 的内存访问可能自然地由 L2 缓存满足，不会产生额外的 DRAM 流量。也就是说，可以让这些 halo cell 继续通过原始 N 元素访问，

而不是加载到 `N_s` 中。这样，共享内存输入瓦片和输出瓦片可以采用相同尺寸，只需把每个输入瓦片的内部元素加载到共享内存。

```

1  #define TILE_DIM 32
2  __constant__ float F_c[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];
3  __global__ void convolution_cached_tiled_2D_const_mem_kernel(float *N,
4                                     float *P,
5                                     int width, int height) {
6      int col = blockIdx.x*TILE_DIM + threadIdx.x;
7      int row = blockIdx.y*TILE_DIM + threadIdx.y;
8      // loading input tile
9      __shared__ float N_s[TILE_DIM][TILE_DIM];
10     if(row<height && col<width) {
11         N_s[threadIdx.y][threadIdx.x] = N[row*width + col];
12     } else {
13         N_s[threadIdx.y][threadIdx.x] = 0.0;
14     }
15     __syncthreads();
16     // Calculating output elements
17     // turning off the threads at the edges of the block
18     if (col < width && row < height) {
19         float Pvalue = 0.0f;
20         for (int fRow = 0; fRow < 2*FILTER_RADIUS+1; fRow++) {
21             for (int fCol = 0; fCol < 2*FILTER_RADIUS+1; fCol++) {
22                 if (threadIdx.x-FILTER_RADIUS+fCol >= 0 &&
23                     threadIdx.x-FILTER_RADIUS+fCol < TILE_DIM &&
24                     threadIdx.y-FILTER_RADIUS+fRow >= 0 &&
25                     threadIdx.y-FILTER_RADIUS+fRow < TILE_DIM){
26                     Pvalue += F_c[fRow][fCol]*
27                         N_s[threadIdx.y-FILTER_RADIUS+fRow]
28                           [threadIdx.x-FILTER_RADIUS+fCol];
29                 } else {
30                     if (row-FILTER_RADIUS+fRow >= 0 &&
31                         row-FILTER_RADIUS+fRow < height &&
32                         col-FILTER_RADIUS+fCol >= 0 &&
33                         col-FILTER_RADIUS+fCol < width) {
34                         Pvalue += F_c[fRow][fCol] *
35                             N[(row-FILTER_RADIUS+fRow)*width+
36                               col-FILTER_RADIUS+fCol];
37                     }
38                 }
39             }
40         }
41         P[row*width+col] = Pvalue;
42     }
43 }
44

```

图 7.15: 使用缓存处理 halo 并使用常量内存存放 F 的分块二维卷积内核（依据原书图 7.15 重绘）

图 7.15 展示了使用缓存处理 halo cell 的二维卷积内核。在这个分块内核中，共享内存数组 `N_s` 只需保存瓦片的内部元素。因此，输入瓦片和输出瓦片尺寸相同，都由常量 `TILE_DIM`（第 01 行）定义。这样，`N_s` 在 x 、 y 两个维度上都声明为 `TILE_DIM` 大小（第 06 行）。

由于输入瓦片和输出瓦片大小相同，线程块可以按输入/输出瓦片的相同大小启动。加载 `N_s` 元素也更简单，因为每个线程只需加载与自己负责的输出元素具有相同行列索引的输入元素（第 04–05、07–11 行）。加载输入元素的条件在第 07 行也得到

简化：内核不再把 halo cell 加载到共享内存，因此不存在加载 ghost cell 的危险；条件只需检查瓦片是否超出输入数据的有效范围。

不过，计算 P 元素的循环体变得更复杂。它需要增加条件，同时检查 halo cell 和 ghost cell。第 17–20 行的条件处理 halo cell，检查输入元素是否落在输入瓦片内部；如果是，就从共享内存访问该元素。否则，第 24–27 行的条件检查 halo cell 是否是 ghost cell。如果是，由于假设 ghost 值为 0，因此不执行任何操作；否则从全局内存访问该元素。读者应当验证，这里处理 ghost cell 的条件与图 7.7 中使用的条件相似。

译者注：底本图 7.15 的常量内存变量声明为 `F_c`，但乘加语句中有一处仍写作 `F`。为使代码示例自洽，本译稿统一使用 `F_c`；若严格按底本转录，应把该处视为变量名笔误。此外，底本访问共享内存时漏掉了 `FILTER_RADIUS` 偏移；本译稿依据前面的边界条件和瓦片坐标定义补齐该偏移，使共享内存下标与对应的输入元素一致。

与图 7.12 中的内核相比，图 7.15 中内核还有一个细微优势：块大小、输入瓦片大小和输出瓦片大小可以相同，并且可以取 2 的幂。由于图 7.12 中输入瓦片和输出瓦片大小不同，该内核执行期间很可能存在更多内存分歧和控制分歧。

7.7 小结

本章研究了卷积这一重要的并行计算模式。卷积广泛用于计算机视觉和视频处理等应用，同时也是许多并行算法所基于的一般模式。例如，可以把偏微分方程 (PDE) 求解器中的模板算法看成卷积的特殊情况，这将是第 8 章的主题。第 19 章讨论卷积神经网络时，还会大量应用本章学到的内容。

我们介绍了一个基本的并行卷积实现，其性能受访问输入和滤波器元素时的 DRAM 带宽限制。随后介绍了常量内存，并对内核和主机代码做了简单修改，以利用常量缓存，几乎消除访问滤波器元素时的所有 DRAM 流量。接着介绍了使用共享内存的分块并行卷积实现；它减少了 DRAM 带宽消耗，但引入了更多控制流分歧和编程复杂性。最后介绍了利用 L1 和 L2 缓存处理 halo cell 的分块并行卷积实现。

我们还用提高算术强度的方式分析了分块的收益。这项分析是第五章引入的一项重要技能，也有助于理解其他模式中分块的收益。通过分析，我们认识到小滤波器和小瓦片尺寸的局限性。

虽然本章只展示了一维和二维卷积示例，但这些技术同样可以直接应用于三维卷积。一般而言，由于维度更高，输入和输出数组的索引计算更复杂；加载瓦片和/或计算输出值时，每个线程也需要更多层循环来遍历多个维度。建议读者把这些高维内核作为课后练习完成。

练习

1. 计算图 7.3 中的 $P[0]$ 值。

2. 对数组

$$N = \{4, 1, 3, 2, 3\}$$

执行一维卷积，滤波器为

$$F = \{2, 1, 4\}.$$

求得到的输出数组。

3. 你认为下面的一维卷积滤波器分别在做什么？

- a. $[0, 1, 0]$
- b. $[0, 0, 1]$
- c. $[1, 0, 0]$
- d. $[-\frac{1}{2}, 0, \frac{1}{2}]$
- e. $[\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$

4. 对大小为 N 的数组执行一维卷积，滤波器大小为 M ：

- a. 总共有多少个 ghost cell？
- b. 如果把 ghost cell 视为乘法（乘以 0），会执行多少次乘法？
- c. 如果不把 ghost cell 视为乘法，会执行多少次乘法？

5. 对大小为 $N \times N$ 的方阵执行二维卷积，方形滤波器大小为 $M \times M$ ：

- a. 总共有多少个 ghost cell？
- b. 如果把 ghost cell 视为乘法（乘以 0），会执行多少次乘法？
- c. 如果不把 ghost cell 视为乘法，会执行多少次乘法？

6. 对大小为 $N_1 \times N_2$ 的矩形矩阵执行二维卷积，矩形滤波器大小为 $M_1 \times M_2$ ：

- a. 总共有多少个 ghost cell？
- b. 如果把 ghost cell 视为乘法（乘以 0），会执行多少次乘法？
- c. 如果不把 ghost cell 视为乘法，会执行多少次乘法？

7. 对大小为 $N \times N$ 的数组执行二维分块卷积，使用图 7.12 所示内核，滤波器大小为 $M \times M$ ，输出瓦片大小为 $T \times T$ ：

- a. 需要多少个线程块？

- b. 每个块需要多少个线程?
 - c. 每个块需要多少共享内存?
 - d. 如果使用图 7.15 中的内核, 重复回答上述问题。
- 8. 修改图 7.7 中的二维内核, 使其执行三维卷积。
 - 9. 修改图 7.9 中的二维内核, 使其执行三维卷积。
 - 10. 修改图 7.12 中的二维分块内核, 使其执行三维卷积。
 - 11. 修改图 7.12 中的分块二维内核, 使每个线程块使用与输出瓦片大小相同数量的线程, 并让每个线程在把输入瓦片加载到共享内存时加载多个元素。

第八章 模板计算与线程粗化

模板 (stencil) 是求解偏微分方程数值方法的基础, 广泛用于流体动力学、热传导、燃烧、天气预报、气候模拟和电磁学等应用领域。基于模板的算法所处理的数据, 是质量、速度、力、加速度、温度、电场和能量等具有物理意义的离散量; 这些量之间的关系由微分方程支配。模板的一种常见用途, 是根据函数在一段输入变量范围内的取值, 近似计算函数的导数值。

模板与卷积十分相似: 二者都会根据另一个多维数组中同一位置及其邻域内元素的当前值, 计算多维数组某个元素的新值。因此, 模板同样需要处理光环单元 (halo cell) 和幽灵单元 (ghost cell)。与卷积不同, 模板计算用于迭代求解关注区域内连续可微函数的值。模板邻域中使用的数据元素及其权重系数由待求解的微分方程决定。有些模板模式适合采用卷积无法使用的优化。在把初始条件迭代传播到整个区域的求解器中, 输出值的计算还可能存在依赖, 必须遵循一定的次序约束。此外, 求解微分方程通常有较高的数值精度要求, 模板处理的数据往往是高精度浮点数; 采用分块技术时, 它们会占用更多片上内存。正是这些差异, 使模板计算催生出不同于卷积的优化方法。

8.1 背景

使用计算机对函数、模型、变量和方程进行数值求值与求解的第一步, 是把它们转换成离散表示。例如, 图 8.1(a) 展示了 $0 \leq x \leq \pi$ 区间内的正弦函数 $y = \sin(x)$ 。图 8.1(b) 给出一个一维规则 (结构化) 网格的设计, 其中七个网格点对应的 x 值以常量 $\pi/6$ 等间距排列。一般来说, 结构化网格使用形状相同的平行多面体覆盖 n 维欧氏空间, 例如一维中的线段、二维中的矩形和三维中的长方体。后文将会看到, 在结构化网格上, 变量的导数可以方便地表示为有限差分, 因此结构化网格主要用于有限差分方法。非结构化网格更加复杂, 常用于有限元方法和有限体积方法。为简单起见, 本章只使用规则网格, 因而也只讨论有限差分方法。

图 8.1(c) 展示了得到的离散表示: 用正弦函数在七个网格点上的值表示该函数, 并把这些值存入一维数组 F 。这里隐含假设 $x = i \cdot \pi/6$, 其中 i 是数组元素的下标。例如, 与元素 $F[2]$ 对应的 x 值为 $2 \cdot \pi/6$, 而 $F[2]$ 的值为该位置的正弦值 0.87。

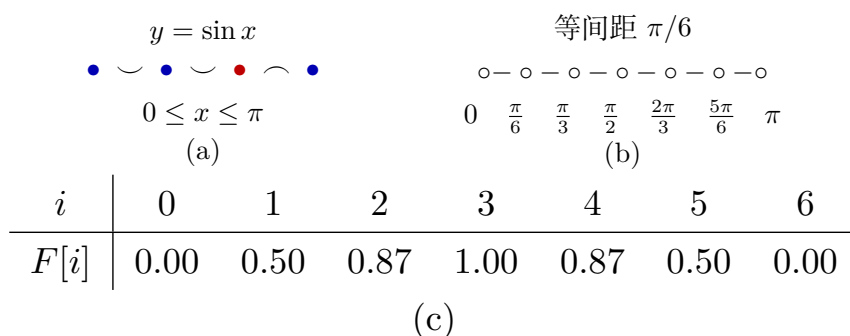


图 8.1: 正弦函数的离散化: (a) $0 \leq x \leq \pi$ 上连续可微的正弦函数; (b) 以 $\pi/6$ 为固定间距的规则网格; (c) 得到的离散表示 (依据原书图 8.1 重绘)

在离散表示中, 如果某个 x 值不对应任何网格点, 就需要使用线性插值或样条插值等技术求出函数在该位置的近似值。表示的保真度, 也就是这些插值技术给出的近似函数值有多准确, 取决于网格点之间的间距: 间距越小, 近似越准确。减小间距可以提高表示的精度, 代价是需要更多存储空间; 而且后文会看到, 求解偏微分方程时还需要更多计算。

离散表示的保真度还取决于所用数值的精度。由于要近似连续函数, 网格点值通常使用浮点数。主流 CPU 和 GPU 支持双精度 (64 位)、单精度 (32 位)、半精度 (16 位) 及其他表示 (见附录 A)。其中, 双精度数具有最高精度, 能给出保真度最高的离散表示。然而, 现代 CPU 和 GPU 的单精度、半精度算术吞吐量通常高得多; 双精度数包含更多位, 读写它们会消耗更多内存带宽, 存储它们也需要更多内存容量。分块技术为了获得较高算术强度, 需要在片上内存和寄存器中保存大量网格点值, 因此上述问题会给分块带来显著挑战, 正如第 7 章所讨论的那样。

下面用更形式化的方式讨论模板。在数学中, 模板是作用于结构化网格各点的一种权重几何模式。它规定如何通过数值近似过程, 从关注点的邻点值推导出该点的值。例如, 模板可以规定如何利用某点及其邻点处函数值的有限差分, 近似该点处的函数导数。偏微分方程表达函数、变量及其导数之间的关系, 而模板为描述有限差分方法如何数值求解偏微分方程提供了一种方便的基础。

假设函数 $f(x)$ 已经离散为一维网格数组 F , 现在希望计算 $f(x)$ 的离散导数 $f'(x)$ 。可以采用经典的一阶导数有限差分近似:

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} + O(h^2).$$

也就是说, 函数在 x 点处的导数, 可以用两个相邻点处函数值之差除以这两个邻点的 x 值之差来近似。 h 是网格相邻点之间的间距。误差项 $O(h^2)$ 表示误差与 h 的平方成正比; 显然, h 越小, 近似越好。在图 8.1 的例子中, h 为 $\pi/6$, 即约 0.52。这个间距还不足以使近似误差可以忽略, 但应当能够给出相当接近的结果。

由于网格间距为 h , 当前对 $f(x-h)$ 、 $f(x)$ 和 $f(x+h)$ 的估计值分别位于 $F[i-1]$ 、

$F[i]$ 和 $F[i+1]$, 其中 $x = i \cdot h$ 。于是, 可以把各网格点的导数值写入输出数组 FD :

$$FD[i] = \frac{F[i+1] - F[i-1]}{2h}.$$

对所有网格点, 这个表达式还可以改写为

$$FD[i] = -\frac{1}{2h} F[i-1] + \frac{1}{2h} F[i+1].$$

因此, 计算网格点 i 处的函数导数估计值, 会涉及网格点 $[i-1, i, i+1]$ 当前的函数估计值, 其系数为 $[-1/(2h), 0, 1/(2h)]$ 。这定义了图 8.2(a) 所示的一维三点模板。如果要近似网格点处更高阶的导数, 就需要使用更高阶有限差分。例如, 如果微分方程包含 $f(x)$ 的二阶导数, 就会使用涉及 $[i-2, i-1, i, i+1, i+2]$ 的一维五点模板, 如图 8.2(b) 所示。一般来说, 如果方程最高包含 n 阶导数, 模板就会涉及中心网格点每侧的 n 个网格点。图 8.2(c) 展示了一维七点模板。中心点每侧的网格点数称为模板的阶数, 因为它反映了所近似导数的阶数。

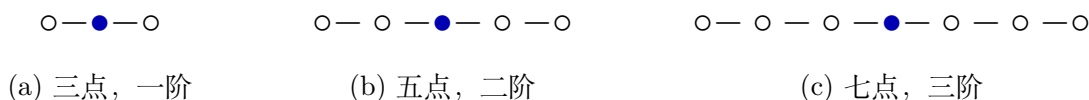


图 8.2: 一维模板示例 (依据原书图 8.2 重绘)

求解包含两个变量的偏微分方程时, 函数值需要离散到二维网格, 并使用二维模板计算偏导数的近似值。如果偏微分方程只含分别对某一个变量求得的偏导数, 例如 $\partial f(x, y)/\partial x$ 和 $\partial f(x, y)/\partial y$, 而不含混合偏导数 $\partial^2 f(x, y)/(\partial x \partial y)$, 就可以使用只沿 x 轴和 y 轴选择网格点的二维模板。例如, 只涉及 x 和 y 的一阶偏导数时, 可以在中心点沿两条坐标轴的每一侧各取一个网格点, 得到图 8.3(a) 的二维五点模板。如果方程最高包含分别对 x 或 y 求得的二阶导数, 则使用图 8.3(b) 的二维九点模板。依照前面的定义, 图 8.3(a)、(b) 中模板的阶数分别为 1 和 2; 图 8.3(c)、(d) 则分别给出一阶三维七点模板和二阶三维十三点模板。

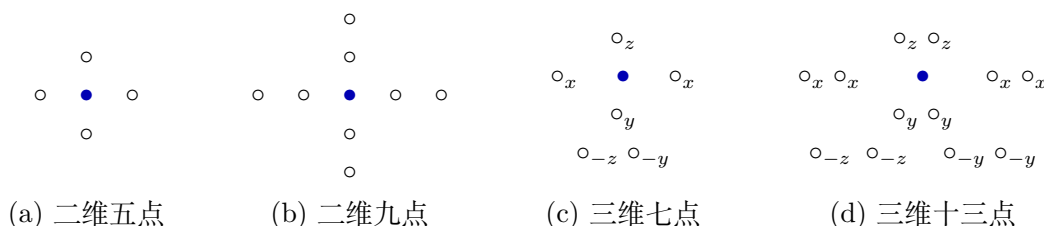


图 8.3: 二维与三维模板示例 (依据原书图 8.3 重绘)

图 8.4 用五点模板说明数值网格及模板在网格点上的应用。函数被离散成存储在多维数组中的网格点值。图中, 一个二元函数被离散成二维网格, 并以二维数组保存。二维模板根据某个网格点自身及其邻点处的函数值, 计算该网格点处导数的近似值 (输出)。本章关注这样一种计算模式: 把模板应用于所有相关输入网格点, 以生成所有网格点的输出值。这个过程称为**模板扫描 (stencil sweep)**。



图 8.4: 二维网格及用一阶五点模板计算不同网格点处的导数近似值（依据原书图 8.4 重绘）

8.2 并行模板：基本内核

首先给出一个执行模板扫掠的基本内核。为简单起见，假设一次模板扫掠生成各输出网格点值时，输出网格点之间不存在依赖；还假设边界网格点存储边界条件，它们的值从输入到输出保持不变，如图 8.5 所示。也就是说，只计算输出网格中着色的内部区域，不计算未着色的边界单元。由于模板主要用于求解带边界条件的微分方程，这是一个合理的假设。

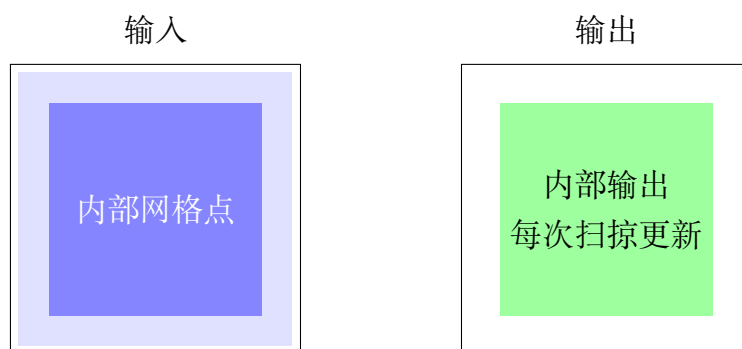


图 8.5: 简化的边界条件：边界单元保存边界条件，迭代之间不更新；每次模板扫掠只计算内部输出网格点（依据原书图 8.5 重绘）

图 8.6 给出执行一次模板扫掠的基本内核。该内核假设每个线程块负责计算一个输出网格值分块，每个线程分配到一个输出网格点。图 8.5 展示的是二维分块示例，其中每个线程块负责一个 4×4 输出分块。不过，大多数实际应用求解三维微分方程，因此图 8.6 的内核假设使用三维网格，以及类似图 8.3(c) 的三维七点模板。

线程到网格点的映射使用熟悉的线性表达式完成，其中涉及 `blockIdx`、`blockDim` 和 `threadIdx` 的 x 、 y 、 z 字段（第 2–4 行）。每个线程映射到一个三维网格点之后，该点及其所有邻点的输入值分别乘以不同的系数，再相加得到输出；这些系数就是第 6–12 行的 `c0` 到 `c6`。根据所需的灵活性，可以把系数硬编码在程序中，也可以存入常量内存。正如第 8.1 节所述，系数值取决于正在求解的具体微分方程。

```

1 __global__ void stencil_kernel(float* in, float* out, unsigned int N) {
2     unsigned int i = blockIdx.z*blockDim.z + threadIdx.z;
3     unsigned int j = blockIdx.y*blockDim.y + threadIdx.y;
4     unsigned int k = blockIdx.x*blockDim.x + threadIdx.x;
5     if(i >= 1 && i < N - 1 && j >= 1 && j < N - 1 && k >= 1 && k < N - 1) {
6         out[i*N*N + j*N + k] = c0*in[i*N*N + j*N + k]
7                               + c1*in[i*N*N + j*N + (k - 1)]
8                               + c2*in[i*N*N + j*N + (k + 1)]
9                               + c3*in[i*N*N + (j - 1)*N + k]
10                              + c4*in[i*N*N + (j + 1)*N + k]
11                              + c5*in[(i - 1)*N*N + j*N + k]
12                              + c6*in[(i + 1)*N*N + j*N + k];
13     }
14 }
15

```

图 8.6: 基本模板扫描内核（依据原书图 8.6 重排）

8.3 内存带宽考量

下面沿用第 5 章介绍并在第 7 章使用的方法，分析三维七点模板扫描的算术强度，进而推断它对内存带宽的需求。假设网格尺寸为 $n \times n \times n$ 。由于只对网格内部点执行模板计算，需要计算的网格点总数为 $(n - 2)^3$ 。计算每个值要执行 13 次浮点运算，即 7 次乘法和 6 次加法，所以算术运算总数为 $13(n - 2)^3$ FLOP。

执行这些计算时需要加载输入网格值，并存储输出网格值。对输入网格而言，除了三维空间各条棱及各个角上的点之外，其余点都会被访问，总计 $n^3 - 12n + 16$ 个值；对输出网格而言，会访问全部内部点，总计 $(n - 2)^3$ 个值。每个值都是 4 B 的单精度浮点数，因此访问的总字节数为

$$4(n^3 - 12n + 16 + (n - 2)^3).$$

理想情况下，每个输入网格点只加载一次，三维七点模板计算的总体算术强度为

$$\frac{13(n - 2)^3}{4(n^3 - 12n + 16 + (n - 2)^3)}.$$

当 n 很大时，这个比值趋近于 $13/8 = 1.625$ FLOP/B。显然，该比值很小，因此三维七点模板计算在现代 GPU 上受内存带宽限制。

再计算图 8.6 基本模板扫描内核的算术强度。内核中的每个线程计算一个输出元素，执行 13 次浮点运算，加载 7 个 4 B 输入值，并存储 1 个 4 B 输出值。因此，该内核的算术强度为

$$\frac{13}{(7 + 1) \cdot 4} = 0.41 \text{ FLOP/B}.$$

这个值显著低于 1.625 FLOP/B 的理想算术强度。实际测得的算术强度可能稍高，因为部分元素或许已经存在于 L1 或 L2 缓存中。尽管如此，仍可以通过优化内存带宽利用率，更加可靠地提高内核的算术强度。下一节将像第 7 章那样使用共享内存分块来提高模板内核的算术强度。

8.4 模板扫掠的共享内存分块

第 5 章已经看到，共享内存分块可以显著提高计算的算术强度。模板的共享内存分块设计几乎与卷积完全相同，不过二者之间存在几个细微却重要的差别。

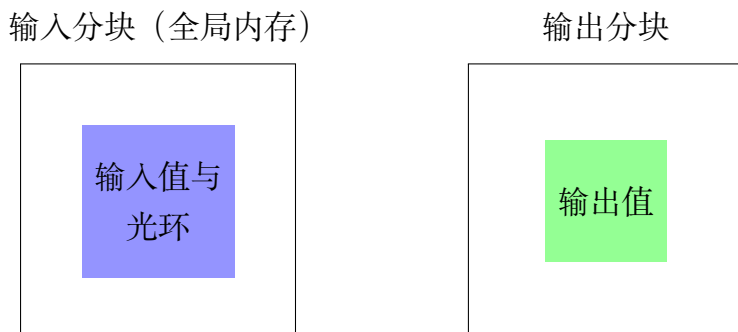


图 8.7: 二维五点模板的输入分块与输出分块（依据原书图 8.7 重绘）

图 8.7 展示在一个小型网格上应用二维五点模板时的输入分块和输出分块。与原书图 7.11 对比，可以看到卷积与模板扫掠之间的小差别：五点模板的输入分块不包含角上的网格点。后文讨论寄存器分块时，这一性质会变得很重要。

就共享内存分块而言，二维五点模板的输入数据复用明显低于 3×3 卷积。第 7 章已经讨论过，二维 3×3 卷积的理想算术强度上界是 $\frac{1}{4} \cdot 3^2 = 2.25$ FLOP/B，而二维五点模板的理想算术强度只有 1.125 FLOP/B。这是因为每个输出网格点只使用 5 个输入网格值，而 3×3 卷积使用 9 个输入像素值。

随着维数和模板阶数增加，二者的差距更加明显。如果把二维模板的阶数从 1（每侧 1 个网格点，共 5 点）增加到 2（每侧 2 个网格点，共 9 点），算术强度上界是 2.125 FLOP/B，而对应的二维 5×5 卷积为 6.25 FLOP/B。若阶数增加到 3（每侧 3 个网格点，共 13 点），上界为 3.125 FLOP/B，而对应的二维 7×7 卷积为 12.25 FLOP/B。

进入三维后，模板扫掠与卷积之间的算术强度差距更为突出。例如，三阶三维模板（每侧 3 点，共 19 点）的上界为 4.625 FLOP/B，而对应的三维 $7 \times 7 \times 7$ 卷积高达 85.75 FLOP/B。也就是说，对模板扫掠而言，把一个输入网格点值加载到共享内存所带来的收益可能远低于卷积，尤其是在模板最常使用的三维情形下。这个看似不大的差别非常重要，它促使我们在第三维采用线程粗化和寄存器分块。

卷积中加载输入分块的各种策略都可直接用于模板扫掠，因此图 8.8 给出的内核与原书图 7.12 的卷积内核类似：线程块与输入分块大小相同，计算输出网格点时会停用一部分线程。该内核由图 8.6 的基本模板扫掠内核改写而来，下面只关注改写之处。

与分块卷积内核一样，分块模板扫掠内核首先计算每个线程负责加载并可能参与计算的网格点的 x 、 y 、 z 坐标。因为内核假设采用三维七点模板，即中心点每侧各有

```

1  __global__ void stencil_kernel(float* in, float* out, unsigned int N) {
2      int i = blockIdx.z*OUT_TILE_DIM + threadIdx.z - 1;
3      int j = blockIdx.y*OUT_TILE_DIM + threadIdx.y - 1;
4      int k = blockIdx.x*OUT_TILE_DIM + threadIdx.x - 1;
5      __shared__ float in_s[IN_TILE_DIM][IN_TILE_DIM][IN_TILE_DIM];
6      if(i >= 0 && i < N && j >= 0 && j < N && k >= 0 && k < N) {
7          in_s[threadIdx.z][threadIdx.y][threadIdx.x] = in[i*N*N + j*N + k];
8      }
9      __syncthreads();
10     if(i >= 1 && i < N-1 && j >= 1 && j < N-1 && k >= 1 && k < N-1) {
11         if(threadIdx.z >= 1 && threadIdx.z < IN_TILE_DIM-1 && threadIdx.y >= 1
12             && threadIdx.y < IN_TILE_DIM-1 && threadIdx.x >= 1 && threadIdx.x < IN_TILE_DIM-1) {
13             out[i*N*N + j*N + k] = c0*in_s[threadIdx.z][threadIdx.y][threadIdx.x]
14                 + c1*in_s[threadIdx.z][threadIdx.y][threadIdx.x-1]
15                 + c2*in_s[threadIdx.z][threadIdx.y][threadIdx.x+1]
16                 + c3*in_s[threadIdx.z][threadIdx.y-1][threadIdx.x]
17                 + c4*in_s[threadIdx.z][threadIdx.y+1][threadIdx.x]
18                 + c5*in_s[threadIdx.z-1][threadIdx.y][threadIdx.x]
19                 + c6*in_s[threadIdx.z+1][threadIdx.y][threadIdx.x];
20         }
21     }
22 }
23

```

图 8.8: 采用共享内存分块的三维七点模板扫描内核（依据原书图 8.8 重排）

一个网格点，所以每个表达式都减去 1（第 2–4 行）。一般来说，所减的值应当等于模板阶数。

内核在共享内存中分配数组 `in_s`，用来保存每个线程块的输入分块（第 5 行）。每个线程加载一个输入元素，也就是包含模板网格点模式的立方输入区域的起始元素。由于第 2–4 行执行了减法，一些线程可能试图加载网格的幽灵单元；第 6 行的 $i \geq 0$ 、 $j \geq 0$ 和 $k \geq 0$ 防止这些越界访问。线程块大于输出分块，处于线程块各维末端的一些线程也可能访问超过网格数组各维上界的幽灵单元；同一行的 $i < N$ 、 $j < N$ 和 $k < N$ 防止这些访问。线程块中的所有线程协作把输入分块加载到共享内存（第 7 行），再通过屏障同步等待所有输入分块网格点都进入共享内存（第 9 行）。

每个线程块在第 10–21 行计算自己的输出分块。第 10 行的条件反映了简化假设：输入与输出网格的边界点保存初始条件，不需要由内核逐次迭代更新。因此，输出点落在这些边界位置的线程会被停用。边界网格点在三维网格表面形成一层。第 11–12 行的条件则停用仅为加载输入分块而额外启动的线程，只允许 i 、 j 、 k 下标落在输出分块内的线程计算相应输出点。最后，每个活跃线程使用七点模板指定的输入网格点计算输出值，但输入值来自共享内存而非全局内存。

可以用内核达到的算术强度来评估共享内存分块的效果。忽略缓存时，原始模板扫描内核达到 0.41 FLOP/B。对共享内存分块内核，假设每个输入分块是各维包含 t 个网格点的立方体，每个输出分块则在各维包含 $t - 2$ 个网格点。因此，每个线程块有 $(t - 2)^3$ 个活跃线程计算输出值，每个活跃线程执行 13 次浮点乘法或加法，总计 $13(t - 2)^3$ 次浮点运算。每个线程块还执行 t^3 次加载来读入输入分块，并执行 $(t - 2)^3$

次 $4B$ 存储来写出输出分块。分块内核的算术强度为

$$\frac{13(t-2)^3}{4t^3 + 4(t-2)^3} = \frac{13}{8} \cdot \frac{1}{\frac{1}{2} \frac{t^3}{(t-2)^3} + \frac{1}{2}} \text{ FLOP/B.}$$

t 越大, 算术强度越高; 直观上, 输出分块越大, 输入网格点值的复用次数就越多。当 t 渐近增大时, 算术强度上界为 $13/8 = 1.625$ FLOP/B, 正是第 8.3 节推导出的理想算术强度。

遗憾的是, 当前硬件对线程块大小的 1024 线程限制, 使图 8.8 的内核难以采用很大的 t 。实际可用的 t 上限是 8, 因为 $8 \times 8 \times 8$ 线程块已有 512 个线程。此外, 输入分块占用的共享内存与 t^3 成正比, 增大 t 会显著增加共享内存消耗。这些硬件限制迫使分块模板内核采用较小的分块。

硬件造成的 t 值上限较小有两个主要缺点。第一个缺点是限制算术强度。当 $t = 8$ 时, 七点模板的算术强度只有 0.96 FLOP/B, 远低于 1.625 FLOP/B 的理想值。随着 t 减小, 光环开销占比上升, 算术强度还会降低。第 7 章已经讨论过, 光环元素的复用次数低于非光环元素; t 越小, 输入分块中光环元素所占比例越大。例如, 对半径为 1 的卷积滤波器, 32×32 二维输入分块包含 1024 个输入元素, 对应的输出分块包含 $30 \times 30 = 900$ 个元素。因此, 输入中有 $1024 - 900 = 124$ 个光环元素, 约占 12%。相比之下, 对一阶三维模板, $8 \times 8 \times 8$ 输入分块包含 512 个元素, 对应输出分块只有 $6 \times 6 \times 6 = 216$ 个元素。输入中有 $512 - 216 = 296$ 个光环元素, 约占 58%!

第二个缺点是小分块会妨碍内存访问合并。对于 $8 \times 8 \times 8$ 分块, 每个包含 32 个线程的 warp 要负责加载分块中 4 个不同的行, 每行包含 8 个输入值。因此, 在同一条加载指令上, warp 中的线程会访问全局内存中至少 4 个相距较远的位置。这些访问无法合并, DRAM 带宽得不到充分利用。可以采用非立方分块来克服这一缺点: 把线程块的 x 维设为 32, 使同一 warp 中的线程访问连续的 32 个元素。例如, 可把线程块的 x 、 y 、 z 三维分别设为 32、8、4。与三维均为 8 的立方线程块相比, 这种设置能改善访问合并, 因而性能更好。然而, 即使重新排列线程块的维度, 输出分块总体上仍然很小, 算术强度依旧较低。为了提高算术强度而扩大分块的需求, 引出了下一节的方法。

8.5 线程粗化

上一节提到, 模板通常用于三维网格, 而且模板模式比较稀疏, 因此模板扫描不像卷积那样容易从共享内存分块中获益。本节通过线程粗化突破线程块大小的限制: 把每个线程的工作从计算一个网格点值, 粗化为计算一系列网格点值, 如图 8.9 所示。回顾第 6 章, 线程粗化会把并行工作单元部分串行化到每个线程中, 从而降低并行化开销。在这里, 细粒度并行化的开销, 是必须把复用率较低的光环元素加载到更多线程块中。

图 8.9 假设每个输入分块包含 $t^3 = 6^3 = 216$ 个网格点。为了能看到输入分块内部，示意图剥去了其前、左、上三层。每个输出分块包含 $(t-2)^3 = 4^3 = 64$ 个网格点。输入分块的每个 $x-y$ 平面包含 $6^2 = 36$ 个网格点，输出分块的每个 $x-y$ 平面包含 $4^2 = 16$ 个网格点。负责该分块的线程块，其线程数与输入分块的一个 $x-y$ 平面相同，即 6×6 ；图中只突出显示实际参与输出计算的内部 4×4 线程。

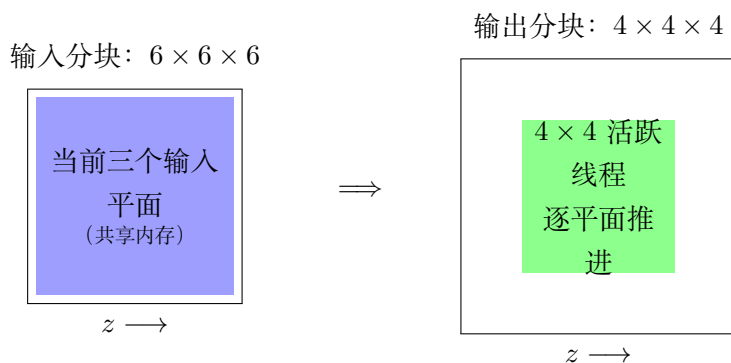


图 8.9: 三维七点模板扫描沿 z 方向的线程粗化（依据原书图 8.9 重绘）

图 8.10 给出沿 z 方向粗化线程的三维七点模板扫描内核。基本思路是让线程块沿 z 方向（即图中深入纸面的方向）迭代，每次迭代计算输出分块中一个 $x-y$ 平面的网格点值。内核首先把每个线程分配给输出的一个 $x-y$ 平面中的网格点（第 3-4 行）。变量 i 是各线程计算的输出分块网格点的 z 下标；每次迭代期间，一个线程块的所有线程处理输出分块的同一个 $x-y$ 平面，因此它们所计算输出点的 z 下标相同。

初始时，每个线程块需要把三个输入分块平面加载到共享内存；它们包含计算图 8.9 中最靠近读者、以线程标出的输出平面所需的全部点。所有线程先把第一层加载到共享内存数组 `inPrev_s`（第 8-10 行），再把第二层加载到 `inCurr_s`（第 11-13 行）。在图中，为了显示内部层，第一层已经从输入分块前侧剥离。

第一次迭代期间，线程块中的所有线程协作把当前输出平面所需的第三个输入层加载到共享内存数组 `inNext_s`（第 15-17 行），然后在屏障处等待所有线程完成加载（第 18 行）。第 19-21 行的条件与图 8.8 共享内存内核中的对应条件作用相同。

随后，每个线程使用 `inCurr_s` 中的四个 $x-y$ 邻点、`inPrev_s` 中的一个 z 邻点以及 `inNext_s` 中的另一个 z 邻点，计算当前输出分块平面中的一个输出网格点。之后线程块中的所有线程再次在屏障处等待，确保每个线程都完成计算，再进入下一个输出分块平面。离开屏障后，所有线程协作把 `inCurr_s` 的内容移入 `inPrev_s`，再把 `inNext_s` 的内容移入 `inCurr_s`。这是因为线程沿 z 方向前进一个输出平面后，各输入分块平面承担的角色也随之改变。每次迭代结束时，线程块已经保存了计算下一输出平面所需的三个输入平面中的两个；进入下一次迭代后，只需再加载第三个输入平面。为计算第二个输出平面而更新后的数组映射如图 8.11 所示。

线程粗化内核的优点是：无需增加线程数就能扩大分块，也无需让输入分块的所有平面同时驻留在共享内存中。此时线程块大小只有 t^2 ，而不是 t^3 ，其中 t 是输入分

```

1  __global__ void stencil_kernel(float* in, float* out, unsigned int N) {
2      int iStart = blockIdx.z*OUT_TILE_DIM;
3      int j = blockIdx.y*OUT_TILE_DIM + threadIdx.y - 1;
4      int k = blockIdx.x*OUT_TILE_DIM + threadIdx.x - 1;
5      __shared__ float inPrev_s[IN_TILE_DIM][IN_TILE_DIM];
6      __shared__ float inCurr_s[IN_TILE_DIM][IN_TILE_DIM];
7      __shared__ float inNext_s[IN_TILE_DIM][IN_TILE_DIM];
8      if(iStart-1 >= 0 && iStart-1 < N && j >= 0 && j < N && k >= 0 && k < N) {
9          inPrev_s[threadIdx.y][threadIdx.x] = in[(iStart - 1)*N*N + j*N + k];
10     }
11     if(iStart >= 0 && iStart < N && j >= 0 && j < N && k >= 0 && k < N) {
12         inCurr_s[threadIdx.y][threadIdx.x] = in[iStart*N*N + j*N + k];
13     }
14     for(int i = iStart; i < iStart + OUT_TILE_DIM; ++i) {
15         if(i + 1 >= 0 && i + 1 < N && j >= 0 && j < N && k >= 0 && k < N) {
16             inNext_s[threadIdx.y][threadIdx.x] = in[(i + 1)*N*N + j*N + k];
17         }
18         __syncthreads();
19         if(i >= 1 && i < N - 1 && j >= 1 && j < N - 1 && k >= 1 && k < N - 1) {
20             if(threadIdx.y >= 1 && threadIdx.y < IN_TILE_DIM - 1
21                && threadIdx.x >= 1 && threadIdx.x < IN_TILE_DIM - 1) {
22                 out[i*N*N + j*N + k] = c0*inCurr_s[threadIdx.y][threadIdx.x]
23                     + c1*inCurr_s[threadIdx.y][threadIdx.x-1]
24                     + c2*inCurr_s[threadIdx.y][threadIdx.x+1]
25                     + c3*inCurr_s[threadIdx.y+1][threadIdx.x]
26                     + c4*inCurr_s[threadIdx.y-1][threadIdx.x]
27                     + c5*inPrev_s[threadIdx.y][threadIdx.x]
28                     + c6*inNext_s[threadIdx.y][threadIdx.x];
29             }
30         }
31         __syncthreads();
32         inPrev_s[threadIdx.y][threadIdx.x] = inCurr_s[threadIdx.y][threadIdx.x];
33         inCurr_s[threadIdx.y][threadIdx.x] = inNext_s[threadIdx.y][threadIdx.x];
34     }
35 }
36

```

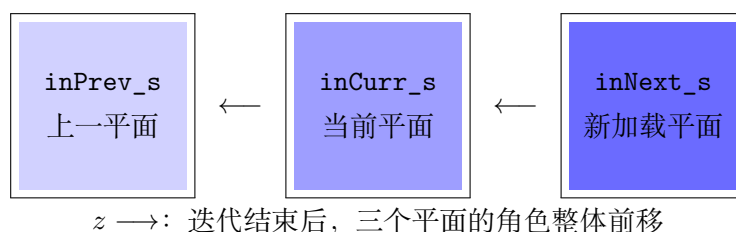
图 8.10: 沿 z 方向执行线程粗化的三维七点模板扫掠内核（依据原书图 8.10 重排）

图 8.11: 第一次迭代后，共享内存数组到输入分块平面的映射（依据原书图 8.11 重绘）

块的维度，因此可以采用大得多的 t 。例如， $t = 32$ 时线程块大小为 $t^2 = 1024$ 。在此 t 值下，内核的算术强度为 1.52 FLOP/B，相比原共享内存分块内核采用 $8 \times 8 \times 8$ 输入分块时的 0.96 FLOP/B 有显著提升，也更接近 1.625 FLOP/B 的理想值。

此外，任意时刻只需把输入分块的三个层放入共享内存。共享内存容量需求从 t^3 个元素降为 $3t^2$ 个元素。对于 $t = 32$ ，每个线程块的共享内存消耗为 $3 \cdot 32^2 \cdot 4 = 12$ KB，处于合理范围内。

8.6 寄存器分块

某些模板模式的特殊性质会带来新的优化机会。这里介绍一种对如下模板尤其有效的优化：模板只涉及中心点沿 x 、 y 、 z 方向的邻点。图 8.3 中的所有模板都符合这一描述，图 8.10 的三维七点模板扫掠内核也体现了该性质。

在计算具有相同 x - y 下标的输出分块网格点时，每个 `inPrev_s` 和 `inNext_s` 元素都只由一个线程使用。只有 `inCurr_s` 元素会被多个线程访问，真正需要放在共享内存中。因此，`inPrev_s` 和 `inNext_s` 中的 z 邻点可以改为保存在唯一使用它们的线程的寄存器中。第 5、6 章已经讨论过，这种把数据留在寄存器中以减少共享内存或全局内存访问的技术称为**寄存器分块 (register tiling)**。

图 8.12 在输入分块的上一平面和下一平面上应用寄存器分块。该内核以图 8.10 的线程粗化内核为基础，只做了几处简单但重要的修改。首先建立三个寄存器变量 `inPrev`、`inCurr` 和 `inNext`（第 5、7、8 行）。寄存器变量 `inPrev` 和 `inNext` 分别替代共享内存数组 `inPrev_s` 和 `inNext_s`，但仍保留 `inCurr_s`，使线程能够共享 x - y 邻点值。因此，该内核使用的共享内存降至图 8.10 内核的三分之一。

译者注： 原书图 8.12 最后一条赋值把右值写为已经被寄存器变量替代的 `inNext_s`；结合本节文字和循环中的变量角色，此处应为 `inNext`，代码清单已按这一含义更正。原书同节还把共享内存用量的比较对象印为图 8.12；从上下文可知应与改写前的图 8.10 比较。

输入分块上一平面和当前平面的初始加载（第 9–15 行），以及每次新迭代前下一平面的加载（第 17–19 行），都以寄存器变量作为目的位置。因此，输入分块“活跃部分”的三个平面分散保存在同一线程块各线程的寄存器中。内核还始终在共享内存中保存输入分块当前平面的一个副本（第 14、36 行），使活跃输入平面上的 x - y 邻点一直可供所有需要它们的线程访问。

图 8.10 和图 8.12 的内核都只把输入分块的“活跃部分”保存在片上内存中。活跃部分的平面数取决于模板阶数；对三维七点模板而言是 3 个平面。图 8.12 的线程粗化与寄存器分块内核，相比图 8.10 只有线程粗化的内核具有两个优点。第一，原来对共享内存的许多读写现在改由寄存器完成。寄存器延迟明显更低、带宽更高，因

```

1  __global__ void stencil_kernel(float* in, float* out, unsigned int N) {
2      int iStart = blockIdx.z*OUT_TILE_DIM;
3      int j = blockIdx.y*OUT_TILE_DIM + threadIdx.y - 1;
4      int k = blockIdx.x*OUT_TILE_DIM + threadIdx.x - 1;
5      float inPrev;
6      __shared__ float inCurr_s[IN_TILE_DIM][IN_TILE_DIM];
7      float inCurr;
8      float inNext;
9      if(iStart-1 >= 0 && iStart-1 < N && j >= 0 && j < N && k >= 0 && k < N) {
10         inPrev = in[(iStart - 1)*N*N + j*N + k];
11     }
12     if(iStart >= 0 && iStart < N && j >= 0 && j < N && k >= 0 && k < N) {
13         inCurr = in[iStart*N*N + j*N + k];
14         inCurr_s[threadIdx.y][threadIdx.x] = inCurr;
15     }
16     for(int i = iStart; i < iStart + OUT_TILE_DIM; ++i) {
17         if(i + 1 >= 0 && i + 1 < N && j >= 0 && j < N && k >= 0 && k < N) {
18             inNext = in[(i + 1)*N*N + j*N + k];
19         }
20         __syncthreads();
21         if(i >= 1 && i < N - 1 && j >= 1 && j < N - 1 && k >= 1 && k < N - 1) {
22             if(threadIdx.y >= 1 && threadIdx.y < IN_TILE_DIM - 1
23                && threadIdx.x >= 1 && threadIdx.x < IN_TILE_DIM - 1) {
24                 out[i*N*N + j*N + k] = c0*inCurr
25                     + c1*inCurr_s[threadIdx.y][threadIdx.x-1]
26                     + c2*inCurr_s[threadIdx.y][threadIdx.x+1]
27                     + c3*inCurr_s[threadIdx.y+1][threadIdx.x]
28                     + c4*inCurr_s[threadIdx.y-1][threadIdx.x]
29                     + c5*inPrev
30                     + c6*inNext;
31             }
32         }
33         __syncthreads();
34         inPrev = inCurr;
35         inCurr = inNext;
36         inCurr_s[threadIdx.y][threadIdx.x] = inNext;
37     }
38 }
39

```

图 8.12: 沿 z 方向同时采用线程粗化与寄存器分块的三维七点模板扫描内核（依据原书图 8.12 重排）

此代码有望运行得更快。第二，每个线程块的共享内存消耗只剩原来的三分之一。

当然，这一改进的代价是每个线程多使用三个寄存器；若采用 32×32 线程块，每个线程块总共会多用 3072 个寄存器。模板阶数更高时，寄存器用量还会进一步上升。如果寄存器用量成为问题，可以改回把部分平面存放在共享内存中。这体现了共享内存用量与寄存器用量之间经常需要做出的权衡。

总体而言，数据复用现在分散在寄存器和共享内存中。全局内存访问次数没有变化；把寄存器和共享内存一并考虑时，总体数据复用率与只用共享内存保存输入分块的线程粗化内核相同。因此，在线程粗化基础上增加寄存器分块，不会改变全局内存带宽消耗。

8.7 小结

本章深入讨论了模板扫掠计算。它看起来只是采用特殊滤波器模式的卷积，但模板源于求解微分方程时对导数的离散化和数值近似，因此具有两项能够促成新优化的特征。

第一，模板扫掠通常在三维网格上执行，而卷积通常用于二维图像或少量二维图像的时间切片。这使二者的分块考量不同，并促使三维模板采用线程粗化，以获得更大的输入分块和更高的算术强度。第二，某些模板模式允许对输入数据执行寄存器分块，从而进一步提高数据访问吞吐量，并缓解共享内存压力。

8.8 练习

1. 考虑在包含边界单元的 $120 \times 120 \times 120$ 网格上执行三维模板计算。
 - a. 每次模板扫掠会计算多少个输出网格点？
 - b. 对图 8.6 的基本内核，假设线程块大小为 $8 \times 8 \times 8$ ，需要多少个线程块？
 - c. 对图 8.8 的共享内存分块内核，假设线程块大小为 $8 \times 8 \times 8$ ，需要多少个线程块？
 - d. 对图 8.10 同时采用共享内存分块和线程粗化的内核，假设线程块大小为 32×32 ，需要多少个线程块？
2. 考虑一个同时采用共享内存分块和线程粗化的三维七点模板实现。该实现与图 8.10 和图 8.12 类似，但分块不是规则立方体；它采用 32×32 的线程块，并把粗化因子设为 16，也就是说，每个线程块沿 z 维处理连续的 16 个输出平面。
 - a. 在线程块的整个生命周期中，它所加载的输入分块大小是多少（以元素个数计）？

- b. 在线程块的整个生命周期中，它所处理的输出分块大小是多少（以元素个数计）？
- c. 该内核的算术强度是多少（以 FLOP/B 计）？
- d. 如果不使用寄存器分块，如图 8.10 所示，每个线程块需要多少字节共享内存？
- e. 如果使用寄存器分块，如图 8.12 所示，每个线程块需要多少字节共享内存？

第九章 直方图

到目前为止，我们介绍的并行计算模式都可以把每个输出元素的计算工作排他地分配给某个线程，也就是说，每个输出元素都有自己的“所有者”。因此，这些模式都适用 **所有者计算规则 (owner-computes rule)**：每个线程只写入指定给自己的输出元素，无须担心其他线程的干扰。本章介绍并行直方图模式。在这种模式中，每个输出元素都有可能被任意线程更新。多个线程更新输出元素时必须彼此协调，以免相互干扰，破坏最终结果。

实际应用中还有许多重要的并行计算模式，其输出干扰并不容易消除。并行直方图模式为这类干扰提供了一个典型例子。我们先考察一种基线方法：用原子操作把对每个元素的更新串行化。这种方法简单，却不够高效，执行速度往往令人失望。随后介绍几种广泛使用的优化技术，其中最重要的是私有化；这些技术在保证正确性的同时，可以显著提高执行速度。它们的成本和收益既取决于底层硬件，也取决于输入数据的特征。因此，开发人员必须理解这些技术的核心思想，并能判断它们在不同情形下是否适用。

9.1 背景

直方图 (histogram) 用来显示数据集中各个数据值的出现次数。最常见的直方图把取值区间画在横轴上，再用从横轴向上延伸的矩形（即柱）的高度表示各区间内数据值的频数。例如，图 9.1(b) 是图 9.1(a) 中树木灰度图像的像素强度直方图。灰度图像的像素强度通常介于 0 和 255 之间，其中 0 表示黑色，255 表示白色。图 9.1(b) 把这一范围分成四个区间，每个区间包含 64 个取值。描绘树干的 6 个黑色像素计入区间 $[0, 63]$ ；描绘树叶的 12 个深灰色像素计入 $[64, 127]$ ；描绘树木周围草地的 14 个浅灰色像素计入 $[128, 191]$ ；描绘天空的 32 个白色像素计入 $[192, 255]$ 。

直方图能为数据集提供有用的概括。在本例中，图像像素大量集中于明亮的高强度区间，而落在较暗的低强度区间中的像素很少。直方图的这种形状有时称为数据集的 **特征 (feature)**，可以帮助我们迅速判断数据中是否存在显著现象，例如图像曝光偏低还是偏高。再举一例，信用卡账户在购买类别和购买地点上的直方图形状可以用于检测欺诈性使用。一旦直方图的形状明显偏离常态，系统便会标记潜在风险。

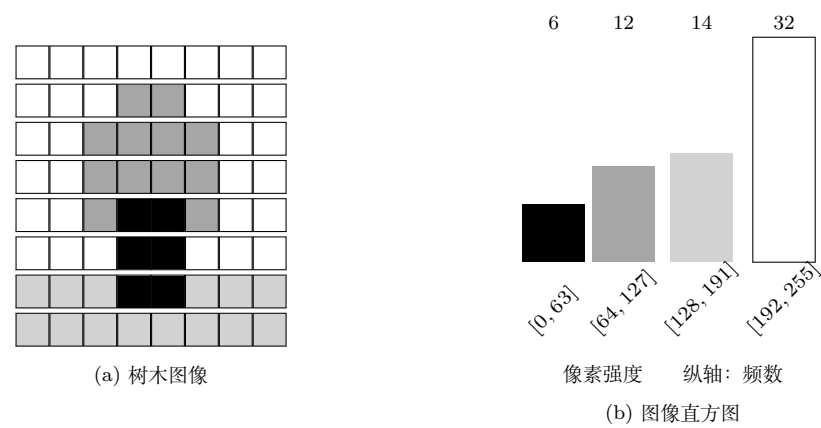


图 9.1: 图像直方图示例

许多应用领域都依靠直方图概括数据集，以便开展数据分析，计算机视觉就是其中之一。不同类型的物体图像（例如人脸和汽车）往往具有不同形状的直方图。正如前例所示，我们既可以绘制整幅图像的像素强度直方图，也可以只绘制图像某一区域的直方图。晴天天空的直方图可能只在光强谱的高强度区间出现少数几根很高的柱。把图像划分为多个子区域并分析各区域的直方图，就能快速找出可能包含目标物体、值得进一步处理的区域。

计算图像子区域的直方图，是计算机视觉中提取特征的一种重要方法；这里的“特征”是指图像中值得关注的模式。一般来说，只要需要从海量数据中提炼有意义的事件，直方图就很可能充当基础计算。信用卡欺诈检测和计算机视觉显然符合这一描述。其他具有类似需求的应用还包括语音识别、网站购买推荐，以及科学数据分析，例如关联天体物理学中不同天体的运动。

直方图很容易按顺序方式计算。图 9.2 给出了计算图像直方图的顺序函数。C 代码假设输入图像存放在 `unsigned char` 数组 `image` 中，每个元素给出一个像素的灰度强度（第 1 行）；直方图写入 `unsigned int` 数组 `bins`（第 1 行）。图像的像素数由函数参数 `width` 和 `height` 指定（第 2 行）。for 循环（第 3 行）依次遍历图像像素。由于像素的二维下标与这项计算无关，代码把图像视为长度为 `width*height` 的一维数组。它从图像中加载像素值（第 4 行），再递增与该像素值对应的直方图分箱 (`bin`)（第 5 行）。这里假设每个分箱只代表一个像素值。如果分箱像图 9.1 那样表示一个像素值区间，那么在索引分箱数组之前，应先用像素值除以区间宽度。

图 9.2 中的 C 代码简单而高效。算法的计算复杂度为 $O(N)$ ，其中 N 是输入数据元素个数。for 循环按顺序访问 `image` 数组元素，因此从系统 DRAM 取入 CPU 缓存行后，缓存行中的数据能够得到充分利用。`bins` 数组很小，完全可以放入 CPU 的一级 (L1) 数据缓存，从而快速更新其中的元素。对大多数现代 CPU 而言，这段代码的执行速度预计会受内存限制，也就是受数据元素从 DRAM 搬入 CPU 缓存的速率限制。

```

1 void histogram_sequential(unsigned char* image, unsigned int* bins,
2                           unsigned int width, unsigned int height) {
3     for(unsigned int i = 0; i < width*height; ++i) {
4         unsigned char b = image[i];
5         ++bins[b];
6     }
7 }
8

```

图 9.2: 计算灰度图像直方图的简单 C 函数

9.2 原子操作与基本直方图内核

并行化直方图计算最直接的办法，是启动与输入元素同样多的线程，让每个线程处理一个输入元素。每个线程读取分配给自己的像素值，并递增对应的分箱。图 9.3 给出了这种并行化策略的示例。多个线程可能分到取值相同的像素，因而需要更新同一个分箱；这种冲突称为**输出干扰 (output interference)**。要在并行代码中可靠地处理此类输出干扰，程序员必须理解竞态条件和原子操作。

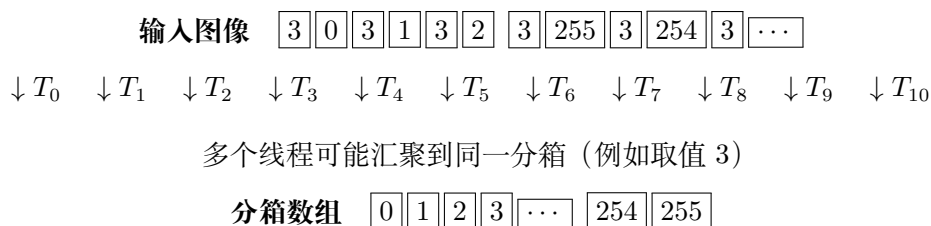


图 9.3: 图像直方图的基本并行化方式

递增 bins 数组中的某个分箱计数器，是对一个内存位置执行的更新操作，也就是**读-改-写 (read-modify-write)** 操作。它先读取内存位置中的值（读），给原值加 1（改），再把新值写回该内存位置（写）。读-改-写经常用于协调多个参与者共同完成的活动。

例如，通过航空公司预订航班时，我们先调出座位图寻找空位（读），选择一个要预订的座位（改），再把座位图中该座位的状态改为不可用（写）。以下情况可能导致错误：

- 两位乘客同时打开同一航班的座位图；
- 两人都选择了同一个座位，例如 9C；
- 两人都把座位 9C 的状态改为不可用。

上述操作完成后，两位乘客都以为自己订到了 9C。登机时，他们才会发现其中一人无法使用预订座位，场面自然很不愉快。尽管听起来不可思议，现实中确实会因

为航空订票软件的缺陷发生这类问题。

再举一个例子。有些商店让顾客取号候客，无须一直排队。顾客从某台取号机领取号码，显示屏则显示下一位接受服务的号码。服务人员空闲后，会请持有对应号码票据的顾客上前，核对票据，再把显示屏上的号码加一。理想情况下，所有顾客都会按进入商店的先后顺序得到服务。一种不希望出现的结果是，两位顾客同时在两台取号机上登记，却拿到了相同号码。当服务人员呼叫这个号码时，两人都会认为应该轮到自己。

这两个例子中的错误结果都源于**读-改-写竞态条件 (read-modify-write race condition)**：两个或更多更新操作同时发生时，结果会随这些操作相对时序的不同而变化，有些时序得到正确结果，有些则得到错误结果。图 9.4 展示了图像直方图示例中两个线程试图更新同一 bins 元素时的竞态条件。图中每一行对应一个时间段，时间从上向下推进。

时间	线程 1	线程 2	时间	线程 1	线程 2
1	(0) old <- bins[x]		1	(0) old <- bins[x]	
2	(1) new <- old + 1		2	(1) new <- old + 1	
3	(1) bins[x] <- new		3		(0) old <- bins[x]
4		(1) old <- bins[x]	4	(1) bins[x] <- new	
5		(2) new <- old + 1	5		(1) new <- old + 1
6		(2) bins[x] <- new	6		(1) bins[x] <- new

(a) 正确：线程 1 先完成

(b) 错误：两个更新交叠

图 9.4: 更新 bins 数组元素时的竞态条件

图 9.4(a) 描绘了这样一种时序：线程 1 在时间段 1-3 完成读-改-写的全部三步，线程 2 到时间段 4 才开始自己的操作序列。每项操作前括号内的数值表示写入目标的值，并假定 bins[x] 的初值为 0。执行完毕后，bins[x] 的值是 2，正是预期结果：两个线程都成功把它递增了一次，元素值从 0 变成 2。

图 9.4(b) 中，两个线程的读-改-写序列发生交叠。线程 1 在时间段 4 才把新值写入 bins[x]；线程 2 在时间段 3 读取 bins[x] 时，看到的仍是 0。因此，它计算并最终写入 bins[x] 的新值是 1，而不是 2。问题在于线程 2 读取 bins[x] 太早，发生在线程 1 完成更新之前。最终 bins[x] 的值为 1，这是错误的；线程 1 的更新丢失了。

并行执行时，线程之间可能以任意相对顺序运行。在本例中，线程 2 完全可能先于线程 1 开始更新。图 9.5 给出了两种这种时序。图 9.5(a) 中，线程 2 在线程 1 开始之前完成更新；图 9.5(b) 中，线程 1 在线程 2 完成之前开始更新。图 9.5(a) 中的序列会得到正确的 bins[x]，而图 9.5(b) 中的序列会得到错误结果。

bins[x] 的最终值随操作相对时序变化，这一事实表明确实存在竞态条件。要消除这种变化，就必须排除线程 1 和线程 2 的操作序列相互交错的可能性。也就是说，我们希望允许图 9.4(a) 和图 9.5(a) 所示的时序，同时排除图 9.4(b) 和图 9.5(b) 的情况。原子操作可以做到这一点。

对某个内存位置执行**原子操作 (atomic operation)**，是指以如下方式完成该位

时间	线程 1	线程 2	时间	线程 1	线程 2
1		(0) old <- bins[x]	1		(0) old <- bins[x]
2		(1) new <- old + 1	2		(1) new <- old + 1
3		(1) bins[x] <- new	3	(0) old <- bins[x]	
4	(1) old <- bins[x]		4		(1) bins[x] <- new
5	(2) new <- old + 1		5	(1) new <- old + 1	
6	(2) bins[x] <- new		6	(1) bins[x] <- new	

(a) 正确：线程 2 先完成

(b) 错误：两个更新交叠

图 9.5: 线程 2 先于线程 1 运行时的竞态条件

置上的读-改-写序列：其他任何针对同一位置的读-改-写序列都不能与它交叠。换句话说，读、改、写三个部分形成一个不可再分的整体，“原子”一名正源于此。实际硬件会锁住该位置，在当前操作完成之前排斥对同一位置的其他操作。在本例中，后到的线程必须等先到的线程完成更新序列才能开始，因此图 9.4(b) 和图 9.5(b) 的情况不再可能发生。

务必记住，原子操作不会在线程之间强制规定某个特定执行顺序。在本例中，图 9.4(a) 和图 9.5(a) 的两种顺序都允许出现：线程 1 既可以先于线程 2，也可以落后于线程 2。原子操作施加的规则是：如果两个线程都对同一内存位置执行原子操作，后到线程的原子操作必须等先到线程的原子操作完成后才能开始。这实际上把针对同一内存位置的原子操作串行化了。

CUDA 可以通过 C++ 原子对象、原子引用对象以及这些对象的方法函数执行原子操作。¹CUDA 扩展了 C++ 原子库，可以这样包含相应头文件：

```
#include <cuda/atomic>
```

包含原子库后，要对象执行原子操作，第一步是用 `cuda::atomic_ref` 类为它创建原子引用。²原子引用使我们能够对所引用的对象执行原子操作。CUDA C++ 库中 `cuda::atomic_ref` 类的定义如下：

```
template <typename T, cuda::thread_scope>
class cuda::atomic_ref;
```

`atomic_ref` 类模板在尖括号内接受两个参数。第一个模板参数 `T` 指定对象的类型，常用类型包括 `int`、`unsigned int` 和 `float`。第二个模板参数用于指定能够通过该类方法函数彼此同步的线程作用域，其名称是 `cuda::thread_scope`。常见作用域包括同一线程块内、同一 GPU 设备内，以及同一多 GPU 系统的所有 GPU 设备。前缀 `cuda::` 表明 `atomic_ref` 声明在 CUDA 命名空间中。更多细节可参阅 CUDA 编程指南。

¹不熟悉 C++ 对象、类和命名空间概念的读者，可以参考 C++ 文档 [1] 深入了解。概括地说，C++ 类规定如何创建某种对象；对象由一组数据和一组能够操作这些数据的方法函数组成。这里的类以模板形式定义，因而是泛型的。直观地说，模板允许库开发者给出通用对象，再由用户应用按数据类型和方法函数选项将其特化。

²另一种做法是使用 `cuda::atomic` 类，在对象创建之初就把它声明为原子对象。这样可以保证对象自创建起所有访问都是原子的。本例使用原子引用，是因为代码的其他部分可能仍需以非原子方式访问 `bins`。

现在可以实现图 9.3 所示的基本并行直方图方法了。图 9.6 给出一个 CUDA 内核，它利用原子操作保证对 bins 元素的并发递增是正确的。代码与图 9.2 的顺序版本相似，主要有两点区别。第一，遍历输入元素的循环改为线程索引计算（第 3 行）和边界检查（第 4 行），从而为每个输入图像元素分配一个线程。第二，图 9.2 中的递增表达式

```
++bins[b];
```

被图 9.6 的第 6-7 行取代。这两行代码保证不同线程对同一 bins 数组元素的并发更新以原子方式正确串行化。

```
1 __global__ void histogram_kernel(unsigned char* image, unsigned int* bins,
2                                 unsigned int width, unsigned int height) {
3     unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
4     if(i < width*height) {
5         unsigned char b = image[i];
6         cuda::atomic_ref<unsigned int, cuda::thread_scope_device> bins_ref(bins[b]);
7         bins_ref.fetch_add(1, cuda::memory_order_relaxed);
8     }
9 }
10
```

图 9.6: 计算直方图的 CUDA 内核

要以原子方式执行更新，先为 bins[b] 构造名为 bins_ref 的原子引用（第 6 行）。回忆一下，cuda::atomic_ref 类是模板。第 6 行的第一个模板参数 unsigned int 表示要对无符号整数对象执行原子操作；这里的对象就是 bins 数组中的一个元素。第二个模板参数 cuda::thread_scope_device 是预定义的编译期常量，表示该原子操作必须允许不同线程块中的线程彼此同步。无论对同一 bins 元素的递增来自同一线程块，还是同一 GPU 设备上的不同线程块，都要串行化。这样便构造出原子引用 bins_ref，供代码对选定的 bins[b] 执行原子操作。第 6 行的构造表达式看起来很复杂，但它实际上主要是告诉 CUDA C++ 编译器如何为第 7 行生成代码，运行时开销很小。

构造原子引用后，就可以调用它的方法函数，对所引用的对象执行原子操作。CUDA 原子引用类提供了加、最小值、最大值、与、或、异或等多种操作。直方图内核需要执行整数递增：先取得旧值，加 1 得到新值，再把新值写入 bins 元素。执行原子加法的方法函数是 fetch_add；CUDA C++ 库把它定义为 cuda::atomic_ref 类的泛型方法函数：

```
T fetch_add(T arg, cuda::std::memory_order order)
```

fetch_add 函数接受两个参数。第一个参数 arg 指定要加上的值，本例中是 1。这个值必须与定义 fetch_add 的原子引用类的第一个模板参数类型 T 相同，从而保证加数与对象本身类型一致。图 9.6 第 7 行对无符号整数对象 bins[b] 调用 fetch_add；编译器会把传入的第一个实参，即常量 1，按无符号整数常量处理。

第二个参数 `order` 是 C++ 标准库定义的编译期常量，用于限制当前原子操作相对于调用线程其他内存访问的重排方式。编译器或硬件可能重排内存访问的执行次序，以优化代码性能；`order` 参数按照程序员意图指定允许哪类重排。本例中，调用线程只有另一项内存访问，即把 `image[i]` 读入局部变量 `b`。由于 `b` 用于选择执行原子操作的对象，编译器无法把原子操作重排到这次内存访问之前。运行时，这种选择关系又会向硬件调度器呈现指令级数据依赖：原子操作自然必须等待读取 `image[i]` 的内存访问完成。因此，无须再增加限制来阻止二者相互重排。图 9.6 第 7 行使用的编译期常量 `cuda::memory_order_relaxed` 表示，除指令级已有约束外不施加更多限制。第 11 章会遇到必须为内存访问重排指定额外限制的用例。

`fetch_add` 函数还会返回一个与第一个模板参数同类型 (`T`) 的值：在把第一个参数原子地加到对象上之前，对象原来的值。图 9.6 第 7 行没有使用这个返回值，因为该线程的任务只是递增所选分箱，并不关心递增前的旧值。第 12 章和第 18 章还会看到需要使用 `fetch_add` 返回值的情形。

9.3 原子操作的延迟与吞吐量

图 9.6 的内核利用原子操作，把针对某个位置同时发生的更新串行化，从而保证更新结果正确。大规模并行程序的任何一部分一旦串行化，都可能显著延长执行时间、降低程序速度。因此，必须尽可能缩短这类串行操作占用的执行时间。

第 5 章已经介绍过，访问 DRAM 数据可能需要数百个时钟周期。第 4 章说明 GPU 如何通过零周期上下文切换容忍这类延迟。第 6 章又指出，只要有许多线程的内存访问延迟能够彼此重叠，执行速度就由内存系统的吞吐量决定。因此，GPU 必须充分利用 DRAM 突发、bank 和通道，才能获得较高的内存访问吞吐量。

读者此时应该已经很清楚：要提高内存访问吞吐量，关键是让许多 DRAM 访问同时进行。遗憾的是，当大量原子操作更新同一个内存位置时，这一策略就会失效。后到线程的读-改-写序列必须等先到线程的读-改-写序列结束后才能开始。如图 9.7 所示，在同一内存位置上，任何时刻只能有一个原子操作正在进行。每个原子操作的持续时间近似等于一次内存加载的延迟（图中原子操作时间的左半段）加上一次内存存储的延迟（右半段）。每个读-改-写操作的这些时间段通常长达数百个时钟周期，决定了服务一个原子操作所需的最短时间，因而限制了原子操作的吞吐量，也就是单位时间能够完成的原子操作数。

例如，假设一个内存系统的每条通道都有 64 位 (8 B) 双倍数据速率 DRAM 接口，共有 8 条通道，时钟频率为 1 GHz，典型访问延迟为 200 个周期。该系统的峰值访问吞吐量为

$$8 \text{ B/传输} \times 2 \text{ 传输}/(\text{周期} \cdot \text{通道}) \times 10^9 \text{ 周期/s} \times 8 \text{ 通道} = 128 \text{ GB/s.}$$

若每个被访问的数据元素占 4 B，则峰值访问吞吐量为每秒 320 亿个数据元素。

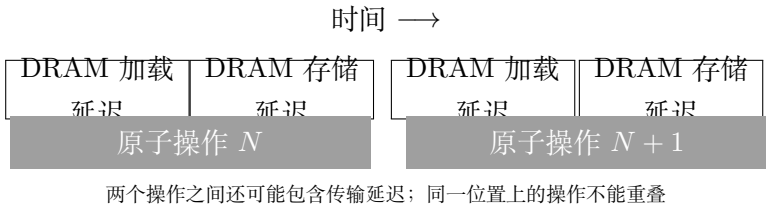


图 9.7: 原子操作的吞吐量由内存访问延迟决定

然而，对某一个特定内存位置执行原子操作时，能达到的最高吞吐量是每 400 个周期完成一次原子操作，其中读取用 200 个周期，写入再用 200 个周期。换算后得到

$$\frac{1}{400} \text{ 原子操作/周期} \times 10^9 \text{ 周期/s} = 2.5 \times 10^6 \text{ 原子操作/s.}$$

这个结果远低于大多数用户对 GPU 内存系统的预期。此外，这串原子操作的长延迟很可能支配内核执行时间，导致内核执行速度大幅下降。

实际程序并不会把所有原子操作都施加到同一个内存位置。图像直方图示例包含 256 个分箱。如果输入像素的亮度均匀分布，原子操作也会均匀分散到各个 bins 元素，吞吐量就可以提高到

$$256 \times 2.5 \times 10^6 = 6.4 \times 10^8 \text{ 原子操作/s.}$$

现实中的提升倍数通常远小于直方图的分箱数，因为像素亮度往往存在偏斜分布。例如，图 9.1 中的像素就明显偏向较亮的强度级。大量更新高亮度分箱的争用流量，很可能显著压低实际可达到的吞吐量。

提高原子操作吞吐量的一种办法，是缩短对高争用位置的访问延迟。缓存正是降低内存访问延迟的主要工具。为此，现代 GPU 允许在所有 SM 共享的末级缓存中执行设备作用域的原子操作。如果待更新变量已经在末级缓存中，原子操作就在缓存内更新它；如果不在，则触发缓存未命中，把变量取入缓存后再更新。由原子操作更新的变量通常会被许多线程频繁访问，因此在线程持续执行原子操作期间，这些变量往往能够留在缓存中。末级缓存的访问时间只有数十个周期，而不是数百个周期，所以这种方法至少可以把原子操作吞吐量提高一个数量级。这也是大多数现代 GPU 在末级缓存中支持原子操作的重要原因。

9.4 私有化

提高原子操作吞吐量的另一种办法，是把访问流量从高争用位置引开，缓解争用。并行计算中经常使用**私有化 (privatization)** 技术解决严重的输出干扰问题。其基本思路是为高争用的输出数据结构复制若干私有副本，让每个线程子集更新自己的副本。私有副本受到的争用少得多，访问延迟也往往更低，因此可以显著提高更新数据结构的吞吐量。代价是计算结束后必须把这些私有副本合并到最终输出数据结构中。

设计时需要仔细权衡争用程度和合并成本。因此，在大规模并行系统中，私有化通常针对线程子集进行，而不是为每个线程都建立一份副本。

在本章的直方图示例中，可以建立多个私有直方图，并指定各线程子集更新其中一份。例如，建立两个私有副本，让索引为偶数的线程块更新一份，索引为奇数的线程块更新另一份。也可以建立四个副本，让索引为 i 的线程块更新编号为 $i \bmod 4$ 的副本。一种常见做法是为每个线程块建立一个私有副本，后文会看到它有多项优点。无论采用哪种分配方式，私有直方图计算完成后，都要把它们逐项相加并合并到公共副本。由于加法满足交换律和结合律，各私有副本加到公共副本中的次序并不重要。

译者注：原书本段把示例写成“text histogram example”，但本章前后及图 9.1、图 9.3 均讨论图像直方图；译稿按上下文译为“本章的直方图示例”。

图 9.8 展示了如何把私有化用于图 9.3 的图像直方图示例。线程被组织成线程块，本玩具示例中每块包含 8 个线程，实际线程块要大得多。每个线程块得到一份由它更新的私有直方图。这样，更新同一分箱时的争用不再发生于所有线程之间，而只发生于同一线程块内的线程之间，以及最后合并私有副本时的不同线程块之间。

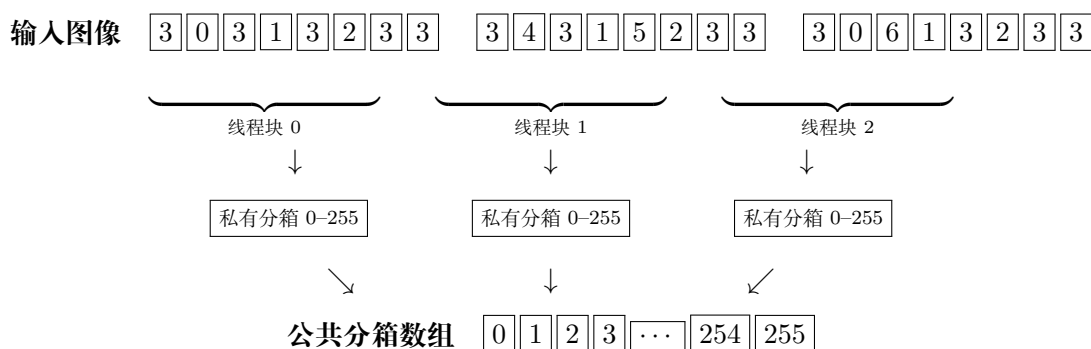


图 9.8: 直方图的私有副本能够减少原子操作的争用

图 9.9 给出一个简单内核，为每个线程块建立并关联一份私有直方图。在这种方案下，同一份私有直方图最多由 1024 个线程协作更新，这是当前硬件允许的每块最大线程数。这个内核把私有直方图放在全局内存中；它们很可能被缓存，从而降低延迟、提高吞吐量。

图 9.9 的内核新增了参数 `bins_pool`（第 2 行），它指向主机分配的一片全局内存，用于存放所有直方图私有副本，总容量为 `gridDim.x*NUM_BINS` 个整数。每个线程在 `bins_pool` 数组上加上偏移 `blockIdx.x*NUM_BINS`，找到所属线程块的私有分箱数组 `bins_priv`。该偏移把位置移动到当前线程所属线程块的私有副本。

译者注：原书图 9.9 没有在内核中初始化 `bins_pool`。要使代码正确，主机端必须在启动内核前把这片 `gridDim.x*NUM_BINS*sizeof(unsigned int)` 字节的内存清零；公共 `bins` 数组也必须按直方图累加的通常约定预先清零。代码清单保留底本结构，这里明确补出其隐含前置条件。

图 9.9 第 6–12 行与图 9.6 的内核相似，主要有两点区别。第一，代码使用

```

1 __global__ void histogram_kernel(unsigned char* image, unsigned int* bins,
2                                 unsigned int* bins_pool,
3                                 unsigned int width, unsigned int height) {
4     unsigned int* bins_priv = &bins_pool[blockIdx.x*NUM_BINS];
5
6     unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
7     if(i < width*height) {
8         unsigned char b = image[i];
9         cuda::atomic_ref<unsigned int, cuda::thread_scope_block>
10         bins_priv_ref(bins_priv[b]);
11         bins_priv_ref.fetch_add(1, cuda::memory_order_relaxed);
12     }
13     __syncthreads();
14
15     for(unsigned int b = threadIdx.x; b < NUM_BINS; b += blockDim.x) {
16         if(bins_priv[b] > 0) {
17             cuda::atomic_ref<unsigned int, cuda::thread_scope_device>
18             bins_ref(bins[b]);
19             bins_ref.fetch_add(bins_priv[b], cuda::memory_order_relaxed);
20         }
21     }
22 }
23

```

图 9.9: 每个线程块在全局内存中拥有私有分箱的直方图内核

`bins_priv` 而不是 `bins`，所以每个线程块只更新自己的私有分箱。这时，争用程度大致按所有 SM 上活动线程块的总数成比例下降，因而可能显著提高内核的更新吞吐量。

第二，原子引用使用 `cuda::thread_scope_block` 作用域（第 9 行），也就是线程块作用域，而不是设备作用域。换句话说，通过这个引用发起的原子操作只需与同一线程块内的其他线程同步，不必覆盖整个设备。具体收益取决于硬件怎样实现原子操作；更窄的作用域可能带来更短的延迟和更高的吞吐量。之所以能缩小作用域，是因为只有同一线程块中的线程会争用该块私有直方图中的分箱。

执行末尾，每个线程块把私有副本中的值提交到 `bins` 指向的公共副本（第 15–21 行）。块内线程先互相等待，直到所有线程都完成对私有副本的更新（第 13 行）。接着，线程遍历私有直方图的分箱（第 15 行），每个线程负责提交一个或多个私有分箱。采用循环是为了支持任意分箱数，包括分箱数大于块内线程数的情形。每次迭代中，线程读取自己负责的私有分箱，检查它是否非零（第 16 行）；如果非零，就以原子加法把该值提交到公共副本（第 17–19 行）。加法必须是原子的，因为多个线程块中的线程可能同时向同一位置相加。在内核执行的这一阶段，每个线程块只有一个线程更新任意给定的 `bins` 数组元素，所以各位置的争用程度不高。

按线程块建立私有直方图有一项好处：提交私有副本之前，块内线程可以使用 `__syncthreads()` 互相等待。另一项好处是，如果直方图的分箱足够少，就可以把私有副本声明在线程块的共享内存中。回忆一下，延迟只要降低，就会直接提高同一内存位置上原子操作的吞吐量。因此，把分箱放入共享内存可以显著缩短访问延迟。共

享内存在物理上位于 SM 内，由各线程块私有使用，访问延迟只有几个周期。这一延迟降幅会直接转化为原子操作吞吐量的大幅提升。

对支持线程块集群和分布式共享内存的 GPU，还可以为每个集群建立一份私有直方图。同一集群中的线程块可以使用集群级屏障同步，在把私有副本提交到公共副本之前互相等待。它们还可以把私有直方图放入分布式共享内存，从而支持更多分箱。

图 9.10 给出了一个私有化直方图内核，它把私有副本放在共享内存中，而不是全局内存中。它与图 9.9 的关键区别是：私有直方图分配在共享内存数组 `bins_s` 中（第 4 行），由块内线程并行初始化为 0（第 5–7 行）。屏障同步（第 8 行）保证私有直方图的全部分箱都正确初始化后，才有线程开始更新它们。其余代码与图 9.9 相同，只有一处差别：第一次原子操作针对共享内存数组 `bins_s` 的元素执行（第 14 行），之后也从该数组读取私有分箱值（第 20 行）。

```

1  __global__ void histogram_kernel(unsigned char* image, unsigned int* bins,
2                                unsigned int width, unsigned int height) {
3
4  __shared__ unsigned int bins_s[NUM_BINS];
5  for(unsigned int b = threadIdx.x; b < NUM_BINS; b += blockDim.x) {
6      bins_s[b] = 0;
7  }
8  __syncthreads();
9
10 unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
11 if(i < width*height) {
12     unsigned char b = image[i];
13     cuda::atomic_ref<unsigned int, cuda::thread_scope_block>
14         bins_s_ref(bins_s[b]);
15     bins_s_ref.fetch_add(1, cuda::memory_order_relaxed);
16 }
17 __syncthreads();
18
19 for(unsigned int b = threadIdx.x; b < NUM_BINS; b += blockDim.x) {
20     if(bins_s[b] > 0) {
21         cuda::atomic_ref<unsigned int, cuda::thread_scope_device>
22             bins_ref(bins[b]);
23         bins_ref.fetch_add(bins_s[b], cuda::memory_order_relaxed);
24     }
25 }
26 }
27

```

图 9.10: 共享内存私有分箱直方图内核

9.5 线程粗化

前文已经看到，私有化能够有效减少原子操作的争用，把私有直方图放入共享内存又能缩短每次原子操作的延迟。不过，私有化需要初始化直方图私有副本，并在最

后把这些副本提交到公共副本，这就是它的额外开销。每个线程块都要执行一次初始化和提交，所以线程块越多，开销越大。如果各线程块能够并行执行，这笔开销通常值得付出；但若启动的线程块数超过硬件能够同时执行的数量，硬件调度器就会把部分线程块串行执行，此时有些私有化开销便没有必要。

线程粗化可以降低私有化开销。具体来说，我们减少线程块数，让每个线程处理多个输入元素，从而减少需要初始化和提交到公共副本的私有副本数。本节考察两种把多个输入元素分配给线程的策略：**连续划分 (contiguous partitioning)** 和 **交错划分 (interleaved partitioning)**。

图 9.11 给出了连续划分策略的示例。输入被切分为多个连续段，每一段分配给一个线程。图中采用粗化因子 4；在第一次迭代中，每个线程访问自己连续段的第一个元素，在第二次迭代中访问第二个元素，后续迭代以此类推。

	$T_0 \downarrow$	$T_1 \downarrow$	$T_2 \downarrow$	$T_3 \downarrow$	$T_4 \downarrow$	$T_5 \downarrow$	$T_6 \downarrow$	$T_7 \downarrow$
第一次迭代	0 1,2,3	4 5,6,7	8 9,10,11	12 13,14,15	16 17,18,19	20 21,22,23	24 25,26,27	28 29,30,31
	$T_0 \downarrow$	$T_1 \downarrow$	$T_2 \downarrow$	$T_3 \downarrow$	$T_4 \downarrow$	$T_5 \downarrow$	$T_6 \downarrow$	$T_7 \downarrow$
第二次迭代	0,1 2,3	4,5 6,7	8,9 10,11	12,13 14,15	16,17 18,19	20,21 22,23	24,25 26,27	28,29 30,31

图 9.11: 输入元素的连续划分

图 9.12 展示了采用连续划分进行线程粗化的直方图内核。它与图 9.10 的区别在第 10–12 行。图 9.10 中，输入元素索引 i 就是全局线程索引，所以每个线程只得到一个输入元素。图 9.12 中，每个线程块分到一个连续的**块段 (block segment)**，段大小等于粗化因子 `COARSE_FACTOR` 乘以块内线程数 `blockDim.x`。块段偏移 `segment` 由线程块索引乘以段大小得到（第 10 行）。在每个块段内部，每个线程又得到一个大小为 `COARSE_FACTOR` 的连续**线程段 (thread segment)**。粗化循环遍历该线程段（第 11 行）。每次循环迭代中，线程的输入元素索引 i 等于块段偏移 `segment`、线程段在块内的偏移 `threadIdx.x*COARSE_FACTOR` 与粗化计数器 `c` 三者之和（第 12 行）。因此，每个线程都会访问一段连续的输入元素。

把数据划分成连续段，在概念上简单直观。CPU 的并行执行通常只涉及少量线程，因此连续划分往往性能最好：每个线程按顺序访问数据，能够充分利用缓存行。每个 CPU 缓存通常只服务少量线程，不同线程之间对缓存的干扰很小。一条缓存行为某线程取入后，其中的数据通常能够一直保留到该线程完成后续访问。

相比之下，在 GPU 上使用连续划分会产生次优的内存访问模式。第 6 章已经说明，SM 中同时活动的大量线程往往会造成严重的缓存干扰，不能指望某线程顺序访问所需的数据始终留在缓存中。我们应当保证一个 warp 中的线程访问连续位置，以便合并内存访问。这一观察引出了交错划分。

图 9.13 给出了交错划分策略的示例。第一次迭代中，8 个线程访问像素 0–7；借助内存访问合并，这些元素只需最少数量的内存事务即可取回。第二次迭代中，8 个线程同样以合并方式访问像素 8–15。之所以称为交错划分，是因为不同线程负责的


```
1 __global__ void histogram_kernel(unsigned char* image, unsigned int* bins,
2                                 unsigned int width, unsigned int height) {
3
4     __shared__ unsigned int bins_s[NUM_BINS];
5     for(unsigned int b = threadIdx.x; b < NUM_BINS; b += blockDim.x) {
6         bins_s[b] = 0;
7     }
8     __syncthreads();
9
10    unsigned int segment = COARSE_FACTOR*blockIdx.x*blockDim.x;
11    for(unsigned int c = 0; c < COARSE_FACTOR; ++c) {
12        unsigned int i = segment + threadIdx.x*COARSE_FACTOR + c;
13        if(i < width*height) {
14            unsigned char b = image[i];
15            cuda::atomic_ref<unsigned int, cuda::thread_scope_block>
16                bins_s_ref(bins_s[b]);
17            bins_s_ref.fetch_add(1, cuda::memory_order_relaxed);
18        }
19    }
20    __syncthreads();
21
22    for(unsigned int b = threadIdx.x; b < NUM_BINS; b += blockDim.x) {
23        if(bins_s[b] > 0) {
24            cuda::atomic_ref<unsigned int, cuda::thread_scope_device>
25                bins_ref(bins[b]);
26            bins_ref.fetch_add(bins_s[b], cuda::memory_order_relaxed);
27        }
28    }
29 }
30
```

图 9.12: 采用连续划分进行线程粗化的直方图内核

分区彼此交错。显然，这只是一个玩具示例，实际程序会有更多线程，也有更细致的性能考量。例如，为充分利用缓存与 SM 之间的互连带宽，每个线程在每次迭代中应处理四个像素，也就是一个 32 位字。

译者注：原书此处把图 9.13 第二次迭代写成“4 个线程访问字符 8–15”，但图中明确画出 8 个线程，处理对象也一直是图像像素。译稿按图示和算法含义改为“8 个线程访问像素 8–15”。

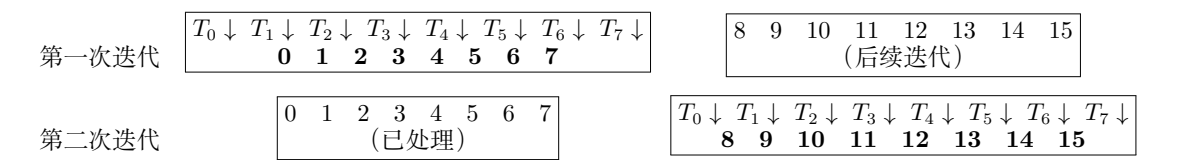


图 9.13: 输入元素的交错划分

图 9.14 给出了采用交错划分进行线程粗化的直方图内核。它与图 9.12 的区别只有第 12 行。循环第一次迭代时 `c` 为 0，每个线程使用自己的局部线程索引 `threadIdx.x` 访问块段中的数据：线程 0 访问 `segment + 0`，线程 1 访问 `segment + 1`，线程 2 访问 `segment + 2`，依此类推。因此，所有线程共同处理块段开头的 `blockDim.x` 个元素。第二次迭代中 `c` 为 1，所有线程都在索引上加 `blockDim.x`，共同处理接下来的 `blockDim.x` 个元素。一般而言，在第 `c` 次迭代中，每个线程都在首元素索引 `segment + threadIdx.x` 上加 `c*blockDim.x`，得到本次应处理的元素。

9.6 线程级私有化

对直方图内核应用线程粗化后，每个线程都会处理多个输入元素，这就为线程层级的另一次私有化创造了机会。我们可以为每个线程建立一份私有直方图，线程更新这份副本时无须使用原子操作。线程处理完自己的所有输入元素后，再用原子操作把线程级私有直方图提交到块级私有直方图。

这项优化受到直方图分箱数的限制。如果分箱很少，为每个线程分配一份直方图是可行的，线程甚至可以把副本放在共享内存或自己的寄存器中。但如果分箱很多，为每个线程都分配一份直方图就会代价过高。

一种折中方法是不为每个分箱都保存私有副本，只保存频繁访问的分箱。某些数据集会在局部区域中集中出现大量相同数据值，因此可以采用这种方法。例如，天空图像中可能有大片取值完全相同的像素。相同值的高度集中会给并行直方图计算造成严重争用，降低吞吐量，却也为线程级私有化提供了很好的机会。对这类数据集，一种简单而有效的优化是：如果一连串更新都针对直方图的同一元素，每个线程就把这些连续更新聚合成一次更新。

线程级私有化让每个线程先在该线程私有的分箱中累计计数，从而减少对高争用直方图分箱的原子操作次数，提高计算的有效吞吐量。

```
1 __global__ void histogram_kernel(unsigned char* image, unsigned int* bins,
2                                 unsigned int width, unsigned int height) {
3
4     __shared__ unsigned int bins_s[NUM_BINS];
5     for(unsigned int b = threadIdx.x; b < NUM_BINS; b += blockDim.x) {
6         bins_s[b] = 0;
7     }
8     __syncthreads();
9
10    unsigned int segment = COARSE_FACTOR*blockIdx.x*blockDim.x;
11    for(unsigned int c = 0; c < COARSE_FACTOR; ++c) {
12        unsigned int i = segment + c*blockDim.x + threadIdx.x;
13        if(i < width*height) {
14            unsigned char b = image[i];
15            cuda::atomic_ref<unsigned int, cuda::thread_scope_block>
16                bins_s_ref(bins_s[b]);
17            bins_s_ref.fetch_add(1, cuda::memory_order_relaxed);
18        }
19    }
20    __syncthreads();
21
22    for(unsigned int b = threadIdx.x; b < NUM_BINS; b += blockDim.x) {
23        if(bins_s[b] > 0) {
24            cuda::atomic_ref<unsigned int, cuda::thread_scope_device>
25                bins_ref(bins[b]);
26            bins_ref.fetch_add(bins_s[b], cuda::memory_order_relaxed);
27        }
28    }
29 }
30
```

图 9.14: 采用交错划分进行线程粗化的直方图内核

图 9.15 给出一个图像直方图内核，其中每个线程只私有化一个直方图分箱，即最近访问的分箱。与图 9.14 相比，主要变化如下。线程先在边界检查（第 11 行）后加载自己负责的第一个像素（第 12 行），并把线程级私有分箱计数 `bin_r` 初始化为 1（第 13 行）。由于第一个输入像素已经加载，粗化循环从 `c = 1` 开始（第 14 行）。

```

1  __global__ void histogram_kernel(unsigned char* image, unsigned int* bins,
2                                unsigned int width, unsigned int height) {
3
4  __shared__ unsigned int bins_s[NUM_BINS];
5  for(unsigned int b = threadIdx.x; b < NUM_BINS; b += blockDim.x) {
6      bins_s[b] = 0;
7  }
8  __syncthreads();
9
10 unsigned int segment = COARSE_FACTOR*blockIdx.x*blockDim.x;
11 if(segment + threadIdx.x < width*height) {
12     unsigned int b = image[segment + threadIdx.x];
13     unsigned int bin_r = 1;
14     for(unsigned int c = 1; c < COARSE_FACTOR; ++c) {
15         unsigned int i = segment + c*blockDim.x + threadIdx.x;
16         if(i < width*height) {
17             unsigned char bNext = image[i];
18             if(bNext == b) {
19                 ++bin_r;
20             } else {
21                 cuda::atomic_ref<unsigned int, cuda::thread_scope_block>
22                     bins_s_ref(bins_s[b]);
23                 bins_s_ref.fetch_add(bin_r, cuda::memory_order_relaxed);
24                 b = bNext;
25                 bin_r = 1;
26             }
27         }
28     }
29     cuda::atomic_ref<unsigned int, cuda::thread_scope_block>
30         bins_s_ref(bins_s[b]);
31     bins_s_ref.fetch_add(bin_r, cuda::memory_order_relaxed);
32 }
33 __syncthreads();
34
35 for(unsigned int b = threadIdx.x; b < NUM_BINS; b += blockDim.x) {
36     if(bins_s[b] > 0) {
37         cuda::atomic_ref<unsigned int, cuda::thread_scope_device>
38             bins_ref(bins[b]);
39         bins_ref.fetch_add(bins_s[b], cuda::memory_order_relaxed);
40     }
41 }
42 }
43

```

图 9.15: 采用线程级私有化的直方图内核

每当新像素加载到 `bNext`（第 17 行），代码就把它与线程级私有分箱所对应的像素值比较（第 18 行）。如果二者相同，线程只需递增线程级私有分箱计数（第 19 行）。如果不同，线程使用原子操作把线程级私有分箱的旧计数加到块级私有分箱中（第 21–23 行），再把线程级私有分箱的像素值更新为刚刚加载的值（第 24 行），并把

分箱计数器重置为 1（第 25 行）。

采用这种方案时，直方图更新总是至少落后一个元素。极端情况下，如果线程处理的像素从未出现相邻相同值，那么每项更新都会恰好落后一次循环迭代。因此，线程处理完所有输入像素并退出循环后，还要为最后一个像素再执行一次原子操作（第 29–31 行）。

需要注意，采用线程级私有化的内核使用了更多语句和变量。因此，如果连续像素值之间很少相同，它可能比前一个内核运行得更慢；但如果数据分布具有很强的重复性，线程级私有化就可能显著提速。新增的 if 语句可能产生控制流分歧。不过，当相似度极低或极高时，控制流分歧都很少，因为各线程要么都递增私有分箱，要么都执行原子操作。即使确实出现控制流分歧，减少争用带来的收益也很可能抵消这部分代价。

9.7 小结

直方图计算对分析大规模数据集十分重要。它也代表了一类重要的并行计算模式：每个线程写入哪个输出位置由数据决定，因此无法应用所有者计算规则。这一模式很自然地引出了读-改-写竞态条件，以及原子操作的实际用法；原子操作能够保证针对同一内存位置的并发读-改-写操作不会破坏数据完整性。

遗憾的是，正如本章所述，原子操作的吞吐量远低于简单的内存读取或写入，因为它近似等于两倍内存延迟的倒数。因此，存在严重争用时，直方图计算的吞吐量可能低得出人意料。私有化是一项重要优化技术，它系统地减少争用，还使程序可以进一步采用低延迟、高吞吐量的共享内存。事实上，让同一线程块中的线程快速执行原子操作，正是共享内存的重要用例之一。

本章还用线程粗化减少需要合并的私有副本数，并比较了连续划分与交错划分两种粗化策略。最后，对于不同数据元素之间相似度很高的数据集，线程级私有化还可以进一步减少原子操作的争用。

练习

1. 假设 DRAM 系统中每次原子操作的总延迟为 100 ns。针对同一个全局内存变量执行原子操作时，能够获得的最大吞吐量是多少？
2. 某处理器支持在 L2 缓存中执行原子操作。假设每次原子操作在 L2 缓存中需要 4 ns，在 DRAM 中需要 100 ns，并且 90% 的原子操作命中 L2 缓存。针对同一个全局内存变量执行原子操作时，近似吞吐量是多少？
3. 在第 1 题的条件下，假设内核每执行一次原子操作就执行 5 次浮点运算。受原

子操作吞吐量限制时，内核执行能达到的最大浮点吞吐量是多少？

4. 在第 1 题的条件下，假设内核把全局内存变量私有化为共享内存变量，共享内存访问延迟为 1 ns。原来的全局内存原子操作全部改成共享内存原子操作。为简化分析，假设把私有变量累加到全局变量所增加的全局内存原子操作，会使总执行时间增加 10%。又假设内核每执行一次原子操作就执行 5 次浮点运算。受原子操作吞吐量限制时，内核执行能达到的最大浮点吞吐量是多少？
5. 某直方图内核处理包含 524,288 个元素的输入，生成包含 128 个分箱的直方图。内核配置为每块 1024 个线程。
 - a. 图 9.6 的内核不使用私有化、共享内存和线程粗化，它会在全局内存上执行多少次原子操作？
 - b. 图 9.10 的内核使用私有化和共享内存，但不使用线程粗化。它在全局内存上最多可能执行多少次原子操作？
 - c. 图 9.14 的内核使用私有化、共享内存和线程粗化，粗化因子为 4。它在全局内存上最多可能执行多少次原子操作？

参考文献

- [1] C++ reference, 2025. <https://cppreference.com>.

第十章 归约

归约 (reduction) 操作从一系列值中导出一个值。这个结果可以是所有输入值的和、最大值、最小值等，输入与结果也可以采用整数、单精度浮点数、双精度浮点数、半精度浮点数、字符等不同类型。各种归约都有相同的计算结构。与直方图一样，归约也是一种重要的计算模式，因为它能从大量数据中提炼出概括信息。并行归约要求并行线程相互协调，才能得到正确结果。协调方式必须精心设计，否则很容易触发并行计算系统中常见的性能瓶颈。因此，并行归约很适合用来说明这些瓶颈，并引出缓解它们的技术。

10.1 背景

归约根据某个二元运算符，把一组元素变换为单个元素。例如，可以用加法运算符定义浮点数集合上的求和归约，结果就是集合中所有浮点数之和。对集合 {7.0, 2.1, 5.3, 9.0, 11.2}，求和归约得到

$$7.0 + 2.1 + 5.3 + 9.0 + 11.2 = 34.6.$$

归约可以顺序遍历数组中的每个元素来完成。图 10.1 给出了一个简单的 C 语言顺序求和归约。代码先用加法的单位元 0.0 初始化结果变量 `sum`，¹再用 for 循环遍历保存输入值的数组。在第 i 次迭代中，代码把 `sum` 的当前值与输入数组的第 i 个元素相加。在本例中，迭代 0 后，`sum` 为 $0.0 + 7.0 = 7.0$ ；迭代 1 后，它为 $7.0 + 2.1 = 9.1$ 。也就是说，每次迭代都把一个输入元素累加到 `sum` 中。处理完 5 个元素后，`sum` 为 34.6，即归约结果。

译者注：底本在这个 5 元素示例中写的是“迭代 5 后”，但循环下标从 0 开始，实际最后一次迭代是迭代 4。本译稿改为“处理完 5 个元素后”，以免把元素个数与迭代下标混淆。

还可以为许多其他运算符定义归约。以单位元 1.0 初始化浮点乘法运算符，就得

¹二元运算符的单位元作为一个操作数时，结果总是另一个操作数。例如，浮点加法的单位元为 0.0，因为任意浮点值 v 与 0.0 相加仍为 v 。代码中使用单位元只是为了方便；如果运算符没有单位元，可以用元素 0 初始化累加量，并让循环从 $i=1$ 开始。

```
1 sum = 0.0f;
2 for(i = 0; i < N; ++i) {
3     sum += input[i];
4 }
5
```

图 10.1: 简单的顺序求和归约代码

到乘积归约，其结果是集合中所有浮点数的乘积。最小值归约可以用返回两个输入中较小者的比较运算符定义；最大值归约则使用返回较大者的比较运算符。对实数而言，最小值运算符的单位元是 $+\infty$ ，最大值运算符的单位元是 $-\infty$ 。因此，浮点数集合的最小值（最大值）归约会生成列表中的最小（最大）数。

译者注：底本把上段写成“minimum (minimum) operator”，但前后文明确分别讨论最小值与最大值运算符。本译稿按上下文改为“最小值（最大值）运算符”。

图 10.2 给出了任意运算符上的顺序归约通式。这里的运算符定义为一个接收两个输入并返回一个值的函数。循环访问某个元素时，执行的动作取决于归约类型。例如，最大值归约的 `Operator` 函数比较两个输入并返回较大者；最小值归约则返回较小者。for 循环访问完所有元素后，顺序算法结束。对包含 N 个元素的集合，循环执行 N 次，退出时得到归约结果。

```
1 acc = IDENTITY;
2 for(i = 0; i < N; ++i) {
3     acc = Operator(acc, input[i]);
4 }
5
```

图 10.2: 顺序归约代码的通式

归约非常常用，一些编程语言已经把它纳入标准库。例如，C++ 标准库中的 `std::accumulate` 和 `std::reduce` 可以执行不同形式的归约。类似地，Thrust[1, 2] 和 CUB[3] 等 CUDA 库也提供了针对 GPU 高度优化的并行归约实现。尽管如此，本章仍从头讲解如何在 GPU 上并行化并优化归约，因为它能清楚展示许多并行化和优化技术。

10.2 归约树

并行归约算法在文献中已有大量研究。图 10.3 展示了它的基本思想：时间沿竖直方向向下推进，每个时间步中线程并行执行的活动沿水平方向排列。

先看图 10.3(a) 的最大值归约树。第一轮（时间步）并行处理原始元素的四对组

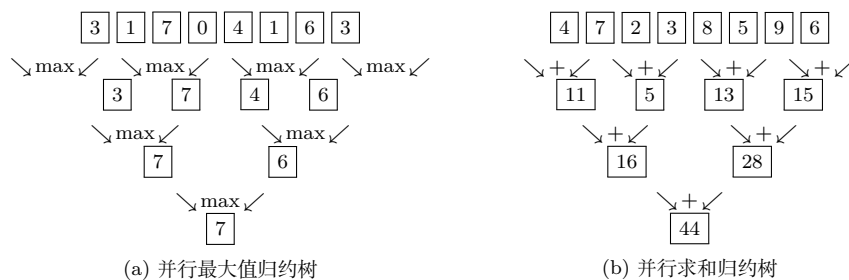


图 10.3: 最大值归约与求和归约的并行归约树示例

合，执行四次最大值运算，得到四个部分归约结果，也就是每一对中的较大值。第二个时间步并行处理这四个部分结果的两对组合，执行两次最大值运算，生成两个更接近最终结果的部分结果。它们分别是原输入前四个元素和后四个元素中的最大值。第三个也是最后一个时间步执行一次最大值运算，得到原输入中的最大值 7。

从顺序归约改为并行归约时，运算的执行次序发生了变化。以图 10.3(a) 顶部的输入为例，图 10.2 那样的顺序最大值归约会先比较 `acc` 中的单位元 $-\infty$ 与输入值 3，把较大的 3 写回 `acc`；再比较 `acc` 中的 3 与输入值 1，结果仍为 3；随后比较 3 与 7，得到 7。图 10.3(a) 的并行归约则先比较输入值 7 与输入值 0，然后才把结果与 3 和 1 的最大值比较。要并行化归约，关键就在于能够这样改变运算次序。

为了重排运算，必须假定把运算符应用于输入值的顺序不会影响结果。最大值归约无论以什么顺序应用运算符，结果都相同。运算符满足**结合律 (associativity)** 时，数学上可以保证这一性质。若运算符 Θ 满足

$$(a \Theta b) \Theta c = a \Theta (b \Theta c),$$

就称它具有结合性。换句话说，对于只含该运算符的表达式，可以在任意位置加括号而不改变结果。借助这种等价关系，我们可以在保持结果等价的同时，把一种运算次序变成另一种。

例如，整数加法满足结合律，即 $(1 + 2) + 3 = 1 + (2 + 3)$ 。²整数减法则不满足结合律，因为 $(1 - 2) - 3 \neq 1 - (2 - 3)$ 。可以把减法变为与第二个操作数的相反数相加，从而获得结合性；例如， $(1 - 2) - 3$ 可以改写为 $(1 + (-2)) + (-3) = 1 + ((-2) + (-3))$ 。

对实数而言，加法在数学意义上满足结合律；但严格来说，由于不同加括号方式可能产生不同的舍入结果，浮点加法并不满足结合律。有些应用只要不同浮点结果之间的差异不超过容许范围，就认为结果相同。凭借这种容差，开发者和编译器优化在实践中可以把浮点加法视为具有结合性。附录 A 将详细讨论这个问题。

把图 10.2 的顺序归约转换为图 10.3(a) 的归约树，要求运算符满足结合律。可以

²从数学意义上说，整数加法满足结合律；但在 C++ 中并非绝对如此，因为运算次序可能影响是否发生溢出或下溢，而有符号整数溢出的行为未定义。如果程序员确定不会溢出或下溢，实际编程时仍可把整数加法视为满足结合律。

把归约看成一系列运算；二者的次序差异来自在同一运算列表的不同位置加入括号。图 10.2 对应

$$((((((3 \max 1) \max 7) \max 0) \max 4) \max 1) \max 6) \max 3,$$

而图 10.3(a) 对应

$$((3 \max 1) \max (7 \max 0)) \max ((4 \max 1) \max (6 \max 3)).$$

第 10.4 节还会采用一种优化，它不仅调整应用运算符的次序，还会调整操作数的次序。要重排操作数，运算符还必须满足**交换律 (commutativity)**。若 $a \ominus b = b \ominus a$ ，就称运算符 $\ominus$ 具有交换性；也就是说，调换表达式中操作数的位置仍得到相同结果。最大值运算符以及最小值、加法和乘法等许多运算符都满足交换律。³显然，并非所有运算符都如此。例如，加法满足 $1 + 2 = 2 + 1$ ，整数减法却不满足 $1 - 2 = 2 - 1$ 。

图 10.3(a) 的并行模式称为**归约树 (reduction tree)**，因为它看起来像一棵以原始输入元素为叶、以最终结果为根的树。这里的归约树不要与树形数据结构混淆；后者通过指针或约定的位置显式或隐式连接结点，归约树中的边则只是概念关系，表示信息从一个时间步中的运算流向下一个时间步。

为了比较顺序归约与并行归约实现的速度，需要引入**工作量 (work)** 和 **跨度 (span)**。实现的工作量是它执行的运算数；算法的跨度则是在执行资源无限时完成这些运算所需的时间。例如，第 2 章的向量加法内核为两个长度为 N 的向量启动 N 个线程。执行时，每个线程完成一次操作，包括读取两个输入元素、执行加法并写出一个输出元素。因此，该内核完成 $O(N)$ 的工作量，即所有线程合计执行 N 次操作。

把一个线程完成一次操作的时间称为一个**步 (step)**。涉及多个线程时，一个步是所有线程或其中一部分线程各自并行执行一次操作的时间段。计算的跨度用实现所需的步数衡量，并假定执行资源数量不受限制。顺序向量加法的跨度为 $O(N)$ ，因为 N 次操作依次执行；并行向量加法内核的跨度则为 $O(1)$ ，因为 N 次操作全部并行执行。

分析顺序算法时，工作量与跨度没有区别，因为操作依次执行，算法所需步数与工作量直接成正比。例如，顺序向量加法完成 $O(N)$ 的工作，也需要 $O(N)$ 个步。

顺序归约同样完成 $O(N)$ 的工作并需要 $O(N)$ 个步。相比之下，图 10.3(a) 的并行归约树用 7 次运算归约 8 个值：第一步 4 次，第二步 2 次，第三步 1 次。一般来说，对 N 个输入值，第一轮执行 $N/2$ 次，第二轮执行 $N/4$ 次，依此类推。因此总运算数为等比级数

³对实数而言，最大值和最小值在数学意义上满足交换律；但对浮点数并非绝对如此，因为其中一个操作数为 NaN 时会有特殊行为。详细规则见 IEEE 浮点标准。如果程序员确定输入中没有 NaN，实际编程时可以把最大值和最小值视为满足交换律。

$$\frac{N}{2} + \frac{N}{4} + \frac{N}{8} + \cdots + \frac{N}{N} = N - 1.$$

换句话说，并行归约树的工作量仍为 $O(N)$ ，与顺序归约相同。第 11 章将看到，并行算法并不总能保持与顺序算法相同的工作量，这会引出工作效率的概念。

跨度方面，归约 N 个输入值的并行归约树需要 $\log_2 N$ 个步。每个步执行一层运算，所以跨度为 $O(\log N)$ 。例如，在执行资源充足时，只需 $\log_2 1024 = 10$ 个步就能归约 $N = 1024$ 个输入值。第一步需要 $N/2 = 512$ 份执行资源，之后所需资源迅速减少，最后一步只需要一份。每一步的并行度等于该步所需的执行单元数。所有步的平均并行度等于总运算数除以步数，即 $(N - 1)/\log_2 N$ 。当 $N = 1024$ 时，10 个步的平均并行度为 102.3，而峰值并行度为第一步的 512。并行度与资源消耗随步骤大幅变化，使归约树成为并行计算系统中颇具挑战性的模式。

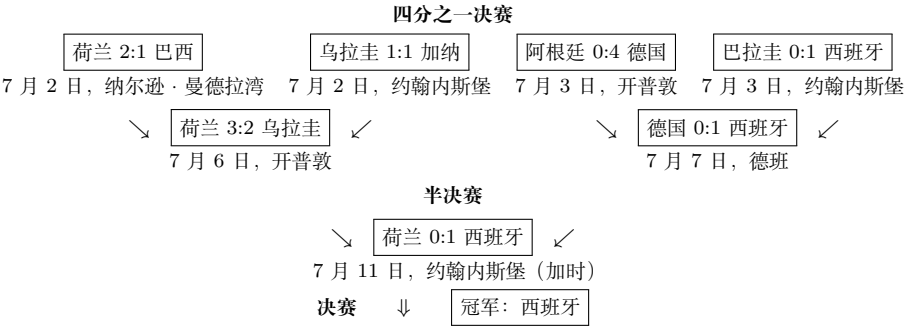


图 10.4: 作为归约树的 2010 年世界杯淘汰赛末段赛程

体育与竞赛中的并行归约。 早在计算机出现之前，体育和竞赛就已经采用并行归约。图 10.4 展示了 2010 年南非世界杯四分之一决赛、半决赛和决赛的赛程，它实际上就是重新排布过的归约树。世界杯的淘汰过程是一种最大值归约，其中“最大值”运算符返回击败另一支球队的球队。赛事归约分多轮进行：球队两两配对，第一轮的各项比赛并行进行，胜者进入第二轮，第二轮胜者再进入第三轮，以此类推。8 支球队参赛时，第一轮（四分之一决赛）产生 4 支胜队，第二轮（半决赛）产生 2 支胜队，第三轮（决赛）产生最终冠军。每一轮都是归约过程中的一个步，因此图 10.4 的淘汰赛跨度为 3 个步。

译者注：底本写成“跨度为 4 个步”，但图中 8 支球队依次经过四分之一决赛、半决赛和决赛三轮，且紧接着的正文也说明 1024 支球队需要 $\log_2 1024 = 10$ 轮。本译稿据此改为 3 个步。

即使有 1024 支球队，也只需 10 轮就能决出冠军。关键在于要有足够资源，让第一轮 的 512 场比赛、第二轮的 256 场、第三轮的 128 场等分别并行进行。资源充足时，即使有 6 万支球队，也只需 16 轮就能产生最终胜者。归约树能大幅加速归约过程，但也消耗大量资源。在世界杯的例子中，一场比赛需要大型足球场、裁判和工作人员，还需要酒店和餐馆接待大量观众。图 10.4 的四场四分之一决赛在三个城市举

行：纳尔逊·曼德拉湾（伊丽莎白港）、开普敦和约翰内斯堡，三座城市合计提供了举办四场比赛所需的资源。约翰内斯堡的两场比赛在不同日期进行，说明两场比赛共享资源会延长归约过程。计算中的归约树也会遇到类似权衡。

图 10.3(b) 给出了 8 个输入值的求和归约树。前面对最大值归约树工作量与跨度的分析同样适用于它。使用最多 4 个加法器及其读写数据所需资源，只需 $\log_2 8 = 3$ 个时间步即可完成归约。下面将用这个例子说明求和归约内核的设计。

10.3 一个简单的归约内核

现在可以着手实现图 10.3(b) 的并行求和归约树了。归约树需要所有线程协作，而整个网格中的线程无法直接完成这种协作，所以先实现只在一个线程块内执行求和归约树的内核。对包含 N 个元素的输入数组，调用这个简单内核时启动一个网格，其中只有一个线程块，块内有 $N/2$ 个线程。线程块最多可有 1024 个线程，因此这种做法最多能处理 2048 个输入元素。第 10.9 节将解除这个限制。

第一步中，全部 $N/2$ 个线程都参与计算，每个线程把两个元素相加，生成 $N/2$ 个部分和。下一步中，一半线程退出，只剩 $N/4$ 个线程继续计算并生成 $N/4$ 个部分和。这个过程不断重复，直到最后一步只剩一个线程，由它产生总和。

```

1 __global__ void SimpleSumReductionKernel(float* input, float* output) {
2     unsigned int i = 2*threadIdx.x;
3     for(unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
4         if(threadIdx.x % stride == 0) {
5             input[i] += input[i + stride];
6         }
7         __syncthreads();
8     }
9     if(threadIdx.x == 0) {
10        *output = input[0];
11    }
12 }
13

```

图 10.5: 简单的求和归约内核

图 10.5 给出了简单的求和内核，图 10.6 则展示了该代码实现的归约树如何执行。图中时间从上向下推进。假设输入数组位于全局内存中，调用内核函数时传入指向它的指针。每个线程分配到位置 $2*\text{threadIdx.x}$ （第 2 行），也就是输入数组的偶数位置：线程 0 对应 `input[0]`，线程 1 对应 `input[2]`，线程 2 对应 `input[4]`，依此类推。每个线程都是所分配位置的“所有者”，只有它可以写入该位置。这种设计遵循所有者计算规则：每个数据位置只归一个线程所有，也只能由该线程更新。

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
线程所有者	T_0	T_1	T_2	T_3	T_4	T_5	T_6	T_7								
步 0, $s = 1$	Σ	Σ	Σ	Σ	Σ	Σ	Σ	Σ	Σ	Σ	Σ	Σ	Σ	Σ	Σ	Σ
步 1, $s = 2$	Σ			Σ				Σ					Σ			
步 2, $s = 4$	Σ							Σ								
步 3, $s = 8$	Σ															

步 0 $\text{input}[2k] \leftarrow \text{input}[2k] + \text{input}[2k+1]$
 步 1 $\text{input}[4k] \leftarrow \text{input}[4k] + \text{input}[4k+2]$
 步 2 $\text{input}[8k] \leftarrow \text{input}[8k] + \text{input}[8k+4]$
 步 3 $\text{input}[0] \leftarrow \text{input}[0] + \text{input}[8]$

图 10.6: 线程“所有者”到输入数组位置的分配, 以及简单归约内核随时间推进的执行过程; 每一层对应一次循环迭代

图 10.6 顶行表示线程到输入数组位置的分配, 后续每一行表示图 10.5 的 for 循环一次迭代结束后写入的位置。每次迭代中被覆盖的位置以 Σ 标出。例如, 第一次迭代结束时, 偶数下标位置被原数组相邻元素对 (0-1、2-3、4-5 等) 的部分和覆盖。第二次迭代结束时, 下标为 4 的倍数的位置被四个相邻原始元素 (0-3、4-7 等) 的部分和覆盖。

图 10.5 使用 `stride` 变量, 让线程找到应当累加到所有者位置的部分和。它从 1 开始 (第 3 行), 每次迭代翻倍, 依次取 1、2、4、8 等, 直到大于块中的线程总数 `blockDim.x`。图 10.6 中, 每个活跃线程都把距离其所有者位置 `stride` 个元素的数组元素累加进去。例如, 在迭代 0 中, 线程 0 以步长 1 把 `input[1]` 加到自己的 `input[0]`, 于是 `input[0]` 变成原始 `input[0]` 与 `input[1]` 的部分和。迭代 1 中, 线程 0 以步长 2 把 `input[2]` 加到 `input[0]`。此时 `input[2]` 已保存原始 `input[2]` 和 `input[3]` 的和, `input[0]` 则保存原始 `input[0]` 和 `input[1]` 的和, 因此迭代 1 结束后, `input[0]` 保存原数组前四个元素之和。最后一次迭代后, `input[0]` 包含全部原始元素之和, 也就是求和归约结果, 线程 0 把它写入最终输出 (第 10 行)。

译者注: 底本把“块中的线程总数”写成 `blockIdx.x`, 但该变量表示块在网格中的下标; 图 10.5 的循环条件实际使用 `blockDim.x`。本译稿按代码改为后者。

再看图 10.5 第 4 行的 if 语句。它的条件用于选出每次迭代中的活跃线程。图 10.6 表明, 迭代 n 中执行加法的是线程下标为 2^n 的倍数的线程。条件 `threadIdx.x \% stride == 0` 正是在检查线程下标是否为步长的倍数, 而 `stride` 在各次迭代中依次为 1, 2, 4, 8, ..., 也就是迭代 n 中的 2^n 。所有线程都执行相同的内核代码; 满足条件的活跃线程执行第 5 行加法, 不满足条件的非活跃线程跳过它。迭代越往后, 活跃线程越少; 最后一次迭代只剩线程 0, 由它生成归约结果。

循环中的 `__syncthreads()` (第 7 行) 保证当前迭代计算出的所有部分和都已经写入输入数组中的目标位置, 任何线程才能开始下一次迭代。这样, 进入下一次迭代的线程才能正确读取上次迭代产生的部分和。例如, 第一次迭代会用成对部分和替换偶数位置; `__syncthreads()` 保证这些结果确实全部就绪, 第二次迭代中的活跃线程

才会使用它们。

还要注意，一个线程写入全局内存的数据不能在内存栅栏的情况下由其他线程读取。这里跨迭代读写相同位置的线程都属于同一线程块，因此每次迭代末尾的 `__syncthreads()` 也充当块内线程的内存栅栏：调用它之前写入的数据，在调用完成后可以被同一块中的线程正确读取。规定线程通过内存访问交换数据时应有何种可观察行为的规则称为**内存一致性模型 (memory consistency model)**。更多细节可参阅 CUDA 编程指南。

10.4 减少控制流分歧

图 10.5 的代码实现了图 10.6 的并行归约树，也能产生预期的求和结果。遗憾的是，它在各次迭代中管理活跃和非活跃线程的方式造成了严重的控制流分歧。例如，图 10.6 中第二次迭代只有 `threadIdx.x` 为偶数的线程执行加法。正如第 4 章所述，控制流分歧会显著降低执行资源利用效率，也就是用于生成有效结果的资源比例。本例中，一个 warp 的 32 个线程都占用执行资源，实际却只有一半活跃，浪费了一半资源。随着时间推进，浪费还会增加：第三次迭代只有四分之一线程活跃，浪费四分之三；到第 6 次迭代，一个 warp 中只有 1 个线程活跃，浪费 31/32 的执行资源。

如果输入数组长度大于 32，第 6 次迭代后会有整个 warp 变为非活跃。以 256 个输入元素为例，内核启动 128 个线程，即 4 个 warp。前 6 次迭代中，四个 warp 都呈现上段所述的分歧模式。第 7 次迭代时，warp 1 和 warp 3 完全非活跃，不再产生控制流分歧；warp 0 和 warp 2 则各只有一个活跃线程，浪费 31/32 的执行资源。第 8 次迭代只有 warp 0 活跃，其中也只有一个线程工作。

一般来说，长度为 N 的输入数组，其执行资源利用效率等于所有迭代中的活跃线程总数与消耗的执行资源总量之比。后者正比于所有迭代中的活跃 warp 总数，因为只要 warp 仍活跃，无论内部有几个线程实际工作，它都要消耗完整的 warp 执行资源。总消耗为

$$\left(\frac{N}{64} \cdot 6 + \frac{N}{64} \cdot \frac{1}{2} + \frac{N}{64} \cdot \frac{1}{4} + \cdots + 1 \right) \cdot 32.$$

这里 $N/64$ 是启动的 warp 总数，因为内核启动 $N/2$ 个线程，每 32 个线程组成一个 warp。 $N/64$ 乘以 6，是因为前 6 次迭代中所有 warp 都活跃；之后每次迭代活跃 warp 数减半。括号中的表达式就是跨全部迭代的活跃 warp 总数，外面的 32 表示每个活跃 warp 总会占用 32 个线程对应的完整执行资源。对 256 个输入元素，消耗量为 $(4 \times 6 + 2 + 1) \times 32 = 864$ 。

活跃线程提交的有效结果数，就是所有迭代中的活跃线程总数：

$$\frac{N}{64}(32 + 16 + 8 + 4 + 2 + 1) + \frac{N}{64} \cdot \frac{1}{2} \cdot 1 + \frac{N}{64} \cdot \frac{1}{4} \cdot 1 + \dots + 1.$$

括号中的各项给出前 6 次迭代里每个 warp 的活跃线程数。从第 7 次迭代开始, 活跃 warp 数每次减半, 每个活跃 warp 中只剩一个活跃线程。对 256 个输入元素, 有效结果总数为 $4(32 + 16 + 8 + 4 + 2 + 1) + 2 + 1 = 255$ 。这也很直观, 因为把 256 个值归约为一个值恰好需要 255 次运算。

结合这两个结果, 输入长度为 256 时, 执行资源利用效率为 $255/864 = 0.30$ 。也就是说, 并行执行资源没有充分发挥加速潜力; 平均只有约 30% 的已消耗资源真正贡献于求和结果。

上述分析表明, 控制流分歧跨越多个 warp, 并且随时间持续存在。更好的线程与输入位置分配方式可以减少分歧并提高利用效率。图 10.6 的问题在于, 部分和的位置随时间越来越分散, 拥有这些位置的活跃线程也越来越分散, 非活跃线程比例和资源闲置程度因而不断上升。

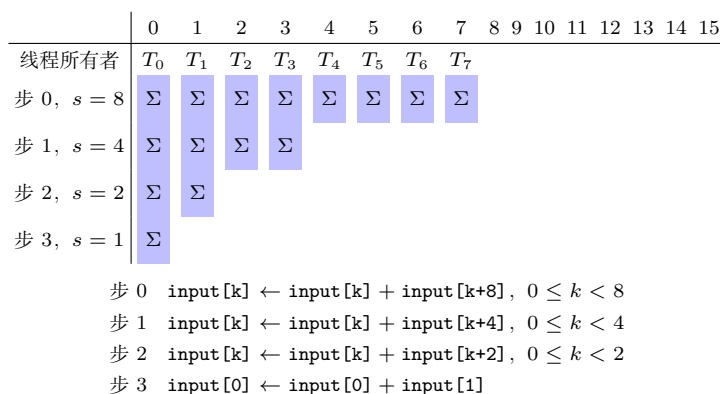


图 10.7: 减少控制流分歧的更优线程到输入数组位置分配方式

确实存在一种能显著减少控制流分歧的分配策略: 安排线程及其所有者位置, 使活跃线程随时间推进仍彼此相邻。换言之, 让步长随时间减小, 而不是增大。图 10.7 对 16 个输入元素展示了新策略。线程分配到数组前半部分的位置。第一次迭代中, 每个线程跨过半个数组, 把对应的另一个元素加到自己的位置: 线程 0 把 $\text{input}[8]$ 加到 $\text{input}[0]$, 线程 1 把 $\text{input}[9]$ 加到 $\text{input}[1]$, 依此类推。

以后的每次迭代中, 一半活跃线程退出, 其余线程把距离所有者位置“当前活跃线程数”个元素的输入加进来。例如迭代 3 时只剩线程 0 和线程 1: 线程 0 把 $\text{input}[2]$ 加到 $\text{input}[0]$, 线程 1 把 $\text{input}[3]$ 加到 $\text{input}[1]$ 。比较图 10.7 与图 10.6 的运算和操作数次序可以发现, 这里实际上重排了输入值列表, 而不只是改变括号位置。有效次序变为

$$\{\text{input}[0], \text{input}[8], \text{input}[1], \text{input}[9], \dots, \text{input}[7], \text{input}[15]\}.$$

要使这种重排后的结果始终不变，运算符除了满足结合律，还必须满足交换律。

```
1 __global__ void ConvergentSumReductionKernel(float* input, float* output
  ) {
2     unsigned int i = threadIdx.x;
3     for(unsigned int stride = blockDim.x; stride >= 1; stride /= 2) {
4         if(threadIdx.x < stride) {
5             input[i] += input[i + stride];
6         }
7         __syncthreads();
8     }
9     if(threadIdx.x == 0) {
10        *output = input[0];
11    }
12 }
13
```

图 10.8: 控制流分歧更少、执行资源利用效率更高的归约内核

图 10.8 实现了图 10.7 的工作分配。它与图 10.5 有几处细微但关键的区别。第一，所有者位置 i 设为 threadIdx.x ，而不是 $2 * \text{threadIdx.x}$ （第 2 行），相邻线程的所有者位置因而也相邻。第二，步长从 blockDim.x 开始，每次减半，直到 1（第 3 行）。第三，每次迭代只有下标小于步长的线程保持活跃（第 4 行），所以所有活跃线程的下标始终连续。

这些改动使第一轮不再把相邻元素相加，而是把相隔半个区段的元素相加；区段大小始终是剩余活跃线程数的两倍。第一轮中各对元素都相距 blockDim.x ，迭代结束后，所有成对部分和都保存在输入数组前半部分。循环随后把步长除以 2，因此第二次迭代的步长为 blockDim.x 的一半，活跃线程所加的元素相隔四分之一一个区段。

图 10.8 的循环中仍有 `if` 语句，每次迭代执行加法的线程数也与图 10.5 相同，为什么分歧会减少？答案在于执行加法的线程与不执行加法的线程如何分布。以 256 个输入元素为例，第一次迭代所有线程都活跃，没有分歧。第二次迭代中，线程 0–63 执行加法，线程 64–127 不执行。warp 由 threadIdx.x 连续的 32 个线程组成，因此 warp 0 和 warp 1 中的所有线程都执行加法，warp 2 和 warp 3 全部不执行；每个 warp 内的线程走同一条控制路径，所以没有控制流分歧。

不过，这个内核没有彻底消除 `if` 语句造成的分歧。对 256 元素的例子，从第 4 次迭代开始，执行加法的线程少于 32；最后五次迭代分别只有 16、8、4、2 和 1 个线程执行加法，因而仍有分歧。但存在分歧的循环迭代数已从原内核中的全部后续阶段减少为 5 次。执行资源总消耗为

$$\left(\frac{N}{64} + \frac{N}{64} \cdot \frac{1}{2} + \cdots + 1 + 5 \right) \cdot 32.$$

括号前半部分表示每次迭代都有一半 warp 完全退出，不再消耗资源，直到只剩一个完整 warp；最后一项 5 表示最后五次迭代中都只有一个活跃 warp，尽管其中仅有部分线程工作，32 个线程对应的资源仍全部被占用。对 256 个输入元素，总消耗为 $(4+2+1+5) \times 32 = 384$ 。活跃线程数没有改变，所以图 10.8 的效率为 $255/384 = 66\%$ ，几乎是图 10.5 的两倍。warp 会在执行资源有限的 SM 上轮流调度，降低资源消耗也会缩短总执行时间。

译者注：底本此处把图 10.5 的资源消耗写成 736，但按前文公式应为 $(4 \times 6 + 2 + 1) \times 32 = 864$ 。本译稿沿用前文可复算的 864；384 约为 864 的 44%，仍支持“显著减少”的结论。

图 10.5 与图 10.8 的代码差别很小，性能影响却可能很大。要有把握做出这种调整，必须清楚理解设备的 SIMD 硬件如何执行线程。

10.5 减少内存访问分歧

图 10.5 的简单内核还有一个性能问题：内存访问分歧。第 5 章已经说明，每个 warp 内应尽量实现内存访问合并，也就是相邻线程访问全局内存中的相邻位置。但图 10.6 中，相邻线程并不访问相邻位置。每次迭代中，每个线程执行两次全局内存读取和一次写入：第一次读所有者位置，第二次读相距 `stride` 的位置，最后写回所有者位置。相邻线程拥有的位置并不相邻，所以访问无法充分合并；一次迭代中，一个 warp 集体访问的位置彼此相隔一个步长。

例如，第一次迭代中一个 warp 的线程执行第一次读取时，访问位置彼此相隔两个元素，因此触发的全局内存事务数是合并访问的两倍，返回的数据也只有一半被线程使用。第二次读取和写入也有同样问题。第二次迭代中，每隔一个线程就会退出，warp 集体访问的位置相隔四个元素，于是需要四倍的全局内存事务，返回数据只有四分之一被使用。步长越大，访问效率就越低。

这个过程一直持续到每个仍活跃的 warp 只剩一个活跃线程。此时一个 warp 才只发出一次全局内存请求。因此，全局内存请求总数为

$$\left(\frac{N}{64} \cdot 5 \cdot 2 + \frac{N}{64} + \frac{N}{64} \cdot \frac{1}{2} + \frac{N}{64} \cdot \frac{1}{4} + \cdots + 1 \right) \cdot 3.$$

第一项对应前五次迭代：全部 $N/64$ 个 warp 中都至少有两个活跃线程，分散访问会触发两个全局内存请求。其余项对应最后几次迭代：每个活跃 warp 只剩一个活跃线程，每次只触发一个请求，而活跃 warp 数逐次减半。最后乘以 3，是因为每

个活跃线程在一次迭代中有两次读和一次写。对 256 个元素，图 10.5 的内核共发出 $(4 \times 5 \times 2 + 4 + 2 + 1) \times 3 = 141$ 次全局内存请求。

图 10.8 的内核中，同一 warp 的相邻线程始终访问相邻的全局内存位置，所以访问始终能够合并。每个 warp 的读写只触发最少数量的全局内存事务，并充分利用返回数据。迭代推进时，整个 warp 逐批退出，非活跃 warp 不再访问全局内存；其余活跃 warp 继续发出合并访问，直到最后五次迭代只剩一个 warp。仅当请求的数据量小于单次全局内存事务的大小时，才会出现资源利用不足。DRAM 带宽有限，因此图 10.8 内核的执行时间很可能显著短于图 10.5 的简单内核。总之，前者既减少了控制流分歧，又改善了内存访问合并，能更高效地利用执行资源和 DRAM 带宽。

译者注：底本本段两次写成“图 10.7 的内核”，但图 10.7 是工作分配示意，实际内核位于图 10.8。本译稿将引用改为图 10.8。

10.6 减少全局内存访问

还可以用共享内存进一步改进图 10.8 中分歧较少的内核。每次迭代都会把部分和写到全局内存，下一次迭代中同一线程或其他线程又会把它们读回来。虽然部分读写可能由末级缓存提供服务，但访问末级缓存仍受明显的延迟和带宽限制。共享内存比末级缓存和全局内存延迟低、带宽高，因此把部分和保留在共享内存中，可以继续提高执行速度。图 10.9 展示了这种思路。

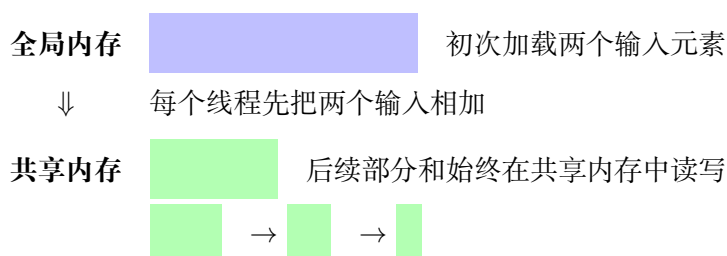


图 10.9: 使用共享内存减少全局内存访问

图 10.10 的内核实现了这项策略。每个线程先从全局内存加载两个原始元素并相加，再把部分和写入共享内存（第 4 行）。第一次归约已经在循环外访问全局内存时完成，所以 for 循环从 `blockDim.x/2` 开始，而不是 `blockDim.x`。把 `__syncthreads()` 移到循环开头，是为了在初次写入共享内存与第一次循环迭代之间同步。后续迭代只读写共享内存（第 8 行）。最后，线程 0 把总和写入 `output`，与前几个内核保持相同行为（第 11–13 行）。

这样，全局内存访问只剩初次加载原输入以及最后写出结果。归约 N 个元素时，全局内存访问次数为 $N + 1$ 。图 10.10 第 5 行的两组全局内存读取都能够合并，因此只发出 $(N/32) + 1$ 次全局内存请求。对 256 个元素，图 10.8 的内核需要 36 次请求，共享内存内核则只需 $8 + 1 = 9$ 次，改善 4 倍。共享内存除了减少全局内存访问，还


```
1 __global__ void SharedMemorySumReductionKernel(float* input, float*
  output) {
2   __shared__ float input_s[BLOCK_DIM];
3   unsigned int t = threadIdx.x;
4   input_s[t] = input[t] + input[t + BLOCK_DIM];
5   for(unsigned int stride = blockDim.x/2; stride >= 1; stride /= 2) {
6     __syncthreads();
7     if(threadIdx.x < stride) {
8       input_s[t] += input_s[t + stride];
9     }
10  }
11  if(threadIdx.x == 0) {
12    *output = input_s[0];
13  }
14 }
15
```

图 10.10: 用共享内存减少全局内存访问的归约内核

有一个好处：输入数组不会被修改。如果程序其他部分还要使用原始数组值，这项性质很重要。

译者注：底本图 10.10 的函数签名只列出 `input` 参数，函数体却写入未声明的 `output`；紧接着的正文又把最后一次写出误写成写到 `input[0]`。根据图 10.5、图 10.8 以及图 10.10 函数体的一致行为，本译稿在签名中补入 `float* output`，并把正文改为“写出结果”。

10.7 用 warp 级原语减少同步开销

图 10.10 的内核中，当步长减到 32 及以下时，线程块里只有一个 warp 仍参与归约树计算。CUDA 提供多种 **warp 级原语 (warp-level primitive)**，让同一 warp 的线程以跨 warp 线程无法使用的方式共享数据和同步。因此，当活跃线程数降至 32 及以下时，可以用这些原语减少共享内存访问和 `__syncthreads()` 的开销。本节先介绍相关 CUDA warp 级原语，再用它们继续优化归约内核。

最简单的 warp 级原语是 `__syncwarp()`。它的作用类似 `__syncthreads()`，但范围只有一个 warp：warp 中所有线程都到达调用位置之后，任何线程才能继续执行。如果 warp 中的线程此前发生分歧，`__syncwarp()` 会保证它们重新汇合。

与本节归约优化尤其相关的另一类原语是 **warp shuffle 函数 (warp shuffle function)**。同一 warp 中的线程可以用它们直接交换 (shuffle) 寄存器里的数据，无须访问共享内存，也无须调用 `__syncthreads()`。其中，`__shfl_down_sync` 能让线

程取得下标比自己大一个给定步长的线程所持数据。图 10.11 说明了它的参数。

```
T __shfl_down_sync(unsigned mask, T var, unsigned int delta);
```

- `mask` 指定执行 shuffle 指令时 warp 中哪些线程处于活跃状态；
- `var` 是发送线程交给接收线程的值；
- `delta` 是发送线程下标相对接收线程下标高出的步长；
- 返回值是接收线程从发送线程取得的值。对线程 i ，若线程 i 与 $i + \text{delta}$ 都在 `mask` 中，则返回线程 $i + \text{delta}$ 的 `var`。

图 10.11: warp 级原语 `__shfl_down_sync`

借助 `__shfl_down_sync`，可以在不访问共享内存、不调用 `__syncthreads()` 的情况下完成 warp 内归约。图 10.12 展示了如何把这种归约接到块内归约的最后阶段，从而减少屏障同步与共享内存访问。为了画面简洁，图中一个线程块由 4 个 warp 构成，每个 warp 只有 4 个线程；实际配置中，1024 线程的块会包含 32 个 warp，每个 warp 有 32 个线程。

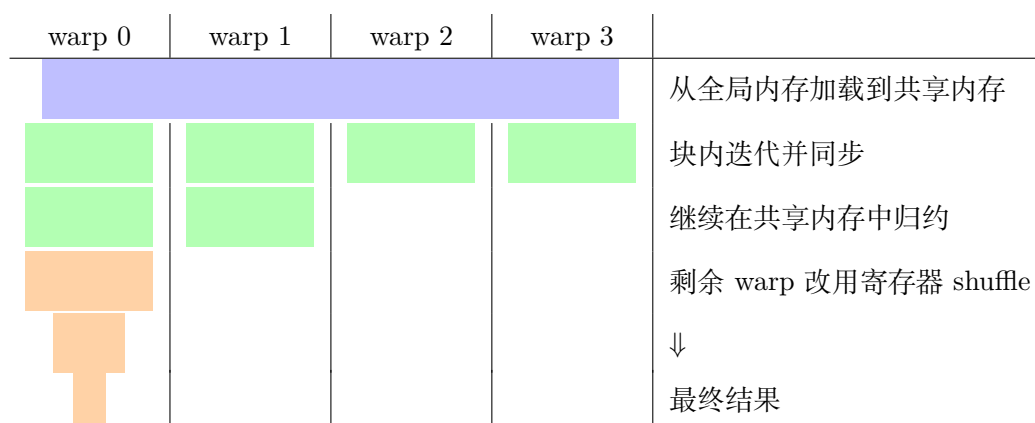


图 10.12: 在最后阶段使用 warp 内归约，以减少同步开销和共享内存访问

图 10.12 的前几次迭代与图 10.9 相同。每个线程从全局内存加载两个元素，相加后把结果存入共享内存；各次迭代之间用 `__syncthreads()` 执行块级屏障同步。每轮结束后，一半线程退出。不同之处是，一旦剩余线程只构成一个 warp，代码便不再使用 `__syncthreads()` 和共享内存，而是把共享内存中的值载入寄存器，通过在寄存器之间 shuffle 数据来执行归约树的剩余迭代。

为了实现图 10.12，先定义两个设备辅助函数，分别确定线程块内的 warp 下标，以及线程在 warp 内的下标：

```
__device__ unsigned int warpIdx() { return threadIdx.x/WARP_SIZE; }
```

这个函数用线程下标除以 32，使每组连续 32 个线程得到同一个 warp 下标。例如，线程 0-31 的 warp 下标都是 0，线程 32-63 的 warp 下标都是 1，依此类推。线程在 warp 内的下标也称 **lane 下标 (lane index)**，定义为：

```
__device__ unsigned int laneIdx() { return threadIdx.x%WARP_SIZE; }
```

它计算线程下标对 warp 大小 32 的余数，所以同一 warp 的连续 32 个线程的 lane 下标为 0-31。例如，块内线程 32 是 warp 1 的第一个线程，lane 下标为 $32 \bmod 32 = 0$ ；线程 33 是第二个，lane 下标为 $33 \bmod 32 = 1$ 。

有了这两个辅助函数，就可以实现 warp 内归约。图 10.13 给出的设备函数为 `warp_reduce`，它通过寄存器 shuffle 完成归约。warp 中所有线程都应调用它，每个线程通过参数 `val` 提供自己要贡献的值。线程先以 `val` 初始化局部部分和 `partialSum` (第 2 行)，再执行归约循环，让步长从 16 逐次减半到 1 (第 3 行)。起始步长为 16，是因为一个 warp 有 32 个线程、需要归约 32 个值；第一次迭代中，前 16 个线程分别累加后 16 个线程的值。

```
1 __device__ __inline__ float warp_reduce(float val) {
2     float partialSum = val;
3     for(unsigned int stride = WARP_SIZE/2; stride > 0; stride /= 2) {
4         partialSum += __shfl_down_sync(0xffffffff, partialSum, stride
5     );
6     }
7     return partialSum;
8 }
```

图 10.13: 使用 warp 级原语执行 warp 内归约的设备函数

归约循环的每次迭代都调用 `__shfl_down_sync`，以较低开销在 warp 内交换数据并同步 (第 4 行)。第一个参数 `0xffffffff` 是所有位均为 1 的 32 位掩码，表示 warp 中全部线程都参与归约。第二个参数表示各发送线程传出寄存器中的 `partialSum`；第三个参数表示接收线程的下标比发送线程低 `stride`。函数返回接收线程从发送线程得到的值。也就是说，对 warp 中的线程 i ，它返回线程 $i + \text{stride}$ 的 `partialSum`，再把该值加到线程 i 的部分和上。循环结束后，warp 第一个线程的 `partialSum` 才是最终结果。函数把值返回给调用方 (第 6 行)，但只有第一个线程的返回值代表真实归约结果。

图 10.14 用刚才的设备函数优化归约内核。前半部分 (第 1-12 行) 与图 10.10 类似，不过第 5 行的循环在步长达到 warp 大小 32 时结束共享内存阶段。从剩余 32 个部分和开始，只需块内第一个 warp 继续执行 (第 13 行)。这个 warp 的每个线程把共享内存中与自身下标对应的元素载入寄存器 `partialSum` (第 14 行)，再共同调用图 10.13 的 `warp_reduce` (第 15 行)。最终和落到 warp 的第一个线程，也就是块内

```
1 __global__ void WarpLevelSumReductionKernel(float* input, float* output) {
2     __shared__ float input_s[BLOCK_DIM];
3     unsigned int t = threadIdx.x;
4     input_s[t] = input[t] + input[t + BLOCK_DIM];
5     for(unsigned int stride = blockDim.x/2; stride >= WARP_SIZE;
6         stride /= 2) {
7         __syncthreads();
8         if(threadIdx.x < stride) {
9             input_s[t] += input_s[t + stride];
10        }
11    }
12    __syncthreads();
13    if(warpIdx() == 0) {
14        float partialSum = input_s[t];
15        partialSum = warp_reduce(partialSum);
16        if(threadIdx.x == 0) {
17            *output = partialSum;
18        }
19    }
20 }
21
```

图 10.14: 在最后阶段采用 warp 内归约，以减少同步开销和共享内存访问的归约内核

线程 0，由它写入全局内存中的输出（第 16–18 行）。这样，最后几个步长阶段不再访问共享内存，也不再执行块级屏障同步。

`__shfl_down_sync` 以及其他 warp 级原语还接受一个可选参数，用来指定协作线程组的宽度。默认宽度为 32，即整个 warp 是一个组；也可以指定更小的宽度，把 warp 划成多个独立协作的小组。例如宽度为 8 时，一个 warp 被视为 4 个 8 线程小组，每组内部协作执行原语，各组互不影响。

CUDA 还提供 `__shfl_sync`、`__shfl_up_sync` 和 `__shfl_xor_sync` 等 warp shuffle 函数，适合需要在同一 warp 的不同线程间交换数据的其他场景。其具体行为见 CUDA C++ 编程指南。除 shuffle 之外，warp 级原语还包括：检查 warp 中哪些线程满足某个条件的 warp 投票函数；检查哪些线程的某个参数值相同的 warp 匹配函数；以及对整数执行 warp 内归约的 warp 归约函数。第 12 章将展示 warp 投票函数的用法。

10.8 用两阶段 warp 内归约进一步减少同步开销

图 10.14 的内核消除了归约树最后几次迭代中的共享内存访问和屏障同步，但最初几轮仍要承担这些开销。还可以在初始阶段也采用 warp 内归约：线程块中的每个 warp 分别归约一段输入，把部分和写入共享内存，再由一个 warp 对这些部分和执行 warp 内归约。

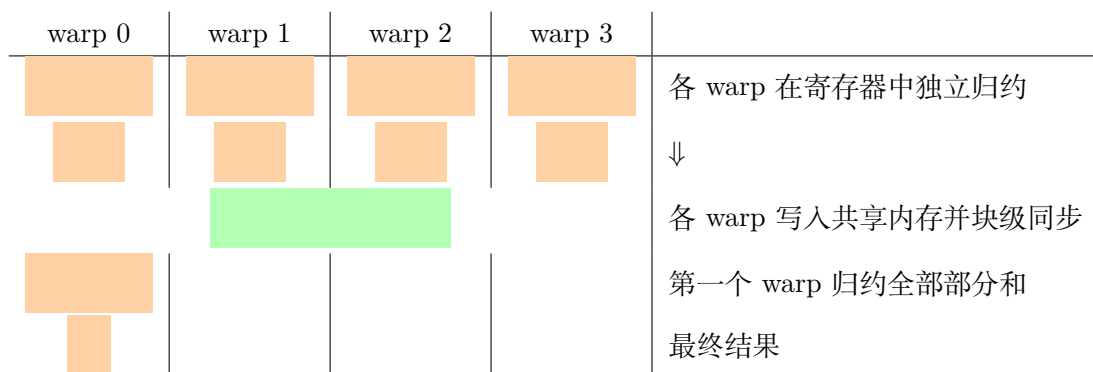


图 10.15: 用两阶段 warp 内归约进一步减少同步开销和共享内存访问

图 10.15 展示了这种两阶段方案。最初，每个 warp 加载分配给自己归约的元素，并执行一次 warp 内归约。第一阶段结束时，每个 warp 的第一个线程保存该 warp 所有输入元素之和。这些线程把各自的和写入共享内存，然后调用 `__syncthreads()` 执行一次块级屏障同步。⁴随后，第一个 warp 读取这些部分和，再执行一次 warp 内归约，得到总和。

⁴这里仍假设只用一个线程块归约最多 2048 个元素，所以所有 warp 可以通过共享内存交换数据。下一节将去掉这项假设。

```
1 __global__ void TwoStageWarpLevelSumReductionKernel(float* input, float*  
    output) {  
2  
3     unsigned int t = threadIdx.x;  
4     float partialSum = input[t] + input[t + BLOCK_DIM];  
5     partialSum = warp_reduce(partialSum);  
6  
7     __shared__ float partialSums_s[BLOCK_DIM/WARP_SIZE];  
8     if(laneIdx() == 0) {  
9         partialSums_s[warpIdx()] = partialSum;  
10    }  
11    __syncthreads();  
12  
13    if(warpIdx() == 0) {  
14        float partialSum = (t < BLOCK_DIM/WARP_SIZE) ? partialSums_s[t] :  
0.0f;  
15        partialSum = warp_reduce(partialSum);  
16        if(threadIdx.x == 0) {  
17            *output = partialSum;  
18        }  
19    }  
20 }  
21
```

图 10.16: 用两阶段 warp 内归约进一步减少同步开销和共享内存访问的归约内核

图 10.16 的内核用两阶段 warp 内归约完成块内归约。每个线程先加载分配给自己的两个输入元素，相加后把结果放在寄存器中（第 4 行），同一 warp 的线程再共同调用 `warp_reduce`（第 5 行）。此时块内所有 warp 都活跃，各自执行独立的归约。随后，每个 warp 中 lane 下标为 0 的第一个线程保存了本 warp 的部分和，它把这个值写入共享内存中与 warp 对应的位置（第 7–10 行）。一次屏障同步保证所有 warp 都完成写入（第 11 行）。

部分和全部就绪后，代码的后半段与图 10.14 类似。只有第一个 warp 继续执行（第 13 行）。⁵第一个 warp 的线程分别从共享内存加载部分和；部分和不足 32 个时，其余 lane 以加法单位元 0.0 参与（第 14 行）。这些线程执行 warp 内归约（第 15 行），最后由第一个线程把总和写入全局内存（第 16–18 行）。

译者注：本节以单线程块归约最多 2048 个元素为主要示例，底本图 10.16 因而可能隐含 `blockDim.x` 与 `BLOCK_DIM` 都等于 1024；在这一前提下，`partialSums_s` 恰有 32 个元素，原代码不会越界。不过，内核本身没有明示只能按这一配置启动。若 `blockDim.x` 与 `BLOCK_DIM` 相等但小于 1024，第一个 warp 中的多余 lane 读取 `partialSums_s[t]` 时会越界。本译稿用加法单位元 0.0 填充这些 lane，在不改变 1024 线程情形的同时，把适用范围扩展到 32–1024 之间、大小为整 warp 倍数且 `blockDim.x` 与 `BLOCK_DIM` 相等的线程块；这是兼容性扩展，并非对 1024 线程示例的纠错。

两阶段实现的主要优点是第一阶段也不再承担共享内存访问和屏障同步开销。只有从第一阶段过渡到第二阶段时，才需要访问一次共享内存并执行一次屏障同步。不过，相比图 10.14，它也有缺点。图 10.14 的第一阶段中，每次迭代都会有一半 warp 完全退出，而且剩余 warp 内没有额外的跨 warp 分歧；图 10.16 的两阶段实现则让所有 warp 在整个第一阶段都保持活跃，但每次 shuffle 并非每个线程都做有用工作。因此，它用更多的 warp 内资源闲置换取更少的共享内存访问和屏障同步。

实践中，这项权衡通常值得。数据从全局内存载入后，归约内核其余部分的性能主要受延迟限制。消除共享内存访问与屏障同步，可以减少内核中的长延迟指令。相比之下，消除分歧 warp 虽然有效，但对延迟受限内核的收益未必明显，因为省下的执行周期很可能会被停顿周期取代。

10.9 任意长度输入的归约

目前的所有内核都假设只启动一个线程块，主要原因是它们用 `__syncthreads()` 在所有活跃线程之间执行屏障同步，而该函数只能同步同一个块中的线程。这就把当前硬件上的并行度限制为 1024 个线程。大型输入数组可能有数百万乃至数十亿个元素，启动更多线程可以继续加速归约。由于没有适合在不同块的线程之间执行屏障同

⁵一个线程块最多有 1024 个线程，而一个 warp 有 32 个线程，因此块内最多有 32 个 warp，第一阶段也最多产生 32 个部分和；一个 warp 足以完成第二阶段。

步的方法，必须允许不同块相互独立地执行。

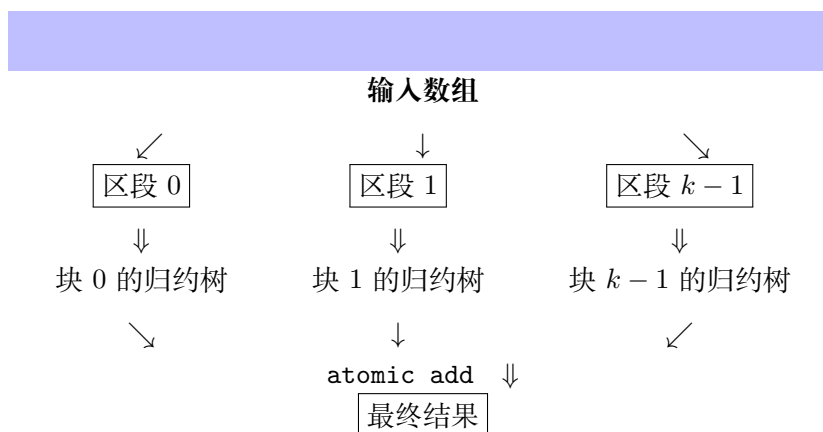


图 10.17: 使用原子操作的多块归约

图 10.17 展示了使用原子操作进行多块归约的思路。先把输入数组划分为多个区段，每个区段的大小适合一个线程块处理；所有块各自独立执行归约树，再用原子加法把各块结果累加到最终输出。

图 10.18 实现了多块归约，与图 10.16 相比只有少量改动。首先，根据线程的块下标给 `segment` 赋不同值（第 3 行），以完成区段划分。每个区段的大小为 $2 \times \text{blockDim.x}$ ，也就是每块处理 $2 \times \text{blockDim.x}$ 个元素。把区段大小乘以块的 `blockIdx.x`，即可得到该块所处理区段的起始位置。例如，每块有 1024 个线程时，区段大小为 $2 \times 1024 = 2048$ ，块 0、块 1、块 2 的起始位置分别是 0、2048 和 4096。

确定每块的起始位置后，块内所有线程就可以把自己的区段当作完整输入来处理。把 `threadIdx.x` 加到块的区段起始位置，得到每个线程的所有者位置（第 4 行）。局部变量 `i` 保存线程在全局输入数组中的所有者位置，取代前面内核里的 `t`；第 5 行也相应改用 `i` 访问全局输入。

执行块内归约的其余代码保持不变，这里可以换入任意一种单块归约实现。最后一项差异是：每个块的线程 0（首领线程）不再直接把部分和写成全局输出值，而是用原子加法把它累加到输出中（第 18–20 行）。原子操作保证各线程块的首领线程不会在累加部分和时发生竞态。

用原子操作把各部分和累加为总和，有一个细微但重要的性质：线程块可以按任意相对次序贡献结果，因为原子操作只保证互斥，并不规定顺序。因此，要用原子操作把归约推广到多个区段，运算符必须同时满足交换律和结合律。

也可以不用原子操作。各线程块的首领线程可以按块下标把结果写入部分和数组，再启动一个只有单块的新内核生成最终结果；还可以把部分和数组复制到主机内存，由 CPU 完成归约。在原子操作不可用或代价过高的系统中，用单块网络或主机完成最后一步更合适。这两种方法也让程序员更容易控制部分和的累加次序。

译者注：底本图 10.18 的参数 `N` 没有在函数体中使用。底本还以 `BLOCK_DIM` 表示预期

```
1 __global__ void reduce_kernel(float* input, float* output, unsigned int N)
2 {
3     unsigned int segment = 2*blockDim.x*blockIdx.x;
4     unsigned int i = segment + threadIdx.x;
5     float partialSum = input[i] + input[i + BLOCK_DIM];
6     partialSum = warp_reduce(partialSum);
7
8     __shared__ float partialSums_s[BLOCK_DIM/WARP_SIZE];
9     if(threadIdx.x%WARP_SIZE == 0) {
10         partialSums_s[threadIdx.x/WARP_SIZE] = partialSum;
11     }
12     __syncthreads();
13
14     if(threadIdx.x < WARP_SIZE) {
15         float partialSum = (threadIdx.x < BLOCK_DIM/WARP_SIZE) ?
16         partialSums_s[threadIdx.x] : 0.0f;
17         partialSum = warp_reduce(partialSum);
18         if(threadIdx.x == 0) {
19             cuda::atomic_ref<float, cuda::thread_scope_device>
20             output_ref(*output);
21             output_ref.fetch_add(partialSum, cuda::memory_order_relaxed);
22         }
23     }
24 }
```

图 10.18: 使用原子操作的多块求和归约内核

块宽；以下讨论假定它等于 `blockDim.x`。在此前提下，代码只能处理若干完整区段。每个区段包含 `2*BLOCK_DIM` 个元素，因此输入长度必须是该区段大小的整数倍。这里的“任意长度”是指输入不再受单线程块 2048 个元素的上限约束，而不是已经处理了末尾不完整区段。底本练习 5 专门要求为非整段长度补上边界处理，本译稿保留这项递进关系，没有替练习预先补写。图中第二阶段的部分和加载沿用图 10.16 的单位元填充兼容性扩展，适用条件也与该图相同。此外，调用方必须先把 `*output` 初始化为加法单位元 0.0；否则原子累加会叠加到原有值上。底本没有明说这项前置条件。

10.10 用线程粗化降低开销

目前的归约内核都试图用尽可能多的线程来最大化并行度：归约 N 个元素时启动 $N/2$ 个线程。每块 1024 个线程时，需要 $N/2048$ 个线程块。然而，处理器的执行资源有限，硬件可能只能并行执行其中一部分块；多出的块会被串行调度，每当一个旧块完成后再执行一个新块。

为了并行化归约，我们把工作分散到多个线程与线程块，实际上付出了很高代价。前几节已经看到，归约树每进入下一阶段，更多线程都会闲置，硬件利用率随之降低；而且每个启动的线程块都会经历这段低利用率阶段。如果各块确实并行运行，这项代价不可避免；但如果硬件原本就要把它们串行化，不如由程序以更高效的方式自行串行化。第 6 章介绍过 **线程粒度粗化 (thread granularity coarsening)**，简称线程粗化：把一部分工作串行地交给更少线程，降低并行化开销。下面先把更多元素分给每个线程块，以展示线程粗化后的并行归约，再说明它如何减轻硬件利用不足。

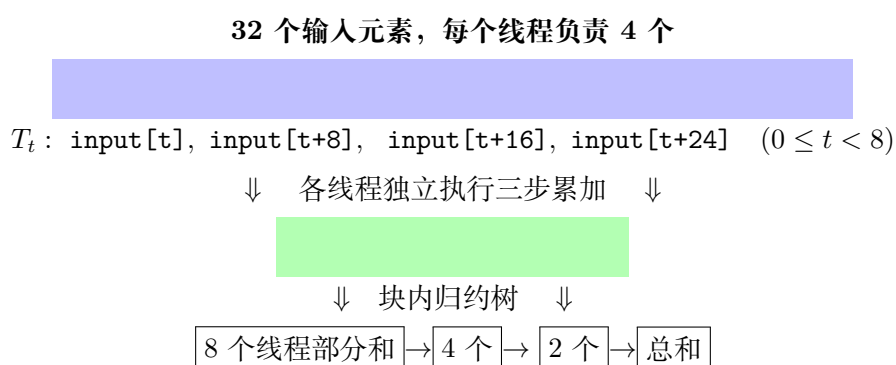


图 10.19: 归约中的线程粗化

图 10.19 把线程粗化应用于图 10.9 的例子。图 10.9 中，每个块接收 16 个元素，也就是每个线程处理 2 个元素。每个线程从全局内存加载两个元素时独立把它们相加，然后所有线程协作执行归约树。图 10.19 的线程块按因子 2 粗化，于是每块接收两倍元素，即 32 个元素，每个线程处理 4 个。线程先在加载时独立累加自己的 4 个元素，再共同执行归约树。累加 4 个元素需要的三个步位于图的最前面，此时所有线程都活跃并完成有效工作；各线程独立处理自己的元素，无须同步。剩余的归约树步

骤与粗化前相同。

```

1 __global__ void CoarsenedSumReductionKernel(float* input, float* output) {
2     ...
3     unsigned int segment = COARSE_FACTOR*2*blockDim.x*blockIdx.x;
4     unsigned int i = segment + threadIdx.x;
5     float partialSum = input[i];
6     for(unsigned int c = 1; c < COARSE_FACTOR*2; ++c) {
7         partialSum += input[i + c*BLOCK_DIM];
8     }
9     ...
10 }
11

```

图 10.20: 采用线程粗化的求和归约内核

图 10.20 展示了多块归约内核中实现线程粗化的关键代码；底本用省略号表示与图 10.18 相同的块内归约和原子累加部分。两者有两项主要区别。第一，计算块内区段起点时乘以 `COARSE_FACTOR`，反映每个区段扩大了该倍数（第 3 行）。第二，不再只把线程负责的两个元素相加，而是用粗化循环按 `COARSE_FACTOR` 遍历并累加这些元素（第 6–8 行）。整个粗化循环中所有线程都活跃，部分和累加到局部变量 `partialSum`；线程彼此独立，所以循环内不调用 `__syncthreads()`。

译者注：底本图 10.20 的函数定义漏写返回类型 `void`，本译稿按本章其他 `__global__` 内核的声明方式补上。代码中的两处省略号属于底本原有的差异展示，并非译稿删节。

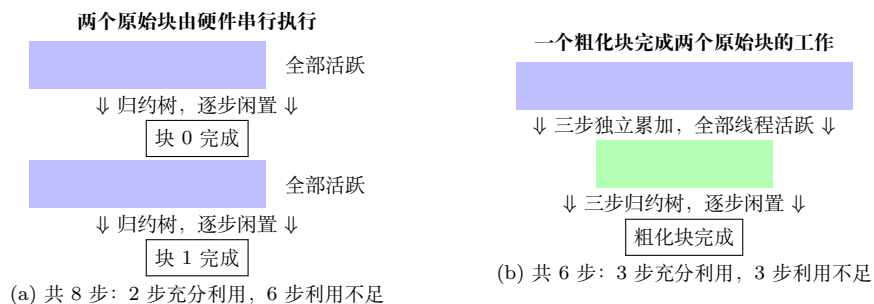


图 10.21: 比较采用与不采用线程粗化的并行归约

图 10.21 比较了两种执行方式：图 10.21(a) 是两个未粗化的原始线程块被硬件串行执行；图 10.21(b) 是一个粗化块完成两个原始块的工作。在图 10.21(a) 中，第一个块先执行一个步，每个线程把自己负责的两个元素相加。此时所有线程都活跃，硬件得到充分利用。接下来的三个步执行归约树，每一步都有一半线程退出，硬件利用不足；每一步还需要屏障同步和共享内存访问。第一个块完成后，硬件再调度第二个块，在另一个数据区段重复同样步骤。两个块合计需要 8 个步，其中只有 2 个步充分利用硬件，另外 6 个步利用不足并需要屏障同步和共享内存访问。


```

步 0  $\text{input}[8+k] \leftarrow \text{input}[8+k] + \text{input}[k], 0 \leq k < 8$ 
步 1  $\text{input}[12+k] \leftarrow \text{input}[12+k] + \text{input}[8+k], 0 \leq k < 4$ 
步 2  $\text{input}[14+k] \leftarrow \text{input}[14+k] + \text{input}[12+k], 0 \leq k < 2$ 
步 3  $\text{input}[15] \leftarrow \text{input}[15] + \text{input}[14]$ 

```

4. 修改图 10.20 的内核，使它执行最大值归约而不是求和归约。
5. 修改图 10.20 的内核，使它能处理不一定为 `COARSE_FACTOR*2*blockDim.x` 整数倍的任意长度输入。为内核增加一个表示输入长度的参数 `N`。
6. 假设要对下面的输入数组执行并行归约：

$\{6, 2, 7, 4, 5, 8, 3, 1\}$.

分别采用以下内核时，画出每次迭代后数组内容的变化：

- a. 图 10.5 中未经优化的内核；
- b. 图 10.8 中针对访问合并与分歧优化的内核。

参考文献

- [1] N. Bell, J. Hoberock, Thrust: a productivity-oriented library for CUDA, in: *GPU Computing Gems Jade Edition*, Elsevier, 2012, pp. 359–371.
- [2] NVIDIA, Thrust: the C++ parallel algorithms library, 2025. <https://nvidia.github.io/cccl/thrust/index.html>.
- [3] NVIDIA, CUB, 2025. <https://nvidia.github.io/cccl/cub/index.html>.

第三部分

高级模式与应用

第十七章 稀疏矩阵计算

本章讨论的并行模式是稀疏矩阵计算。所谓稀疏矩阵，是指其中绝大多数元素为零的矩阵。把这些零元素原封不动地存下来并参与运算，会同时浪费内存容量、内存带宽、执行时间和能耗。现实世界中大量重要问题都归结为稀疏矩阵计算，因此人们发展出了多种稀疏矩阵存储格式以及与之配套的处理方法，并在工程实践中广泛使用。

这些方法的共同思路是用某种**紧凑化**（compaction）手段，把零元素从表示中挤出去，从而不必存储它们、也不必对它们做无意义的乘加。代价是数据表示变得不再规整：元素的位置需要显式记录，行与行之间的长度参差不齐。而不规整性正是并行计算的天敌——它会带来内存带宽利用不足、控制流分歧和负载不均衡。

因此，稀疏矩阵格式设计的核心是在**紧凑化**与**规整化**（regularization）之间取得平衡。有的格式把紧凑化做到极致，但表示极不规整；有的格式只做适度紧凑，却换来更规整的结构。更重要的是，究竟哪种格式跑得快，严重依赖于具体矩阵中非零元素的分布——这与我们前面各章讨论的绝大多数内核都不同，那些内核的性能主要由算法和硬件决定，而不由输入数据的取值决定。理解稀疏矩阵存储格式及其并行算法，能为读者提供处理这类“紧凑化与规整化”矛盾的一般性思路。

17.1 背景

稀疏矩阵在科学计算、工程仿真和金融建模中随处可见。一个典型来源是线性方程组：矩阵的每一行对应一个方程，每一列对应一个未知量，矩阵元素就是方程中的系数。在许多科学与工程问题里，虽然未知量和方程的总数极其庞大，但它们之间只是**稀疏耦合**的——每个方程只牵涉到少数几个未知量。于是矩阵的每一行都只有少量非零元素。

以图 17.1 中的 4×4 矩阵为例。第 0 行只在第 0 列和第 1 列上有非零元素，说明方程 0 只包含未知量 x_0 和 x_1 ；第 1 行在第 0、2、3 列上非零，说明方程 1 包含 x_0 、 x_2 和 x_3 ；第 2 行涉及 x_1 和 x_2 ；第 3 行则只涉及 x_3 。整个矩阵的 16 个元素中只有 8 个非零，一半的存储被浪费在零上。真实问题中的稀疏度往往还要极端得多：非零元素占比常常低于 1%。

	x_0	x_1	x_2	x_3
行 0	1	7	0	0
行 1	5	0	3	9
行 2	0	2	8	0
行 3	0	0	0	6

图 17.1: 一个简单的稀疏矩阵示例。着色格子为非零元素；本章后续各节都以这个矩阵作为统一示例，展示不同存储格式的代表方式

用矩阵求解含 N 个未知量、 M 个方程的线性方程组时，问题通常写成

$$A \times X + Y = 0,$$

其中 A 是 $M \times N$ 矩阵， X 是 N 维未知量向量， Y 是 M 维常数向量。目标是求出满足所有方程的 X 。最直观的思路是求逆： $X = A^{-1} \times (-Y)$ 。对中等规模的矩阵，高斯消元一类方法确实可以做到这一点。但把它套用到真实的稀疏矩阵上有两个致命问题。第一，稀疏矩阵的规模往往大到让这种直接法完全不可行。第二，稀疏矩阵求逆的结果通常远比原矩阵稠密——消元过程会在原本为零的位置上生成大量新的非零元素，这些新增元素称为**填充元** (fill-in)。因此在实际问题中，计算并存储逆矩阵通常是不现实的。

替代方案是迭代法。当稀疏矩阵 A 正定（即对 $\mathbb{R}^n$ 中任意非零向量 x 都有 $x^T A x > 0$ ）时，可以用共轭梯度法 [1] 迭代求解对应的线性方程组，并且保证收敛。共轭梯度法的基本循环是：先猜一个 X ，算出 $A \times X + Y$ ，看结果是否足够接近零向量；若不够接近，则用梯度公式修正 X ，再算一轮 $A \times X + Y$ 。这类线性方程组的迭代解法，与第 八 章介绍的微分方程迭代解法在思想上一脉相承。

译者注：第 八 章即模板计算一章。本译稿目前尚未译出第 11–16 章，本章的章节交叉引用只指向已完成的章节，其余仅以章号文字说明。

迭代解法中最耗时的部分，正是反复计算 $A \times X + Y$ 。当 A 为稀疏矩阵、而 X 和 Y 通常是稠密向量（即绝大多数元素非零）时，这个运算被称为 **SpMV**（Sparse Matrix-Vector multiplication and accumulation，稀疏矩阵向量乘加）。由于它的重要性，标准库普遍以 SpMV 之名提供该接口。本章就用 SpMV 作为载体，来比较不同存储格式之间的取舍。

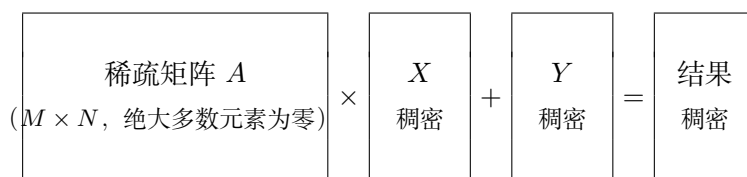


图 17.2: 矩阵向量乘加运算示意。矩阵 A 稀疏，而向量 X 与 Y 通常稠密

各种稀疏矩阵存储格式的首要目标，都是把零元素从表示中彻底剔除。这样做的

收益不止于省下存储空间：它同时省去了从内存取回这些零元素的开销，也省去了与它们做无用乘加的计算开销，从而显著降低对内存带宽和计算资源的消耗。对超大规模矩阵而言，还有一层更实际的意义——压缩后的矩阵可能刚好能整个装进单个计算节点的内存。否则就只能求助于外存算法或分布式计算，而这两者都要付出高昂的 I/O 或网络通信延迟。

设计一种稀疏矩阵存储格式时，需要权衡的关键因素包括：

- **空间效率** (space efficiency, 或称紧凑度)：用该格式表示矩阵需要占用多少内存容量。
- **灵活性** (flexibility)：向矩阵中增删非零元素有多容易。
- **可访问性** (accessibility)：该格式让哪些类型的数据变得易于访问——例如“给定一个非零元素，取它的行号和列号”，或者“给定一行，取该行全部非零元素”。
- **内存访问效率** (memory access efficiency)：对某项具体计算而言，该格式能否支撑高效的内存访问模式。这是规整化的一个侧面。
- **负载均衡** (load balance)：对某项具体计算而言，该格式能否让不同线程的工作量大致相当。这是规整化的另一个侧面。

本章接下来逐一介绍几种存储格式，并在每一节末尾用上述五个维度作统一比较。

17.2 COO 格式与最简单的 SpMV 内核

第一种格式是**坐标格式** (Coordinate, 简称 COO)。它的想法直白到近乎朴素：既然要把零元素扔掉，那就把每个非零元素连同它的“坐标”一起记下来。于是 COO 用三个等长的一维数组来表示矩阵——value 存非零值，rowIdx 存对应的行号，colIdx 存对应的列号。三个数组中下标相同的三个元素共同描述同一个非零元素。

图 17.3 给出了图 17.1 中矩阵的 COO 表示。例如 $A[0,0] = 1$ 被存放在 value[0]，它的行号 0 存在 rowIdx[0]，列号 0 存在 colIdx[0]。

	0	1	2	3	4	5	6	7
rowIdx	0	0	1	1	1	2	2	3
colIdx	0	1	0	2	3	1	2	3
value	1	7	5	3	9	2	8	6

图 17.3: 图 17.1 中矩阵的 COO 表示。三个数组等长，长度等于非零元素个数

COO 把零元素清除得干干净净，但它引入了 rowIdx 和 colIdx 两个额外数组作为开销。对图 17.1 这样零元素并不比非零元素多多少的小例子，这份开销甚至超过

了省下的空间。但只要矩阵足够稀疏，收益就会压倒开销：如果一个矩阵只有 1% 的元素非零，那么包含全部索引开销在内，COO 表示大约只占稠密表示的 3%。

图 17.3 中的非零元素是先按行号、再按列号排好序的。实践中 COO 数据往往确实是有序的，但**排序并非 COO 的要求**。本节给出的 SpMV 内核对无序的 COO 数据同样正确。

并行化：一个线程负责一个非零元素

用 COO 表示做并行 SpMV 的一种自然方式，是给每个非零元素分配一个线程。图 17.4 展示了这种划分：8 个非零元素对应 8 个线程，线程 i 只处理 `value[i]`。

	T_0	T_1	T_2	T_3	T_4	T_5	T_6	T_7
rowIdx	0	0	1	1	1	2	2	3
colIdx	0	1	0	2	3	1	2	3
value	1	7	5	3	9	2	8	6

T_0, T_1 → 都要更新 `y[0]`，必须用原子加
 T_2, T_3, T_4 → 都要更新 `y[1]`，必须用原子加
 T_5, T_6 → 都要更新 `y[2]`，必须用原子加
 T_7 → 更新 `y[3]`

图 17.4: 以 COO 格式并行化 SpMV：每个线程负责一个非零元素。同色的线程映射到同一行，因而会写同一个输出元素

对应的内核见图 17.5。线程先算出自己负责的非零元素下标并做边界检查，然后分别从 `rowIdx`、`colIdx` 和 `value` 三个数组中取出该元素的行号、列号和数值，接着用列号去输入向量 `x` 中取值、与非零值相乘，最后把乘积累加到行号对应的输出元素上。

```

1 __global__ void spmv_coo_kernel(COOmatrix cooMatrix, float* x, float* y
  ) {
2     unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
3     if(i < cooMatrix.numNonzeros) {
4         unsigned int row = cooMatrix.rowIdx[i];
5         unsigned int col = cooMatrix.colIdx[i];
6         float value = cooMatrix.value[i];
7         atomicAdd(&y[row], x[col]*value);
8     }
9 }
10

```

图 17.5: 并行 SpMV/COO 内核

累加**必须**使用原子操作。原因在图 17.4 中一目了然：映射到同一行的多个线程会同时更新同一个输出元素， T_0 和 T_1 都要写 $y[0]$ 。

由于 SpMV 的代码总是与它所假设的存储格式绑死，本章统一在内核名字里带上格式后缀，把图 17.5 的代码称为 SpMV/COO。

译者注：上面的内核只做累加、不做初始化，因此调用前必须保证 y 已按 Y 的值（或按 0）填好。这与 $A \times X + Y$ 的“乘加”语义一致，但很容易被忽略。底本对此未作说明，本译稿在此提醒。

COO 的五维评价

空间效率：留到介绍完其他格式后再一并比较。

灵活性：COO 极为灵活。只要三个数组同步地重排，非零元素在数组中的顺序可以任意打乱而不丢失任何信息。图 17.6 演示了这一点：重排之后 $value[0]$ 装的是原矩阵第 1 行第 3 列的元素，但因为行号和列号跟着一起搬了家，它在原矩阵中的位置仍能被正确还原。

	0	1	2	3	4	5	6	7
rowIdx	1	3	0	2	1	0	2	1
colIdx	3	3	1	2	0	0	1	2
value	9	6	7	8	5	1	2	3

图 17.6: 重排后的 COO 表示。三个数组同步重排，表示的矩阵与图 17.3 完全相同

换句话说，SpMV/COO 可以按任意顺序处理这些元素：只要保证 $value$ 中每个元素都被处理过一次， $rowIdx[i]$ 指定的那个 y 元素总会收到 $value[i] \times x[colIdx[i]]$ 这份贡献，最终结果与处理顺序无关。

读者也许会问：为什么要重排？一个原因是数据可能来自某个不按特定顺序给出非零元素的文件，而我们仍需要一种一致的表示方式；正因为如此，COO 是矩阵**构建阶段**的常用格式。另一个原因是，不要求任何顺序保证，就意味着新增非零元素只需在三个数组末尾各追加一项即可；正因为如此，COO 也是矩阵在计算过程中**需要被修改**时的常用格式。第 17.5 节还会展示 COO 灵活性的另一处妙用。

可访问性：COO 让“给定一个非零元素，取它的行号和列号”变得非常容易，这正是 SpMV/COO 能够按非零元素并行的前提。反过来，COO 并不方便“给定一行（或一列），取出该行（列）的全部非零元素”。因此，如果计算需要按行或按列遍历矩阵，COO 就不是好选择。

内存访问效率：参见图 17.4 中线程与数组元素的对应关系——相邻线程访问三个数组中的相邻元素。因此 SpMV/COO 对矩阵数据的访问是**合并的**。

负载均衡：每个线程只负责一个非零元素，工作量完全相同，因此除边界线程外

不会出现控制流分歧。

SpMV/COO 的主要缺陷是必须使用原子操作，而这有两重麻烦。其一，正如前面各章所见，原子操作延迟高、吞吐量低。其二，多个原子操作作用于同一输出元素的**先后顺序是不确定的**；当操作对象是浮点数时，这种不确定性会带来数值稳定性问题（参见附录 A）。

要避免原子操作，就得让同一行的所有非零元素都归同一个线程处理，使该线程成为对应输出元素的唯一写者。但 COO 恰恰不提供这种可访问性。下一节介绍的格式正好补上这一块。

17.3 用 CSR 格式按行分组非零元素

上一节的结论是：只要能“给定一行，取出该行全部非零元素”，就能把整行交给同一个线程，从而彻底摆脱原子操作。提供这种可访问性的格式是**压缩稀疏行格式**（Compressed Sparse Row，简称 CSR）。

CSR 与 COO 一样，把非零值放在一维的 `value` 数组里，把每个非零值的列号放在 `colIdx` 数组的相同位置上。区别在于两点。第一，CSR 中的非零元素**按行分组**：先放第 0 行的全部非零元素，再放第 1 行的，依此类推。第二，CSR 用一个 `rowPtrs` 数组取代了 COO 的 `rowIdx` 数组。

图 17.7 给出了图 17.1 中矩形的 CSR 表示。第 0 行的非零元素（1 和 7）排在最前，接着是第 1 行的（5、3、9），然后是第 2 行的（2、8），最后是第 3 行的（6）。

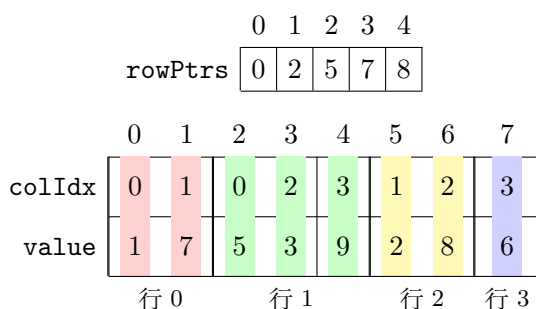


图 17.7: 图 17.1 中矩形的 CSR 表示。非零元素按行分组，`rowPtrs` 记录每行在 `value/colIdx` 中的起始偏移

数组 `rowPtrs` 的第 r 项，给出第 r 行的非零元素在 `colIdx` 和 `value` 中的起始位置。例如它的第 0 项为 0，表示第 0 行从 `value[0]` 开始；第 1 项为 2，表示第 1 行从 `value[2]` 开始，依此类推。注意 `rowPtrs` 的长度是行数加一：最后一项的值 8，记录的是一个并不存在的“第 4 行”的起始位置。这个额外的哨兵项纯粹是为了方便——许多算法需要用“下一行的起始位置”来界定当前行的结束位置，有了它，第 3 行的结束位置就无须特殊处理。

图 17.7 中每一行内部的非零元素还按列号升序排好了序。这样排序能带来更好的内存访问模式，但**同样不是必需的**：行内非零元素乱序时，本节的内核依然正确。当行内确实按列号有序时，value 数组（以及 colIdx 数组）的布局可以理解为“把所有零元素剔除之后的矩阵行主序布局”。

并行化：一个线程负责一行

用 CSR 做并行 SpMV 的自然方式是给每一行分配一个线程，如图 17.8 所示。

	0	1	2	3	4	5	6	7
value	1	7	5	3	9	2	8	6
	↑		↑			↑		↑
	T_0		T_1			T_2		T_3

各线程在点积循环中依次访问的下标

T_0 : 0, 1 T_1 : 2, 3, 4 T_2 : 5, 6 T_3 : 7

图 17.8: 以 CSR 格式并行化 SpMV：每个线程负责一行。第一次迭代时，四个线程分别访问 value[0]、value[2]、value[5] 和 value[7]，彼此相隔很远

对应的内核见图 17.9。线程先确定自己负责的行号并做边界检查，然后用 rowPtrs 中相邻的两项分别找到该行非零元素的起点和终点，在这个区间上做点积循环。循环体中，线程取出当前非零元素的列号和数值，用列号去 x 中查值、相乘，并累加到局部变量 sum 上。循环开始前 sum 被初始化为 0，循环结束后再一次性写回输出向量。

```

1  __global__ void spmv_csr_kernel(CSRMatrix csrMatrix, float* x, float* y
    ) {
2      unsigned int row = blockIdx.x*blockDim.x + threadIdx.x;
3      if(row < csrMatrix.numRows) {
4          float sum = 0.0f;
5          for(unsigned int i = csrMatrix.rowPtrs[row];
6              i < csrMatrix.rowPtrs[row + 1]; ++i) {
7              unsigned int col = csrMatrix.colIdx[i];
8              float value = csrMatrix.value[i];
9              sum += x[col]*value;
10         }
11         y[row] += sum;
12     }
13 }
14

```

图 17.9: 并行 SpMV/CSR 内核

请注意，把 `sum` 累加到输出向量时**不需要原子操作**：每一行只由一个线程遍历，因此每个线程写的都是互不相同的输出元素。这正是 CSR 相对 COO 的核心收益。

CSR 的五维评价

空间效率：CSR 比 COO 更省空间。COO 需要 `rowIdx`、`colIdx` 和 `value` 三个数组，长度都等于非零元素个数。CSR 只有 `colIdx` 和 `value` 两个数组的长度等于非零元素个数，第三个数组 `rowPtrs` 的长度只是行数加一，通常远小于非零元素个数。

灵活性：在增加非零元素这件事上，CSR 不如 COO 灵活。COO 只需在数组末尾追加（前提是预留了足够空间），而 CSR 必须把新元素插进它所属的那一行——这意味着后续所有行的非零元素都要向后搬移，`rowPtrs` 中后续所有行的偏移也都要递增。因此向 CSR 矩阵中添加非零元素的代价远高于 COO。

可访问性：CSR 让“给定一行，取出该行全部非零元素”变得容易，这正是 SpMV/CSR 能够按行并行、从而免去原子操作的基础。在真实的稀疏矩阵应用中，矩阵通常有数千到数百万行，每行含有几十到几百个非零元素，按行并行看起来非常合适：线程数量足够多，每个线程的工作量也足够可观。但也有例外——某些应用中矩阵的行数可能不足以喂饱 GPU 的全部线程，这时 COO 反而能榨出更多并行度，因为非零元素总是比行多。此外，CSR 并不方便“给定一列，取出该列全部非零元素”；若应用确实需要按列访问，可能得额外维护一份按列组织的表示，例如第 17.7 节介绍的 CSC 格式。

内存访问效率：这是 CSR 的软肋。看图 17.8 中点积循环第一次迭代时的访问模式——相邻线程访问的数据元素相距甚远：线程 0、1、2、3 分别访问 `value[0]`、`value[2]`、`value[5]` 和 `value[7]`；第二次迭代它们分别访问 `value[1]`、`value[3]`、`value[6]`，而线程 3 已经无事可做。结果是 SpMV/CSR 对矩阵的访问**无法合并**，内存带宽利用效率低下。

负载均衡：这是 CSR 的另一个软肋。线程在点积循环中的迭代次数取决于它负责的那一行有多少非零元素。而非零元素在各行之间的分布可能相当随机，相邻的两行完全可能一行有 3 个非零元素、另一行有 300 个。结果是大多数甚至全部 warp 中都可能出现严重的控制流分歧。

小结一下：CSR 相对 COO 的优势是空间效率更高，并且提供了“按行访问”的能力，从而让 SpMV/CSR 能够按行并行、避开原子操作；劣势则是增删非零元素不灵活、内存访问不合并、控制分歧严重。接下来几节讨论的格式，会用一部分空间效率去交换更好的访问合并与更低的控制分歧。

顺带一提，在 GPU 上把 COO 转换成 CSR 是一道很好的练习题：它恰好要用到直方图（统计每行的非零元素个数）和前缀和（把计数转成偏移）这两个基础并行原语，参见本章练习 3。

17.4 用 ELL 格式改善内存访问合并

SpMV/CSR 访问不合并的根源在于：每一行的长度不同，所以各行的起始偏移在 `value` 数组中毫无规律，相邻线程自然落在相距甚远的位置上。要修好这一点，就得先把“行长参差”这个不规整性消掉。办法是**填充** (padding) 加**转置** (transposition)。这正是 ELL 格式的思路，它的名字来自求解椭圆边值问题的软件包 ELLPACK 中的稀疏矩阵模块 [2]。

理解 ELL 最方便的途径是从 CSR 出发，如图 17.10 所示。第一步，找出非零元素最多的那一行，然后给其余各行在末尾补上**填充元素**，让所有行等长。这样，原本参差不齐的行就被拉成了一个规整的矩形。在我们的小例子里，第 1 行最长（3 个非零元素），于是给第 0 行补 1 个、第 2 行补 1 个、第 3 行补 2 个填充元素，图中用带 * 的格子表示。注意 `colIdx` 数组也要同步填充，以保持与 `value` 的一一对应。

第二步，把这个矩形按**列主序**铺开：先把第 0 列的所有元素放在连续的内存位置上，接着放第 1 列的所有元素，依此类推。这等价于把矩形按 C 语言的行主序做一次转置。在我们的例子里，转置之后 `value[0]` 到 `value[3]` 依次是 1、5、2、6，也就是各行的第 0 个元素；`colIdx[0]` 到 `colIdx[3]` 则是这些元素的列号。

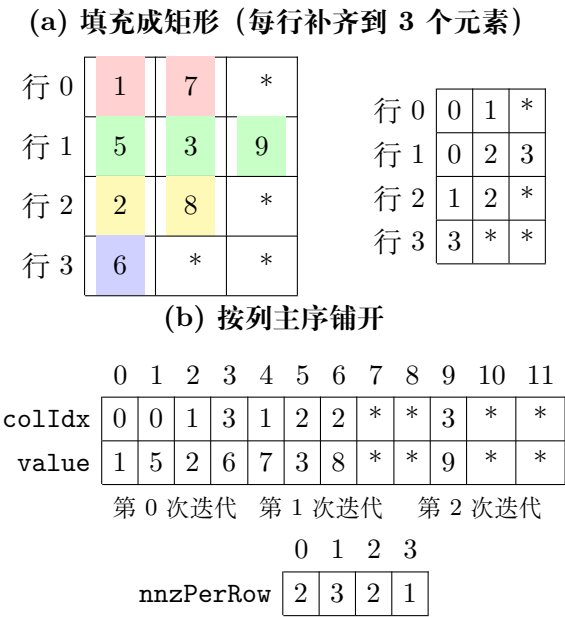


图 17.10: ELL 存储格式示例。先把各行填充到等长，再按列主序铺开；* 表示填充元素

由于所有行等长，`rowPtrs` 数组不再需要了：第 r 行的起点就是 `value[r]`。而从第 r 行的当前元素走到下一个元素，只需把下标加上原矩阵的行数。例如第 2 行的第 0 个元素在 `value[2]`，它的下一个元素就在 `value[2+4]`，即 `value[6]`——这里的 4 就是原矩阵的行数。

并行化：一个线程负责一行，但按列主序取数

图 17.11 展示了 SpMV/ELL 的并行划分，图 17.12 给出对应内核。与 CSR 一样，每个线程负责一行；不同之处在于线程取数的下标算法。

	0	1	2	3	4	5	6	7	8	9	10	11
value	1	5	2	6	7	3	8	*	*	9	*	*
	T_0	T_1	T_2	T_3	T_0	T_1	T_2			T_1		

图 17.11: 以 ELL 格式并行化 SpMV。第 0 次迭代中四个线程访问 value[0..3]，地址连续，访问得以合并

```

1  __global__ void spmv_ell_kernel(ELLMatrix ellMatrix, float* x, float* y
    ) {
2      unsigned int row = blockIdx.x*blockDim.x + threadIdx.x;
3      if(row < ellMatrix.numRows) {
4          float sum = 0.0f;
5          for(unsigned int t = 0; t < ellMatrix.nnzPerRow[row]; ++t) {
6              unsigned int i = t*ellMatrix.numRows + row;
7              unsigned int col = ellMatrix.colIdx[i];
8              float value = ellMatrix.value[i];
9              sum += x[col]*value;
10         }
11         y[row] += sum;
12     }
13 }
14

```

图 17.12: 并行 SpMV/ELL 内核

内核假设输入矩阵带有一个 nnzPerRow 向量，记录每行真实的非零元素个数，使线程只需在自己那一行的有效元素上迭代。即使没有这个向量也不影响正确性——线程可以一律迭代到最大行长，因为填充元素的值为 0，乘进去不会改变结果，只是白白多做了几轮乘加。

由于压缩后的矩阵按列主序存放，第 t 次迭代中该元素在一维数组里的下标为

$$i = t \times \text{numRows} + \text{row}.$$

row 是由 threadIdx.x 直接导出的，所以**相邻线程的 i 也相邻**——对 value 和 colIdx 的访问因此是合并的。这正是 ELL 相对 CSR 的核心收益。

ELL 的五维评价

空间效率：ELL 因填充元素而比 CSR 更费空间，而且开销的大小高度依赖非零元素的分布。最糟糕的情形是极少数行的非零元素特别多——这时所有其他行都要被填充到那个极端长度，产生海量填充元素。

举个更贴近现实的例子。设一个 1000×1000 的稀疏矩阵有 1% 的元素非零，即平均每行 10 个非零元素。算上索引开销，CSR 表示大约只占稠密表示的 2%。现在假设其中恰好有一行含 200 个非零值，而其余各行都不到 10 个。用 ELL 就得把所有行都填充到 200 个元素，表示规模一举膨胀到稠密表示的约 40%，是 CSR 表示的 20 倍。

这说明我们需要一种手段，在从 CSR 转换到 ELL 时控制填充元素的数量。第 17.5 节给出的正是这样一种手段。

灵活性：在增加非零元素这件事上，ELL 反而比 CSR 灵活。CSR 中插入一个非零元素要搬移后续所有行、并更新它们的行偏移；而在 ELL 中，只要目标行还有填充元素，直接把某个填充元素替换成真实值即可。更进一步，如果给 ELL 数组预留了余量，那么即使向最长的那一行添加元素，也只需把矩形整体扩出一列、给其他行各补一个填充元素，无须搬移任何已有元素。在这种情形下，ELL 的灵活性与 COO 相当。

可访问性：ELL 同时具备 CSR 和 COO 两种可访问性。图 17.12 已经展示了如何“给定行号，访问该行的非零元素”。而“给定非零元素的下标，反推它的行号和列号”同样可行：列号可以直接从 `colIdx[i]` 读出；行号则可以利用填充后表示的规整性算出来。由下标公式 $i = t \times \text{numRows} + \text{row}$ 以及 $\text{row} < \text{numRows}$ 可得

$$\text{row} = i \bmod \text{numRows}.$$

这种双向可访问性让 ELL 既能按行并行、也能按非零元素并行。不过，按非零元素并行时会有一部分线程被分配到填充元素上，白做无用功——这是它相对 COO 的一处劣势。

内存访问效率：如前所述，按列主序排布让相邻线程访问相邻内存位置，访问得以合并，内存带宽利用率因而显著提升。某些 GPU 架构（尤其是较老的世代）对访问合并有更严格的地址对齐要求；这时可以在转置之前给矩阵末尾补上几行，强制 SpMV/ELL 内核的每一次迭代都对齐到架构规定的对齐单位（例如 64 字节）。

负载均衡：ELL 在这方面毫无改善。每个线程仍然要按自己那一行的非零元素个数循环，因此 SpMV/ELL 的控制分歧与 SpMV/CSR 一样严重。

小结：ELL 相对 CSR 的改进在于增删更灵活、可访问性更强，以及最关键的——为 SpMV/ELL 带来了内存访问合并。代价是空间效率不如 CSR，而控制分歧问题依然存在。下一节将同时改善这两点。

译者注：底本在推导 $\text{row} = i \bmod \text{numRows}$ 时把下标公式的出处误标为“图 17.9” (Sp-

MV/CSR 内核)，实际应为图 17.12 (SpMV/ELL 内核)。本译稿已改为指向对应的 SpMV/ELL 内核，即图 17.12。

17.5 用混合 ELL-COO 格式调节填充量

ELL 的两个毛病——空间效率低和控制分歧——有一个共同的病灶：极少数超长行拖累了全体。只要有办法把这些超长行的“超出部分”**挪走**，填充量和迭代次数就会同时下降。而挪到哪里去呢？答案正是 COO——第 17.2 节埋下的伏笔在这里揭晓。

具体做法是：在把稀疏矩阵转成 ELL 之前，先设定一个每行保留的元素上限 K ；超过 K 的那些非零元素被摘出来，单独放进一份 COO 表示。剩下的部分用 SpMV/ELL 计算，摘出来的部分用 SpMV/COO 计算，两者的结果累加到同一个输出向量上即可——回忆一下，COO 允许以任意顺序处理元素，而且它本来就用原子加写输出，所以两份贡献能安全地汇合。这种“用两种格式协同完成一次计算”的思路，通称**混合方法** (hybrid method)。

图 17.13 用一个 7 行的矩阵演示了这一过程。若单用 ELL，第 1 行（5 个非零元素）和第 6 行（4 个非零元素）拉高了行长上限，其余各行要被填充到 5 个元素，全表共 $7 \times 5 = 35$ 个槽位承载 17 个非零元素，填充元素多达 18 个，而且所有线程都得循环 5 次。取 $K = 2$ ，把第 1 行末尾的 3 个元素和第 6 行末尾的 2 个元素移入 COO 后，ELL 部分缩成 $7 \times 2 = 14$ 个槽位，填充元素只剩 2 个，所有线程只需循环 2 次。

读者可能会担心：为了分离出 COO 部分而额外做的这些工作，会不会得不偿失？答案是**看情况**。如果一个稀疏矩阵只参与一次 SpMV，这份额外开销确实可能相当可观。但在大量真实应用中，SpMV 是在迭代求解器里**对同一个稀疏矩阵反复执行**的：每次迭代变化的只是 x 和 y 向量，而矩阵元素对应的是被求解线性方程组的系数，逐次迭代并不改变。因此，构造混合表示的一次性开销可以被摊薄到成百上千次迭代上。第 17.6 节还会再次用到这个“预处理开销可摊薄”的论证。

混合 ELL-COO 的五星评价

空间效率：优于纯 ELL，因为填充量被显著压低了。

灵活性：优于纯 ELL。两种格式都可以通过“替换填充元素”来向有余量的行添加非零元素；区别在于当目标行已经没有填充元素时——纯 ELL 必须整体扩出一列，而混合格式只需往 COO 部分末尾追加一项。

可访问性：这是混合格式付出的代价。“给定行号，取出该行全部非零元素”不再总是可行了——只有完全装进 ELL 部分的行才能这样访问；一旦某行溢出到了 COO 部分，要凑齐它的全部非零元素就得去 COO 里做一次昂贵的搜索。

(a) 原矩阵各行的非零元素 (列号: 值)

行	非零元素	个数
0	(0:3) (3:1)	2
1	(0:2) (1:4) (2:1) (5:5) (7:7)	5
2	(4:9)	1
3	(2:6) (6:2)	2
4	(1:8) (5:3)	2
5	(7:4)	1
6	(0:1) (2:5) (3:2) (6:6)	4

着色部分被移入 COO 部分

(b) ELL 部分 ($K = 2$, 按列主序)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
colIdx	0	0	4	2	1	7	0	3	1	*	6	5	*	2
value	3	2	9	6	8	4	1	1	4	*	2	3	*	5

第 0 次迭代

第 1 次迭代

(c) COO 部分 (被摘出的 5 个元素)

	0	1	2	3	4
rowIdx	1	1	1	6	6
colIdx	2	5	7	3	6
value	1	5	7	2	6

图 17.13: 混合 ELL-COO 表示示例。把超长行的尾部元素移入 COO 后, ELL 部分的填充元素从 18 个降到 2 个, 迭代次数从 5 次降到 2 次

内存访问效率：SpMV/ELL 和 SpMV/COO 对矩阵的访问都是合并的，两者的组合自然也是合并的。

负载均衡：把超长行的尾巴摘掉之后，ELL 部分各行的长度趋于一致，SpMV/ELL 的控制分歧随之下降；而被摘走的那些元素落到了 COO 部分，那里本来就是“一个线程一个非零元素”，不存在控制分歧。两头都受益。

小结：相对于纯 ELL，混合 ELL-COO 格式减少了填充从而提升了空间效率，保留了添加非零元素的灵活性，保留了合并的内存访问模式，并降低了控制分歧。付出的代价是可访问性上的一点损失：对溢出到 COO 部分的行，要取齐它的全部非零元素变得困难。

译者注：底本在描述这一节的示例时，先说“第 1 行和第 6 行的非零元素最多”，随后又说“移走第 2 行末尾的 3 个元素和第 6 行末尾的 2 个元素”，行号前后不一致。本译稿改用自建的 7 行示例，行号、元素个数和填充量前后自治，读者对照原书图 17.13 时请注意这一处编辑疏漏。

17.6 用 JDS 格式降低控制分歧

上一节用“摘掉超长行的尾巴”来同时改善空间效率和控制分歧。本节介绍另一条路：**完全不做填充**，而是把行**按长度排序**后再按列主序铺开。排序之后的矩阵形似一个三角形（各行长度单调变化，边缘呈锯齿状），这种格式因此得名**锯齿对角存储格式**（Jagged Diagonal Storage，简称 JDS）。

图 17.14 演示了如何用 JDS 存储图 17.1 中的矩阵。步骤如下。

第一步，和 CSR、ELL 一样，把非零元素按行分组。

第二步，按每行的非零元素个数对行排序（这里采用从多到少的降序）。排序时要额外维护一个 `row` 数组，记录每个位置上的行在**原矩阵**中的行号；每交换两行，就同步交换 `row` 数组中对应的两项。这样就始终能追溯回原始位置。在我们的例子中，各行的非零元素个数依次为 2、3、2、1，降序排序后的顺序是原来的第 1、0、2、3 行，于是 `row = [1, 0, 2, 3]`。

第三步，把排好序的非零元素及其列号**按列主序**存入 `value` 和 `colIdx`。因为没有填充，各“列”的长度并不相同，所以需要有一个 `iterPtr` 数组来标记每一次迭代所对应的那一段非零元素从哪里开始。

图 17.15 展示了 SpMV/JDS 的并行划分。每个线程负责排序后的一行，沿着该行的非零元素做点积，并用 `iterPtr` 定位每次迭代的起点。从图中可以看出，同一次迭代中各线程访问的是 `value` 和 `colIdx` 中的**连续位置**，因此访问是合并的。SpMV/JDS 内核的实现留作练习（见本章练习 5）。



图 17.14: JDS 存储格式示例。行按长度排序后按列主序铺开, 无须任何填充元素; iterPtr 标记每次迭代对应的段起点, row 保存原始行号

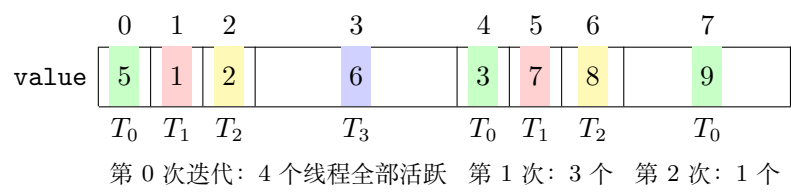


图 17.15: 以 JDS 格式并行化 SpMV。因为行已按长度排序, 同一 warp 内的线程迭代次数接近, 控制分歧显著降低

分段变体

JDS 还有一个常见变体：行排好序之后，把它们**切分成若干段**。由于已经排过序，同一段内各行的非零元素个数大致相当。于是可以为**每一段**单独生成一份 ELL 表示——段内只需把各行填充到**本段最长行**的长度即可。相比对整个矩阵做一份 ELL，这样能大幅削减填充元素。在这个变体中，`iterPtr` 不再需要，取而代之的是一个记录各 ELL 段起始位置的段偏移数组。

排序会不会算错？开销值不值？

读者应当追问两个问题。

第一个问题：把行重新排序，会不会导致线性方程组的解出错？不会。回忆线性代数中的基本事实——**任意交换方程组中方程的次序，解不变**。矩阵的每一行就是一个方程，只要在重排行的同时把 y 向量的对应元素一起重排，就等价于重排了方程的次序，解依然正确。唯一需要补做的一步是：算完之后用 `row` 数组把结果向量还原回原来的次序。

第二个问题：排序的开销会不会太大？这里的答案与上一节的混合方法如出一辙——只要 SpMV/JDS 是被放在迭代求解器里反复调用的，排序以及最终结果的还原都只需做一次，其成本可以摊薄到求解器的成百上千次迭代上。

JDS 的五维评价

空间效率：JDS 比 ELL 更省空间，因为它完全不做填充。分段 ELL 变体虽然有填充，但填充量也远少于对整个矩阵做一份 ELL。

灵活性：JDS 在添加非零元素上很不方便，甚至比 CSR 还差——添加元素会改变行的长度，而行长一变就可能需要重新排序。

可访问性：JDS 类似 CSR，允许“给定行号，访问该行的非零元素”；但不像 COO 和 ELL 那样，能“给定一个非零元素，直接反推它的行号和列号”。

内存访问效率：JDS 和 ELL 一样按列主序存放非零元素，因此访问同样是合并的。但两者有一点微妙差别：JDS 不做填充，各次迭代的起始地址（见图 17.15 中 `iterPtr` 标出的段边界）可以落在任意位置，没有简单廉价的办法把它们强制对齐到架构规定的对齐边界上。因此在对对齐敏感的架构上，JDS 的访存效率可能略逊于 ELL。

负载均衡：这是 JDS 最独特的价值。行按长度排序之后，同一个 warp 内的线程所负责的行长度相近，迭代次数因而接近一致。所以 JDS 在**降低控制分歧**方面非常有效。

译者注：底本在本节开头说“把行按长度排序，比如从最长到最短”，随后描述具体步骤时却写成“按每行非零元素个数升序排序”，前后不一致；而其后“排序后的矩阵形似三角矩阵”的说法，以及 JDS 这一名称的由来，都对应降序排列。本译稿统一采用降序（从最长到最短），图 17.14 与图 17.15 均按此绘制。需要说明的是，就 SpMV 的正确性和降低控制分歧的效果而言，升序与降序并无本质差别，关键只在于让长度相近的行落进同一个 warp。

17.7 用 CSC 格式支持按列访问

到目前为止，COO 和 ELL 让我们能“给定一个非零元素，取它的行号和列号”，而 CSR、ELL 和 JDS 让我们能“给定一行，取该行的全部非零元素”。但对 SpMV 之外的某些计算，我们关心的是**按列遍历**——给定一列，取出该列的全部非零元素。这时该用 **压缩稀疏列格式**（Compressed Sparse Column，简称 CSC）。

CSC 与 CSR 高度对称，只是把行和列的角色互换了。非零值仍然存在一维的 value 数组里，但这次是**按列分组**：先放第 0 列的非零元素，再放第 1 列的，依此类推。每个非零值的行号存在 rowIdx 数组的相同位置上。最后，一个 colPtrs 数组记录每一列的非零元素在 rowIdx/value 中的起始偏移，其长度为列数加一，末项同样是界定最后一列结束位置的哨兵。

图 17.16 给出了图 17.1 中矩阵的 CSC 表示：第 0 列的非零元素是 1 和 5，第 1 列是 7 和 2，第 2 列是 3 和 8，第 3 列是 9 和 6。与 CSR 一样，列内非零元素按行号升序排列并非必需。

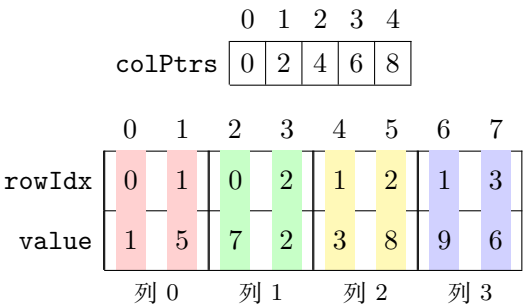


图 17.16: 图 17.1 中矩阵的 CSC 表示。与 CSR 对称，只是行列角色互换

用 CSC 做 SpMV：一个反面教材

CSC 本来就不是为 SpMV 设计的。但把它用在 SpMV 上仍有价值——作为一个对照，它能把前面几节反复出现的各项取舍集中呈现出来。

用 CSC 做 SpMV 时，给每一列分配一个线程。线程先取出该列对应的输入向量元素 $x[col]$ ，然后沿着该列的非零元素循环：对每个非零元素，取出它的行号，把

$x[col]$ 与非零值相乘，再**原子地**累加到行号对应的输出元素上。内核见图 17.17。

```

1  __global__ void spmv_csc_kernel(CSCMatrix cscMatrix, float* x, float* y
    ) {
2      unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;
3      if(col < cscMatrix.numCols) {
4          float x_val = x[col];
5          for(unsigned int i = cscMatrix.colPtrs[col];
6              i < cscMatrix.colPtrs[col + 1]; ++i) {
7              unsigned int row = cscMatrix.rowIdx[i];
8              float value = cscMatrix.value[i];
9              atomicAdd(&y[row], x_val*value);
10         }
11     }
12 }
13

```

图 17.17: 并行 SpMV/CSC 内核

这里必须用原子操作：遍历不同列的线程完全可能同时去更新同一个输出元素——在图 17.1 的矩阵中，负责第 0 列和第 1 列的两个线程就都要写 $y[0]$ 。

译者注：底本中这段 SpMV/CSC 内核所配的图题（图 17.18）误写成“A parallel SpMV/CSR kernel”，与代码内容不符；本译稿已在图 17.17 的标题中改正为 Sp-MV/CSC。

CSC 的五维评价

空间效率：在行数与列数相当的前提下，CSC 与 CSR 相同——两个长度等于非零元素个数的数组（`rowIdx` 和 `value`），加上一个长度为列数加一的 `colPtrs`。

灵活性：与 CSR 一样不灵活。新增的非零元素必须插进它所属的那一列，后续所有列的元素都要搬移，`colPtrs` 中后续所有偏移都要递增。

可访问性：CSC 让“给定一列，取该列的全部非零元素”变得容易。如前所述，这种可访问性对 SpMV 并无帮助（SpMV 不需要按列遍历），但对另一些计算很有用，稍后再谈。

内存访问效率：与 CSR 同病相怜。看图 17.1 的 CSC 表示，线程 0、1、2、3 在第一次迭代中分别访问 `value[0]`、`value[2]`、`value[4]` 和 `value[6]`，第二次迭代分别访问 `value[1]`、`value[3]`、`value[5]` 和 `value[7]`——相邻线程访问的位置彼此相隔很远，因此 SpMV/CSC 对矩阵的访问**不合并**。此外，对输出向量的访问还需要原子操作。

负载均衡：与 CSR 同样糟糕。线程的迭代次数取决于它负责的那一列有多少非零元素，而非零元素在各列间的分布可能相当随机，于是大多数甚至全部 warp 中都可能出现严重的控制分歧。（图 17.1 这个小例子恰好每列都是 2 个非零元素，掩盖了这个问题；真实矩阵中各列长度差异可以非常大。）

综合来看，用 CSC 做 SpMV 简直是把 COO 和 CSR 的缺点集于一身：像 COO 一样需要对输出向量做原子操作，又像 CSR 一样对输入矩阵访问不合并、且控制分歧严重。后两个问题可以借鉴 ELL 和 JDS 的手法（填充加转置、按长度排序）来部分缓解，但对输出向量的原子操作是躲不掉的。

不过 CSC 在 SpMV 中也并非一无是处，它有一个独特优点：**对输入向量的访问模式极好**。其他所有格式的 SpMV 内核都是用列号去 x 中随机取值，而 SpMV/CSC 是相邻线程负责相邻列、因而加载 x 中相邻的值（图 17.17 第 4 行），访问是合并的；更妙的是， x 中每个值只被加载一次，然后被该列的所有非零元素复用。可惜这一点优点抵不过对矩阵和输出向量的种种不利，更别提控制分歧，所以就 SpMV 而言 CSC 整体上仍是不合适的格式。

CSC 真正的用武之地

CSC 适用于那些**按列遍历更自然**的计算。举两个例子。

其一，若要做的是**向量矩阵乘法**而非矩阵向量乘法，即输入向量作第一个操作数、矩阵作第二个操作数，那么输出向量的每个元素就是输入向量与矩阵对应**列**的点积。这本质上就是一次按列遍历，CSC 天然契合。

其二，若做的仍是矩阵向量乘法，但**输入向量本身也是稀疏的**——这种运算称为稀疏矩阵稀疏向量乘法（SpMSpV）——那么我们可以利用输入向量的稀疏性：凡是输入向量中取值为零的分量，其对应的**整列**矩阵元素都可以整体跳过。而“整列跳过”之所以可行，正是因为 CSC 把同一列的非零元素聚在了一起。

17.8 小结

本章把稀疏矩阵计算作为一种重要的并行模式加以介绍。稀疏矩阵广泛出现在各类复杂现象的建模中。由于零元素数量巨大，人们用紧凑化技术削减存储、内存访问和无效计算；而紧凑化带来的不规整性，又要靠**规整化**手段来补救——本章展示的两类规整化手段分别是 **混合方法**（ELL-COO）与**排序/分段**（JDS）。有意思的是，某些规整化技术（如 ELL 的填充）反而会把零元素重新塞回压缩表示中；而混合方法的作用，恰恰是遏制“塞回去的零太多”这种病态情形。

各格式在五个设计维度上的对比汇总于表 17.1。

需要强调的是：并行 SpMV 内核的执行效率和内存带宽效率，都**取决于输入矩**

维度	COO	CSR	ELL	混合 ELL-COO	JDS
空间效率	中	高	低（填充）	较高	高（无填充）
增删灵活性	高	低	较高	高	很低（需重排序）
按非零访问行列号	是	否	是	部分	否
按行取全部非零	否	是	是	部分	是
访问是否合并	是	否	是	是	是（难对齐）
控制分歧	无	严重	严重	较低	低
是否需要原子操作	是	否	否	是（COO 部分）	否

表 17.1: 本章各稀疏矩阵存储格式在 SpMV 场景下的对比。CSC 未列入，因为它并非为 SpMV 设计；就 SpMV 而言它兼有 COO 的原子操作开销与 CSR 的访问不合并和控制分歧

阵中数据的分布。这与本书此前研究的绝大多数内核截然不同，但这种“性能依赖数据”的行为在真实应用中相当普遍。这正是并行 SpMV 之所以是一个重要并行模式的原因之一——它足够简单，却揭示了许多复杂并行应用中的关键行为。建议读者用不同的稀疏数据集实际做些实验 [3]，亲身体会本章各个 SpMV 内核在性能上的数据依赖性。

还要指出，稀疏表示相对稠密表示的优势，只有在矩阵**非常稀疏**时才真正兑现。稠密矩阵的好处是下标是隐式的，而且很容易做分块以优化数据在存储层次中的搬运；稀疏格式则必须显式存储并访问索引，也更难施加分块技术。只有当稀疏度足够高、被剔除的零元素足够多时，付出这些代价才划算。

一个常被提及的现象是：无论在 CPU 还是 GPU 上，稀疏矩阵计算能达到的 FLOPS 数值都远低于稠密矩阵计算，低到常常令人意外。读完本章之后，就不该再感到意外了。

最后，鉴于稀疏矩阵计算的重要性，cuSPARSE 库 [4] 为多种稀疏矩阵存储格式提供了 GPU 加速的基础线性代数例程。本章的内容为读者有效使用这类库、或者针对自己关心的特定稀疏计算开发更精巧的格式与例程，打下了必要的基础。

练习

1. 考虑下面这个稀疏矩阵：

$$\begin{bmatrix} 1 & 0 & 7 & 0 \\ 0 & 0 & 8 & 0 \\ 0 & 4 & 3 & 0 \\ 2 & 0 & 0 & 1 \end{bmatrix}$$

- 分别写出它在以下格式下的表示：(a) COO；(b) CSR；(c) ELL；(d) JDS；(e) CSC。
2. 设有一个整数稀疏矩阵，含 m 行、 n 列、 z 个非零元素。分别写出用 (a) COO、(b) CSR、(c) ELL、(d) JDS 表示该矩阵各需要多少个整数。若已知信息不足以作答，请指出还缺少哪些信息。
 3. 用直方图和前缀和这两个基础并行原语，实现从 COO 到 CSR 的格式转换。
 4. 实现构造混合 ELL-COO 表示的主机端代码，并用它完成一次 SpMV：ELL 部分在设备上通过内核计算，COO 部分的贡献在主机上计算。
 5. 实现一个使用 JDS 格式的并行 SpMV 内核。
 6. 第 17.6 节指出，SpMV/JDS 各次迭代的起始地址难以对齐到架构规定的边界。请说明：如果把 JDS 改造成第 17.6 节提到的“分段 ELL”变体，这个对齐问题能否得到缓解？代价是什么？
 7. 第 17.2 节提到，SpMV/COO 中原子操作的执行顺序不确定，可能带来浮点数值稳定性问题。请说明：SpMV/CSR 是否也存在同样的问题？为什么？
 8. 混合 ELL-COO 格式中，每行保留的元素上限 K 是一个可调参数。请分析： K 取得过大或过小分别会带来什么后果？如果已知矩阵各行长度的分布，你会如何选择 K ？

参考文献

- [1] M.R. Hestenes, E. Stiefel, et al., *Methods of Conjugate Gradients for Solving Linear Systems*, vol. 49, NBS, Washington, DC, 1952.
- [2] J.R. Rice, R.F. Boisvert, *Solving Elliptic Problems Using ELLPACK*, vol. 2, Springer Science & Business Media, 2012.
- [3] N. Bell, M. Garland, Implementing sparse matrix-vector multiplication on throughput-oriented processors, in: *Proc. Conf. on High Performance Computing Networking, Storage and Analysis*, 2009, pp. 1–11.
- [4] NVIDIA, cuSPARSE, 2025. <http://docs.nvidia.com/cuda/cusparse>.

术语表（待扩充）

GPU	图形处理器（Graphics Processing Unit）
CPU	中央处理器（Central Processing Unit）
CUDA	NVIDIA 的通用并行计算编程平台与模型
throughput	吞吐量
latency	延迟
data parallelism	数据并行
memory bandwidth	内存带宽
speedup	加速比
SM	流式多处理器（Streaming Multiprocessor）
GPC	GPU 处理集群（GPU Processing Cluster）
warp	CUDA 中以 32 个线程为常见实现大小的线程调度单元
occupancy	占用率
global memory	全局内存
shared memory	共享内存
arithmetic intensity	算术强度
tiling	分块

Roofline model

Roofline 模型

memory coalescing

内存访问合并

DRAM burst

DRAM 突发

cache line

缓存行

channel 内存通道**bank conflict**

bank 冲突

vector load/store

向量加载/存储

thread coarsening

线程粗化

coarsening factor

粗化因子

stencil 模板; 差分模板**stencil sweep**

模板扫掠

structured grid

结构化网格

finite difference

有限差分

boundary condition

边界条件

register tiling

寄存器分块

loop unrolling

循环展开

instruction scheduling

指令调度

double-buffering

双缓冲

true dependence

真依赖

false dependence

假依赖

privatization

私有化

histogram

直方图

histogram bin

分箱

owner-computes rule

所有者计算规则

output interference

输出干扰

race condition

竞态条件

atomic operation

原子操作

atomic reference

原子引用

contention

争用

contiguous partitioning

连续划分

interleaved partitioning

交错划分

warp-level primitive

warp 级原语

warp shuffle

warp shuffle (warp 内线程寄存器数据交换)

lane index

lane 下标 (线程在 warp 内的下标)

reduction

归约

reduction tree

归约树

associativity

结合律; 结合性

commutativity

交换律; 交换性

work 工作量**span** 跨度**memory consistency model**

内存一致性模型

bottleneck

瓶颈

convolution

卷积

convolution filter

卷积滤波器

filter radius

滤波器半径

ghost cell

ghost cell (幽灵单元)

halo cell halo cell (晕影单元)**input tile**

输入瓦片

output tile

输出瓦片

constant memory

常量内存

constant cache

常量缓存

sparse matrix

稀疏矩阵

SpMV 稀疏矩阵向量乘加 (Sparse Matrix-Vector multiplication and accumulation)

compaction

紧凑化

regularization

规整化

space efficiency

空间效率

flexibility

灵活性

accessibility

可访问性

load balance

负载均衡

fill-in 填充元 (矩阵求逆或消元过程中新产生的非零元素)

Conjugate Gradient method

共轭梯度法

COO 坐标格式 (Coordinate format)

CSR 压缩稀疏行格式 (Compressed Sparse Row)

CSC 压缩稀疏列格式 (Compressed Sparse Column)

ELL ELL 格式 (源自 ELLPACK 软件包)

JDS 锯齿对角存储格式 (Jagged Diagonal Storage)

hybrid method

混合方法

padding element

填充元素

column-major order

列主序

transposition

转置

SpMSpV 稀疏矩阵稀疏向量乘法